

```
<h1>오늘의 메뉴</h1>
```

```
<ul> <li class="menu">아메리카노</li>
```

```
</ul>
```

```
.menu { color: #2F6B57; font-size: 18px; }
```

```
.card { display: flex; gap: 12px; }
```

```
@media (max-width: 768px) { .card { display: block; } }
```

2 권 · 구조와 CSS 기초

HTML 과 CSS

HTML 로 글에 이름표를 붙이고, CSS 로 그 모양을 정합니다.

적힌 결과는 전부 진짜 크롬으로 열어서 재어 본 값입니다.

5

단원

27

개념

173

직접 해보기

140

문제

차례

06	시맨틱 구조	11
01	div 와 span	12
02	시맨틱 태그란	22
03	페이지 뼈대 태그	33
04	본문 구획 태그	40
05	전체 구조 만들기	52
07	CSS 시작하기	75
01	CSS 란 무엇인가	76
02	CSS 를 HTML 에 연결하는 세 가지 방법	87
03	규칙의 문법	97
04	색 표기법	111
05	상속과 기본값	127
08	선택자	153
01	기본 선택자	154
02	여러 개 고르기	167
03	관계로 고르기	180
04	상태로 고르기	194
05	순서로 고르기	209
06	명시도	222
09	텍스트와 폰트	251
01	글자 크기와 단위	252
02	글꼴	265
03	줄과 간격	280
04	정렬과 꾸미기	296
05	넘치는 글자	311

10	박스모델	337
01	모든 것은 상자다	338
02	padding (안쪽 여백)	351
03	border (테두리)	363
04	margin (바깥 여백)	378
05	width 와 box-sizing	394
06	배경과 넘침	409

부록 가	연습문제·종합연습 정답	437
------	---------------------	-----

부록 나	심화문제 정답	499
------	----------------	-----

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

시작 전 준비 (처음 한 번만)

자바스크립트 자료와 달리 **설치할 프로그램이 거의 없습니다**. HTML 과 CSS 는 브라우저만 있으면 바로 돌아갑니다.

1. 브라우저

크롬(Chrome) 또는 **엣지(Edge)** 를 쓰세요. 이 자료의 설명은 크롬 기준입니다. 이미 깔려 있을 것입니다.

2. VS Code 설치

코드를 쓰는 편집기입니다. <https://code.visualstudio.com> 에서 받아 설치하세요.

설치 후 VS Code 를 열고 **파일 → 폴더 열기**로 이 **HTMLCSS자료** 폴더를 통째로 여세요. 파일 하나만 여는 것보다 폴더째 여는 편이 훨씬 편합니다.

꼭 하세요 — 화면 나누기 HTML/CSS 는 **코드와 결과를 동시에 봐야** 배워집니다. 화면 왼쪽 절반에 VS Code, 오른쪽 절반에 브라우저를 띄워 두세요. 윈도우에서는 창을 잡고 화면 왼쪽·오른쪽 끝으로 밀면 반씩 채워집니다.

3. 파일을 브라우저로 여는 법

.html 파일을 **더블클릭**하면 브라우저가 열립니다. 그게 전부입니다.

더블클릭했는데 **VS Code** 가 열린다면 — 파일 우클릭 → 연결 프로그램 → 크롬(또는 엣지) → **"항상 이 앱으로 열기"** 체크. VS Code 를 설치하면 자주 이렇게 됩니다.

이것만은 먼저 (제일 많이 하는 실수)

.html 파일은 **저장한다고 화면이 저절로 바뀌지 않습니다**.

VS Code 에서 고친다 → Ctrl + S 로 저장 → 브라우저로 가서 F5 (새로고침)

이 세 단계를 매번 해야 합니다. **"코드는 고쳤는데 화면이 그대로예요"**의 1순위 원인이 **저장을 안 했거나 새로고침을 안 한 것**입니다. 이 자료의  **직접 해보기**는 전부 이 과정을 요구합니다.

파일 읽는 법

개념 파일 (개념01_..., 개념02_...)

브라우저와 VS Code 로 동시에 여세요. 둘을 번갈아 보는 것이 이 자료를 쓰는 방법입니다.

- 브라우저 화면에는 섹션 제목, 짧은 요약, 그리고 실제 결과가 보입니다.
- VS Code 소스에는 그 결과가 어떻게 나왔는지 자세한 설명이 주석으로 들어 있습니다.

파일은 항상 같은 순서로 되어 있습니다.

1. 맨 위 주석 — 이 파일이 무엇을 다루는지, 앞 파일과 어떻게 이어지는지
2. 섹션 N – 제목 · 주제가 바뀌는 지점
3. 점선 상자 — 실제 결과가 그려지는 곳. 소스에서는 `<div class="doc-demo">` 입니다
4.  직접 해보기 · 거의 모든 섹션에 하나씩 있는 짧은 과제. 파란 상자로 화면에도 보입니다. (자주 하는 실수 와 정리 섹션에는 없습니다) 정답은 파일 맨 아래 직접 해보기 정답 주석 블록에 있습니다. 먼저 스스로 해 보세요
5. 자주 하는 실수 · 학생들이 실제로 자주 하는 실수 모음. 빨간 상자(잘못된 예)와 초록 상자(올바른 예)가 나란히 나옵니다
6. — 정리 — · 그 파일의 핵심을 대여섯 줄로 추린 것. 복습할 때 여기만 봐도 됩니다

결과를 적어 둔 주석

주석	뜻	쓰는 곳
<code><!-- 화면: ... --></code>	이 코드가 브라우저에 어떻게 보이는지	HTML 안
<code>/* 화면: ... */</code>	이 규칙이 화면을 어떻게 바꾸는지	<code><style></code> 안

전부 "열어 보면 이렇게 나온다" 는 뜻입니다. 자료에 적힌 결과는 전부 진짜 크롬으로 열어서 재어 본 값입니다. 다르게 나오면 어딘가 잘못 친 것입니다.

파일 맨 아래에 아래는 자료 자동 검사용입니다 라고 적힌 주석 블록이 있습니다. 자료가 스스로 틀리지 않았는지 검사하는 장치입니다. 학습 내용이 아니니 넘어가세요.

연습 파일 (연습문제.html)

TODO가 적힌 빈칸을 채우는 파일입니다. 각 문제는 이렇게 생겼습니다.

```
<!-- —— 문제 1 —— (개념01 섹션2)
```

```
h1 태그로 "우리 동네 빵집" 이라는 제목을 만드세요.
```

```
기대 화면: 큰 굵은 글씨로 "우리 동네 빵집" 이 한 줄 보입니다.
```

```
이렇게 되면 틀린 것: 글씨 크기가 그대로면 h1 이 아니라 p 를 쓴 것입니다.
```

```
-->
```

- 기대 화면과 똑같이 나오면 정답입니다.
- 막히면 연습문제_정답.html 을 보세요. 다만 먼저 10분은 혼자 고민하고 보세요.
- 문제는 기본 → [응용] → [도전] 순서로 어려워집니다. [도전]은 못 풀어도 괜찮습니다.
- 맨 마지막 문제는 [확인] 문항입니다. 일부러 틀리게 써 둔 코드를 주고 "왜 아무 일도 안 일어나는지 찾아서 고치세요"를 시킵니다.

HTML 과 CSS 에는 에러 메시지가 없습니다

이 자료에서 가장 중요한 이야기입니다.

다른 프로그래밍 언어는 틀리면 빨간 글씨로 알려 줍니다. HTML 과 CSS 는 그러지 않습니다. 틀린 부분을 조용히 무시하고, 나머지만 그립니다.

```
태그 이름을 틀림 → 그 태그가 없는 셈 치고 안의 글자만 나옵니다
CSS 속성 이름을 틀림 → 그 한 줄만 버려집니다. 나머지는 멀쩡히 적용됩니다
세미콜론을 빠뜨림 → 그 줄과 다음 줄이 같이 버려지기도 합니다
클래스 이름이 안 맞음 → 아무 일도 일어나지 않습니다
```

그래서 배우는 방법이 다릅니다.

에러를 읽는 게 아니라, 화면이 생각과 다른지 눈으로 알아채야 합니다.

"아무 일도 안 일어난다"가 이 자료에서 가장 자주 만날 증상입니다. 당황하지 말고 아래 순서로 확인하세요.

- 1. 저장했나? (Ctrl + S) → 새로고침했나? (F5)
- 2. 오타는 없나? 특히 클래스 이름, 속성 이름, 콜론과 세미콜론
- 3. F12 를 눌러 개발자도구에서 실제로 어떻게 적용됐는지 본다 (바로 아래)

개발자도구(F12) — 이 자료의 필수 도구

브라우저에서 F12 를 누르면 개발자도구가 열립니다. HTML/CSS 는 이 도구 없이는 배우기 어렵습니다. 10만원부터는 사실상 필수입니다.

하고 싶은 것	방법
이 글자가 어느 태그인지 보기	화면에서 우클릭 → 검사
지금 적용된 CSS 보기	Elements 탭 → 오른쪽 Styles
실제 계산된 값 보기	Elements 탭 → 오른쪽 Computed
박스 모델(여백) 그림 보기	Computed 탭 맨 위의 네모 그림
취소선 그어진 CSS	다른 규칙에 밀려서 안 먹은 것 (08단원 명시도)
휴대폰 크기로 보기	Ctrl + Shift + M (13단원)

F12 를 눌러도 아무 일이 없다면 — 노트북은 Fn + F12 이거나, Ctrl + Shift + I 로도 열립니다. 화면 아무 곳이나 우클릭 → 검사 도 됩니다.

Styles 패널에서는 값을 직접 고쳐 볼 수도 있습니다. 숫자를 클릭하고 위아래 화살표를 누르면 화면이 실시간으로 바뀝니다. 새로고침하면 원래대로 돌아오니 마음껏 만져 보세요.

혼자 공부할 때

1. 브라우저와 VS Code 를 나란히 띄웁니다. 이것부터 하세요.
2. 브라우저에서 결과를 보고, "이건 어느 코드에서 나온 거지?" 를 소스에서 찾습니다.
3. 소스의 주석 설명을 읽습니다. 여기가 진짜 본문입니다.
4.  직접 해보기 는 건너뛰지 마세요. 읽는 것과 직접 고쳐 보는 것은 완전히 다릅니다. 고치고 → 저장하고 → 새로고침하는 습관이 이 자료의 목표 중 하나입니다.
5. 자주 하는 실수 는 빨간 상자와 초록 상자를 비교하면서 읽습니다.
6. 단원이 끝나면 연습문제 를 풀고, 그다음 연습문제_정답 과 비교합니다.
7. 뭔가 이상하면 F12 를 먼저 누르세요. 답은 거의 항상 거기 있습니다.

파일을 망가뜨렸을까 봐 걱정된다면 — 이 폴더를 통째로 한 번 복사해 두고 시작하세요. 마음껏 고칠 수 있어야 빨리 늡니다.

시맨틱 구조

OPEN 06_시맨틱_구조

독서 모임 안내

매주 목요일 저녁 7시에 모입니다.

장소는 3층 열람실입니다.

첫 번째 div 입니다.

01 div 와 span

02 시맨틱 태그란

03 페이지 뼈대 태그

04 본문 구획 태그

05 전체 구조 만들기

- 연습문제

div 와 span

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — div 와 span

01단원부터 05단원까지 배운 태그에는 전부 “뜻” 이 있었습니다.

```
<h1>      이 글자는 제목이다
<p>       이 글자 덩어리는 문단이다
<ul>      이건 순서 없는 목록이다
<img />   여기에 그림이 하나 있다
<input /> 여기에 사람이 값을 적어 넣는다
```

그래서 태그를 고를 때 “이 글자가 무엇인가” 만 생각하면 됐습니다.

그런데 뜻이 하나도 필요 없는 순간이 있습니다. “이 세 줄을 그냥 한 덩어리로 묶어 두고 싶다” 같은 때입니다. 묶는 것 말고는 아무 뜻도 붙이고 싶지 않습니다.

그때 쓰는 태그가 div 와 span 입니다. 이 둘은 뜻이 없습니다. 화면도 아무것도 바꾸지 않습니다. 하는 일은 딱 하나, 여러 요소를 묶어서 손잡이를 만드는 것입니다.

묶기 전 <pre><h3>제목</h3> <p>본문</p> <p>날짜</p></pre>	→	묶은 뒤 <pre><div> <h3>제목</h3> <p>본문</p> <p>날짜</p> </div></pre>
따로 노는 세 조각		한 덩어리 + 그 안의 세 조각

왜 묶느냐고요? 07단원에서 CSS 를 배우면 답이 나옵니다. “이 덩어리 전체를 회색 배경으로”, “이 덩어리를 통째로 오른쪽으로” 같은 주문은 잡을 덩어리가 있어야 걸 수 있습니다. div 가 그 덩어리입니다.

지금은 CSS 를 아직 안 배웠으니, 이 파일에서 만드는 div 와 span 은 화면을 하나도 바꾸지 않습니다. 그게 정상입니다. 지금 배우는 것은 “나중에 잡을 손잡이를 미리 달아 두는 법” 입니다.

이 파일에서 만드는 태그들은 화면을 거의 바꾸지 않습니다. 그래서 결과 상자들이 미묘해 보입니다. 그게 맞습니다. 대신 코드가 어떻게 묶이는지를 보세요. 묶어 두면 07단원부터 그 묶음을 통째로 잡아 꾸밀 수 있게 됩니다.

뜻이 없는 상자가 왜 필요한가

섹션 1

<div> 는 여러 요소를 하나로 묶기만 하는 상자입니다. 묶어도 화면은 그대로입니다.

아래 두 상자에 들어 있는 글자는 완전히 같습니다. 다른 것은 코드뿐입니다.

— 묶기 전

```
<h3>독서 모임 안내</h3>
<p>매주 목요일 저녁 7시에 모입니다.</p>
<p>장소는 3층 열람실입니다.</p>
```

— 묶은 뒤

```
<div> <h3>독서 모임 안내</h3> <p>매주 목요일 저녁 7시에 모입니다.</p> <p>장소는 3층
열람실입니다.</p>
</div>
```

div 를 씌웠는데도 글자 크기, 줄 간격, 순서가 전부 그대로입니다. 높이까지 정확히 같습니다.

“그럼 왜 씌우느냐” 가 이 단원 전체의 질문입니다. 답은 두 가지입니다.

1) 사람이 읽기 좋아집니다.

코드가 200줄쯤 되면 "여기부터 여기까지가 안내문 한 덩어리" 라는 표시가 없으면 어디가 어디인지 못 찾습니다.

- 2) 나중에 통째로 다룰 수 있습니다.
07단원에서 "이 덩어리에 테두리를 둘러라" 라고 시킬 때,
시킬 대상이 있어야 합니다. div 가 그 대상입니다.

div 는 division(구획) 의 줄임말입니다. "여기부터 여기까지 한 칸" 이라는 뜻입니다.

화면 굵고 조금 큰 글씨로 "독서 모임 안내" 가 보이고,

그 아래 두 문단이 한 줄씩 차례로 보입니다

독서 모임 안내
매주 목요일 저녁 7시에 모입니다.
장소는 3층 열람실입니다.

화면 위 상자와 완전히 똑같이 보입니다. 달라진 곳이 한 군데도 없습니다

독서 모임 안내
매주 목요일 저녁 7시에 모입니다.
장소는 3층 열람실입니다.

두 상자의 높이는 같습니다. 눈으로도 자로 재도 같습니다. div 는 "묶는다" 는 것 말고는 아무 일도 하지 않기 때문입니다.

▣ 직접 해보기

1

두 번째 상자의 <div> 와 </div> 두 줄을 지웠다 새로고침(F5) 해 보세요. 화면이 하나도 안 바뀌는 것을 확인한 뒤 다시 되돌려 놓으세요.

div 는 한 줄을 통째로 차지한다

섹션 2

<div> 는 블록 요소입니다. 가로 한 줄을 혼자 다 쓰고, 다음 것은 아래로 내려갑니다.

02단원 개념04 에서 블록과 인라인을 배웠습니다. 다시 한 번 정의합니다.

블록(block) : 가로 한 줄을 혼자 다 차지합니다.
폭이 남아도 옆에 아무도 못 옵니다.
다음 요소는 아래로 내려갑니다.
h1~h6, p, ul, li, hr, table 이 블록입니다.

인라인(inline): 글자 딱 그만큼만 차지합니다.
옆에 다른 글자가 계속 이어붙습니다.
strong, em, a, img 가 인라인입니다.

div 는 블록입니다. 그래서 div 를 세 개 쓰면 세 줄이 됩니다.

여기서 p 와 헛갈리기 쉬운 점이 하나 있습니다. p 도 블록이라 세 줄이 되지만, p 에는 위아래 여백이 기본으로 붙어 있습니다. div 에는 그 여백이 없습니다. 그래서 div 세 개는 줄이 딱 붙어 나옵니다.

```
<p> → 블록 + 위아래 여백 16px 씩
<div> → 블록 + 여백 0
```

“왜 붙어 나오지?” 하고 놀라지 마세요. div 는 원래 이렇습니다. 여백은 10단원에서 CSS 로 직접 줍니다.

화면 세 줄이 위에서 아래로 바짝 붙어서 보입니다.

줄과 줄 사이에 빈 공간이 없습니다

```
첫 번째 div 입니다.
두 번째 div 입니다.
세 번째 div 입니다.
```

화면 세 줄이 보이는데, 줄과 줄 사이가 눈에 띄게 벌어져 있습니다

```
첫 번째 p 입니다.
두 번째 p 입니다.
세 번째 p 입니다.
```

두 상자 모두 “세 줄” 이라는 점은 같습니다. 둘 다 블록이기 때문입니다. 다른 것은 여백뿐입니다.

▫ 직접 해보기

2

위쪽 상자에 <div>네 번째 div 입니다.</div> 를 한 줄 더 넣어 보세요. 줄이 네 개가 되고, 여전히 서로 붙어 있는지 확인하세요.

span 은 글줄 안의 한 조각

섹션 3

 은 인라인 요소입니다. 문장 한가운데의 몇 글자만 콕 집어 묶을 때 씁니다.

span 은 02단원 개념04 에서 인라인과 함께 이미 배웠습니다. 여기서는 div 와 나란히 놓고 “덩어리냐 글자 조각이냐” 를 가릅니다.

```
<p>저는 <span>가운데</span> 조각입니다.</p>
```

span 을 씌운 “가운데” 세 글자는 겉보기에 아무 변화가 없습니다. 굵어지지도, 색이 바뀌지도 않습니다. 줄도 안 바뀝니다.

strong 과 비교하면 차이가 분명합니다.

`<strong>가운데</strong>` → 뜻이 있습니다. "이건 중요한 말이다"
 그래서 브라우저가 굵게 그려 줍니다.

`<span>가운데</span>` → 뜻이 없습니다. 아무 일도 안 일어납니다.
 "여기부터 여기까지" 라는 표시만 남습니다.

그럼 `span` 은 언제 쓸까요? "중요하지도 강조하지도 않는데, 이 몇 글자만 나중에 따로 꾸미고 싶을 때" 입니다.

`<p>가격 <span>12,000원</span></p>`
 → 12,000원 은 중요한 말이 아닙니다. 그냥 값입니다.
 그런데 나중에 이 숫자만 빨간색으로 하고 싶습니다. 그래서 `span` 을 씌워 둡니다.

`span` 은 인라인이라 옆에 계속 이어붙습니다. `span` 을 세 개 나란히 쓰면 세 줄이 아니라 한 줄이 됩니다. 이게 `div` 와 가장 큰 차이입니다.

화면 "저는 가운데 조각입니다." 가 한 줄로 보입니다.

"가운데" 도 나머지 글자와 똑같은 굵기, 똑같은 색입니다

저는 가운데 조각입니다.

화면 "빨강파랑노랑" 이 한 줄에 붙어서 보입니다. 세 줄이 되지 않습니다

빨강파랑노랑

`<span>` 사이에 띄어쓰기를 넣지 않으면 글자도 붙어 나옵니다. 띄우고 싶으면 태그 사이에 진짜 띄어쓰기를 한 칸 넣으세요.

▹ 직접 해보기 3

위 "빨강파랑노랑" 상자에서 `span` 과 `span` 사이에 띄어쓰기를 한 칸씩 넣어 "빨강 파랑 노랑" 으로 만들어 보세요.

같은 글자, `div` 와 `span`

섹션 4

둘 다 뜻이 없고 둘 다 화면을 안 바꿉니다. 다른 것은 한 줄을 다 먹느냐 아니냐뿐입니다.

말로만 하면 안 와닿습니다. 눈으로 보는 방법이 있습니다. 상자 뒤에 글자를 하나 더 붙여 보는 것입니다.

```
<div>안녕하세요</div>뒤에 붙인 글자
→ div 가 한 줄을 다 먹으므로 "뒤에 붙인 글자" 는 다음 줄로 밀려납니다.

<span>안녕하세요</span>뒤에 붙인 글자
→ span 은 글자 폭만 먹으므로 그 자리에서 바로 이어집니다.
```

이것이 블록과 인라인의 실제 차이입니다. 아래 두 상자에서 줄 수를 세어 보세요. 위는 두 줄, 아래는 한 줄입니다.

폭을 자로 재면 이렇게 나옵니다.

```
div → 824px (상자 안쪽 폭을 끝까지 다 씹니다)
span → 80px ("안녕하세요" 다섯 글자 폭만 씹니다)
```

배경색이 없어서 눈에는 안 보이지만, 차지한 자리는 이렇게 다릅니다.

화면 "안녕하세요" 가 한 줄, "뒤에 붙인 글자" 가 그 아래 줄.

모두 두 줄로 보입니다

안녕하세요뒤에 붙인 글자

화면 "안녕하세요뒤에 붙인 글자" 가 한 줄로 이어져 보입니다

안녕하세요뒤에 붙인 글자

고를 때 기준은 간단합니다. 덩어리를 묶으면 div, 글줄 속 몇 글자를 묶으면 span 입니다.

▫ 직접 해보기 4

위쪽 상자의 <div> 를 으로 바꿔 보세요(답는 태그도 같이). 두 줄이던 것이 한 줄로 합쳐집니다.

이름표 붙여 두기

섹션 5

묶기만 하면 나중에 "그 덩어리" 를 부를 방법이 없습니다. 그래서 class 라는 속성으로 이름을 붙여 둡니다.

class 는 01단원에서 배운 '속성' 중 하나입니다. 이렇게 씁니다.

```
<div class="post-card"> ... </div>
  ↳ 속성 이름   ↳ 값(내가 지은 이름)
```

이 이름은 화면에 안 보입니다. 사람이 지어 붙이는 꼬리표일 뿐입니다. 07·08단원에서 CSS 를 배우면 이 이름을 불러서 꾸미게 됩니다.

08단원에서 이렇게 씁니다(지금은 이런 게 있다는 것만 보세요).

```
.post-card { background-color: #eee; }  
→ "post-card 라는 이름표가 붙은 덩어리를 회색 배경으로"
```

이름 짓는 방법에도 약속이 있습니다.

- 영어 소문자로 씁니다.
- 단어 사이는 하이픈 한 개로 잇습니다. post-card, menu-item
- 생김새가 아니라 역할을 적습니다.
red-text 처럼 색을 적으면, 나중에 파랑으로 바꿀 때 이름이 거짓말이 됩니다.
price, warning 처럼 역할을 적으면 색이 바뀌어도 이름은 그대로 맞습니다.

하나의 요소에 class 를 붙이는 것도 됩니다. span 에도 붙일 수 있습니다. 아래 상자에서 확인하세요. class 를 붙여도 화면은 여전히 그대로입니다.

화면 굵고 조금 큰 글씨 "오늘 점심" 아래에 두 문단이 보입니다.

"가격 7,000원" 은 한 줄이고, 7,000원 도 보통 글씨입니다

오늘 점심
학교 앞 국숫집에 갔습니다.
가격 7,000원

class 이름에는 doc- 을 붙이지 마세요. doc- 으로 시작하는 이름은 이 수업 자료의 걸모양 전용입니다.

▫ 직접 해보기 5

위 상자의 두 번째 문단 전체를 class="place" 를 붙인 <div> 로 한 번 더 감싸 보세요. 화면이 그대로인지 확인하세요.

자주 하는 실수

섹션 6

div 와 span 은 아무 뜻도 없기 때문에 잘못 써도 화면이 멀쩡합니다. 그래서 틀린 줄 모르고 지나갑니다. 아래 다섯 가지를 눈에 익혀 두세요.

이런 실수를 합니다

div 로 제목을 만들

이런 실수를 합니다

실수 1 — 제목을 div 로 만들

```
<div>오늘의 공지</div>
```

오늘의 공지 글자가 보통 크기 그대로입니다. 제목처럼 보이지 않습니다. “나중에 CSS 로 크게 만들면 되지” 라고 생각하기 쉬운데, 크게 만들어도 제목이 되지는 않습니다. 검색엔진과 화면 낭독기에게는 여전히 그냥 글자 한 줄입니다. 자세한 이유는 개념02 에서 봅니다.

이렇게 쓰세요

```
<h3>오늘의 공지</h3>
```

이런 실수를 합니다

span 안에 div 를 넣음

이런 실수를 합니다

실수 2 — span 안에 div 를 넣음

```
<span>앞 글자 <div>가운데</div> 뒤 글자</span>
```

앞 글자 가운데 뒤 글자 한 줄이길 바랐는데 세 줄로 쪼개집니다. 에러는 안 납니다. 안에 넣은 div 가 블록이라 한 줄을 통째로 차지하기 때문입니다. 인라인 안에는 인라인만 넣으세요. 덩어리를 넣고 싶으면 바깥을 div 로 바꾸세요.

이런 실수를 합니다

줄을 바꾸려고 div 를 씀

이런 실수를 합니다

실수 3 — 줄을 바꾸려고 빈 div 를 씀

```
<div></div>
<div></div>
```

빈 div 는 높이가 0 입니다. 아무 일도 안 일어납니다. 여백을 만들려고 빈 div 를 늘어놓는 사람이 많은데, 화면은 그대로이고 코드만 지저분해집니다. 여백은 10단원에서 CSS 로 줍니다.

이런 실수를 합니다

div 로만 도배

이런 실수를 합니다

실수 4 — 전부 div 로만 씀

```
<div>
<div>우리 반 소식</div>
<div> <div>체육대회</div> <div>다음 주 화요일입니다.</div>
</div>
</div>
```

화면은 잘 나옵니다. 그래서 이게 실수라는 걸 알기가 가장 어렵습니다. 문제는 여는 div 가 열 개쯤 되면 어느 것이 어느 것의 짝인지 못 찾는다는 것입니다. 이 상태를 흔히 "div 수프" 라고 부릅니다. 제목은 h1~h6, 목록은 ul, 구획은 개념03·04 에서 배울 시맨틱 태그로 바꾸면 코드가 저절로 읽힙니다.

이런 실수를 합니다

div 에 뜻이 있다고 착각

이런 실수를 합니다

실수 5 — div 이름에 뜻이 담긴다고 생각함

```
<div class="header">우리 가게</div>
```

class="header" 라고 적었으니 브라우저가 "아, 머리말이구나" 하고 알아들을 것 같지만 전혀 그렇지 않습니다. class 값은 내가 지은 글자일 뿐이라 브라우저에게는 아무 뜻이 없습니다. class="banana" 라고 써도 결과는 똑같습니다. 진짜로 뜻을 전하려면 개념03 의 <header> 태그를 써야 합니다.

정리

▹ 직접 해보기 6

정답

1) 섹션 1 의 두 번째 상자에서 이 두 줄을 지웁니다.

```
<div id="noticeBox">
</div>
```

→ 화면이 하나도 안 바뀝니다. 글자 크기도 줄 간격도 그대로입니다.

div 는 묶는 일만 하기 때문입니다. 확인했으면 다시 붙여 두세요.

2) 섹션 2 위쪽 상자의 마지막 div 아래에 넣습니다.

```
<div>네 번째 div 입니다.</div>
```


→ 줄이 네 개가 되고, 네 줄이 여전히 서로 바짝 붙어 있습니다.

아래쪽 p 상자와 견주면 여백 차이가 더 잘 보입니다.

3) 섹션 3 아래쪽 상자를 이렇게 고칩니다.

```
<p id="spanRow"><span>빨강</span> <span>파랑</span> <span>노랑</span></p>
```

→ “빨강 파랑 노랑” 으로 사이가 한 칸씩 벌어집니다.

띄운 것은 span 이 아니라 태그와 태그 사이에 넣은 진짜 띄어쓰기입니다.

4) 섹션 4 위쪽 상자를 이렇게 고칩니다.

```
<span id="wrapDiv">안녕하세요</span>뒤에 붙인 글자
```

→ 두 줄이던 것이 “안녕하세요뒤에 붙인 글자” 한 줄로 합쳐집니다.

span 은 글자 폭만 차지하므로 뒤 글자가 그 자리에서 바로 이어집니다.

5) 섹션 5 상자의 두 번째 문단을 이렇게 감쌉니다.

```
<div class="place"> <p>학교 앞 국숫집에 갔습니다.</p>
</div>
```

→ 화면은 그대로입니다. 덩어리가 하나 더 생겼을 뿐입니다.

나중에 “장소 부분만 회색으로” 같은 주문을 걸 자리가 생긴 것입니다.

시맨틱 태그란

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 시맨틱 태그란

개념01 에서 div 는 “뜻이 없는 상자” 라고 했습니다. 뜻이 없다는 말은, 브라우저도 검색엔진도 그 상자를 보고 아무것도 짐작하지 못한다는 뜻입니다.

```
<div>우리 가게</div>  
→ 브라우저: "글자 한 줄이 있구나." 끝입니다.
```

그런데 실제 웹페이지에는 뜻이 분명한 구역들이 있습니다. 맨 위 이름 부분, 메뉴 줄, 본문, 맨 아래 연락처. 사람은 보자마자 압니다. 브라우저는 모릅니다. div 라서 그렇습니다.

그 구역에 이름을 붙여 주는 태그들이 있습니다.

```
<header> 이 구역은 머리말이다
<nav> 이 구역은 길 안내다
<main> 이 구역이 이 페이지의 알맹이다
<footer> 이 구역은 꼬리말이다
```

이렇게 “이름 자체에 뜻이 담긴 태그” 를 시맨틱(semantic) 태그라고 부릅니다. 시맨틱은 ‘의미의’ 라는 뜻의 영어 단어입니다.

★ 이 파일에서 가장 중요한 한 문장을 먼저 말합니다.

```
시맨틱 태그는 화면을 하나도 바꾸지 않습니다.
```

정말입니다. div 를 header 로 바꿔도 글자 크기, 색, 위치, 여백이 전부 그대로입니다. 이 파일에서 그걸 직접 눈으로 보고, 자로도 재 봅니다.

“화면이 안 바뀌는데 왜 굳이 바꾸느냐” 가 이 파일의 진짜 주제입니다.

이 파일에는 같은 화면을 만드는 두 별의 코드가 나옵니다. 하나는 div 만 썼고, 하나는 시맨틱 태그를 썼습니다. 두 결과 상자를 번갈아 보면서 다른 곳을 찾아보세요. 찾을 수 없습니다. 그게 정답입니다.

같은 화면, 두 가지 코드

섹션 1

아래 A 와 B 는 글자도 순서도 완전히 같은 페이지 조각입니다. 태그 이름만 다릅니다.

A 는 이렇게 썼습니다. 전부 div 입니다.

```
<div> <div> <h3>동네 도서관 소식</h3> </div> <div> <p>새 열람실이 열렸습니다.</p> <p>토요일에도 밤 9시까지 엽니다.</p> </div> <div> <p>글쓴이 : 편집실</p> </div>
</div>
```

B 는 바깥쪽 상자 이름만 바꿨습니다. 안쪽 h3, p 는 손대지 않았습니

```
<main> <header> <h3>동네 도서관 소식</h3> </header> <section> <p>새 열람실이
열렸습니다.</p> <p>토요일에도 밤 9시까지 엽니다.</p> </section> <footer> <p>글쓴이 :
편집실</p> </footer>
</main>
```

바뀐 것은 딱 이것뿐입니다.

div → main	바깥 상자
div → header	첫째 상자
div → section	둘째 상자
div → footer	셋째 상자

header, main, section, footer 가 각각 무슨 뜻인지는 개념03 과 개념04 에서 하나씩 배웁니다. 지금은 “이름만 다른 상자” 라고 생각하고 결과만 보세요.

화면 굵고 조금 큰 글씨로 "동네 도서관 소식",

그 아래 문단 두 줄, 그 아래 “글쓴이 : 편집실” 한 줄

동네 도서관 소식

새 열람실이 열렸습니다.
토요일에도 밤 9시까지 엽니다.

글쓴이 : 편집실

화면 위 A 상자과 글자, 크기, 줄 간격, 순서가 전부 똑같습니다

동네 도서관 소식

새 열람실이 열렸습니다.
토요일에도 밤 9시까지 엽니다.

글쓴이 : 편집실

두 상자를 아무리 봐도 다른 점이 없습니다. “내가 뭘 잘못 넣었나” 싶겠지만 맞게 한 것입니다.

◦ 직접 해보기 1

B 상자의 <header> 와 </header> 를 <div> 와 </div> 로 바꿔 보세요. 저장하고 새로고침(F5) 해도 화면이 그대로인 것을 확인한 뒤 되돌려 놓으세요.

화면은 정말 똑같습니다

섹션 2

눈으로만 보면 못 믿을 수 있으니 자로 재 봤습니다. A 와 B 는 높이도 폭도 같은 숫자입니다.

브라우저에는 크롬 개발자 도구라는 자가 들어 있습니다. 그걸로 A 상자과 B 상자를 재면 이렇게 나옵니다.

A 의 바깥 상자 폭 824px 높이 164px

B 의 바깥 상자 폭 824px 높이 164px

글자 크기도 재 보면 이렇게합니다.

A 의 h3	18.72px	B 의 h3	18.72px
A 의 div	display: block	B 의 section	display: block
A 의 div	display: block	B 의 header	display: block

display 는 11단원에서 제대로 배우는 이름인데, 지금은 “이 요소가 한 줄을 통째로 쓰는가(block) 아닌가 (inline)” 를 가리키는 값이라고만 알면 됩니다.

div 도 block, header 도 block, section 도 block, footer 도 block, main 도 block. 전부 같습니다. 그래서 화면이 같을 수밖에 없습니다.

★ 왜 이렇게 만들었을까요?

시맨틱 태그는 “생김새” 를 위한 것이 아니라 “뜻” 을 위한 것이기 때문입니다. 생김새는 07단원부터 CSS 가 말합니다. 역할이 완전히 갈려 있습니다.

HTML → 이게 무엇인가 (뜻)
CSS → 어떻게 보일까 (생김새)

시맨틱 태그가 화면까지 바뀌 버리면 이 구분이 무너집니다. “제목처럼 보이게 하려고 header 를 쓴다” 같은 잘못된 습관이 생기니까요.

화면	두 줄이 위아래로 바짝 붙어서 보입니다.
----	------------------------

글자 크기도 색도 여백도 두 줄이 완전히 같습니다

저는 div 안의 글자입니다.
저는 section 안의 글자입니다.

시맨틱 태그를 썼는데 화면이 안 바뀌면 잘한 것입니다. 바뀌었다면 그건 시맨틱 태그 때문이 아니라 다른 곳을 같이 고친 것입니다.

직접 해보기

2

위 상자의 <section> 을 <article> 로 바꿔 보세요. 역시 아무것도 안 바뀝니다. <aside>, <nav> 로도 한 번씩 바꿔 보세요.

이 페이지를 읽는 네 종류의 독자

섹션 3

웹페이지를 보는 것은 눈으로 보는 사람만이 아닙니다. 화면을 안 보고 읽는 쪽이 더 많습니다.

화면이 같은데도 태그를 나누는 이유는, 화면을 안 보고 읽는 쪽이 있기 때문입니다. 아래 넷을 하나씩 봅시다.

[1] 검색엔진

구글, 네이버의 수집 프로그램은 화면을 눈으로 보지 않습니다.
HTML 글자만 읽어 갑니다. div 뿐이면 "글자 뭉치" 로만 읽힙니다.
main 이 있으면 "이 페이지의 알맹이는 여기" 라고 알 수 있습니다.
메뉴에 있는 글자와 본문에 있는 글자를 구별해서 셀 수 있게 됩니다.

[2] 화면 낭독기를 쓰는 사람

눈이 안 보이는 사람은 화면을 소리로 듣습니다.
이때 시맨틱 태그가 없으면 페이지 맨 위부터 끝까지
메뉴 스무 개를 다 듣고 나서야 본문에 닿습니다. 매 페이지마다요.
시맨틱 태그가 있으면 "본문으로 건너뛰기" 를 쓸 수 있습니다. 섹션 4 에서 봅니다.

[3] 같이 일하는 동료

남이 쓴 코드를 열었을 때 div 가 40개면 어디가 어디인지 모릅니다.
header, main, footer 가 보이면 스크롤만 내려도 지도가 그려집니다.

[4] 반년 뒤의 나

사실 가장 자주 겪는 쪽입니다.
내가 쓴 코드도 반년 지나면 남이 쓴 코드와 똑같습니다.
그때 `<div class="a1">` 을 보면 아무 기억이 안 납니다.

정리하면 이렇습니다. 시맨틱 태그는 눈에 보이는 사람을 위한 것이 아니라 화면을 못 보거나 안 보는 쪽을 위한 것입니다. 그래서 화면이 안 바뀌는 게 당연합니다.

화면 3칸짜리 표가 하나 보입니다. 맨 윗줄은 굵은 글씨 제목 칸이고

아래로 4줄이 이어집니다. 칸에는 선이 없습니다

읽는 쪽
div 만 썼을 때
시맨틱 태그를 썼을 때

검색엔진
글자 뭉치로만 읽힘
본문이 어디인지 알아냄

화면 낭독기 사용자
메뉴부터 끝까지 다 들어야 함
본문으로 바로 건너뛰

같이 일하는 동료
코드를 한참 읽어야 구조가 보임
태그 이름만 봐도 구조가 보임

반년 뒤의 나
내가 쓴 코드인데 기억이 안 남
다시 읽어도 바로 이해됨

표의 칸에 선이 없는 것이 정상입니다. 표 선은 CSS 로 그립니다(10단원). 지금은 03단원에서 배운 `<table>` 을 복습하는 셈으로 보세요.

▣ 직접 해보기 3

위 표에 줄을 하나 더 넣어 보세요. `<tr>` 안에 `<td>` 세 개를 넣으면 됩니다. 읽는 쪽은 “글자를 키워 쓰는 사람” 이라고 적어 보세요.

본문으로 건너뛰기

섹션 4

화면 낭독기는 시맨틱 태그를 만나면 구역 목록을 만들어 줍니다. 사용자는 그 목록에서 원하는 구역으로 바로 뛩니다.

화면 낭독기는 01단원 개념03 에서 말한 그 프로그램입니다. 화면의 글자를 소리로 읽어 줍니다. 윈도우에는 NVDA, 맥에는 VoiceOver 가 있습니다.

이 프로그램들은 페이지를 읽기 전에 먼저 훑어서 header, nav, main, aside, footer 같은 태그를 찾아 목록을 만듭니다. 이 구역들을 랜드마크(landmark, 이정표) 라고 부릅니다.

— div 만 있는 페이지

읽기 시작 → 로고 글자 → 메뉴1 → 메뉴2 → ... → 메뉴20
→ 광고 문구 → 그제서야 본문 첫 글자

페이지를 옮길 때마다 이 과정을 다시 겪습니다.
메뉴는 모든 페이지에 똑같이 있으니까요.
듣는 사람 입장에서는 매번 같은 소리를 20번씩 다시 듣는 셈입니다.

— 시맨틱 태그가 있는 페이지

읽기 시작 → 프로그램이 알려 줍니다.
"이 페이지에는 구역이 5개 있습니다."
사용자가 단축키를 누르면 아래처럼 목록이 뜹니다.
거기서 '본문' 을 고르면 그 자리로 바로 갑니다.

여기서 main 태그가 왜 페이지에 하나뿐이어야 하는지도 나옵니다. “본문으로 건너뛰기” 를 눌렀는데 본문이 세 개면 어디로 가야 할까요. 하나뿐이어야 건너될 곳이 정해집니다.

★ “우리 사이트에는 그런 사용자가 안 올 것 같은데요?” 그렇지 않습니다. 그리고 그 사용자가 한 명도 없더라도 검색엔진과 동료와 미래의 나는 늘 있습니다. 셋 다 같은 태그를 보고 페이지를 이해합니다.

화면 4칸짜리 표가 보이고 아래로 다섯 줄이 이어집니다.

맨 윗줄만 굵은 글씨이고 가운데 정렬되어 있습니다

번호
구역 이름
태그
거기서 들리는 첫 글자

1배너header동네 도서관
2내비게이션nav공지 · 자료 · 오시는 길
3본문main새 열람실이 열렸습니다.
4보충aside많이 본 글
5바닥글footer연락처 02

이 표는 낭독기 화면을 흉내 낸 것이지 진짜 낭독기가 아닙니다. 중요한 것은 이런 목록은 시맨틱 태그가 있어야 만들어진다는 점입니다. <div> 는 아무리 많아도 이 목록에 한 줄도 못 올라갑니다.

▹ 직접 해보기

4

위 표의 마지막 줄 아래에 줄을 하나 더 넣고 번호 6, 구역 이름 “찾기”, 태그 search, 첫 글자 “검색어 입력” 을 적어 보세요.

앞으로 배울 시맨틱 태그

섹션 5

이 단원에서 배우는 시맨틱 태그는 일곱 개입니다. 여기서 한 번에 훑고, 개념03·개념04 에서 하나씩 자세히 봅니다.

사실 여러분은 시맨틱 태그를 이미 여러 개 써 봤습니다. h1, p, ul, li, table, form, label 도 전부 시맨틱 태그입니다. 이름 자체에 뜻이 있으니까요.

<h1> 이건 제목이다
 이건 순서 없는 목록이다
<th> 이건 표의 제목 칸이다

06단원에서 새로 배우는 것은 “글” 이 아니라 “구역” 에 이름을 붙이는 태그들입니다.

페이지 뼈대 → header nav main footer (개념03)
본문 구획 → section article aside (개념04)

아래 표를 지금 다 외울 필요는 없습니다. 개념03, 개념04 를 보고 나서 이 표로 돌아오면 정리가 됩니다.

화면 3칸짜리 표가 보이고 일곱 줄이 이어집니다

태그
뜻
배우는 곳

- header그 구역의 머리말개념03
- nav주요 길 안내 링크 묶음개념03
- main이 페이지만의 알맹이 (하나만)개념03
- footer그 구역의 꼬리말개념03
- section주제로 묶은 덩어리개념04
- article떼어 내도 말이 되는 덩어리개념04
- aside곁다리 내용개념04

일곱 개 어디에도 안 맞는 덩어리는 그냥 <div> 입니다. 억지로 끼워 맞추지 마세요. div 를 쓰는 것이 맞는 경우가 아주 많습니다.

➤ 직접 해보기 5

위 표에서 "이 페이지만의 알맹이 (하나만)" 라고 적힌 줄을 찾아 태그 이름을 소리 내어 읽어 보세요.
그리고 그 줄의 <td> 세 칸이 코드에서 어디인지 VS Code 에서 짚어 보세요.

자주 하는 실수

섹션 6

시맨틱 태그의 실수는 대부분 “화면을 바꿔 줄 것”이라는 기대에서 나옵니다.

이런 실수를 합니다

화면이 바뀔 거라고 기대함

이런 실수를 합니다

실수 1 — header 로 바꿨는데 아무 일도 안 일어남

```
<header>우리 가게</header>
```

우리 가게 “머리말이라고 했으니 크고 굵게 나오겠지” 하고 기대하면 실망합니다. 보통 크기 글자 한 줄이 그대로 나옵니다. 이건 고장이 아닙니다. 시맨틱 태그는 원래 화면을 안 건드립니다. 크게 만들고 싶으면 안에 `<h1>` 을 넣거나 07단원부터 CSS 로 하세요.

이런 실수를 합니다

class 이름으로 뜻을 전하려 함

이런 실수를 합니다

실수 2 — class 이름을 시맨틱이라고 생각함

```
<div class="main">본문입니다</div>
```

`class="main"` 은 사람만 읽는 글자입니다. 브라우저에게도 검색엔진에게도 화면 낭독기에게도 아무 뜻이 없습니다. `class="tomato"` 라고 써도 결과가 똑같습니다. 뜻을 전하는 것은 태그 이름뿐입니다.

이런 실수를 합니다

모든 div 를 없애려 함

이런 실수를 합니다

실수 3 — div 를 하나도 안 쓰려고 함

```
<article>
<section> <section>가격 7,000원</section>
</section>
</article>
```

“div 는 나쁜 것” 이라고 배우면 이렇게 됩니다. 뜻이 없는 자리에 뜻 있는 태그를 억지로 끼우면 거짓말을 하는 코드가 됩니다. 아무 뜻 없이 묶는 자리는 div 가 정답입니다. div 는 나쁜 태그가 아니라 뜻이 없을 때 쓰는 정직한 태그입니다.

이런 실수를 합니다

구역만 나누고 제목을 안 붙임

이런 실수를 합니다

실수 4 — 구역만 나누고 제목을 안 붙임

```
<section>
<p>화요일은 쉽니다.</p>
</section>
```

구역을 나눴지만 그 구역이 무슨 주제인지는 아무도 모릅니다. 화면 낭독기가 목록을 만들어도 이름 없는 칸이 됩니다. <section> 에는 <h2> 같은 제목을 같이 넣으세요. 자세한 것은 개념04 에서 봅니다.

이런 실수를 합니다

낭독기 쓰는 사람이 없다고 생각함

이런 실수를 합니다

실수 5 — “우리 사이트엔 그런 사용자 없어요”

설령 화면 낭독기 사용자가 한 명도 없더라도 검색엔진은 매일 옵니다. 동료도 옵니다. 반년 뒤의 나도 옵니다. 시맨틱 태그를 쓰는 데 드는 시간은 div 를 쓰는 것과 똑같이 0초입니다. 안 쓸 이유가 없습니다.

06

정리

▢ 직접 해보기 6

정답

1) 섹션 1 의 B 상자를 이렇게 고칩니다.

```
<div> <h3>동네 도서관 소식</h3>
</div>
```

→ 화면이 전혀 안 바뀝니다. 높이도 그대로 164px 입니다.

바뀐 것은 "이 구역이 머리말이다" 라는 정보가 사라진 것뿐인데, 그건 화면에 안 보입니다. 이 단원의 핵심이 이 지점입니다.

2) 섹션 2 의 상자를 이렇게 고칩니다.

```
<article id="plainSection">저는 section 안의 글자입니다.</article>
```

→ 역시 아무것도 안 바뀝니다. aside, nav 로 바뀌도 같습니다.

일곱 개 시맨틱 태그가 전부 display: block 이라서 그렇습니다.

3) 섹션 3 표의 마지막 <tr> 아래에 넣습니다.

```
<tr> <td>글자를 키워 쓰는 사람</td> <td>어디가 본문인지 못 찾음</td> <td>본문만 골라  
크게 볼 수 있음</td>  
</tr>
```

→ 표에 다섯 번째 줄이 생깁니다. 칸 수(3개) 가 맞아야 줄이 안 어긋납니다.

4) 섹션 4 표의 마지막 <tr> 아래에 넣습니다.

```
<tr><td>6</td><td>찾기</td><td>search</td><td>검색어 입력</td></tr>
```

→ 표에 여섯 번째 줄이 생깁니다.

search 는 아주 최근에 생긴 시맨틱 태그라 이 자료에서는 다루지 않습니다.
이런 태그가 계속 늘어난다는 것만 알아 두세요.

5) "이 페이지만의 알맹이 (하나만)" 줄의 태그 이름은 main 입니다.

```
<tr><td>main</td><td>이 페이지만의 알맹이 (하나만)</td><td>개념03</td></tr>
```

→ 화면은 아무것도 안 바뀝니다. 코드를 눈으로 짚는 연습이 목적입니다.

td 세 칸이 표의 세 칸과 하나씩 짝지어진다는 것을 확인하세요.

페이지 뼈대 태그

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 페이지 뼈대 태그

개념02 에서 시맨틱 태그가 무엇인지 봤습니다. 이제 실제로 쓰는 법을 배웁니다. 먼저 페이지 전체의 뼈대부터입니다.

여러분이 오늘 들어가 본 웹사이트를 떠올려 보세요. 거의 모든 사이트가 이 네 덩어리로 되어 있습니다.

맨 위	사이트 이름과 로고
그 아래	메뉴 줄
가운데	이 페이지에만 있는 내용
맨 아래	연락처, 저작권 표시

이 네 덩어리에 이름을 붙이는 태그가 있습니다.

<header>	머리말
<nav>	길 안내
<main>	알맹이
<footer>	꼬리말

네 개 다 div 와 똑같이 생겼습니다. 화면도 안 바뀝니다. 달라지는 것은 “이 구역이 무엇인지” 를 브라우저가 알게 된다는 점뿐입니다.

한 가지만 미리 못 봐야 합니다. 이 단원에서 제일 많이 틀리는 곳입니다.

main 은 페이지에 딱 하나입니다.
header 와 footer 는 여러 번 나와도 됩니다.

왜 그런지는 섹션 4 와 섹션 6 에서 설명합니다.

이 파일에서 만드는 결과 상자들은 전부 멍멍한 글 덩어리로 보입니다. 네 태그가 화면을 안 바꾸기 때문입니다. 그러니 화면보다 코드를 보세요. 어느 글자가 어느 태그 안에 들어 있는지 VS Code 에서 짚어가며 읽는 것이 오늘 할 일입니다.

페이지를 네 덩어리로

섹션 1

먼저 뼈대 그림을 눈에 익히세요. 앞으로 만드는 모든 페이지가 이 모양입니다.

아래 그림은 위에서 아래로 쌓인 네 개의 가로 띠입니다. 네 태그 모두 블록이라, 그냥 순서대로 쓰면 이렇게 위아래로 쌓입니다. 좌우로 나누거나 겹치는 것은 11~13단원에서 CSS 로 합니다.

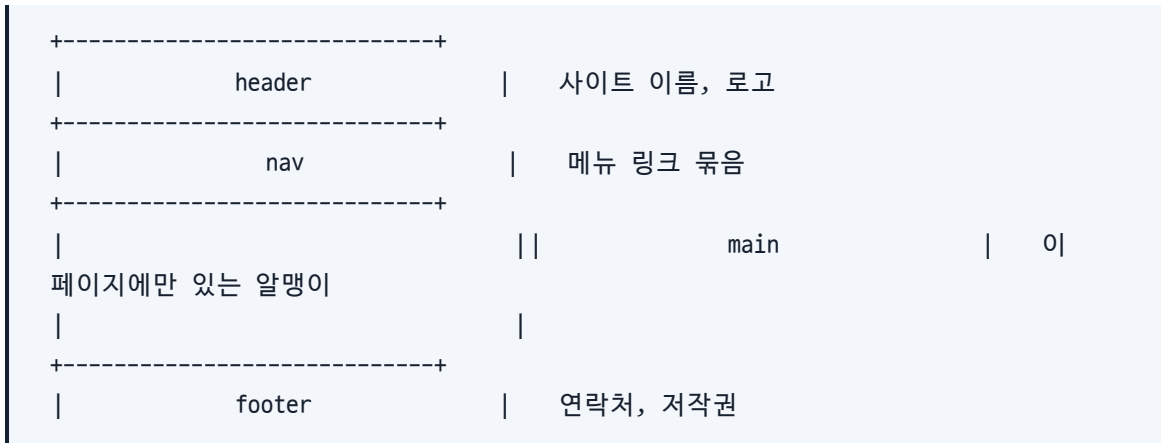
각 덩어리의 이름과 역할을 한 줄로 정리하면 이렇습니다.

header	이 구역의 머리말.	무엇에 대한 곳인지 알려 줍니다.
nav	길 안내.	다른 페이지로 가는 주요 링크 묶음입니다.
main	이 페이지만의 알맹이.	페이지마다 내용이 다릅니다. 하나만 씁니다.
footer	이 구역의 꼬리말.	연락처, 저작권 같은 마무리입니다.

여기서 header, nav, footer 는 사이트의 모든 페이지에 똑같이 들어갑니다. main 만 페이지마다 다릅니다. 그래서 이름이 main 입니다.

화면 검은 상자 안에 흰 글씨로 네모 그림이 보입니다.

네모 오른쪽에 한글 설명이 붙어 있습니다



이 그림은 글자로 그린 것이지 진짜 테두리가 아닙니다. 지금 코드를 그대로 쓰면 테두리 없이 글자만 위아래로 쌓입니다. 테두리는 10단원에서 CSS 로 그립니다.

▹ 직접 해보기

1

오늘 본 웹사이트를 하나 떠올려서 그 페이지의 header, nav, main, footer 가 각각 어디였는지 종이에 그려 보세요. 네 개가 다 있었나요? 없는 것도 있었을 것입니다.

06

header

섹션 2

<header> 는 어떤 구역의 시작을 알리는 소개 부분입니다.

header 안에 보통 들어가는 것들입니다.

- 사이트 이름이나 글 제목 (h1 ~ h6)
- 로고 그림
- 한 줄 설명
- 때로는 검색창이나 메뉴

중요한 것은 header 가 “페이지 맨 위” 를 뜻하는 태그가 아니라는 점입니다. header 는 자기가 들어 있는 구역의 머리말입니다.

body 바로 안에 있으면	→ 사이트 전체의 머리말
article 안에 있으면	→ 그 글 하나의 머리말

이건 섹션 6 에서 다시 자세히 봅니다. 초보자가 가장 많이 오해하는 곳입니다.

header 를 썼다고 글자가 커지지 않습니다. 제목을 크게 만드는 것은 h1~h6 입니다. 그래서 둘을 같이 씁니다.

```
<header>
  <h1>동네 도서관</h1>      ← 크게 보이는 것은 h1 덕분
  <p>책과 사람이 만나는 곳</p>
</header>
```

화면 굵고 큰 글씨로 "동네 도서관", 그 아래 보통 글씨로

"책과 사람이 만나는 곳" 이 한 줄 보입니다

```
동네 도서관
책과 사람이 만나는 곳
```

위 상자에서 <header> 를 <div> 로 바꿔도 화면은 똑같습니다. 큰 글씨는 <h2> 가 만든 것이지 <header> 가 만든 것이 아닙니다.

⇒ 직접 해보기 2

위 상자의 <header> 안에 <p>오늘도 문을 엽니다.</p> 를 한 줄 더 넣어 보세요. 머리말 안에 문단이 두 개가 됩니다.

nav

섹션 3

<nav> 는 다른 곳으로 가는 주요 링크 묶음입니다. navigation(길 안내) 의 줄임말입니다.

nav 안에는 보통 목록과 링크를 같이 씁니다. 03단원과 04단원의 복습입니다.

```
<nav> <ul> <li><a href="#">공지</a></li> <li><a href="#">자료실</a></li> </ul>
</nav>
```

왜 ul 을 쓰냐면, 메뉴는 실제로 "항목이 여러 개인 목록" 이기 때문입니다. 낭독기도 "목록, 항목 3개" 라고 읽어 줍니다. 몇 개인지 미리 알 수 있습니다.

★ 여기서 제일 중요한 규칙입니다. 링크가 있다고 해서 다 nav 가 아닙니다.

nav 를 씁니다	사이트 위쪽 메뉴 줄 옆에 붙는 카테고리 목록 페이지 아래쪽의 사이트맵 링크 묶음
nav 를 안 씁니다	본문 문장 속에 끼어 있는 링크 한두 개 "자세히 보기" 버튼 하나 footer 맨 아래 개인정보처리방침 링크 하나

기준은 "이게 이 페이지의 주요 길 안내인가" 입니다. 모든 링크에 nav 를 씌우면 낭독기의 구역 목록이 nav 로만 가득 차서 오히려 아무 도움이 안 됩니다. 이정표가 스무 개면 이정표가 없는 것과 같습니다.

아래 데모의 링크는 같은 폴더의 다른 자료 파일로 갑니다. 눌러 보면 진짜로 이동합니다. 돌아올 때는 브라우저 뒤로 가기를 쓰세요.

화면 점(•) 이 붙은 목록 세 줄이 보이고, 글자는 파란색 밑줄 링크입니다

div 와 span
시맨틱 태그란
본문 구획 태그

<nav> 를 씌워도 목록 모양은 그대로입니다. 메뉴를 가로로 눕히는 것은 12단원 Flexbox 에서 합니다. 지금 세로로 쌓여 있는 것이 정상입니다.

▣ 직접 해보기 3

위 목록에 를 하나 더 넣어 개념05_전체_구조_만들기.html 로 가는 링크를 만들어 보세요. 글자는 "전체 구조 만들기" 로 하세요.

main

섹션 4

<main> 은 이 페이지에만 있는 내용을 감쌉니다. 한 페이지에 딱 하나만 씁니다.

main 에 넣을 것과 넣지 말 것을 가르는 기준은 하나입니다.

"옆 페이지에도 똑같이 있는가?"

똑같이 있다 → main 밖 (header, nav, footer 로 갑니다)

여기만 있다 → main 안

예를 들어 도서관 사이트라면

사이트 로고	모든 페이지에 있음	→ main 밖
상단 메뉴 줄	모든 페이지에 있음	→ main 밖
"새 열람실" 기사	이 페이지에만 있음	→ main 안
아래 연락처	모든 페이지에 있음	→ main 밖

★ main 이 하나뿐이어야 하는 이유

개념02 에서 본 "본문으로 건너뛰기" 때문입니다. 낭독기 사용자가 그 기능을 누르면 브라우저는 main 으로 데려다줍니다. main 이 셋이면 어디로 데려가야 할지 정할 수 없습니다. 그래서 규칙으로 하나만 쓰기로 정해져 있습니다.

★ 이 자료 파일 이야기

이 파일에도 살아 있는 main 은 지금 이 섹션의 상자 하나뿐입니다. 섹션마다 main 데모를 넣고 싶었지만, 그러면 이 파일이 방금 배운 규칙을 어기게 됩니다. 그래서 뒤 섹션에서는 코드만 글자로 보여 줍니다.

★ main 을 넣으면 안 되는 자리

main 은 header, footer, nav, article, aside 안에 들어가면 안 됩니다. "본문" 이 "머리말" 안에 들어 있다는 말이 되니까요. 앞뒤가 안 맞습니다. main 은 보통 body 바로 아래에 둡니다.

화면 굵고 큰 글씨로 "새 열람실이 열렸습니다",

그 아래 문단 두 줄이 차례로 보입니다

새 열람실이 열렸습니다
3층에 조용한 열람실이 새로 생겼습니다.
토요일에도 밤 9시까지 이용할 수 있습니다.

<main> 은 화면에 아무 표시도 남기지 않습니다. 위 상자만 봐서는 main 이 있는지 없는지 알 수 없습니다. 코드를 봐야 합니다. 그게 이 태그가 하는 일의 전부입니다.

▹ 직접 해보기

4

위 <main> 안에 <p>문제는 1층 안내데스크로 오세요.</p> 를 한 줄 더 넣어 보세요. 문단이 세 줄이 됩니다.

footer

섹션 5

<footer> 는 어떤 구역의 마무리 부분입니다.

footer 에 보통 들어가는 것들입니다.

- 연락처, 주소
- 저작권 표시
- 만든 사람, 쓴 날짜
- 관련 문서 링크 묶음

header 와 마찬가지로 footer 도 "페이지 맨 아래" 를 뜻하는 태그가 아닙니다. 자기가 들어 있는 구역의 꼬리말입니다.

★ 가장 흔한 오해

"footer 를 쓰면 화면 맨 아래에 착 달라붙나요?" 아닙니다. 아무 일도 안 일어납니다. footer 는 코드에 적힌 그 자리에 그냥 놓입니다. 코드 한가운데에 footer 를 쓰면 화면 한가운데에 나옵니다.

화면 맨 아래에 붙이는 것은 11단원의 position 이나 12단원의 flex 가 하는 일입니다. 태그는 뜻만 정하고, 위치는 CSS 가 정합니다. 개념02 에서 본 그 구분 그대로입니다.

화면 문단 두 줄이 보이고, 그 아래에

"이 줄은 footer 뒤에 쓴 문단입니다." 가 한 줄 더 보입니다

동네 도서관 · 서울시 어딘가 12길 3
문의 02-000-0000

이 줄은 footer 뒤에 쓴 문단입니다.

footer 뒤에 쓴 문단이 footer 아래에 그대로 나옵니다. footer 가 화면 맨 아래로 내려가지 않는다는 증거입니다.

▹ 직접 해보기 5

위 상자에서 <footer> 덩어리를 <p id="afterFooter"> 아래로 옮겨 보세요. 코드 순서를 바꾼 대로 화면 순서도 바뀝니다.

header 와 footer 는 여러 번

섹션 6

<header> 는 페이지 맨 위 전용 태그가 아닙니다. 글 하나하나에도 자기만의 머리말과 꼬리말이 있습니다.

본문 구획 태그

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 본문 구획 태그

개념03 에서 페이지를 네 덩어리로 나눴습니다. header, nav, main, footer 입니다.

그런데 main 안이 넓습니다. 글이 다섯 편 들어갈 수도 있고 “소개 / 이용 안내 / 오시는 길” 처럼 주제가 여러 개일 수도 있습니다. 그 안을 또 나눠야 합니다.

그때 쓰는 태그가 셋입니다.

```
<section> 주제로 묶은 덩어리
<article> 떼어 내도 말이 되는 덩어리
<aside>   곁다리
```

셋 다 화면을 안 바꿉니다. 셋 다 div 와 똑같이 생겼습니다. 그래서 “언제 무엇을 쓰나” 만 배우면 됩니다. 그게 이 파일의 전부입니다.

미리 한 줄로 답을 말해 두면 이렇습니다.

떼어 내도 말이 되면 → article
 겹다리라 빼도 되면 → aside
 제목을 붙일 주제 묶음이면 → section
 아무 뜻도 없으면 → div

이 네 줄을 섹션 5 에서 흐름도로 다시 정리합니다. 그 전에 하나씩 봅시다.

section, article, aside 는 생김새가 완전히 같습니다. 화면으로는 절대 구별할 수 없습니다. 그러니 결과 상자에서 차이를 찾으려 하지 마세요. “이 덩어리는 어떤 성격인가” 를 판단하는 연습을 하는 파일입니다.

main 안을 또 나뉘야 할 때

섹션 1

내용이 길어지면 <main> 안을 다시 나눕니다. 나누는 방법이 세 가지 있고, 뜻이 각각 다릅니다.

도서관 사이트의 첫 화면을 생각해 봅시다. main 안에 이런 것들이 들어갑니다.

- 공지 글 세 편
- 이용 안내 (여는 시간, 대출 규칙)
- 오른쪽에 붙는 "이번 주 인기 도서" 목록

이걸 전부 div 로 감싸도 화면은 잘 나옵니다. 하지만 셋의 성격이 완전히 다릅니다.

공지 글 한 편 → 이것만 잘라서 다른 페이지에 붙여도 말이 됩니다.
 이용 안내 → 잘라내면 뜬금없습니다. 이 페이지의 한 부분입니다.
 인기 도서 목록 → 없어도 페이지가 멀쩡합니다. 겹다리입니다.

이 성격 차이를 코드에 적어 두는 것이 section, article, aside 입니다.

지금부터 셋을 하나씩 봅시다. 순서는 판단하기 쉬운 것부터입니다. article → section → aside 순서로 갑니다.

화면 굵은 소제목과 문단이 번갈아 여섯 줄로 보입니다

공지 : 휴관일 안내매월 첫째 월요일은 쉽니다.
 이용 안내평일은 9시부터 18시까지입니다.
 이번 주 인기 도서1위 어린 왕자

화면만 보면 세 덩어리가 다 똑같아 보입니다. 성격이 다르다는 사실이 코드 어디에도 안 적혀 있기 때문입니다. 이 파일이 끝나면 이 세 덩어리를 각각 다른 태그로 바꾸게 됩니다.

▫ 직접 해보기 1

위 세 덩어리를 보고, 각각 어떤 성격인지 종이에 적어 보세요. “떼어 내도 되는 것 / 이 페이지의 한 부분 / 없어도 되는 겹다리” 중에서 고르세요.

<article> 은 통째로 잘라내서 다른 곳에 붙여도 그 자체로 말이 되는 덩어리입니다.

판단하는 방법이 아주 간단합니다. 이렇게 물어보세요.

"이 덩어리만 잘라서 다른 사이트에 올려도 말이 되나?"

말이 되면 article 입니다. 이걸 '떼어 내기 검사' 라고 부르겠습니다.

article 이 되는 것들

블로그 글 한 편	→ 잘라서 다른 데 올려도 글 한 편입니다.
신문 기사 하나	→ 그렇습니다.
상품 목록의 상품 카드	→ 상품 하나의 정보라 그 자체로 완결입니다.
댓글 하나	→ 누가 언제 무슨 말을 했는지 다 들어 있습니다.
제품 후기 하나	→ 그렇습니다.

article 이 아닌 것들

"이용 안내" 구역	→ 잘라내면 무엇에 대한 안내인지 알 수 없습니다.
페이지 소개 문단	→ 이 페이지 안에서만 뜻이 있습니다.
레이아웃용 껍데기	→ 아무 내용도 아닙니다. div 입니다.

article 은 이름 때문에 오해를 많이 삽니다. article 은 '기사' 라는 뜻이지만, 신문 기사에만 쓰는 태그가 아닙니다. 댓글 하나에도, 상품 카드 하나에도 씁니다.

article 안에는 제목을 넣는 것이 좋습니다. 떼어 냈을 때 무엇인지 알려면 제목이 있어야 하니까요. 머리말과 꼬리말이 있으면 개념03 에서 본 대로 header, footer 를 안에 넣습니다.

article 을 여러 개 나란히 쓰는 것도 아주 흔합니다. 블로그 목록 페이지가 그렇습니다. 글 한 편이 article 하나입니다.

화면 굵은 제목 · 날짜 · 본문 · 글쓴이 순서의 덩어리가 두 벌 보입니다.

두 덩어리 사이에 특별한 구분선은 없습니다

고구마 맛탕 만들기
2026년 8월 3일

고구마를 깍둑 썰어 기름에 튀깁니다.
글쓴이 : 김민준

여름 이불 세탁법
2026년 8월 9일

미지근한 물에 30분 담근 뒤 행굽니다.
글쓴이 : 이서연

두 글 사이에 선이 없어 어디까지가 첫 글인지 눈으로는 모릅니다. 그게 정상입니다. 선을 긋는 것은 10단원 CSS 의 일입니다. 지금은 코드에서 `</article>` 이 어디인지 찾아보세요.

⇒ 직접 해보기 2

위 상자 아래에 세 번째 `<article>` 을 만들어 제목 "선풍기 청소하기", 날짜 "2026년 8월 10일", 본문 한 줄을 넣어 보세요. header 와 footer 도 같이 넣으세요.

section

섹션 3

`<section>` 은 하나의 주제로 묶인 부분입니다. 떼어 내면 말이 안 되지만, 페이지 안에서는 한 덩어리입니다.

section 은 책의 '장(章)' 이라고 생각하면 쉽습니다. 3장만 뜯어내면 뜬금없지만, 책 안에서는 분명히 한 덩어리입니다.

section 에는 지켜야 하는 관례가 하나 있습니다.

★ section 에는 제목을 붙입니다.

h1~h6 중 하나를 section 안 맨 앞에 넣습니다. 왜냐하면 section 은 "주제로 묶은 덩어리" 인데, 그 주제가 무엇인지 적지 않으면 묶은 의미가 없기 때문입니다.

```
<section>
  <h2>이용 안내</h2>      ← 이 줄이 이 구역의 이름표입니다
  <p>평일은 9시부터 18시까지입니다.</p>
</section>
```

제목 붙이면 좋은 점이 하나 더 있습니다. 낭독기가 페이지의 목차를 만들어 줄 수 있게 됩니다. 제목이 없는 section 은 이름 없는 칸으로 남습니다.

★ "제목 붙일 수 없다면?" 그건 section 이 아니라는 신호입니다. div 를 쓰세요. 이게 section 과 div 를 가르는 가장 쉬운 기준입니다.

section 을 여러 개 쓰는 것도 정상입니다. 한 페이지가 "소개 / 이용 안내 / 오시는 길" 세 주제로 되어 있으면 section 세 개를 나란히 씁니다.

화면 "이용 안내" 아래 문단 두 줄, "오시는 길" 아래 문단 한 줄이

차례로 보입니다. 두 구역 사이에 선은 없습니다

이용 안내
평일은 9시부터 18시까지입니다.
대출은 한 번에 다섯 권까지입니다.

오시는 길
2호선 어딘가역 3번 출구에서 걸어서 5분입니다.

<section> 을 <div> 로 바꿔도 화면은 똑같습니다. 달라지는 것은 “이 두 덩어리가 각각 하나의 주제다”라는 정보뿐입니다.

▫ 직접 해보기 3

위 상자에 세 번째 <section> 을 만들어 제목 “주차 안내” 와 문단 한 줄을 넣어 보세요. 제목은 <h3> 로 쓰세요.

aside

섹션 4

<aside> 는 빼도 본문이 멀쩡한 걸다리입니다. 본문과 관련은 있지만 본문의 일부는 아닙니다. 판단하는 방법입니다.

“이걸 통째로 지워도 본문을 읽는 데 지장이 없나?”

지장이 없으면 aside 입니다.

aside 가 되는 것들

관련 글 목록	없어도 본문은 읽힙니다.
광고	그렇습니다.
용어 풀이 상자	본문 이해에 도움은 되지만 없어도 됩니다.
글쓴이 소개 상자	그렇습니다.
인기 글 순위	그렇습니다.

aside 가 아닌 것들

본문 문단	빼면 글이 안 됩니다.
글 제목	빼면 무슨 글인지 모릅니다.
상품 가격	상품 정보의 핵심입니다.

★ 가장 흔한 오해

aside 는 ‘옆’ 이라는 뜻이라, 오른쪽에 붙는 사이드바 전용이라고 오해합니다. 아닙니다. aside 는 위치가 아니라 관계를 나타냅니다.

본문 위에 있어도 aside 일 수 있습니다.
 본문 아래에 있어도 aside 일 수 있습니다.
 문단 사이에 끼어 있어도 aside 일 수 있습니다.

그리고 aside 를 써도 오른쪽으로 안 갑니다. 그냥 아래에 쌓입니다. 오른쪽으로 보내는 것은 12단원 Flexbox 의 일입니다.

aside 에도 제목을 붙이면 좋습니다. 무엇에 대한 결다리인지 알려 주니까요.

화면 제목과 문단이 나온 뒤, 그 아래에 "같이 보면 좋은 글" 과

점(•) 목록 두 줄이 이어집니다. 옆으로 가지 않고 아래에 쌓입니다

여름에 읽기 좋은 책
 더운 날에는 짧은 소설이 잘 읽힙니다.

같이 보면 좋은 글

겨울에 읽기 좋은 책
 비 오는 날의 책

<aside> 가 오른쪽으로 가지 않고 아래에 쌓였습니다. aside 도 블록이기 때문입니다. 이름이 '옆' 이라고 옆으로 가지는 않습니다.

⇒ 직접 해보기 4

위 <aside> 를 <article id="mainPost"> 위로 옮겨 보세요. 결다리가 본문 위에 와도 여전히 aside 라는 것을 확인하세요.

고르는 순서

섹션 5

헷갈릴 때는 아래 네 개의 질문을 위에서부터 차례로 던지세요. 처음 "예" 가 나오는 곳에서 멈추면 됩니다.

이 순서에는 이유가 있습니다. 판단이 가장 확실한 것부터 물어보게 되어 있습니다.

article 은 '때어 내기 검사' 라는 분명한 기준이 있습니다. → 1번
 aside 는 '지워도 되는가' 라는 분명한 기준이 있습니다. → 2번
 section 은 '제목 붙일 수 있는가' 로 판단합니다. → 3번
 셋 다 아니면 남는 것은 div 입니다. → 4번

헷갈리는 경우를 몇 개 미리 풀어 둡니다.

블로그 글 한 편 안의 "재료" 부분

떼어 내면 무슨 요리의 재료인지 모릅니다 → article 아님

지워도 되나? 아니요, 본문의 일부입니다 → aside 아님

제목 "재료" 를 붙일 수 있나? 예 → section 입니다

상품 목록 페이지의 상품 카드 하나

떼어 내도 상품 정보 하나로 말이 됩니다 → article 입니다

페이지 맨 아래 "이 글을 SNS 에 공유하기" 버튼 묶음

떼어 내면 무엇을 공유하는지 모릅니다 → article 아님

지워도 본문은 멀쩡합니다 → aside 입니다

(그 글의 꼬리말로 보고 footer 를 써도 됩니다)

제목을 붙일 수 없는 그냥 껍데기

셋 다 아닙니다 → div 입니다

마지막 줄이 중요합니다. div 로 끝나는 것이 실패가 아닙니다. 정확한 판단입니다.

화면 검은 상자 안에 흰 글씨로 번호 매긴 질문 네 개와

화살표가 붙은 답이 보입니다

1. 통째로 잘라내 다른 곳에 붙여도 말이 되는가?

예 → `<article>` (블로그 글, 상품 카드, 댓글 하나)

아니오 → 2번으로

2. 지워도 본문을 읽는 데 지장이 없는 걸다리인가?

예 → `<aside>` (관련 글, 광고, 용어 풀이)

아니오 → 3번으로

3. 제목을 하나 붙일 수 있는 주제 묶음인가?

예 → `<section>` (이용 안내, 오시는 길, 소개)

아니오 → 4번으로

4. 뜻은 없고 그냥 묶기만 하면 되는가?

예 → `<div>` (여기서 끝나는 것도 정답입니다)

이 흐름도를 외우려 하지 마세요. 헛갈릴 때마다 돌아와서 보면 됩니다. 몇 번 하다 보면 1번 질문만으로 대부분 갈립니다.

▹ 직접 해보기

5

아래 네 가지에 무엇을 쓸지 흐름도로 판단해 보세요. (가) 쇼핑몰의 상품 후기 하나 (나) 회사 소개 페이지의 "연혁" 부분 (다) 글 옆에 붙는 광고 배너 (라) 두 문단을 그냥 한 번 묶는 껍데기

div 와 갈라지는 지점

섹션 6

제목을 붙일 수 있으면 section, 못 붙이면 div 입니다. 이 한 줄이 실전에서 가장 많이 쓰입니다.

아래 두 상자는 코드가 딱 한 글자만 다릅니다. 하나는 section, 하나는 div 입니다. 결과는 완전히 같습니다.

```
<section> <div> <h4>대출 규칙</h4> <h4>대출 규칙</h4> <p>다섯 권까지입니다.</p> <p>
다섯 권까지입니다.</p>
</section> </div>
```

여기서는 제목이 있으니 section 이 맞습니다. 그런데 아래 같은 경우라면 div 가 맞습니다.

```
<div> <p>왼쪽에 놓을 것들</p> <p>이 두 문단을 나란히 두려고 한 번 묶었을 뿐</p>
</div>
```

붙일 제목이 떠오르지 않습니다. 주제가 있는 묶음이 아니라 배치를 위해 한 번 감싼 것뿐입니다. 이럴 때 section 을 쓰면 “여기 주제가 하나 있습니다” 라고 거짓말을 하는 셈이 됩니다.

★ 정리하면 이렇습니다.

```
뜻이 있는 묶음 → section / article / aside
뜻이 없는 묶음 → div
```

div 를 쓰는 것을 부끄러워하지 마세요. 잘 만든 웹페이지에도 div 는 아주 많습니다.

화면 굵은 소제목 "대출 규칙" 아래 문단 한 줄이 보입니다

대출 규칙
한 번에 다섯 권까지입니다.

화면 위 상자와 완전히 똑같이 보입니다

대출 규칙
한 번에 다섯 권까지입니다.

두 상자의 높이가 같습니다. 화면으로는 영원히 구별할 수 없습니다. 그래서 고르는 기준을 머리에 넣어 두는 것 말고는 방법이 없습니다.

▹ 직접 해보기

6

섹션 1 의 id="allDiv" 상자로 돌아가서 안쪽 <div> 세 개를 각각 알맞은 태그로 바꿔 보세요. 공지 글, 이용 안내, 인기 도서 순서입니다.

셋 다 화면을 안 바꾸기 때문에 잘못 골라도 티가 안 납니다. 그래서 아래 다섯 가지가 아주 오래 살아남습니다.

이런 실수를 합니다

제목 없는 section

이런 실수를 합니다

실수 1 — 제목 없이 section 만 씀

```
<section>
<p>평일은 9시부터 18시까지입니다.</p>
<p>주말은 10시부터 17시까지입니다.</p>
</section>
```

화면은 멀쩡합니다. 그런데 무슨 주제의 구역인지 아무 데도 안 적혀 있습니다. 낭독기가 목차를 만들 때 이름 없는 칸이 하나 생깁니다. 제목을 붙이거나, 붙일 제목이 없으면 <div> 로 바꾸세요.

이렇게 쓰세요

```
<section>
<h2>여는 시간</h2>
<p>평일은 9시부터 18시까지입니다.</p>
</section>
```

이런 실수를 합니다

아무 데나 section

이런 실수를 합니다

실수 2 — 뭉기만 하면 될 자리에 section 을 씀

```
<section>
<section> <section><p>가격 7,000원</p></section>
</section>
</section>
```

“div 보다 있어 보여서” section 을 쓰는 경우입니다. 주제가 셋이라고 말한 셈인데 실제로는 문단 하나뿐입니다. 코드가 사실과 다른 말을 하고 있습니다. 이럴 때는 div 가 정답입니다.

이런 실수를 합니다

aside 를 위치로 이해

이런 실수를 합니다

실수 3 — `aside` 를 쓰면 오른쪽으로 갈 거라 기대함

```
<aside>인기 도서</aside>
```

인기 도서 오른쪽으로 안 갑니다. 그냥 그 자리에 한 줄로 놓입니다. `aside` 는 위치를 정하는 태그가 아니라 관계를 적는 태그입니다. “본문의 결다리” 라는 뜻일 뿐입니다. 오른쪽에 놓는 것은 12단원 `Flexbox` 에서 합니다.

이런 실수를 합니다

`article` 을 기사에만 쓴다고 생각

이런 실수를 합니다

실수 4 — `article` 은 신문 기사에만 쓰는 줄 알

```
<div class="review">
<h4>배송이 빨라요</h4>
<p>주문 다음 날 왔습니다.</p>
</div>
```

후기 하나는 떼어 내도 그 자체로 말이 되는 완결된 덩어리입니다. `article` 이 맞습니다. 댓글 하나, 상품 카드 하나도 마찬가지입니다. `article` 을 ‘기사’ 라고만 새기면 이런 자리를 전부 놓칩니다.

이런 실수를 합니다

태그를 바꾸면 화면이 바뀔 거라 생각

이런 실수를 합니다

실수 5 — 태그를 바꿨는데 화면이 그대로라 잘못된 줄 알

```
<div> → <section> 으로 바꿈
```

바꾸고 새로고침했는데 아무 변화가 없어서 “안 먹었나?” 하고 되돌리는 경우가 정말 많습니다. 변화가 없는 것이 성공입니다. 개념02 에서 본 그대로입니다. 되돌리지 마세요.

정리

▫ 직접 해보기

7

정답

1) 세 덩어리의 성격은 이렇습니다.

공지 : 휴관일 안내 → 떼어 내도 되는 것 (글 한 편)
이용 안내 → 이 페이지의 한 부분
이번 주 인기 도서 → 없어도 되는 걸다리

→ 종이에 적는 문제라 화면은 안 바뀝니다.

섹션 6 의  6 에서 이걸 실제 태그로 바꿉니다.

2) 섹션 2 상자의 두 번째 article 아래에 넣습니다.

```
<article> <header> <h3>선풍기 청소하기</h3> <p>2026년 8월 10일</p> </header> <p>
날개를 빼서 미지근한 물로 닦습니다.</p> <footer><p>글쓴이 : 박지훈</p></footer>
</article>
```

→ 같은 모양의 덩어리가 세 벌이 됩니다.

여전히 구분선은 없습니다. 코드로만 경계를 알 수 있습니다.

3) 섹션 3 상자의 두 번째 section 아래에 넣습니다.

```
<section> <h3>주차 안내</h3> <p>건물 뒤편에 스무 자리가 있습니다.</p>
</section>
```

→ 굵은 소제목 “주차 안내” 와 문단 한 줄이 아래에 더 붙습니다.

4) 섹션 4 상자에서 aside 덩어리를 article 위로 옮깁니다.

```
<aside id="sidePart"> ... </aside>
<article id="mainPost"> ... </article>
```

→ “같이 보면 좋은 글” 과 목록이 맨 위로 올라옵니다.

위에 있어도 aside 입니다. 지워도 본문이 멀쩡하니까요.

5) 흐름도로 판단한 결과입니다.

(가) 상품 후기 하나 → article
후기 하나만 떼어 내도 말이 됩니다. 1번에서 멈춥니다.

(나) 회사 연혁 부분 → section
떼어 내면 어느 회사 연혁인지 모릅니다(1번 아시오).
지우면 소개가 허전합니다(2번 아시오).
"연혁"이라는 제목을 붙일 수 있습니다(3번 예).

(다) 광고 배너 → aside
지워도 본문이 멀쩡합니다. 2번에서 멈춥니다.

(라) 그냥 묶는 껍데기 → div
넷째 질문까지 내려옵니다. 이게 정답입니다.

6) 섹션 1의 상자를 이렇게 바꿉니다.

```
<div id="allDiv"> <article><h4>공지 : 휴관일 안내</h4><p>매월 첫째 월요일은 쉽니다.
</p></article> <section><h4>이용 안내</h4><p>평일은 9시부터 18시까지입니다.</p>
</section> <aside><h4>이번 주 인기 도서</h4><p>1위 어린 왕자</p></aside>
</div>
```

→ 화면은 하나도 안 바뀝니다. 여섯 줄 그대로입니다.

바뀐 것은 코드가 세 덩어리의 성격을 말해 주게 되었다는 점입니다.
바깥의 div 는 그대로 두는 것이 맞습니다. 그냥 묶는 껍데기니까요.

전체 구조 만들기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 전체 구조 만들기

드디어 마지막 파일입니다. HTML 파트의 끝이기도 합니다.

지금까지 배운 것을 한 번 세어 볼까요.

- 01단원 html, head, body, meta, title, 주석
- 02단원 h1~h6, p, br, hr, strong, em, span, 특수문자
- 03단원 ul, ol, li, dl, table, tr, th, td
- 04단원 a, img, figure, figcaption
- 05단원 form, label, input, select, button
- 06단원 div, header, nav, main, section, article, aside, footer

이걸 전부 써서 블로그 글 페이지 한 장을 통째로 만들어 봅니다. 새로 배우는 태그는 하나도 없습니다. 조립만 합니다.

만드는 순서가 중요합니다. 초보자는 보통 위에서부터 한 줄씩 쓰다가 중간에 길을 잃습니다. 그러지 않는 방법이 있습니다.

- 1 종이에 네모를 그립니다. 바깥 뼈대부터.
- 2 바깥 네 덩어리를 먼저 코드로 씁니다. 내용은 비워 둡니다.
- 3 그 다음에 안을 채웁니다.

“큰 상자부터, 안쪽은 나중에” 입니다. 이 파일도 그 순서대로 갑니다.

이 파일은 조립 실습입니다. 새 태그는 나오지 않습니다. 뼈대 그림 → 코드 → 실제 결과 순서로 한 단계씩 쌓아 올립니다. 마지막 섹션에 완성된 페이지 한 장이 통째로 들어 있습니다. 끝까지 따라온 뒤 그 코드를 새 파일에 옮겨 적어 보세요.

만들 페이지 그려 보기

섹션 1

코드를 쓰기 전에 네모를 그립니다. 이게 설계도입니다.

만들 것은 요리 블로그의 글 한 편 페이지입니다.

블로그 이름은 “민준의 부엌”, 글 제목은 “고구마 맛탕 만들기” 로 하겠습니다.

그림을 그릴 때는 안쪽부터 그리지 않습니다. 바깥 네모부터 그립니다. 그래야 어디에 무엇이 들어갈지 정해집니다.

아래 그림에서 안쪽으로 들어갈수록 한 칸씩 들여쓴 것을 보세요. 코드의 들여쓰기도 정확히 이 모양이 됩니다.

그림을 보면서 확인할 것 두 가지입니다.

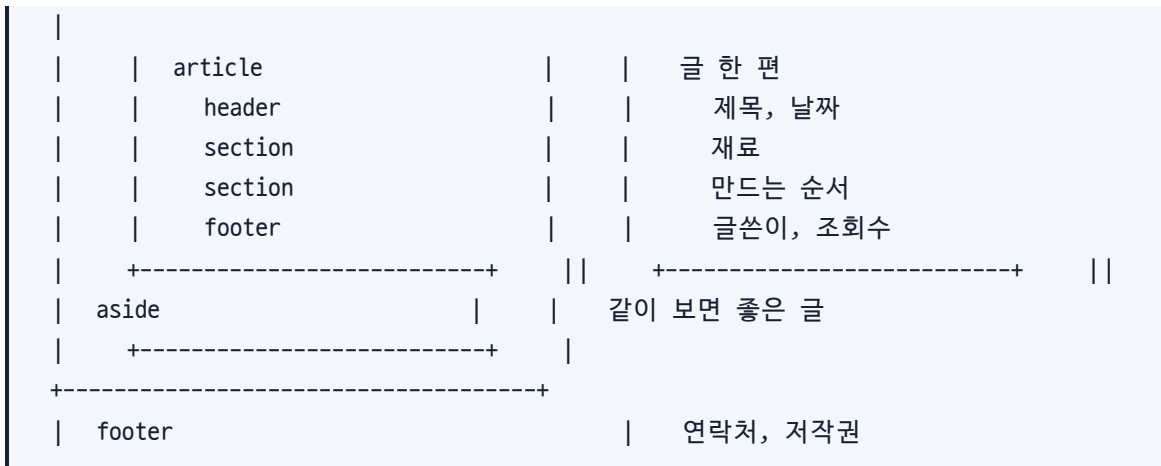
- main 은 몇 개인가요? 하나입니다.
- header 와 footer 는 몇 개인가요? 각각 두 개입니다.
- 하나는 블로그 전체의 것, 하나는 글 한 편의 것입니다.

개념03 섹션 6 에서 배운 그대로입니다.

화면 검은 상자 안에 흰 글씨로 네모 설계도가 보이고,

오른쪽에 한글 설명이 붙어 있습니다

```
+-----+
| header                | | 블로그 이름, 한 줄 소개, 검색창
+-----+
| nav                   | | 전체 글 / 반찬 / 간식
+-----+
| main                  | | +-----+
```



이 그림에는 <div> 가 하나도 없습니다. 이 페이지의 모든 덩어리에 뜻이 있기 때문입니다. 뜻 없는 묶음이 필요해지면 그때 div 를 넣으면 됩니다.

직접 해보기
1

위 설계를 종이에 한 번 그대로 옮겨 그려 보세요. 그리면서 main 이 몇 개인지, header 가 몇 개인지 세어 보세요.

1단계 바깥 뼈대

섹션 2

내용은 나중에입니다. 큰 상자 네 개를 먼저 세웁니다.

body 안에 이렇게 씁니다. 아직 내용은 한 줄씩만 넣었습니다.

```

<body> <header> <h2>민준의 부엌</h2> <p>혼자 사는 사람의 밥상 기록
</p> </header> <nav> <ul> <li><a href="#">전체 글</a></li> <li><a href="#">반찬
</a></li> <li><a href="#">간식</a></li> </ul> </nav> <main>
  (여기는 3단계에서 채웁니다)
</main> <footer> <p>민준의 부엌 · 문의 minjun 앳 example.com</p> </footer>
</body>

```

네 덩어리가 body 바로 아래에 나란히 놓였습니다. 서로 안에 들어가지 않습니다. 형제입니다.

05단원에서 배운 검색창도 header 에 넣어 보겠습니다. 블로그 이름 옆의 검색창은 사이트 전체에 대한 것이므로 header 가 맞습니다.

```

<form> <label for="keyword">검색어
</label> <input type="text" id="keyword" name="keyword" /> <button type="submit">찾기
</button>
</form>

```

아래 상자에는 main 을 뺀 세 덩어리를 살아 있는 요소로 넣었습니다. main 은 이 파일에 하나만 두기로 했으므로 섹션 5 의 완성본에만 있습니다.

화면 굵고 큰 글씨 "민준의 부엌", 그 아래 소개 한 줄,

그 아래 "검색어" 글자와 입력칸과 [찾기] 단추가 한 줄로, 그 아래 점(.) 이 붙은 링크 세 줄, 마지막에 연락처 한 줄

민준의 부엌
혼자 사는 사람의 밥상 기록

검색어

찾기

전체 글
반찬
간식

민준의 부엌 · 문의 minjun 앳 example.com

검색창은 보낼 곳을 정하지 않아서 [찾기] 를 눌러도 이 페이지가 다시 열릴 뿐입니다. 어디로 보낼지 정하는 것은 서버가 있어야 하는 일이라 이 자료의 범위 밖입니다.

▫ 직접 해보기 2

위 <nav> 의 목록에 를 하나 더 넣어 "국물 요리" 메뉴를 만들어 보세요. 링크 주소는 옆의 것과 같은 것을 그대로 쓰세요.

2단계 글 한 편

섹션 3

글 한 편은 떼어 내도 말이 되므로 <article> 입니다. 그 글만의 머리말과 꼬리말도 안에 넣습니다.

main 안에 들어갈 첫 번째 덩어리입니다.

```
<main> <article> <header> <h3>고구마 맛탕 만들기</h3> <p>2026년 8월 11일 · 간식
</p> </header> <p>남은 고구마로 십 분이면 됩니다.</p> <footer> <p>글쓴이 : 김민준 ·
조희 128회</p> </footer> </article>
</main>
```

여기서 header 는 개념03 섹션 6 에서 본 '글 한 편의 머리말' 입니다. 섹션 2 에서 만든 블로그 전체의 header 와는 다른 header 입니다. 같은 태그를 두 번 썼지만 들어 있는 자리가 달라서 뜻이 다릅니다.

★ 제목 단계를 지키세요

이 페이지의 제목 순서는 이렇게 갑니다.

h2	민준의 부엌	블로그 이름
h3	고구마 맛탕 만들기	글 제목
h4	재료	글 안의 소제목
h4	만드는 순서	글 안의 소제목

h2 다음에 h5 로 건너뛰면 안 됩니다. 한 단계씩 내려가야 화면 낭독기가 만드는 목차가 어긋나지 않습니다.

위 표대로 이 파일의 데모도 h2 → h3 → h4 로 씁니다. h1 만 안 씁니다. 이 자료 파일 맨 위의 큰 제목이 이미 h1 이라서, 데모에 h1 을 또 넣으면 한 페이지에 h1 이 두 개가 되기 때문입니다. (여러분이 새 파일을 만들 때는 맨 위 제목을 h1 으로 쓰세요. 그러면 h1 → h2 → h3 이 됩니다. 🖋 5 의 정답이 그 모양입니다)

화면 굵고 조금 큰 글씨 "고구마 맛탕 만들기", 그 아래 낱자 한 줄,

본문 한 줄, 마지막에 "글쓴이 : 김민준 · 조회 128회" 한 줄

고구마 맛탕 만들기
2026년 8월 11일 · 간식

남은 고구마로 십 분이면 됩니다.

글쓴이 : 김민준 · 조회 128회

글 제목 · 낱자 · 본문 · 글쓴이가 그냥 네 줄로 보입니다. 어디까지가 머리말인지 화면으로는 알 수 없습니다. 코드에서 </header> 가 어디인지 찾아보세요.

▫ 직접 해보기 3

위 <footer id="step2PostFooter"> 안에 <p>댓글 4개</p> 를 한 줄 더 넣어 보세요. 글 꼬리말이 두 줄이 됩니다.

3단계 안을 나누고 결다리 붙이기

섹션 4

글 안의 "재료" 와 "만드는 순서" 는 제목을 붙일 수 있는 주제 묶음이므로 <section> 입니다. 관련 글 목록은 <aside> 입니다.

개념04 의 흐름도로 판단해 봅시다.

"재료" 부분

- 떼어 내면 무슨 요리의 재료인지 모릅니다 → article 아님
- 지우면 요리를 못 만듭니다 → aside 아님
- "재료" 라는 제목을 붙일 수 있습니다 → section 입니다

"같이 보면 좋은 글" 목록

- 떼어 내면 어느 글의 관련 글인지 모릅니다 → article 아님
- 지워도 맛탕 만드는 데 아무 지장 없습니다 → aside 입니다

코드는 이렇게 됩니다.

```
<article> <header> ... </header> <section> <h4>재료</h4> <ul> <li>고구마 2개
</li> <li>설탕 3큰술</li> </ul> </section> <section> <h4>만드는 순서</h4> <ol> <li>
고구마를 깍둑 썰니다.</li> <li>기름에 노릇하게 튀깁니다.
</li> </ol> </section> <footer> ... </footer>
</article> <aside> <h4>같이 보면 좋은 글</h4> <ul> ... </ul>
</aside>
```

재료는 순서가 없으니 ul, 만드는 순서는 순서가 있으니 ol 입니다. 03단원에서 배운 그대로입니다. 목록 태그도 시맨틱 태그입니다.

aside 는 main 안에 article 과 나란히 두었습니다. 이 글에 딸린 결다리라서 그렇습니다. 사이트 전체에 대한 결다리라면 main 밖에 두는 것이 맞습니다.

화면 "재료" 아래에 점(•) 목록 세 줄, "만드는 순서" 아래에

1. 2. 3. 번호 목록 세 줄, "같이 보면 좋은 글" 아래에 점(•) 목록 두 줄이 차례로 보입니다

재료

고구마 2개
설탕 3큰술
식용유 적당량

만드는 순서

고구마를 깍둑 썰니다.
기름에 노릇하게 튀깁니다.
줄인 설탕물에 버무립니다.

같이 보면 좋은 글

세 덩어리가 전부 왼쪽에 세로로 쌓였습니다. <aside> 도 옆으로 안 갑니다. 개념04 에서 본 그대로입니다.

▸ 직접 해보기 4

<section id="step3Ingredient"> 의 목록에 물엿 1큰술 을 하나 더 넣어 보세요. 재료가 네 줄이 됩니다.

완성

섹션 5

1단계부터 3단계까지를 한 덩어리로 합쳤습니다. 이것이 06단원의 결승선입니다.

아래 상자가 완성본입니다. 이 안에 이 파일의 유일한 main 이 들어 있습니다.

코드에서 확인할 것들입니다.

- header 가 두 번 나옵니다. 블로그 것 하나, 글 한 편 것 하나.
- footer 가 두 번 나옵니다. 블로그 것 하나, 글 한 편 것 하나.
- main 은 한 번뿐입니다.
- nav 는 위쪽 메뉴 한 곳에만 있습니다.
aside 안의 관련 글 목록에는 nav 를 쓰지 않았습니다.
그건 주요 길 안내가 아니라 걸다리 목록이기 때문입니다.
- div 는 하나도 없습니다. 모든 덩어리에 뜻이 있어서 그렇습니다.

들여쓰기를 눈으로 따라가 보세요. 안으로 들어갈수록 두 칸씩 밀려 있습니다. 브라우저는 들여쓰기를 안 보지만, 사람은 들여쓰기로 구조를 읽습니다.

★ 화면을 보고 실망하지 마세요.

아주 밋밋한 글 덩어리로 보입니다. 선도 없고 색도 없고 배치도 없습니다. 그게 정상입니다. HTML 은 "무엇인가" 만 정합니다. 07단원부터 CSS 를 배우면 이 페이지가 눈에 띄게 달라집니다. 그때 지금 만든 뼈대를 그대로 다시 씁니다. 버리지 마세요.

화면 블로그 이름과 소개, 검색칸 한 줄, 점(•) 메뉴 세 줄,

글 제목과 날짜, 본문 한 줄("십 분" 만 굵게), "재료" 와 점 목록 세 줄, "만드는 순서" 와 번호 목록 세 줄, 글쓴이 한 줄, "같이 보면 좋은 글" 과 점 목록 두 줄, 마지막에 연락처와 저작권 두 줄이 위에서 아래로 이어집니다

민준의 부엌

혼자 사는 사람의 밥상 기록

검색어

찾기

전체 글

반찬

간식

고구마 맛탕 만들기

2026년 8월 11일 · 간식

남은 고구마로 십 분이면 됩니다.

재료

고구마 2개

설탕 3큰술

식용유 적당량

만드는 순서

고구마를 깍둑 썰니다.
기름에 노릇하게 튀깁니다.
줄인 설탕물에 버무립니다.

글쓴이 : 김민준 · 조회 128회

같이 보면 좋은 글

감자 조림 만들기
남은 고구마 보관법

민준의 부엌 · 문의 minjun.ott@example.com
© 2026 민준의 부엌

위 완성본 안에는 구역 태그가 열 개 들어 있습니다. (header 2개 · footer 2개 · section 2개 · nav · main · article · aside) 그런데 화면에는 그 흔적이 하나도 없습니다. HTML 이 하는 일이 무엇인지 이보다 잘 보여 주는 그림은 없습니다.

▸ 직접 해보기 5

새 파일 내블로그.html 을 만들어 위 완성본을 참고해 여러분의 글 페이지를 만들어 보세요. <!DOCTYPE html> 부터 직접 쓰고, 주제는 마음대로 정하세요. <link> 로 ../공통.css 를 연결하면 글꼴이 이 파일과 같아집니다.

자주 하는 실수

섹션 6

태그 하나하나를 알겠는데 합치면 헛갈리는 지점들입니다.

이런 실수를 합니다
달는 순서를 어김

이런 실수를 합니다

실수 1 — 닫는 태그 순서가 뒤엎킴

```
<main>
<article> <p>본문</p>
</main>
</article>
```

에러는 안 납니다. 브라우저가 자기 나름대로 고쳐서 그림니다. 그런데 고친 결과가 내 생각과 다릅니다. article 이 main 밖으로 밀려나거나 엉뚱한 곳에 붙습니다. 화면은 비슷해 보여서 한참 뒤에야 알아챱니다. 나중에 연 태그를 먼저 닫으세요. 들여쓰기를 지키면 눈에 보입니다.

이런 실수를 합니다

공통 부분을 main 안에

이런 실수를 합니다

실수 2 — header 와 nav 를 main 안에 넣음

```
<main>
<header>민준의 부엌</header>
<nav>전체 글 / 반찬</nav>
<article> ... </article>
</main>
```

“다 안에 넣으면 깔끔하겠지” 하고 이렇게 씁니다. 화면은 똑같습니다. 하지만 “이 페이지의 본문은 블로그 이름부터입니다” 라고 말한 셈입니다. 본문으로 건너뛰어도 또 메뉴부터 듣게 됩니다. 모든 페이지에 똑같이 있는 것은 main 밖에 두세요.

이런 실수를 합니다

제목 단계 건너뛰기

이런 실수를 합니다

실수 3 — 제목 단계를 건너뛴

```
<h2>민준의 부엌</h2>
<h5>고구마 맛탕 만들기</h5>
```

“h5 가 크기가 딱 맞아서” 골랐다면 잘못 고른 것입니다. h1~h6 은 크기를 고르는 태그가 아니라 단계를 적는 태그입니다. h2 다음은 h3 입니다. 건너뛰면 낭독기가 만드는 목차에 빈 칸이 생겨 “여기서 두 단계를 건너뛰었는데 뭘 빠뜨렸나” 하고 헛갈리게 됩니다. 크기는 09단원에서 CSS 로 정합니다.

이런 실수를 합니다

화면이 밋밋해서 잘못된 줄 앎

이런 실수를 합니다

실수 4 — 화면이 밋밋해서 틀린 줄 알고 다 지움

```
<div><div><div> ... 로 되돌리기
```

완성본을 열어 보고 “아무 일도 안 일어났네” 하며 시맨틱 태그를 전부 지우는 경우입니다. 밋밋한 것이 정상입니다. 06단원까지는 CSS 를 하나도 안 썼으니까요. 07단원부터 이 뼈대에 색과 배치를 입힙니다. 그때 이 구조가 없으면 꾸밀 곳을 잡을 수가 없습니다. 지우지 마세요.

이런 실수를 합니다

들여쓰기 포기

이런 실수를 합니다

실수 5 — 들여쓰기를 안 함

```
<main>
<article>
<header>
<h3>고구마 맛탕</h3>
</header>
</article>
</main>
```

브라우저는 들여쓰기를 안 봅니다. 그래서 화면은 완벽하게 똑같습니다. 문제는 사람입니다. 태그가 서른 개쯤 되면 어느 닫는 태그가 어느 여는 태그의 짝인지 못 찾습니다. 실수 1 이 바로 여기서 나옵니다. 한 단계 들어갈 때마다 스페이스 두 칸을 지키세요.

시맨틱 구조

푸는 법

- 1) 이 파일을 VS Code 로 열고, 동시에 브라우저로도 여세요.
- 2) 각 문제 상자 안의 주석 자리에 코드를 씁니다.
- 3) 저장(Ctrl+S) 하고 브라우저에서 새로고침(F5) 합니다.
- 4) 주석에 적힌 '기대 화면' 과 같은지 눈으로 대조합니다.

실행하면 나오는 화면 — 시맨틱 구조

★ 이 단원의 문제는 다른 단원과 성격이 다릅니다.

06단원 태그는 화면을 거의 바꾸지 않습니다. 그래서 “화면이 예뻐졌는가” 라는 정답을 알 수 없습니다. 대신 이렇게 확인하세요.

- 글자와 줄 수가 기대한 대로 나왔는가
- 태그를 알맞게 골랐는가 (정답 파일과 코드를 대조)

화면이 맛있하다고 틀린 것이 아닙니다. 그게 이 단원의 결과물입니다.

문제 1~9 기본 문제 10~12 [응용] 문제 13~14 [도전] 문제 15 [확인]

각 상자 안에 코드를 쓰고 저장한 뒤 새로고침(F5) 하세요. 자세한 문제 설명과 기대하는 결과는 코드 주석에 적혀 있습니다. VS Code 에서 이 파일을 열어 문제 번호를 찾아 읽으세요. 화면이 멋지게 나오는 것이 정상입니다.

문제 1 (개념01 섹션1)

제목 한 줄과 문단 두 줄을 div 하나로 묶으세요.

제목 : 동네 축구 모임 (h3 를 쓰세요) 문단1 : 매주 토요일 아침 8시에 모입니다. 문단2 : 초보자도 환영합니다.

기대 출력 굵고 조금 큰 글씨로 "동네 축구 모임" 이 보이고
그 아래 문단 두 줄이 차례로 보입니다. 모두 세 줄입니다.
이렇게 되면 틀린 것: 제목이 보통 크기면 h3 대신 div 나 p 를 쓴 것입니다.
div 를 씌워도 글자 모양은 안 바뀌니, 코드로 확인하세요.

문제 1 — 세 줄을 div 로 묶기

제목 한 줄과 문단 두 줄을 <div> 하나로 묶으세요.

여기에 코드를 쓰세요

문제 2 (개념01 섹션3)

아래 문장에서 가격 부분만 span 으로 묶고, class 이름을 price 로 붙이세요.

오늘의 국수는 7,000원입니다.

묶을 곳은 "7,000원" 다섯 글자입니다.

기대 출력 "오늘의 국수는 7,000원입니다." 가 한 줄로 보입니다.
span 을 씌운 부분도 나머지와 똑같은 보통 글씨입니다.
이렇게 되면 틀린 것: 한 줄이던 문장이 "오늘의 국수는" / "7,000원" /
"입니다."
세 줄로 쪼개지면 span 이 아니라 div 를 쓴 것입니다.

문제 2 — 문장 속 값을 span 으로 묶기

가격 부분만 `<span class="price">` 로 묶으세요.

여기에 코드를 쓰세요

문제 3 (개념02 섹션2)

같은 내용을 두 번 쓰세요. 한 번은 `div` 로, 한 번은 `section` 으로 감쌉니다.

소제목 : 여는 시간 (h4 를 쓰세요) 문단 : 평일은 9시부터 18시까지입니다.

다 쓰고 나면 두 상자를 비교해 보세요.

기대 출력 굵은 소제목과 문단 한 줄로 된 덩어리가 두 벌 보입니다.
두 덩어리는 글자 크기, 줄 간격, 여백이 완전히 같습니다.
이렇게 되면 틀린 것: 두 덩어리가 달라 보이면 안쪽 태그를 서로 다르게 쓴 것입니다.
바깥 태그(`div`, `section`)는 화면을 절대 바꾸지 않습니다.

문제 3 — div 판과 section 판을 같이 써 보기

같은 내용을 `<div>` 와 `<section>` 으로 각각 감싸고 결과가 같은지 확인하세요.

여기에 코드를 쓰세요

문제 4 (개념03 섹션2)

가게 사이트의 머리말을 만드세요. header 를 쓰세요.

가게 이름 : 골목 분식 (h3 를 쓰세요) 한 줄 소개 : 1998년부터 한자리에서

기대 출력 굵고 조금 큰 글씨로 "골목 분식",
그 아래 보통 글씨로 "1998년부터 한자리에서" 한 줄이 보입니다.
이렇게 되면 틀린 것: "골목 분식" 이 보통 크기면 h3 를 안 쓴 것입니다.
header 는 글자를 크게 만들지 않습니다.

문제 4 — header 만들기

가게 이름과 한 줄 소개를 <header> 로 묶으세요.

여기에 코드를 쓰세요

문제 5 (개념03 섹션3)

가게 사이트의 메뉴 줄을 만드세요. nav 안에 ul 과 li 와 a 를 씁니다.

메뉴 세 개 : 메뉴판 / 오시는 길 / 사진

링크 주소는 셋 다 개념03_페이지_빠대_태그.html 로 하세요.

기대 출력 점(•) 이 붙은 세 줄이 보이고, 글자는 파란색 밑줄 링크입니다.
눌러 보면 개념03 파일로 이동합니다.
이렇게 되면 틀린 것: 점이 안 보이면 ul 과 li 를 안 쓴 것입니다.
글자가 검은색이면 a 태그를 안 쓴 것입니다.

문제 5 — nav 로 메뉴 만들기

<nav> 안에 , , <a> 로 메뉴 세 개를 만드세요.

여기에 코드를 쓰세요

문제 6 (개념03 섹션5)

가게 사이트의 꼬리말을 만드세요. footer 를 쓰세요.

문단1 : 골목 분식 · 서울시 어딘가 5길 12 문단2 : 저작권 기호와 함께 “2026 골목 분식”

저작권 기호는 02단원에서 배운 특수문자로 씁니다.

기대 출력 보통 글씨로 두 줄이 보입니다.
 둘째 줄은 동그라미 안에 c 가 든 기호로 시작합니다.
 이렇게 되면 틀린 것: 기호 대신 &cop; 같은 글자가 그대로 보이면
 특수문자 이름을 잘못 적은 것입니다. copy 라고 정확히 쓰세요.
 (없는 이름을 적으면 브라우저는 고쳐 주지 않고 글자로 그립니다)

문제 6 — footer 만들기

주소와 저작권 표시를 <footer> 로 묶으세요.

여기에 코드를 쓰세요

문제 7 (개념04 섹션3)

“주차 안내” 라는 주제 묶음을 만드세요. section 을 쓰고 제목을 꼭 넣으세요.

제목 : 주차 안내 (h3 를 쓰세요) 문단 : 가게 뒤편에 세 자리가 있습니다.

기대 출력 굵고 조금 큰 글씨 “주차 안내” 아래에 문단 한 줄이 보입니다.
 이렇게 되면 틀린 것: 제목을 빼면 화면은 문단 한 줄만 남습니다.
 section 인데 제목이 없으면 개념04 섹션7 의 실수 1 입니다.

문제 7 — 제목이 있는 section 만들기

<section> 을 쓰고 제목을 반드시 같이 넣으세요.

여기에 코드를 쓰세요

문제 8 (개념04 섹션2)

후기 하나를 만드세요. 떼어 내도 말이 되는 덩어리이므로 article 을 씁니다.

제목 : 김밥이 맛있어요 (h3 를 쓰세요)
날짜 : 2026년 8월 5일
본문 : 재료가 아주 신선했습니다.

기대 출력 굵고 조금 큰 글씨 "김밥이 맛있어요" 아래에
날짜 한 줄과 본문 한 줄이 차례로 보입니다. 모두 세 줄입니다.
이렇게 되면 틀린 것: 화면은 div 로 감싸도 똑같이 나옵니다.
코드에서 article 을 썼는지 직접 확인하세요.

문제 8 — article 로 후기 하나 만들기

후기 하나를 <article> 로 감싸세요.

여기에 코드를 쓰세요

문제 9 (개념04 섹션4)

본문 옆에 붙는 겹다리를 만드세요. aside 를 씁니다.

소제목 : 오늘의 추천 (h4 를 쓰세요) 목록 : 라면 / 김밥 / 떡볶이 (ul 과 li 세 개)

기대 출력 굵은 소제목 "오늘의 추천" 아래에 점(•) 이 붙은 세 줄이 보입니다.
오른쪽으로 가지 않고 왼쪽에 그대로 쌓입니다.
이렇게 되면 틀린 것: 오른쪽으로 갈 거라 기대했다면 개념04 섹션7 의 실수 3
입니다.
aside 는 위치를 바꾸지 않습니다.

문제 9 — aside 로 결다리 만들기

추천 목록을 <aside> 로 감싸세요.

여기에 코드를 쓰세요

문제 10 [응용] (개념02 섹션1)

아래 코드는 전부 div 로만 되어 있습니다. 알맞은 시맨틱 태그로 바꿔 쓰세요. 안쪽의 h4 와 p 는 그대로 두고, div 만 바꾸면 됩니다. main 은 쓰지 마세요. 이 파일에서는 문제 14 에만 씁니다.

```
<div>
```

```
<div> <h4>골목 분식</h4>
</div>
<div> <h4>오늘의 국수</h4> <p>멸치 육수로 끓였습니다.</p>
</div>
<div> <p>문의 02-000-0000</p>
</div>
```

```
</div>
```

힌트: 바깥 div 는 그냥 묶는 껍데기라 그대로 두세요.

안쪽 셋은 각각 머리말 / 주제 묶음 / 꼬리말입니다.

기대 출력 바꾸기 전과 완전히 똑같습니다.
굵은 소제목 두 개와 문단 두 줄이 모두 네 줄로 보입니다.
이렇게 되면 틀린 것: 화면이 달라졌다면 h4 나 p 를 같이 건드린 것입니다.
시맨틱 태그로 바꾼 것만으로는 화면이 절대 안 바뀝니다.

문제 10 [응용] — div 만 있는 코드를 시맨틱으로

<div> 만 쓴 코드를 알맞은 시맨틱 태그로 바꿔 쓰세요. 화면은 하나도 안 바뀌어야 합니다.

여기에 코드를 쓰세요

문제 11 [응용] (개념04 섹션5)

아래 네 가지에 무엇을 쓸지 고르세요. article / section / aside / div 중 하나씩입니다. 코드를 쓰는 문제가 아니라 판단하는 문제입니다. 답은 종이나 메모장에 적고, 정답 파일과 대조하세요.

(가) 쇼핑몰 상품 목록에 있는 상품 카드 하나

(사진, 상품 이름, 가격, 담기 단추가 들어 있음)

(나) 회사 소개 페이지의 “찾아오시는 길” 부분

(다) 글 아래에 붙는 “이 글을 쓴 사람” 소개 상자

(라) 두 문단을 나란히 두려고 한 번 감싼 껍데기

풀 때는 개념04 섹션5 의 흐름도를 위에서부터 차례로 던지세요. 1. 떼어 내도 말이 되나? 2. 지워도 되나? 3. 제목을 붙일 수 있나? 4. 아니면 div

기대 출력 이 문제는 화면에 아무것도 안 나옵니다. 상자가 비어 있는 것이 정상입니다. 이렇게 되면 틀린 것: 네 개를 전부 section 으로 골랐다면 흐름도의 1번과 2번 질문을 건너뛴 것입니다.

문제 11 [응용] — 어떤 태그를 쓸지 고르기

네 가지 상황에 article · section · aside · div 중 무엇을 쓸지 고르세요. 답은 종이에 적으세요.

문제 12 [응용] (개념03 섹션6)

글 한 편에 자기만의 머리말과 꼬리말을 붙이세요. article 안에 header 와 footer 를 각각 하나씩 넣습니다.

머리말 안 : 제목 “떡볶이 국물 남기지 않는 법” (h3)

날짜 “2026년 8월 8일”

본문 : 마지막에 밥을 볶으면 됩니다. 꼬리말 안 : 글쓴이 : 최은지

기대 출력 굵고 조금 큰 글씨 제목 아래에 날짜 한 줄, 본문 한 줄,
"글쓴이 : 최은지" 한 줄이 차례로 보입니다. 모두 네 줄입니다.
이렇게 되면 틀린 것: header 와 footer 를 article 밖에 두면 화면은 같지만
"이 글의 머리말" 이 아니라 "페이지의 머리말" 이 되어 버립니다.

문제 12 [응용] — 글 한 편에 머리말과 꼬리말 붙이기

<article> 안에 <header> 와 <footer> 를 넣으세요.

여기에 코드를 쓰세요

문제 13 [도전] (개념04 섹션2)

글 목록을 만드세요. article 두 개를 나란히 쓰고, 각 글에 머리말과 꼬리말을 붙입니다. 그 아래에 결다리 하나를 붙입니다.

첫째 글 : 제목 "여름 김치 담그기" (h3) / 날짜 "2026년 7월 30일"

본문 "소금 양을 줄이는 것이 요령입니다."
꼬리말 "글쓴이 : 김민준"

둘째 글 : 제목 "냉면 육수 내기" (h3) / 날짜 "2026년 8월 2일"

본문 "동치미 국물을 반 섞으면 시원합니다."
꼬리말 "글쓴이 : 이서연"

결다리 : 소제목 "많이 본 글" (h4) / 목록 두 줄 "김밥 싸기", "된장 끓이기"

main 은 쓰지 마세요. 문제 14 에서 씁니다.

기대 출력 제목 · 날짜 · 본문 · 글쓴이 순서의 덩어리가 두 벌 이어서 보이고,
그 아래에 굵은 소제목 "많이 본 글" 과 점(•) 목록 두 줄이 보입니다.
두 글 사이에 구분선은 없습니다.
이렇게 되면 틀린 것: 구분선이 없어서 어디까지가 첫 글인지 안 보이는 것이
정상입니다.
선을 그으려고 hr 을 넣지 마세요. 선은 10단원에서 CSS 로 긋습니다.

문제 13 [도전] — 글 목록 만들기

<article> 두 개와 <aside> 하나로 글 목록을 만드세요.

여기에 코드를 쓰세요

문제 14 [도전] (개념05 섹션5)

페이지 한 장을 통째로 조립하세요. 이 파일에서 main 을 쓰는 유일한 문제입니다.

만들 것 : 골목 분식의 메뉴 소개 페이지

```
header  가게 이름 "골목 분식" (h3) / 소개 "1998년부터 한자리에서"
nav      메뉴판 / 오시는 길 / 사진      (ul + li + a,
      링크 주소는 셋 다 개념05_전체_구조_만들기.html)
```

main

```
article
  header  제목 "오늘의 국수" (h3) / 날짜 "2026년 8월 11일"
  section 소제목 "재료" (h4) + ul 세 줄 "멸치", "다시마", "소면"
  section 소제목 "끓이는 순서" (h4) + ol 두 줄
      "육수를 20분 끓입니다.", "면을 3분 삶습니다."
  footer  "글쓴이 : 사장님"
  aside    소제목 "겉들이면 좋아요" (h4) + ul 두 줄 "김밥", "단무지"
  footer  주소 한 줄 + 저작권 한 줄
```

먼저 종이에 네모를 그리고 나서 코드를 쓰세요. 개념05 섹션1 의 설계도를 참고하세요. 들여쓰기를 두 칸씩 지켜야 닫는 태그를 안 헛갈립니다.

기대 출력 가게 이름과 소개, 점(•) 메뉴 세 줄, 글 제목과 날짜,
"재료" 와 점 목록 세 줄, "끓이는 순서" 와 1. 2. 번호 목록 두 줄,
글쓴이 한 줄, "겉들이면 좋아요" 와 점 목록 두 줄,
마지막에 주소와 저작권 두 줄이 위에서 아래로 이어집니다.
이렇게 되면 틀린 것: 화면이 미미한 글 덩어리로 보이면 맞게 한 것입니다.
색이나 배치가 필요하면 07단원부터 CSS 로 합니다.

문제 14 [도전] — 페이지 한 장 통째로 조립하기

header · nav · main · article · section · aside · footer 를 모두 써서 페이지 한 장을 만드세요.

여기에 코드를 쓰세요

문제 15 [확인] (개념03 섹션7, 개념04 섹션7)

아래 코드는 브라우저에서 에러 없이 잘 그려집니다. 화면도 멀쩡해 보입니다. 그런데 잘못된 곳이 네 군데 있습니다. 찾아서 고치세요.

```
<main>
```

```
<header>골목 분식</header>
<span> <div>오늘의 국수</div>
</span>
```

```
</main>
<section>
```

```
<p>멸치 육수로 끓였습니다.</p>
```

```
</section>
<main>
```

```
<p>문의 02-000-0000</p>
```

```
</main>
```

찾을 것 네 가지입니다. (1) 한 페이지에 하나만 있어야 하는 태그가 두 개입니다. (2) 페이지 전체에 공통으로 들어가는 부분이 잘못된 자리에 있습니다. (3) 인라인 요소 안에 블록 요소가 들어가서 줄이 쪼개집니다. (4) 어떤 구역에 있어야 할 것이 빠져서 무슨 주제인지 알 수 없습니다.

고친 코드는 아래 상자에 글자 그대로 적으세요. 살아 있는 코드로 쓰면 이 파일에 main 이 두 개가 되어 버립니다. 글자 그대로 쓸 때는 꺾쇠를 < 와 > 로 바꿔서 doc-code 상자에 넣습니다.

기대 출력 검은 상자 안에 흰 글씨로 고친 코드가 보입니다.
이렇게 되면 틀린 것: 코드가 글자로 안 보이고 실제로 그려지면
꺾쇠를 < > 로 안 바꾼 것입니다.

문제 15 [확인] — 잘못된 곳 네 군데 찾아 고치기

에러도 안 나고 화면도 멀쩡한데 잘못된 코드입니다. 네 군데를 찾아 고친 코드를 적으세요.

여기에 코드를 쓰세요

06단원 마무리 시맨틱 구조

자주 넘어지는 자리

- 1 시맨틱 태그로 바뀌도 **화면이 안 바뀐다**는 것. 이걸 미리 말 안 하면 배신감을 느낍니다

CSS 시작하기

OPEN 07_CSS_시작하기

저는 문단입니다. 내용은 HTML 이 정했습니다.

아메리카노 4000원

카페라떼 4500원

딸기라떼 5500원

01 CSS 란 무엇인가

02 CSS 를 HTML 에 연결하는 세 가지 방법

03 규칙의 문법

04 색 표기법

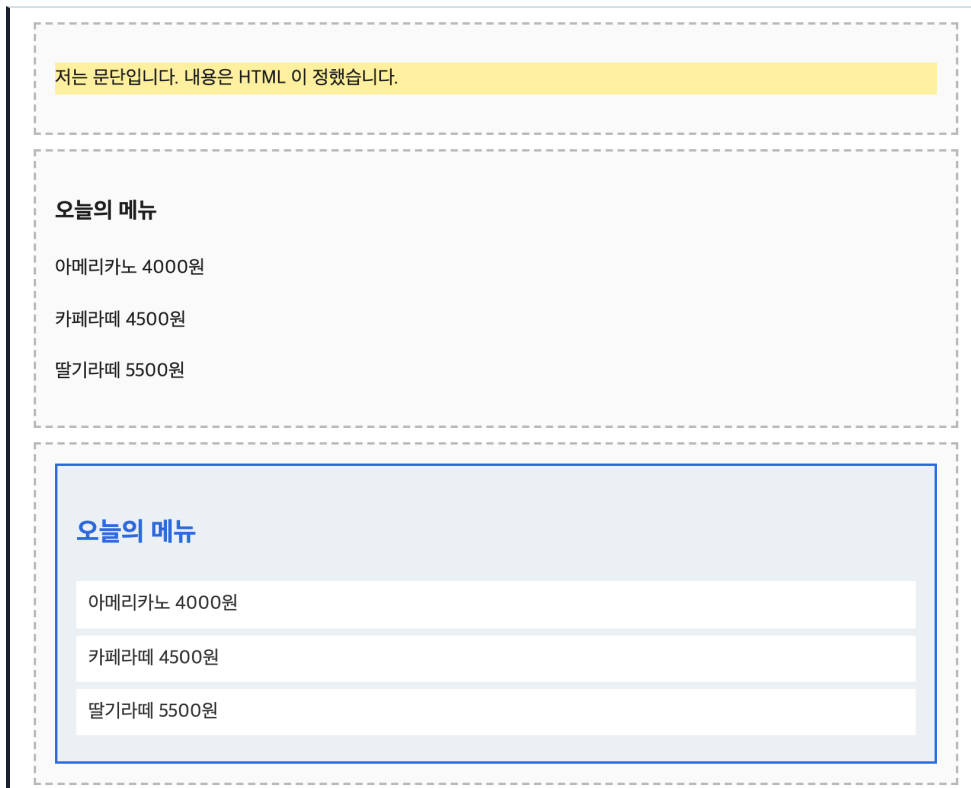
05 상속과 기본값

- 연습문제

CSS 란 무엇인가

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — CSS 란 무엇인가

06단원까지 우리는 HTML 만 썼습니다. HTML 로 한 일은 딱 하나였습니다. 글에 이름표를 붙이는 일입니다.

<code><h1>오늘의 메뉴</h1></code>	이 글자는 큰 제목이다
<code><p>아메리카노 4000원</p></code>	이 글자는 문단이다
<code>...</code>	이건 목록이다

이름표를 붙이면 브라우저가 나름대로 그려 줍니다. 제목은 크고 굵게, 문단은 아래위로 조금 띄워서. 그런데 그건 어디까지나 브라우저가 정한 모양입니다. “제목은 파란색으로”, “메뉴 줄마다 흰 상자라” 같은 요구는 HTML 로 못 합니다.

그래서 언어가 하나 더 있습니다. CSS 입니다.

HTML → 이것은 "무엇인가" (제목이다 / 문단이다 / 목록이다)
 CSS → 그것이 "어떻게 보이는가" (파랗다 / 크다 / 여백이 넓다)

CSS는 Cascading Style Sheets의 줄임말입니다. '스타일(모양)을 적어 둔 종이'라고 생각하면 됩니다. 앞의 Cascading은 "규칙이 여러 개 겹쳤을 때 어느 것이 이기는가"를 뜻하는데, 그건 08단원에서 따로 배웁니다. 지금은 신경 쓰지 마세요.

이 파일에서는 CSS를 쓰는 방법을 배우지 않습니다. "CSS가 있으면 무엇이 달라지는가"를 눈으로 확인하는 것이 전부입니다. 쓰는 방법은 개념02, 문법은 개념03에서 배웁니다.

이 파일은 같은 HTML에 CSS를 붙이기 전과 후를 나란히 보여 줍니다. 아래로 내려가면서 "글자는 똑같은데 모양만 달라졌다"는 점을 계속 확인하세요. 코드를 외울 필요는 없습니다. 개념02부터 하나씩 배웁니다.

내용은 HTML, 모양은 CSS

섹션 1

역할이 딱 나뉩니다. 무슨 글자가 보이는지는 HTML이 정하고, 그 글자가 어떻게 보이는지는 CSS가 정합니다.

아래 상자 안에는 이렇게 썼습니다.

```
<p id="roleText">저는 문단입니다. 내용은 HTML이 정했습니다.</p>
```

그리고 이 파일 맨 위 style 안에 이렇게 한 줄을 적어 두었습니다.

```
#roleText { background-color: #fff3a0; }
```

결과를 보면 배경만 노랗게 칠해졌고 글자는 하나도 안 바뀌었습니다. CSS는 글자를 바꾸지 못합니다. 칠하기만 합니다.

반대로 HTML의 글자를 바꾸면 화면의 글자가 바뀌지만 노란 배경은 그대로 남습니다. 두 가지가 서로 상관없이 움직입니다.

이 구분을 왜 이렇게 강조하느냐면, 초보자가 가장 많이 하는 질문이 "CSS로 글자를 바꾸려면 어떻게 해요?"이기 때문입니다. 답은 "CSS로는 못 합니다. HTML을 고치세요"입니다.

화면 한 줄짜리 문단인데 글자 뒤가 연한 노란색으로 칠해져 있습니다

저는 문단입니다. 내용은 HTML이 정했습니다.

▹ 직접 해보기 1

위 문단의 글자를 여러분 이름으로 바꾸고 저장(Ctrl+S)한 뒤 브라우저에서 새로고침(F5)하세요. 글자는 바뀌지만 노란 배경은 그대로 남습니다.

아래 두 상자의 HTML 글자는 완전히 같습니다. 위쪽에는 CSS 를 하나도 안 붙였고, 아래쪽에만 붙였습니다.

위 상자(CSS 없음) 의 HTML 은 이렇습니다.

```
<div> <h3>오늘의 메뉴</h3> <p>아메리카노 4000원</p> <p>카페라떼 4500원</p> <p>
딸기라떼 5500원</p>
</div>
```

아래 상자(CSS 있음) 의 HTML 은 여기에 class 이름표만 붙인 것입니다.

```
<div class="menu-card"> <h3 class="menu-title">오늘의 메뉴</h3> <p class="menu-item">
아메리카노 4000원</p>
...
</div>
```

class 는 01단원 개념04 에서 배운 '속성' 중 하나이고, 06단원 개념01 섹션5 에서 이름 짓는 규칙까지 배워 두었습니다. 요소에 붙이는 별명입니다. 그때는 "나중에 잡을 손잡이" 라고만 했는데, 지금부터 그 손잡이를 실제로 잡습니다. CSS 가 "이 별명이 붙은 요소를 이렇게 그려라" 라고 지시할 때 씁니다. 별명을 고르는 방법(선택자) 은 개념03 에서 제대로 배웁니다.

style 에 적은 규칙은 이렇습니다.

```
.menu-card { background-color: #eef2f7; padding: 16px; border: 2px solid #2d6cdf; }
.menu-title { color: #2d6cdf; font-size: 22px; }
.menu-item { color: #333333; background-color: #ffffff; padding: 6px 10px; }
```

여기서 padding, border, font-size 는 이 단원 범위가 아닙니다. 지금은 이런 게 있다는 것만 보세요. 여백과 테두리는 10단원, 글자 크기는 09단원에서 배웁니다. 이 단원에서 여러분이 직접 쓰는 것은 color 와 background-color 두 개뿐입니다.

화면 검은 글씨로 "오늘의 메뉴" 가 조금 크고 굵게 나오고,

그 아래 검은 글씨 세 줄이 사이를 띄우고 놓입니다. 색은 전부 같습니다

```
오늘의 메뉴
아메리카노 4000원
카페라떼 4500원
딸기라떼 5500원
```

화면 파란 테두리를 두른 연한 파란 상자가 생기고, 그 안에 파란 제목이

더 크게 나옵니다. 메뉴 세 줄은 각각 흰 상자에 담겨 떨어져 놓입니다

오늘의 메뉴
아메리카노 4000원
카페라떼 4500원
딸기라떼 5500원

두 상자를 번갈아 보세요. 글자는 한 글자도 다르지 않습니다. 달라진 것은 색과 여백과 크기뿐입니다. 이것이 역할을 나눈다는 말의 뜻입니다.

◦ 직접 해보기 2

위 .menu-title 규칙의 color: #2d6cdf; 를 color: #c0392b; 로 바꿔 보세요. 제목만 빨간색이 됩니다. 글자는 그대로입니다.

같은 뼈대에 다른 CSS

섹션 3

역할을 나눠 두면 좋은 점이 여기서 나옵니다. HTML 을 그대로 두고 CSS 만 갈아 끼우면 완전히 다른 페이지가 됩니다.

아래 두 상자의 HTML 뼈대는 똑같습니다. div 하나, h3 하나, p 하나입니다. 글자도 똑같습니다. 붙인 class 이름만 다릅니다.

```
A 안 <div class="card-a"><h3 class="title-a">...</h3><p class="row-a">...</p></div>
B 안 <div class="card-b"><h3 class="title-b">...</h3><p class="row-b">...</p></div>
```

실제 웹사이트에서는 여기서 한 걸음 더 나갑니다. 바깥 상자의 class 하나만 바꾸면 안쪽까지 통째로 바뀌게 만듭니다. 그렇게 하려면 “어떤 것 안에 있는 어떤 것” 을 고르는 방법이 필요한데, 그건 08단원 선택자에서 배웁니다. 지금은 이름표를 하나씩 다 붙여서 같은 효과를 냈습니다.

중요한 것은 이것입니다. 디자인을 바꾸려고 HTML 을 다시 쓴 곳이 한 군데도 없습니다. 글(내용) 과 모양 (디자인) 을 따로 관리할 수 있다는 뜻입니다. 글을 쓰는 사람과 디자인하는 사람이 서로 방해하지 않고 일할 수 있습니다.

화면 진한 파란 상자 안에 흰 제목과 연한 하늘색 글자 한 줄이 보입니다

오늘의 메뉴
아메리카노 4000원

화면 크림색 상자 안에 진한 노란빛 갈색 제목과 짙은 갈색 글자 한 줄이 보입니다

오늘의 메뉴
아메리카노 4000원

▣ 직접 해보기 3

B 안의 .card-b 규칙에서 background-color: #fff7e6; 를 background-color: #222222; 로 바꿔 보세요. 배경이 검게 바뀌면서 갈색 글씨가 잘 안 보이게 됩니다. 색을 고를 때 배경과 글자를 같이 생각해야 하는 이유입니다. (개념04 에서 다룹니다)

CSS 가 맡는 네 가지

섹션 4

CSS 로 할 수 있는 일은 아주 많지만, 크게 네 덩어리로 묶입니다. 이 단원에서는 그중 '색' 하나만 배웁니다.

앞으로 13단원까지 배울 내용을 미리 지도로 그려 두면 길을 잃지 않습니다.

색	글자색, 배경색	07단원 (지금 여기)
크기	글자 크기, 상자의 너비	09단원 · 10단원
간격	안쪽 여백, 바깥 여백	10단원
배치	가로로 나란히, 위에 겹치기	11단원 · 12단원 · 13단원

네 가지 모두 쓰는 문법은 똑같습니다. 개념03 에서 배울

```
선택자 { 속성: 값; }
```

이 한 가지 모양만 계속 반복됩니다. 그래서 07단원을 제대로 해 두면 뒤가 훨씬 수월합니다. 새로 배우는 것은 문법이 아니라 '속성 이름' 뿐이기 때문입니다.

화면 용어 네 개가 왼쪽에, 설명이 한 칸 들여쓴 채로 그 아래에 놓입니다

색
글자색과 배경색 - 07단원, 바로 지금 배웁니다
크기
글자 크기는 09단원, 상자의 너비와 높이는 10단원
간격
안쪽 여백과 바깥 여백 - 10단원
배치
가로로 나란히 놓기, 위에 겹치기 - 11단원부터 13단원

화면 보라색 배경 위에 흰 글자 한 줄이 보입니다

글자색은 color, 배경색은 background-color 로 정합니다.

▣ 직접 해보기 4

#mapColor 규칙의 background-color: #8e44ad; 를 background-color: #c0392b; 로 바꿔 보세요. 배경만 빨강게 바뀌고 글자는 계속 흰색입니다.

첫 규칙 읽어 보기

섹션 5

아래가 이 단원에서 여러분이 계속 쓰게 될 모양입니다. 지금은 소리 내어 읽을 수 있으면 충분합니다.

style 안에 이렇게 적어 두었습니다.

```
#firstRule {
  color: #ffffff;
  background-color: #27853f;
}
```

읽는 법은 이렇습니다.

"id 가 firstRule 인 요소를, 글자색은 흰색으로, 배경색은 초록색으로 그려라"

#firstRule 이 "누구를"에 해당하고, 중괄호 안이 "어떻게"에 해당합니다. 각 부분의 정확한 이름(선택자·선언·속성·값)은 개념03에서 볼입니다.

#ffffff 와 #27853f 는 색을 적는 방법입니다. 처음 보면 암호 같지만 개념04에서 규칙을 알고 나면 읽을 수 있게 됩니다. 지금은 "색을 이렇게도 적는구나" 정도로 넘어가세요.

화면 초록색 배경 위에 흰 글자 한 줄이 보입니다

이 줄은 CSS 규칙 하나로 이렇게 바뀌었습니다.

이 파일에 있는 CSS 는 전부 <head> 안의 <style> 에 들어 있습니다. VS Code 에서 파일 맨 위를 보세요. CSS 를 HTML 에 붙이는 방법이 몇 가지 더 있는데, 개념02에서 세 가지를 다 봅니다.

⇒ 직접 해보기 5

#firstRule 규칙에서 color: #ffffff; 줄을 통째로 지워 보세요. 배경은 초록색 그대로지만 글자가 검은색으로 돌아와 읽기 힘들어집니다. 확인했으면 다시 되돌려 놓으세요.

자주 하는 실수

섹션 6

HTML 과 마찬가지로 CSS 도 틀려도 에러가 나지 않습니다. 아무 일도 안 일어날 뿐입니다. 그래서 "왜 안 되지?" 를 스스로 찾아야 합니다.

이런 실수를 합니다

실수 1 — CSS 로 글자 내용을 바꾸려고 함

```
#roleText { text: "안녕하세요"; }
```

아무 일도 안 일어납니다. text 라는 속성은 없습니다. 화면에 보이는 글자는 HTML 이 정합니다. 글자를 바꾸려면 <p> 안의 글자를 직접 고치세요.

이런 실수를 합니다

실수 2 — 저장하지 않고 새로고침

화면이 그대로입니다. 브라우저는 저장된 파일을 읽습니다. VS Code 에서 Ctrl+S 로 저장한 다음 브라우저에서 F5 를 누르세요. 탭 제목 옆에 점이 떠 있으면 아직 저장 전입니다.

이런 실수를 합니다

실수 3 — CSS 만 써 놓고 HTML 에 연결하지 않음

아무 일도 안 일어납니다. CSS 는 저절로 적용되지 않습니다. HTML 파일이 "이 CSS 를 쓰겠다" 고 밝혀줘야 합니다. 연결하는 방법 세 가지를 개념02 에서 배웁니다. 초보자가 겪는 "왜 색이 안 변해요?" 의 절반은 이것입니다.

이런 실수를 합니다

실수 4 — 클릭하면 움직이게 하려고 함

CSS 로는 못 합니다. "버튼을 누르면 목록이 늘어난다" 같은 동작은 자바스크립트가 맡습니다. CSS 는 모양만 정합니다. 역할이 셋으로 나뉘어 있다고 기억하세요. HTML 은 무엇인가, CSS 는 어떻게 보이는가, 자바스크립트는 무엇을 하는가 입니다.

이런 실수를 합니다

실수 5 — 브라우저가 원래 넣어 둔 모양을 자기가 넣은 줄 앎

섹션 2 의 첫 번째 상자를 다시 보세요. CSS 를 하나도 안 붙였는데도 제목이 크고 굵었고 문단 사이가 벌어져 있었습니다. 그건 브라우저가 원래 갖고 있는 기본 모양입니다. 내가 쓴 CSS 때문인지 브라우저 기본값 때문인지 헷갈리면 원인을 못 찾습니다. 이 이야기는 개념05 에서 자세히 봅니다.

정리

▹ 직접 해보기

6

정답

1) 섹션 1 의 문단을 이렇게 고칩니다.

```
<p id="roleText">김민준</p>
```

→ 화면의 글자만 "김민준"으로 바뀌고 노란 배경은 그대로 남습니다.

CSS 규칙은 "id 가 roleText 인 요소"를 가리킬 뿐,
그 안에 무슨 글자가 있는지는 상관하지 않기 때문입니다.

2) 파일 맨 위 style 안의 .menu-title 규칙을 이렇게 고칩니다.

```
.menu-title {
  color: #c0392b;
  font-size: 22px;
}
```

→ 두 번째 상자의 "오늘의 메뉴"만 빨간색이 됩니다.

첫 번째 상자의 제목은 class 가 없으니 그대로 검은색입니다.

3) style 안의 .card-b 규칙을 이렇게 고칩니다.

```
.card-b {
  background-color: #222222;
  padding: 14px;
}
```

→ B 안 상자가 거의 검은색이 되고, 그 위의 갈색 글자가 잘 안 보입니다.

배경만 바꾸면 글자색도 같이 손봐야 한다는 것을 알 수 있습니다.
확인했으면 #fff7e6로 되돌려 놓으세요.

4) style 안의 #mapColor 규칙을 이렇게 고칩니다.

```
#mapColor {
  color: #ffffff;
  background-color: #c0392b;
}
```

→ 배경만 빨갳게 바꿉니다. color 줄은 건드리지 않았으니 글자는 계속 흰색입니다.

한 규칙 안의 두 줄이 서로 따로 논다는 것을 확인할 수 있습니다.

5) style 안의 #firstRule에서 color 줄을 지웁니다.

```
#firstRule {
  background-color: #27853f;
}
```

→ 배경은 초록색 그대로인데 글자가 검은색으로 돌아옵니다.

지정하지 않은 속성은 원래 값으로 남는다는 뜻입니다.
초록 배경에 검은 글씨라 읽기 불편합니다. 다시 color: #ffffff; 를 넣으세요.

글자 내용은 HTML 이 정합니다

섹션 1

```
#roleText {  
  background-color: #fff3a0;  
}
```

화면 그 문단의 배경만 노랗게 칠해집니다. 글자 내용은 그대로입니다

같은 글에 CSS 를 붙였을 때

섹션 2

아래 padding / border / font-size 는 이 단원에서 배우지 않습니다.
지금은 "이런 게 있다는 것만" 보세요. 여백과 테두리는 10단원,

글자 크기는 09단원에서 제대로 배웁니다.

```
.menu-card {  
  background-color: #eef2f7;
```

padding: 16px; /* 10단원 border: 2px solid #2d6cdf; /* 10단원

```
}  
.menu-title {  
  color: #2d6cdf;
```

font-size: 22px; /* 09단원

```
}  
.menu-item {  
  color: #333333;  
  background-color: #ffffff;
```

padding: 6px 10px; /* 10단원 margin: 6px 0; /* 10단원

```
}
```

화면 연한 회색빛 파란 상자 안에 파란 제목이 크게 나오고,

그 아래 흰 줄 세 개가 조금씩 떨어져 놓입니다

같은 뼈대에 서로 다른 CSS

섹션 3

```
.card-a {
  background-color: #2d6cdf;
```

padding: 14px; /* 10단원

```
}
.title-a {
  color: #ffffff;
}
.row-a {
  color: #dbe7ff;
}
```

화면 진한 파란 상자에 흰 제목, 연한 하늘색 줄이 보입니다

```
.card-b {
  background-color: #fff7e6;
```

padding: 14px; /* 10단원

```
}
.title-b {
  color: #b7791f;
}
.row-b {
  color: #6b5323;
}
```

화면 크림색 상자에 진한 노란빛 갈색 제목, 짙은 갈색 줄이 보입니다

CSS가 말하는 네 가지 중 '색' 만 미리

섹션 4

```
#mapColor {
  color: #ffffff;
  background-color: #8e44ad;
}
```

화면 보라색 배경 위에 흰 글자 한 줄이 보입니다

```
#firstRule {  
  color: #ffffff;  
  background-color: #27853f;  
}
```

화면 초록색 배경 위에 흰 글자 한 줄이 보입니다

CSS 를 HTML 에 연결하는 세 가지 방법

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

같은 폴더의 연습용.css 도 VS Code 로 함께 열어 두세요.



실행하면 나오는 화면 — CSS 를 HTML 에 연결하는 세 가지 방법

개념01 에서 CSS 가 무엇인지 봤습니다. 그런데 CSS 를 아무리 잘 써도, HTML 파일이 그 CSS 를 읽지 않으면 화면은 하나도 안 바뀝니다. 저절로 붙는 일은 없습니다.

CSS 를 HTML 에 붙이는 방법은 세 가지입니다.

- ① 인라인 태그 안에 `style="..."` 을 직접 씀
- ② style 태그 head 안에 `<style>` 을 열고 그 안에 규칙을 씀
- ③ 외부 파일 .css 파일을 따로 만들고 `<link>` 로 불러옴

셋 다 화면을 바꾸는 힘은 같습니다. 다른 것은 '관리하기 좋은 정도' 입니다. 결론부터 말하면 ③ 외부 파일이 정답이고, ① 인라인은 거의 안 씁니다. 왜 그런지는 섹션 5 에서 이유를 하나씩 봅니다.

지금 이 파일은 세 가지를 전부 쓰고 있습니다.

head 안에 <link rel="stylesheet" href="../공통.css" />	← ③ 외부 파일
head 안에 <link rel="stylesheet" href="연습용.css" />	← ③ 외부 파일
head 안에 <style> ... </style>	← ②
본문 안에 <p style="color: #c0392b;">	← ①

VS Code 로 이 파일 맨 위를 열어서 실제로 그렇게 되어 있는지 보세요.

이 파일에서는 세 가지 연결 방법을 전부 실제로 동작시켜 보여 줍니다. 같은 폴더에 있는 연습용.css 파일도 함께 열어 두면 “다른 파일에 적은 규칙이 이 화면에 나타난다” 는 것을 확인하기 좋습니다.

세 가지 방법 한눈에

섹션 1

CSS 를 적는 자리가 세 군데 있습니다. 문법은 자리마다 조금씩 다릅니다.

먼저 세 가지가 각각 어떻게 생겼는지만 눈에 익혀 두겠습니다. 섹션 2 부터 하나씩 실제로 동작시켜 볼 것이니 지금 외우지 마세요.

- ① 인라인


```
<p style="color: #c0392b;">글자</p>
```

 → 중괄호가 없습니다. 선택자도 없습니다. 속성과 값만 적습니다.
이미 "이 요소" 라고 정해진 자리라서 누구를 고를 필요가 없습니다.
- ② style 태그


```
<style>
  p { color: #c0392b; }
</style>
```

 → 선택자와 중괄호가 있습니다. 한 파일 안에서만 쓰입니다.
- ③ 외부 파일
 style.css 파일 안에 p { color: #c0392b; }
 HTML 의 head 안에 <link rel="stylesheet" href="style.css" />
 → 규칙 모양은 ② 와 똑같습니다. 사는 곳만 다른 파일입니다.

세 가지의 차이를 한 줄로 정리하면 이렇습니다.

① 요소 하나에만	② 파일 하나 안에서만	③ 여러 파일이 함께
-----------	--------------	-------------

화면 세 줄이 보입니다. 첫 줄은 빨간 글씨, 둘째 줄은 파란 배경에 흰 글씨,

셋째 줄은 빨간 배경에 흰 글씨입니다

- ① 인라인으로 빨갱게 칠한 줄입니다.
- ② style 태그로 칠한 줄입니다.
- ③ 외부 파일 연습용.css 로 칠한 줄입니다.

▹ 직접 해보기 1

위 세 줄이 각각 파일의 어느 자리에서 색을 받아 왔는지 VS Code 에서 찾아보세요. 첫 줄은 그 줄 자체에, 둘째 줄은 파일 맨 위 <style> 안에, 셋째 줄은 연습용.css 안에 있습니다.

방법 ① 인라인

섹션 2

태그 안에 style 이라는 속성을 붙이고, 그 값으로 스타일을 적습니다. 선택자도 종괄호도 없습니다.

아래 상자 안에는 이렇게 썼습니다.

```
<p id="inlineDemo" style="color: #c0392b; background-color: #fff3a0;">
  저는 인라인으로 칠해졌습니다.
</p>
```

생김새를 뜯어보면 이렇습니다.

```
style="color: #c0392b; background-color: #fff3a0;"
  └─ 속성: 값; ─┬── 속성: 값; ───┬──
```

속성과 값 사이에는 콜론(:), 선언과 선언 사이에는 세미콜론(;) 을 찍습니다. 이 문법 자체는 개념03 에서 제대로 다룹니다.

인라인은 “이 요소 하나”에만 적용됩니다. 옆에 똑같이 생긴 문단이 열 개 있어도 그 열 개는 하나도 안 바뀝니다. 같은 모양을 열 곳에 주려면 style=”...” 을 열 번 복사해 붙여야 합니다. 고칠 일이 생기면 열 곳을 전부 찾아 고쳐야 합니다. 이것이 인라인의 문제입니다.

그래도 인라인이 완전히 쓸모없지는 않습니다. 언제 쓰는지는 섹션 5 에서 봅니다.

화면 첫 줄만 노란 배경에 빨간 글씨이고, 둘째 줄은 평범한 검은 글씨입니다

저는 인라인으로 칠해졌습니다.

저는 바로 옆 문단이지만 아무것도 안 붙었습니다.

style 속성 값 안에서는 큰따옴표를 다시 쓰면 안 됩니다. 값을 감싸는 큰따옴표가 거기서 끝나 버리기 때문입니다. (01단원 개념04 섹션4 에서 배운 따옴표 규칙이 여기서도 그대로 적용됩니다. 안쪽에 또 큰따옴표가 필요하면 작은따옴표를 쓰세요)

▹ 직접 해보기 2

위 상자의 둘째 문단에도 style=”color: #2d6cdf;” 를 붙여 파란 글씨로 만들어 보세요. <p 와 > 사이에 한 칸 띄고 씁니다.

<head> 안에 <style> 을 열고 그 안에 규칙을 적습니다. 그 파일 전체에 적용됩니다.

이 파일 맨 위 head 안에 이렇게 적혀 있습니다.

```
<style>#styleTagDemo {
  color: #ffffff;
  background-color: #2d6cdf;
}
</style>
```

그리고 본문에는 이렇게 썼습니다.

```
<p id="styleTagDemo">저는 style 태그에서 색을 받았습니다.</p>
```

인라인과 달리 여기에는 선택자(#styleTagDemo) 와 중괄호가 있습니다. "누구를 꾸밀지" 를 먼저 고르고, 중괄호 안에 "어떻게" 를 적는 방식입니다.

style 태그의 장점은 한 곳에 모여 있다는 것입니다. 색을 바꾸고 싶으면 파일 맨 위 style 만 보면 됩니다. 본문을 뒤질 필요가 없습니다.

단점은 '그 파일 안에서만' 이라는 점입니다. 페이지가 열 개면 열 개 파일의 style 을 전부 고쳐야 합니다. 그래서 실제 사이트에서는 ③ 외부 파일을 씁니다.

style 태그는 head 안에 넣는 것이 약속입니다. body 안에 넣어도 브라우저는 대충 알아듣지만, 남이 코드를 읽을 때 "스타일이 어디 있지?" 하고 헤매게 됩니다.

화면 파란 배경 위에 흰 글자 한 줄이 보입니다

저는 style 태그에서 색을 받았습니다.

▸ 직접 해보기 3

파일 맨 위 <style> 안의 #styleTagDemo 규칙에서 background-color 값을 #8e44ad 로 바꿔 보세요. 배경이 보라색이 됩니다.

방법 ③ 외부 파일

CSS 만 들어 있는 따로 된 파일을 만들고, HTML 에서 <link> 로 불러옵니다. 실무에서 쓰는 방법입니다.

이 폴더에는 연습용.css 라는 파일이 하나 더 있습니다. VS Code 에서 열어 보세요. 안에 이렇게만 적혀 있습니다.

```
.notice {
  color: #ffffff;
  background-color: #2d6cdf;
}
.notice-warn {
  color: #ffffff;
  background-color: #c0392b;
}
.notice-ok {
  color: #ffffff;
  background-color: #27853f;
}
```

중요한 점이 두 가지 있습니다.

1. <style> 태그가 없습니다. .css 파일 안에는 규칙만 적습니다.

거기에 <style> 을 또 쓰면 파일 전체가 망가집니다.
2. <html> 도 <body> 도 없습니다. HTML 문서가 아니기 때문입니다.

이 파일을 불러오는 줄은 이 HTML 의 head 안에 있습니다.

```
<link rel="stylesheet" href="연습용.css" />
```

rel="stylesheet" 이 파일이 '스타일 시트' 라는 뜻입니다. 빠뜨리면 안 붙습니다.
href="연습용.css" 불러올 파일의 경로입니다. 04단원에서 배운 상대 경로입니다.

<link> 는 닫는 태그가 없는 빈 태그입니다. (01단원 개념02 의
 과 같은 종류)

경로는 이 HTML 파일이 있는 자리를 기준으로 씁니다. 연습용.css 는 같은 폴더에 있으니 파일 이름만 적으면 됩니다. 한 단계 위 폴더에 있는 공통.css 는 ../공통.css 라고 적습니다. 이 파일 맨 위에 그 줄도 같이 있습니다. 찾아보세요.

화면 첫 줄은 파란 배경에 흰 글씨, 둘째 줄은 빨간 배경에 흰 글씨입니다

저는 연습용.css 의 .notice 규칙으로 칠해졌습니다.
저는 .notice-warn 규칙으로 칠해졌습니다.

```
<link rel="stylesheet" href="연습용.css" />
```

위 <link> 줄은 <head> 안에 넣습니다. 브라우저가 본문을 그리기 전에 스타일을 먼저 읽어야 글자가 잠깐 안 꾸며진 채로 번쩍하는 일이 없습니다.

연습용.css 파일을 열어 .notice 의 background-color 를 #8e44ad 로 바꾸고 저장하세요. 그다음 이 HTML 을 새로고침하면 첫 줄이 보라색이 됩니다. HTML 파일은 한 글자도 안 고쳤는데 화면이 바뀝니다.

왜 외부 파일이 정답인가

섹션 5

이유는 두 가지입니다. 여러 페이지가 나눠 쓸 수 있고, 고칠 곳이 한 군데입니다.

이 자료 자체가 가장 좋은 예입니다.

여러분이 지금 보고 있는 노란 안내 상자, 파란 제목 띠, 점선 데모 상자는 전부 공통.css 라는 파일 하나에 적혀 있습니다. 01단원부터 14단원까지 모든 파일이 그 파일 하나를 함께 씁니다.

```
01단원/개념01.html  ↗
01단원/개념02.html  ↳ ../공통.css   (규칙은 여기 한 곳에만 있습니다)
07단원/개념02.html  ↘
```

제목 띠의 파란색을 바꾸고 싶다고 합시다.

외부 파일이면	공통.css 한 줄을 고칩니다. 끝입니다.
style 태그면	파일마다 들어가서 같은 줄을 고칩니다. 백 번 고칩니다.
인라인이면	제목마다 들어가서 고칩니다. 수백 번 고칩니다.

하나 더 있습니다. 브라우저는 한 번 받은 .css 파일을 기억해 둡니다. 그래서 두 번째 페이지부터는 다시 받지 않아 화면이 빨리 뜹니다. 인라인은 페이지를 열 때마다 같은 글자를 계속 다시 받습니다.

그러면 인라인은 언제 쓰나요? 아주 좁게 씁니다.

- 딱 한 요소에만, 다시는 안 쓸 예외적인 값을 줄 때
- 자바스크립트가 화면을 보고 그때그때 값을 계산해 넣을 때 (14단원 범위 밖)
- 메일 본문처럼 <style> 이나 <link> 를 못 쓰는 특수한 자리

수업 중에 “일단 빨리 확인만” 할 때 인라인을 쓰는 것은 괜찮습니다. 다만 확인이 끝나면 style 이나 외부 파일로 옮기세요.

화면 첫 줄과 셋째 줄이 똑같은 초록 배경에 흰 글씨이고,

가운데 둘째 줄만 평범한 검은 글씨입니다

첫 번째 완료 안내입니다.
가운데에 아무것도 안 붙은 줄을 하나 끼워 두었습니다.
두 번째 완료 안내입니다.

위 두 줄은 연습용.css 의 .notice-ok 규칙 한 개에서 색을 받아 왔습니다. 그 규칙 한 줄만 고치면 두 줄이 동시에 바뀝니다. 인라인이었다면 두 곳을 따로 고쳐야 합니다.

직접 해보기

5

연습용.css 의 .notice-ok 의 background-color 를 #b7791f 로 바꾸고 저장한 뒤 새로고침하세요. 위 상자의 두 줄이 한꺼번에 바뀝니다.

자주 하는 실수

섹션 6

연결이 안 됐을 때 브라우저는 아무 말도 하지 않습니다. 그냥 안 꾸며진 화면이 나올 뿐입니다. 아래 다섯 가지를 먼저 의심하세요.

이런 실수를 합니다

실수 1 — rel="stylesheet" 를 빠뜨림

```
<link href="연습용.css" />
```

아무 일도 안 일어납니다. 파일 경로는 맞는데도 그렇습니다. rel 은 “이 파일이 나와 어떤 관계인가” 를 알려 주는 속성입니다. rel="stylesheet" 가 없으면 브라우저는 이 파일을 스타일로 취급하지 않습니다. 에러도 안 납니다.

이렇게 쓰세요

```
<link rel="stylesheet" href="연습용.css" />
```

이런 실수를 합니다

실수 2 — href 경로나 파일 이름이 틀림

```
<link rel="stylesheet" href="연습용css" />
```

아무 일도 안 일어납니다. 위 예에서는 점 하나가 빠졌습니다. style.css 를 styles.css 로 쓰거나, 파일은 하위 폴더에 있는데 폴더 이름을 안 적은 경우도 같습니다. F12 를 눌러 Network 탭을 보면 그 파일이 빨강게 표시됩니다. 화면만 봐서는 알 수 없으니 이때만 F12 를 쓰세요.

이런 실수를 합니다

실수 3 — 인라인 style 안에 선택자와 중괄호를 씌

```
<p style="p { color: #c0392b; }">글자</p>
```

아무 일도 안 일어납니다. 아래 상자의 첫 줄이 그 결과입니다. 인라인 자리는 이미 “이 요소” 라고 정해져 있어서 선택자를 쓸 자리가 없습니다. 중괄호 없이 속성: 값; 만 적습니다. 선택자를 넣어 봤습니다. 이쪽은 속성과 값만 적었습니다.

이런 실수를 합니다

실수 4 — .css 파일 안에 <style> 태그를 씀

```
<style>
.notice { color: #ffffff; }
</style>
```

그 파일의 규칙이 전부 안 먹습니다. .css 파일은 처음부터 끝까지 CSS 로 읽힙니다. 거기에 HTML 태그가 들어오면 브라우저는 그 자리에서 헛갈려서 뒤에 적은 규칙까지 같이 버립니다. .css 파일 안에는 규칙만 적으세요.

이런 실수를 합니다

실수 5 — 인라인이 이기는 줄 모르고 style 태그만 고침

```
<p class="mix-blue" style="color: #c0392b;">
어느 쪽이 이길까요
</p>
```

아래 줄에는 .mix-blue 클래스(파란 글씨) 와 인라인 style(빨간 글씨) 이 둘 다 붙어 있습니다. 실제로는 빨간 글씨로 보입니다. 인라인이 이깁니다. <style> 만 열심히 고치는 동안 화면이 안 바뀌면 그 요소에 인라인이 붙어 있는지 먼저 확인하세요. 규칙이 겹칠 때 누가 이기는지는 08단원 명시도에서 배웁니다. 어느 쪽이 이길까요

정리

▸ 직접 해보기 6

정답

1) 찾아본 결과는 이렇습니다.

첫 줄 → 그 <p> 태그 안의 style="color: #c0392b"
둘째 줄 → 이 파일 head 안 <style> 의 #styleTagDemo2 규칙
셋째 줄 → 연습용.css 의 .notice-warn 규칙

→ 같은 화면 안의 색이 서로 다른 세 자리에서 온다는 것을 확인하는 것이 목적입니다.

2) 섹션 2 상자의 둘째 문단을 이렇게 고칩니다.

```
<p style="color: #2d6cdf;">저는 바로 옆 문단이지만 아무것도 안 붙었습니다.</p>
```

→ 그 줄만 파란 글씨가 됩니다. 첫 줄은 그대로 노란 배경에 빨간 글씨입니다.

인라인은 붙인 요소 하나에만 효과가 있다는 뜻입니다.

3) 파일 맨 위 style 안을 이렇게 고칩니다.

```
#styleTagDemo {
  color: #ffffff;
  background-color: #8e44ad;
}
```

→ 섹션 3 상자의 배경이 파란색에서 보라색으로 바뀝니다.

섹션 1 의 둘째 줄은 #styleTagDemo2 라는 다른 규칙을 쓰므로 그대로 파랑습니다.

4) 연습용.css 를 이렇게 고칩니다.

```
.notice {
  color: #ffffff;
  background-color: #8e44ad;
}
```

→ 섹션 4 상자의 첫 줄이 보라색이 됩니다.

HTML 파일은 손대지 않았는데 화면이 바뀌었다는 점이 핵심입니다.
확인했으면 #2d6cdf 로 되돌려 놓으세요.

5) 연습용.css 를 이렇게 고칩니다.

```
.notice-ok {
  color: #ffffff;
  background-color: #b7791f;
}
```

→ 섹션 5 상자의 첫 줄과 셋째 줄이 동시에 갈색으로 바뀝니다.

가운데 줄은 class 가 없어서 그대로입니다.
규칙 한 개가 여러 곳을 담당한다는 것이 외부 파일의 힘입니다.
확인했으면 #27853f 로 되돌려 놓으세요.

style 태그로 쓴 규칙

섹션 3

```
#styleTagDemo {
  color: #ffffff;
  background-color: #2d6cdf;
}
```

화면 파란 배경 위에 흰 글자 한 줄이 보입니다

```
#styleTagDemo2 {
  color: #ffffff;
  background-color: #2d6cdf;
}
```

화면 위와 똑같이 파란 배경에 흰 글자입니다.

id 는 한 페이지에 하나뿐이라 #styleTagDemo 규칙 하나로는
두 줄을 칠할 수 없어서 규칙을 하나 더 썼습니다.

클래스였다면 규칙 하나로 두 줄을 다 칠할 수 있습니다(섹션 5)

섹션 6 [실수 5]: 인라인과 style 태그가 같은 요소를 건드릴 때

```
.mix-blue {
  color: #2d6cdf;
}
```

화면 이 규칙은 파란 글자를 지시하지만, 같은 요소에 붙은 인라인 style 이

이겨서 실제로는 빨간 글자로 보입니다

을 열고 그 안에 규칙을 씀

③ 외부 파일 .css 파일을 따로 만들고 <link> 로 불러옴

셋 다 화면을 바꾸는 힘은 같습니다. 다른 것은 '관리하기 좋은 정도' 입니다.
결론부터 말하면 ③ 외부 파일이 정답이고, ① 인라인은 거의 안 씁니다.
왜 그런지는 섹션 5 에서 이유를 하나씩 봅니다.

지금 이 파일은 세 가지를 전부 쓰고 있습니다.

파일 head 안에 <link rel="stylesheet" href="../공통.css" /> ← ③ 외부

파일 head 안에 <link rel="stylesheet" href="연습용.css" /> ← ③ 외부

파일 head 안에 <style> ...

```
p { color: #c0392b; }
```

```
#styleTagDemo {
  color: #ffffff;
  background-color: #2d6cdf;
}
```

규칙의 문법

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 파일 맨 위 <style> 안을 함께 보세요.



실행하면 나오는 화면 — 규칙의 문법

개념02 에서 CSS 를 붙이는 자리 세 군데를 봤습니다. 이제 그 자리에 무엇을 어떻게 적는지를 배웁니다.

CSS 에서 적는 것은 딱 한 가지 모양입니다. 이것 하나뿐입니다.

선택자 { 속성: 값; }

예를 들면 이렇습니다.

```
p { color: #2d6cdf; }
└─┬──────────┘
  │             └─ 중괄호 안: 어떻게 그릴지
  └─ 선택자: 누구를 그릴지
```

부품마다 이름이 있습니다. 앞으로 계속 나오니 여기서 정리합니다.

규칙(rule)	위 한 덩어리 전체
선택자(selector)	중괄호 앞에 적는 부분. 누구를 꾸밀지 고릅니다
선언(declaration)	중괄호 안의 "속성: 값;" 한 줄
속성(property)	무엇을 바꿀지 (color, background-color ...)
값(value)	어떻게 바꿀지 (#2d6cdf, red ...)

이 다섯 단어는 08단원부터 13단원까지 계속 쓰입니다. 지금 확실히 구분해 두면 나중에 설명을 훨씬 편하게 읽을 수 있습니다.

그리고 이 파일에서 가장 중요한 이야기가 섹션 5 에 있습니다. 01단원 개념02 에서 "HTML 은 틀려도 에러가 안 난다" 고 했습니다. CSS 도 똑같습니다. 오타를 내면 빨간 글씨가 뜨는 게 아니라 그 줄이 조용히 사라집니다. 그래서 눈으로 알아채는 연습이 필요합니다.

이 파일은 CSS 규칙을 이루는 부품들의 이름과 틀렸을 때 무슨 일이 일어나는지를 다룹니다. 특히 섹션 5 의 상자 네 개는 오타를 일부러 낸 채로 두었습니다. 결과가 어떻게 다른지 눈으로 비교하세요.

규칙 한 덩어리의 부품 이름

섹션 1

선택자 { 속성: 값; } 이 모양 하나만 계속 반복됩니다. 부품마다 이름을 붙여 두면 설명을 읽기가 편해집니다.

아래 상자를 만든 규칙은 이렇습니다.

```
#ruleParts {
  color: #ffffff;
  background-color: #2d6cdf;
}
```

뜯어보면 이렇습니다.

#ruleParts	선택자
{ }	중괄호. 선언을 담는 그릇
color: #ffffff;	선언 하나
color	속성
#ffffff	값
:	속성과 값을 잇는 콜론
;	선언 하나가 끝났다는 표시
위 전체 덩어리	규칙

읽는 순서는 사람이 말하는 순서와 같습니다.

"id 가 ruleParts 인 것을 / 글자색은 흰색으로 / 배경색은 파란색으로"

선언은 몇 개든 쓸 수 있습니다. 위에서는 두 개를 썼습니다. 선언을 하나도 안 쓰고 빈 중괄호만 뒤도 에러는 안 납니다. 아무 일도 안 일어날 뿐입니다.

줄바꿈과 띄어쓰기는 브라우저가 신경 쓰지 않습니다. 한 줄에 다 붙여 써도 되고 여러 줄로 나뉘어도 됩니다. 다만 사람이 읽기 좋게 선언마다 한 줄씩 쓰는 것이 관례입니다.

화면 파란 배경 위에 흰 글자 한 줄이 보입니다

이 줄은 규칙 하나로 만들어졌습니다.

```
#ruleParts { color: #ffffff; background-color: #2d6cdf; }
```

위 한 줄을 왼쪽부터 읽으면 이렇습니다. #ruleParts 가 선택자, color 가 속성, #ffffff 가 값입니다. color: #ffffff; 한 줄이 선언 하나이고, 중괄호까지 포함한 덩어리 전체가 규칙 하나입니다.

▣ 직접 해보기 1

파일 맨 위 <style> 의 #ruleParts 규칙에 선언을 하나 더 넣어 보세요. 아직 배운 속성이 두 개뿐이니 background-color 값을 #c0392b 로 바꿔 보는 것으로 충분합니다.

선택자 세 가지

섹션 2

선택자는 누구를 꾸밀지 고르는 부분입니다. 고르는 방법이 여러 가지인데, 이 단원에서는 기본 세 가지만 씁니다.

세 가지는 이렇게 적습니다.

```
li      { ... }   태그 이름 그대로. 아무 기호도 안 붙입니다
.notice-line { ... } 클래스. 점(.) 을 앞에 붙입니다
#pickMe { ... }   id. 우물 정(#) 을 앞에 붙입니다
```

HTML 쪽에는 이렇게 씁니다. 여기서는 점도 #도 붙이지 않습니다.

```
<li>목록 항목</li>
<p class="notice-line">클래스가 붙은 문단</p>
<p id="pickMe">id 가 붙은 문단</p>
```

점과 # 은 CSS 안에서만 붙입니다. 여기서 초보자가 가장 많이 헛갈립니다. 섹션 6 에서 두 방향의 실수를 다 살아 있는 예로 보여 드립니다.

셋의 성격이 다릅니다.

```
태그 이름  그 태그 전부. 하나도 빠짐없이 전부입니다
클래스     같은 별명을 붙인 요소 전부. 몇 개든 붙일 수 있습니다
id         딱 하나. 한 페이지에 같은 id 를 두 번 쓰지 않습니다
```

태그 이름 선택자는 미치는 범위가 아주 넓습니다. p { color: red; } 라고 쓰면 그 파일의 설명 글, 안내 상자, 정리 상자까지 모조리 빨개집니다. 그래서 이 자료에서는 데모 상자 안에만 있는 태그(ii) 로 보여 줍니다. 여러분 파일에서도 태그 이름 선택자는 조심해서 쓰세요.

여기서 ‘넓다’ 와 ‘세다’ 를 헛갈리지 마세요. 둘은 다른 이야기입니다. 넓다 = 한 규칙이 많은 요소에 걸린다
세다 = 규칙이 겹쳤을 때 이긴다 태그 이름 선택자는 넓지만, 겹쳤을 때는 오히려 가장 약한 축입니다.
셋이 한 요소에 겹쳐 붙었을 때 누가 이기는지는 08단원 명시도에서 배웁니다. 지금은 각각 무엇을
고르지만 알면 됩니다.

화면	목록 세 줄이 전부 보라색 글자로 보입니다. 하나만 고를 수 없습니다
사과 바나나 포도	
화면	첫 줄과 셋째 줄만 배경이 노랑고, 가운데 줄은 그대로입니다
저는 notice-line 클래스가 붙었습니다. 저는 아무것도 안 붙었습니다. 저도 notice-line 클래스가 붙었습니다.	
화면	첫 줄만 초록 배경에 흰 글자이고, 둘째 줄은 평범한 검은 글자입니다
저는 id 가 pickMe 입니다. 저는 id 가 없습니다.	

이름을 지을 때는 보이는 모양이 아니라 뜻으로 지으세요. .red-text 라고 지어 두면 나중에 파란색으로 바꿀
때 이름이 거짓말이 됩니다. .notice-line, .menu-item 처럼 역할로 지으면 색이 바뀌어도 이름이 계속
맞습니다.

직접 해보기

2

위 ㉔ 상자의 가운데 문단에도 class="notice-line" 을 붙여 세 줄이 모두 노란 배경이 되게 하세요. CSS
는 한 글자도 고치지 않습니다.

선언 여러 개 쓰기

섹션 3

중괄호 안에는 선언을 몇 개든 쓸 수 있습니다. 선언마다 끝에 세미콜론(;) 을 찍어 구분합니다.
선언이 하나인 규칙과 둘인 규칙을 나란히 보겠습니다.

```
#oneDecl {
  color: #c0392b;
}

#manyDecl {
  color: #ffffff;
  background-color: #c0392b;
}
```


#oneDecl 은 글자색만 정했습니다. 배경색은 건드리지 않았으니 원래 상태 그대로 남습니다. 지정하지 않은 것은 바뀌지 않습니다.

세미콜론은 "이 선언은 여기서 끝" 이라는 표시입니다. 마침표라고 생각하면 됩니다. 마지막 선언 뒤의 세미콜론은 없어도 동작하지만, 항상 찍는 습관을 들이세요. 나중에 그 아래에 선언을 한 줄 더 붙일 때 사고가 납니다. (섹션 5 에서 봅니다)

줄바꿈은 브라우저에게 아무 의미가 없습니다. 아래 두 규칙은 완전히 같습니다.

```
#oneLine { color: #ffffff; background-color: #8e44ad; }

#oneLine {
  color: #ffffff;
  background-color: #8e44ad;
}
```

브라우저를 기준으로 보면 선언을 나누는 것은 오직 세미콜론뿐입니다. 줄을 아무리 예쁘게 나눠도 세미콜론이 없으면 한 덩어리로 읽힙니다. 이 사실이 섹션 5 의 세 번째 상자에서 그대로 사고로 이어집니다.

화면 빨간 글자 한 줄. 배경은 데모 상자의 연한 회색 그대로입니다

배경은 건드리지 않았습니다.

화면 빨간 배경 위에 흰 글자 한 줄이 보입니다

둘 다 정했습니다.

화면 보라색 배경 위에 흰 글자 한 줄이 보입니다

줄바꿈은 브라우저에게 의미가 없습니다.

◦ 직접 해보기 3

#oneDecl 규칙에 background-color: #fff3a0; 한 줄을 더 넣어 보세요. 앞 줄 끝에 세미콜론이 있는지 꼭 확인하고 넣으세요.

CSS 주석

섹션 4

/* 와 */ 사이에 쓴 글은 브라우저가 읽지 않습니다. 설명을 남기거나 규칙을 잠시 꺼 둘 때 씁니다.

HTML 주석과 CSS 주석은 기호가 다릅니다. 자리마다 맞는 것을 써야 합니다.

HTML 안에서	<!-- 주석 -->	(기호를 붙여 씁니다)
CSS 안에서	/* 주석 */	

위에서 HTML 주석 기호를 일부러 띄어 썼습니다. 붙여 쓰면 지금 읽고 있는 이 주석이 거기서 끝나 버리기 때문입니다.

CSS 주석은 두 가지 일에 씁니다.

1. 설명 남기기

```
/* 알림 상자 색 */
.notice { color: #ffffff; }

2. 규칙을 잠시 꺼 두기 (주석 처리)
#commentDemo {
  color: #2d6cdf;
  /* background-color: #fff3a0; */
}
```

2번이 특히 유용합니다. 지우지 않고 잠시 꺼 두면 “이 줄 때문인가?” 를 한 줄씩 확인할 수 있습니다. 화면이 이상할 때 범인을 찾는 가장 빠른 방법입니다.

주석은 겹쳐 쓸 수 없습니다. /* 안에 또 /* 를 쓰면 먼저 나온 */ 에서 주석이 끝나 버려서 뒤가 어긋납니다.

화면 파란 글자 한 줄. 배경은 데모 상자의 연한 회색 그대로입니다

글자색만 적용되고 배경은 안 칠해집니다.

```
#commentDemo {
  color: #2d6cdf;
  /* background-color: #fff3a0; */
}
```

▹ 직접 해보기 4

#commentDemo 안의 /* 와 */ 를 지워 주석을 풀어 보세요. 배경이 노랑게 칠해집니다. 확인했으면 다시 주석으로 감싸 두세요.

오타를 내면 어떻게 되나

섹션 5

이 파일에서 가장 중요한 섹션입니다. CSS 는 틀려도 에러 메시지를 안 냅니다. 틀린 부분만 버리고 나머지는 그대로 실행합니다.

브라우저가 CSS 를 읽는 방식은 이렇습니다.

- 1 규칙을 하나씩 읽는다
- 2 읽다가 이해 못 하는 선언이 나오면 그 선언만 버린다
- 3 그리고 아무 말 없이 다음 선언으로 넘어간다

“그 선언만”이라는 부분이 중요합니다. 어디까지가 한 선언인지에 따라 피해 범위가 완전히 달라집니다. 아래 네 상자를 비교해 보세요.

- | | |
|------------|---------------------------------|
| ① 정상 | color 와 background-color 둘 다 적용 |
| ② 속성 이름 오타 | colr 은 없는 속성 → 그 줄만 버림 → 배경은 정상 |
| ③ 값 오타 | redd 는 없는 색 → 그 줄만 버림 → 배경은 정상 |
| ④ 세미콜론 빠뜨림 | 두 줄이 한 선언으로 묶임 → 두 줄이 같이 죽음 |

②와 ③은 피해가 한 줄입니다. 그래서 “배경은 됐는데 글자색만 안 되네?” 하고 비교적 빨리 알아챱니다.

④가 무섭습니다. 실제로 style 에 적힌 것은 이렇습니다.

```
#typoSemi {
  color: #ffffff
  background-color: #c0392b;
}
```

브라우저는 세미콜론이 나올 때까지를 한 선언으로 봅니다. 그래서 이렇게 읽습니다.

```
속성 = color
값   = #ffffff background-color: #c0392b
```

그런 값은 없으니 선언 하나를 통째로 버립니다. 내가 건드린 것은 한 줄인데 두 줄이 죽습니다. “분명히 아래 줄은 제대로 썼는데 왜 안 되지?” 의 진짜 원인이 이것입니다.

범인을 찾는 요령: 안 먹는 줄을 찾았으면 그 줄이 아니라 ‘바로 윗줄’을 보세요.

화면 빨간 배경 위에 흰 글자 한 줄이 보입니다

글자색과 배경색이 모두 적용됩니다.

화면 노란 배경 위에 검은 글자입니다. 빨간 글자가 아닙니다

글자색만 안 먹었습니다.

화면 노란 배경 위에 검은 글자입니다. ② 와 겉모습이 똑같습니다

이쪽도 글자색만 안 먹었습니다.

화면 배경도 안 칠해지고 글자도 검은색입니다. 상자의 연한 회색 위에

평범한 검은 글씨 한 줄만 보입니다

글자색도 배경색도 안 먹었습니다.

네 상자를 위아래로 훑어보세요. ②와 ③은 배경만 살아남았고, ④는 둘 다 죽었습니다. 어디까지 망가지는지가 다릅니다. 그런데 브라우저는 네 경우 모두 아무 말도 하지 않습니다.

직접 해보기

5

#typoSemi 규칙의 color: #ffffff 뒤에 세미콜론 하나만 찍어 보세요. 두 줄이 동시에 살아나 ① 과 똑같은 모양이 됩니다. 확인했으면 다시 지워서 원래대로 두세요.

자주 하는 실수

섹션 6

아래 여섯 가지는 전부 에러 없이 조용히 무시됩니다. 각 상자 안에 진짜로 잘못 쓴 결과를 그대로 넣어 두었습니다.

이런 실수를 합니다

실수 1 — 선언 중간에서 세미콜론을 빠뜨림

```
color: #ffffff
background-color: #27853f;
```

섹션 5의 ④에서 본 그대로입니다. 두 줄이 같이 죽습니다. 다만 마지막 선언 뒤의 세미콜론은 없어도 잘 동작합니다. 아래 상자가 그 경우입니다. 그래도 항상 찍으세요. 나중에 그 아래 줄을 하나 더 붙이는 순간 사고가 납니다. 마지막 선언의 세미콜론만 뺐습니다.

이런 실수를 합니다

실수 2 — 콜론 대신 등호를 씀

```
#typoEqual {
color = #c0392b;
background-color: #fff3a0;
}
```

그 줄만 무시됩니다. CSS에서 속성과 값을 잇는 기호는 콜론(:)입니다. 등호는 HTML의 속성에서 쓰는 기호입니다 (class="menu"). 두 언어가 섞여서 나는 실수입니다. 등호를 썼습니다.

이런 실수를 합니다

실수 3 — 속성 이름을 잘못 씀

```
bakground-color: #fff3a0;
```

그 줄만 무시됩니다. background의 c와 k 순서가 바뀌었습니다. backgroud, colour도 혼합니다. VS Code에서 속성 이름을 치면 자동완성 목록이 뜹니다. 목록에 안 뜨면 그 이름은 없는 것이니 그때 알아채면 됩니다. 배경만 안 먹었습니다.

이런 실수를 합니다

실수 4 — CSS 에서 클래스 이름 앞에 점을 안 붙임

```
price-tag { color: #c0392b; }
```

아무 일도 안 일어납니다. 점이 없으면 "price-tag 라는 이름의 태그를 찾아라" 라는 뜻이 됩니다. 그런 태그는 없으니 아무도 안 골립니다. HTML 에는 class="price-tag" 라고 제대로 써 두었는데도 그렇습니다. class="price-tag" 가 붙어 있습니다.

이렇게 쓰세요

```
.price-tag { color: #c0392b; }
```

이런 실수를 합니다

실수 5 — HTML 의 class 값에 점을 붙임

```
<p class=".sale-tag">할인</p>
```

아무 일도 안 일어납니다. 실수 4 와 정반대 방향입니다. 이 요소의 클래스 이름은 sale-tag 가 아니라 점까지 포함한 이상한 이름이 되어 버립니다. 그래서 CSS 의 .sale-tag 규칙과 만나지 못합니다. 점은 CSS 안에서만, HTML 의 class 값에는 이름만 씁니다. class 값에 점을 붙였습니다.

이렇게 쓰세요

```
<p class="sale-tag">할인</p>
```

이런 실수를 합니다

실수 6 — 닫는 중괄호를 빠뜨림

```
#braceOpen {
color: #c0392b;

#braceVictim {
background-color: #fff3a0;
}
```

그 아래에 쓴 규칙들이 전부 죽습니다. 중괄호를 안 닫으면 브라우저는 아래에 이어 쓴 규칙을 "앞 규칙 안에 들어 있는 것" 으로 읽어 버립니다. 그래서 #braceVictim 은 아무 데도 적용되지 않습니다. 피해가 파일 아래쪽 전체로 번지는 유일한 실수라 가장 위험합니다. "CSS 를 한 줄 고쳤더니 페이지 절반이 망가졌다" 면 중괄호부터 세어 보세요. 저는 중괄호를 안 닫은 규칙의 대상입니다. 저는 그 아래에 쓰인 규칙의 대상입니다.

정답

1) 파일 맨 위 style 안의 #ruleParts 를 이렇게 고칩니다.

```
#ruleParts {
  color: #ffffff;
  background-color: #c0392b;
}
```

→ 섹션 1 상자의 배경이 파란색에서 빨간색으로 바뀝니다.

글자색 선언은 건드리지 않았으니 흰 글자 그대로입니다.
확인했으면 #2d6cdf 로 되돌려 놓으세요.

2) 섹션 2 의 ㉡ 상자 가운데 문단을 이렇게 고칩니다.

```
<p class="notice-line">저는 아무것도 안 붙었습니다.</p>
```

→ 세 줄 모두 노란 배경이 됩니다.

CSS 규칙은 하나뿐인데 그 규칙이 세 곳을 담당하게 되었습니다.
클래스는 몇 개에든 붙일 수 있다는 뜻입니다.

3) style 안의 #oneDecl 을 이렇게 고칩니다.

```
#oneDecl {
  color: #c0392b;
  background-color: #fff3a0;
}
```

→ 섹션 3 첫 상자의 배경이 노랗게 칠해집니다.

앞 줄 끝의 세미콜론이 없으면 두 줄이 같이 죽습니다. 꼭 확인하세요.

4) style 안의 #commentDemo 를 이렇게 고칩니다.

```
#commentDemo {
  color: #2d6cdf;
  background-color: #fff3a0;
}
```

→ 섹션 4 상자의 배경이 노랗게 칠해집니다.

주석을 풀었다는 것은 그 줄을 다시 살렸다는 뜻입니다.
확인했으면 다시 `/* */` 로 감싸 두세요.

5) style 안의 `#typoSemi` 를 이렇게 고칩니다.

```
#typoSemi {
  color: #ffffff;
  background-color: #c0392b;
}
```

→ 섹션 5 의 ④ 상자가 ① 상자와 똑같이 빨간 배경에 흰 글자가 됩니다.

세미콜론 하나가 두 줄의 생사를 갈랐다는 것을 확인하는 것이 목적입니다.
확인했으면 다시 세미콜론을 지워서 원래대로 두세요.

규칙 한 덩어리

섹션 1

```
#ruleParts {
  color: #ffffff;
  background-color: #2d6cdf;
}
```

화면 파란 배경 위에 흰 글자 한 줄이 보입니다

선택자 세 가지

섹션 2

```
li {
  color: #8e44ad;
}
```

화면 이 파일 안의 모든 `li` 가 보라색 글자가 됩니다.

이 파일에서는 섹션 2 데모 상자 안에만 `li` 가 있습니다

```
.notice-line {
  background-color: #fff3a0;
}
```

화면 `class="notice-line"` 이 붙은 두 줄의 배경만 노랗게 칠해집니다

```
#pickMe {
  color: #ffffff;
  background-color: #27853f;
}
```

화면 id="pickMe" 인 줄 하나만 초록 배경에 흰 글자가 됩니다

선언을 여러 개 쓰기

섹션 3

```
#oneDecl {
  color: #c0392b;
}
```

화면 빨간 글자 한 줄. 배경은 건드리지 않았으니 그대로입니다

```
#manyDecl {
  color: #ffffff;
  background-color: #c0392b;
}
```

화면 빨간 배경 위에 흰 글자 한 줄이 보입니다

```
#oneLine { color: #ffffff; background-color: #8e44ad; }
```

화면 보라색 배경 위에 흰 글자. 위 규칙과 쓰는 방식만 다르고 결과는 같은 종류입니다

CSS 주석

섹션 4

```
#commentDemo {
  color: #2d6cdf;
```

```
background-color: #fff3a0;
```

```
}
```

화면 파란 글자만 적용되고 배경은 칠해지지 않습니다.

주석 안에 들어간 줄은 없는 것과 같습니다

오타를 냈을 때

섹션 5

```
#typo0k {
  color: #ffffff;
  background-color: #c0392b;
}
```

화면 빨간 배경에 흰 글자. 비교 기준이 되는 정상 규칙입니다

```
#typoProp {
  colr: #c0392b;
  background-color: #fff3a0;
}
```

화면 글자는 원래 검은색 그대로이고 배경만 노랗게 칠해집니다.

colr이라는 속성은 없어서 그 줄만 버려졌습니다

```
#typoValue {
  color: redd;
  background-color: #fff3a0;
}
```

화면 위와 똑같습니다. 글자는 검은색 그대로, 배경만 노란색입니다.

redd라는 색이 없어서 그 줄만 버려졌습니다

```
#typoSemi {
  color: #ffffff
  background-color: #c0392b;
}
```

화면 글자도 검은색 그대로, 배경도 안 칠해집니다.

세미콜론 하나가 없어서 두 줄이 한 덩어리로 묶여 통째로 버려졌습니다

자주 하는 실수

섹션 6

```
#semiLast {
  color: #ffffff;
  background-color: #27853f
}
```

화면 초록 배경에 흰 글자. 마지막 선언의 세미콜론은 없어도 동작합니다

```
#typoEqual {
  color = #c0392b;
  background-color: #fff3a0;
}
```

화면 글자는 검은색 그대로이고 배경만 노랗게 칠해집니다

```
#typoBg {
  color: #c0392b;
  background-color: #fff3a0;
}
```

화면 글자만 빨강고 배경은 안 칠해집니다

```
price-tag {
  color: #c0392b;
}
```

화면 아무 데도 안 칠해집니다. 점을 안 붙이면 'price-tag 라는 이름의 태그' 를

찾는 뜻이 되는데, 그런 태그는 세상에 없습니다

```
.sale-tag {
  background-color: #fff3a0;
}
```

화면 아무 데도 안 칠해집니다. HTML 쪽에서 class 값에 점을 붙여 두었기 때문에

클래스 이름이 '.sale-tag' 라는 이상한 이름이 되어 버렸습니다

이런 실수를 합니다

닫는 중괄호를 빠뜨린 상태를 그대로 재현했습니다.

이 블록은 뒤에 오는 규칙까지 잡아먹기 때문에 style 의 맨 끝에 두었습니다.

```
#braceOpen {
  color: #c0392b;
}
```

바로 이 자리에 닫는 중괄호가 없습니다

```
#braceVictim {
  background-color: #fff3a0;
}
}
```

화면 #braceOpen 은 빨간 글자가 되지만 #braceVictim 은 아무 일도 안 일어납니다

색 표기법

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

F12 를 눌러 개발자도구를 여는 연습도 섹션 5 에서 합니다.



실행하면 나오는 화면 — 색 표기법

지금까지 색을 적을 때 #2d6cdf 같은 암호 비슷한 글자를 계속 봤습니다. 이 파일에서 그 정체를 밝힙니다.

CSS 에서 색을 적는 방법은 여러 가지인데, 다 같은 것을 가리킵니다. 한국어로 “빨강”, 영어로 “red”, 물감 번호로 “#FF0000” 이라고 부르는 것과 같은 차이입니다.

red	이름으로
#c0392b	16진수 여섯 자리로
#f60	16진수 세 자리로 (줄여 쓴 것)
rgb(192, 57, 43)	빨강·초록·파랑 값을 숫자로
rgba(192, 57, 43, 0.5)	거기에 투명도까지
hsl(210, 70%, 45%)	색상·선명함·밝기로

여섯 가지가 있지만 실무에서 제일 많이 쓰는 것은 #rrggbb 입니다. 나머지는 “읽을 줄만 알아도” 충분합니다.

왜 하나로 통일하지 않았느냐면, 각 표기가 잘하는 일이 다르기 때문입니다.

이름 외우기 쉽다. 대신 종류가 적고 미세 조정이 안 된다
 16진수 짧다. 디자인 도구가 이 형태로 알려 준다
 rgb 숫자라서 계산하기 좋다. 투명도를 붙일 수 있다
 hsl "이 색을 조금만 더 어둡게" 를 숫자 하나로 할 수 있다

이 파일에서 배우는 속성은 계속 두 개뿐입니다.

color 글자색
 background-color 배경색

색을 적는 방법 여섯 가지(이름 · #rrggbb · #rgb · rgb() · rgba() · hsl())를 하나씩 실제로 칠해 보며 익힙니다. 다 외울 필요는 없습니다. #rrggbb 하나만 손에 익히고 나머지는 남의 코드에서 봤을 때 알아볼 수 있으면 충분합니다.

색 이름

섹션 1

red, gold 처럼 영어 이름을 그대로 적습니다. 미리 정해진 이름이 140개쯤 있습니다.

style 에 이렇게 적었습니다.

```
#nameRed { color: red; }
#nameTomato { color: #ffffff; background-color: tomato; }
#nameSeagreen { color: seagreen; }
#nameGold { background-color: gold; }
```

여기서 두 속성의 차이를 확실히 해 둡니다.

color 글자 자체의 색
 background-color 글자 뒤쪽 바탕의 색

#nameGold 를 보세요. 배경만 정했더니 글자는 원래 색(검은색) 그대로입니다. 지정하지 않은 것은 바뀌지 않습니다. 두 속성은 서로 따로 놓입니다.

색 이름은 쓰기 쉽지만 한계가 뚜렷합니다.

1 종류가 적습니다. "우리 회사 파란색" 같은 건 이름이 없습니다.

2 이름만 봐서는 어떤 색인지 잘 모릅니다.

tomato 가 어떤 빨강인지, seagreen 이 어떤 초록인지는 봐야 압니다.
 3. 조금만 더 어둡게 같은 조절이 안 됩니다.

그래서 연습할 때는 색 이름을 쓰고, 실제 작업에서는 #rrggbb 를 씁니다. 참고로 이름은 대소문자를 가리지 않습니다. Red 와 red 는 같습니다.

화면 첫 줄은 빨간 글자, 둘째 줄은 주황빛 붉은 배경에 흰 글자,

셋째 줄은 짙은 초록 글자, 넷째 줄은 노란 배경에 검은 글자입니다

```
color: red;
color: #ffffff; background-color: tomato;
color: seagreen;
background-color: gold; (글자색은 안 건드렸습니다)
```

많이 쓰는 이름 몇 가지입니다. red orange gold green seagreen blue navy purple white black gray lightgray tomato 전부 외울 필요는 없습니다. 검색하면 목록이 나옵니다.

▹ 직접 해보기 1

#nameGold 규칙에 color: navy; 한 줄을 더 넣어 보세요. 노란 배경 위의 글자가 짙은 남색으로 바뀝니다.

16진수

섹션 2

실무에서 가장 많이 쓰는 방법입니다. # 뒤에 여섯 글자를 적습니다.

여섯 글자는 두 글자씩 세 덩어리입니다.

```
#2d6cdf
| | |
| | └─ 파랑(Blue) 의 양
| └─ 초록(Green) 의 양
└─ 빨강(Red) 의 양
```

화면의 모든 색은 빨강·초록·파랑 세 가지 빛을 섞어서 만듭니다. 각 빛을 얼마나 곁지 0부터 255까지로 정합니다.

그런데 왜 숫자가 아니라 2d, 6c 같은 글자가 나올까요? 16진수라는 표기법으로 적었기 때문입니다. 0부터 9까지 쓰고, 그다음 10부터 15는 a b c d e f 로 씁니다.

```
0 1 2 3 4 5 6 7 8 9 a b c d e f
10 11 12 13 14 15
```

두 자리면 00 부터 ff 까지, 즉 0 부터 255 까지 적을 수 있습니다. 그래서 두 글자씩 세 덩어리로 색 하나를 표현합니다.

여기까지 이해 못 해도 괜찮습니다. 실제로 쓸 때는 이렇게만 알면 됩니다.

숫자가 클수록(f 에 가까울수록) 그 빛이 세다
#ffffff 는 전부 최대 → 흰색
#000000 은 전부 0 → 검은색
#ff0000 은 빨강만 최대 → 빨간색

세 자리 축약도 있습니다. 두 글자가 같은 글자로 반복될 때만 줄일 수 있습니다.

#ffffff → #fff ff, ff, ff 라서 한 글자씩만 남깁니다
#ff6600 → #f60
#0088aa → #08a
#2d6cdf → 줄일 수 없습니다. 2d 는 두 글자가 다르기 때문입니다

축약형은 짧아서 편하지만 표현할 수 있는 색이 적습니다. 헛갈리면 항상 여섯 자리로 쓰세요. 그게 안전합니다.

대소문자는 가리지 않습니다. #2D6CDF 와 #2d6cdf 는 같은 색입니다. 이 자료는 소문자로 통일합니다.

화면 첫 줄은 파란 배경에 흰 글자, 둘째 줄은 벽돌색 글자입니다

```
background-color: #2d6cdf; (빨강 2d, 초록 6c, 파랑 df)
color: #c0392b;
```

화면 첫 줄은 주황 배경에 흰 글자, 둘째 줄은 청록빛 파란 글자입니다

```
background-color: #f60; (#ff6600 과 같습니다)
color: #08a; (#0088aa 와 같습니다)
```

색 값을 직접 만들어 낼 필요는 없습니다. 그림판, 피그마 같은 디자인 도구나 브라우저 개발자도구에서 색을 고르면 #rrggbb 형태로 알려 줍니다. 여러분은 그걸 복사해 붙이면 됩니다.

▸ 직접 해보기 2

#hex3 의 #f60 을 여섯 자리 #ff6600 으로 바꿔 보세요. 화면이 하나도 안 바뀝니다. 같은 색이기 때문입니다.

rgb() 와 rgba()

섹션 3

16진수와 똑같은 빨강·초록·파랑을 10진수 숫자로 적는 방법입니다. 여기에 투명도를 하나 더 붙일 수 있습니다.

모양은 이렇습니다.

```
rgb(45, 108, 223)
└─┬─┬─
  │ │ └─ 파랑 0 ~ 255
  │ └─ 초록 0 ~ 255
  └─ 빨강 0 ~ 255
```

#2d6cdf 와 rgb(45, 108, 223) 은 완전히 같은 색입니다. 16진수 2d 가 10진수로 45, 6c 가 108, df 가 223 이기 때문입니다.

rgba 는 뒤에 값을 하나 더 붙입니다. 투명도(알파) 입니다.

```
rgba(45, 108, 223, 0.15)
└─ 0 = 완전히 투명, 1 = 전혀 안 비침
```

0.15 면 15%만 진하게, 즉 아주 연하게 보입니다.
뒤에 있는 것이 비쳐 보입니다.

투명도는 배경색에 쓸 때 특히 쓸모 있습니다. 연한 색을 만들려고 색을 새로 고르지 않아도, 같은 색에 투명도만 낮추면 아래 배경과 자연스럽게 어울리는 연한 색이 나옵니다.

글자색에 투명도를 쓰는 것은 조심하세요. 아래 상자의 두 번째 줄이 그 예입니다. 흐려서 읽기 힘듭니다. 글자는 진해야 합니다.

참고로 요즘 CSS 에서는 rgb 로 투명도까지 한꺼번에 쓸 수 있고 쉼표 없이 rgb(45 108 223) 이라고 써도 동작합니다. 남의 코드에서 보면 놀라지 마세요. 이 자료는 쉼표를 쓴 옛 방식으로 통일합니다.

화면 초록 배경 위에 흰 글자 한 줄입니다

```
background-color: rgb(39, 133, 63);
```

화면 첫 줄은 진한 파란 배경에 흰 글자, 둘째 줄은 아주 연한 하늘색 배경에

검은 글자, 셋째 줄은 흐릿한 분홍빛 글자라 잘 안 읽힙니다

```
background-color: rgba(45, 108, 223, 1); (전혀 안 비침)
background-color: rgba(45, 108, 223, 0.15); (많이 비침)
color: rgba(192, 57, 43, 0.35); (글자에 쓰면 읽기 힘듭니다)
```

▹ 직접 해보기 3

#rgbaBg 의 투명도 0.15 를 0.6 으로 바꿔 보세요. 배경이 훨씬 진해집니다. 그다음 0 으로 바꾸면 배경이 아예 안 보입니다.

빛을 섞는 대신 색상 · 선명함 · 밝기 세 가지로 색을 적습니다. 지금은 읽을 줄만 알면 됩니다.

모양은 이렇습니다.

```
hsl(210, 70%, 45%)
┌ ┌ ┌
│ │ ┌ 밝기(Lightness)  0% 검정 ~ 100% 흰색
│ ┌ 선명함(Saturation)  0% 회색 ~ 100% 짙은 색
└ 색상(Hue)  0 ~ 360. 색을 둥근 원판 위의 각도로 봅니다
```

색상 각도는 이렇게 기억하면 편합니다.

0 은 빨강, 120 은 초록, 240 은 파랑, 360 은 다시 빨강

hsl 이 편한 순간이 있습니다. “같은 색인데 조금만 더 어둡게” 를 숫자 하나로 할 수 있습니다. 아래 세 줄이 그 예입니다. 앞의 두 숫자는 그대로 두고 마지막 밝기만 85% → 45% → 25% 로 낮췄습니다.

```
hsl(210, 70%, 85%)  연한 하늘색
hsl(210, 70%, 45%)  진한 파랑
hsl(210, 70%, 25%)  아주 어두운 남색
```

같은 일을 16진수로 하려면 세 덩어리를 다 다시 계산해야 합니다. 그래서 색 계열을 만들 때 hsl 을 씁니다.

% 기호를 빠뜨리면 안 됩니다. 두 번째와 세 번째 값에는 반드시 % 를 붙입니다. 섹션 7 에서 빠뜨렸을 때 어떻게 되는지 봅니다.

화면 위에서 아래로 갈수록 같은 계열의 파란색이 점점 어두워집니다.

첫 줄은 연한 하늘색 배경에 검은 글자, 아래 두 줄은 흰 글자입니다

```
hsl(210, 70%, 85%)
hsl(210, 70%, 45%)
hsl(210, 70%, 25%)
```

hsl 은 지금 외우지 않아도 됩니다. “이런 표기도 있고, 밝기만 바꾸기 편하구나” 정도로 넘어가세요. 실제로 자주 쓰게 되는 것은 #rrggbb 입니다.

▫ 직접 해보기 4

#hslMid 의 첫 번째 숫자 210 을 120 으로 바꿔 보세요. 밝기는 그대로인데 색만 초록으로 바뀝니다. 0 으로 바꾸면 빨강이 됩니다.

브라우저는 rgb 로 돌려줍니다

섹션 5

어떤 표기로 적었든 브라우저 안에서는 전부 같은 형태로 저장됩니다. 개발자도구를 열면 rgb(...) 로 나옵니다.

확인하는 방법입니다. 지금 바로 해 보세요.

- 1 F12 를 누릅니다 (또는 화면에서 마우스 오른쪽 버튼 → 검사)
- 2 왼쪽 위의 화살표 아이콘을 누르고 아래 상자의 첫 줄을 클릭합니다
- 3 오른쪽 패널에서 Computed 탭을 누릅니다
- 4 color 항목을 찾습니다

style 에는 이렇게 적었습니다.

```
#sameHex { color: #2d6cdf; }
#sameRgb { color: rgb(45, 108, 223); }
```

그런데 Computed 탭에는 둘 다 이렇게 나옵니다.

```
rgb(45, 108, 223)
```

#2d6cdf 라고 쓴 쪽도 rgb 로 바뀌어 나옵니다. 브라우저는 색을 읽자마자 자기 방식으로 바뀌서 기억하기 때문입니다. 빨강 45, 초록 108, 파랑 223 이라는 정보만 남기고 표기법은 버립니다.

이걸 알아 두면 좋은 점이 있습니다.

- 1 "#2d6cdf 라고 썼는데 왜 rgb 로 나오지?" 하고 당황하지 않습니다
- 2 두 요소의 색이 정말 같은지 비교할 때 편합니다.

표기가 달라도 Computed 값이 같으면 같은 색입니다
 3. 색이 안 먹었는지 확인할 수 있습니다.
 내가 지정한 색이 아니라 rgb(34, 34, 34) 같은 엉뚱한 값이 나오면
 그 규칙이 적용되지 않은 것입니다

참고로 이 자료의 각 파일 맨 아래에는 자료가 제대로 만들어졌는지 자동으로 확인하는 목록이 주석으로 들어 있습니다. 거기에 적힌 색도 전부 rgb(...) 형태입니다. 이유가 바로 이것입니다. 브라우저에게 물으면 그 형태로 대답하기 때문입니다.

화면 두 줄이 눈으로 구분할 수 없을 만큼 똑같은 파란 글자입니다

```
color: #2d6cdf; 라고 적었습니다.
color: rgb(45, 108, 223); 라고 적었습니다.
```

투명도가 있는 색은 rgba(...) 형태로 나옵니다. 단 투명도가 1 이면 비칠 일이 없으니 브라우저가 알아서 rgb(...) 로 줄여서 보여 줍니다.

▣ 직접 해보기 5

F12 를 눌러 위 상자의 두 줄을 각각 검사하고 Computed 탭의 color 값을 확인하세요. 둘 다 rgb(45, 108, 223) 이면 맞게 본 것입니다.

읽기 쉬운 색 고르기

섹션 6

색을 고를 때 가장 중요한 것은 예쁨이 아니라 글자가 읽히는가입니다. 글자색과 배경색의 차이를 대비라고 합니다.

아래 세 줄을 눈을 조금 가늘게 뜨고 보세요. 첫 줄과 셋째 줄은 잘 읽히는데 가운데 줄은 읽기 힘듭니다.

```
#contrastGood { color: #ffffff; background-color: #2d6cdf; }
#contrastBad  { color: #f5f5f5; background-color: #fff3a0; }
#contrastFix   { color: #6b5323; background-color: #fff3a0; }
```

가운데 줄은 문법이 하나도 안 틀렸습니다. 에러도 없습니다. 그런데 못 읽습니다. 이런 실수는 검사 도구가 안 잡아 줍니다.

실용적인 요령 네 가지입니다.

1. 배경이 어두우면 글자는 밝게, 배경이 밝으면 글자는 어둡게.

둘 다 밝거나 둘 다 어두우면 안 읽힙니다

2. 흰 배경에 회색 글자는 생각보다 잘 안 읽힙니다.

멋있어 보여서 자주 쓰는데, 나이가 있는 분이나 밝은 곳에서는 힘듭니다

3. 색만으로 정보를 전달하지 마세요.

"빨간 글씨가 품질입니다" 라고 하면 색을 구분 못 하는 사람은 알 수 없습니다.
글자나 표시를 같이 넣으세요

4. 배경색을 정했으면 글자색도 같이 정하세요.

배경만 바꾸고 글자를 안 건드리면 원래 색과 부딪히는 일이 생깁니다.
개념01 의 직접 해보기 3 에서 이미 한 번 겪어 봤습니다

얼마나 차이 나야 충분한지는 계산하는 기준이 따로 있습니다. 지금은 "눈을 가늘게 떼서 안 읽히면 다시 고른다" 로 충분합니다.

화면 진한 파란 배경 위에 흰 글자. 또렷하게 읽힙니다

흰 글자 + 진한 파란 배경 - 잘 읽힙니다.

화면 연한 노란 배경 위에 거의 흰 글자. 눈을 가까이 대야 겨우 읽힙니다

거의 흰 글자 + 연한 노란 배경 - 읽기 힘듭니다.

화면 위와 같은 노란 배경인데 짙은 갈색 글자라 또렷하게 읽힙니다

짙은 갈색 글자 + 같은 노란 배경 - 잘 읽힙니다.

⇒ 직접 해보기

6

#contrastBad 의 color: #f5f5f5; 를 color: #222222; 로 바꿔 보세요. 배경은 그대로인데 글자가 확 읽히게 됩니다.

자주 하는 실수

섹션 7

색 값이 조금이라도 이상하면 그 선언은 통째로 버려집니다. 아래 상자들은 전부 배경색만 살아남았습니다.

이런 실수를 합니다

실수 1 — # 을 빠뜨림

```
color: 2d6cdf;
```

글자색만 안 먹습니다. # 이 없으면 2d6cdf 라는 이름의 색을 찾습니다. 그런 이름은 없습니다. 디자인 도구에서 색 값을 복사할 때 # 이 빠져 오는 일이 잦습니다. 배경만 칠해졌습니다.

이런 실수를 합니다

실수 2 — 16진수 자리 수를 어중간하게 씀

```
color: #2d6cd;
```

글자색만 안 먹습니다. 위 값은 다섯 자리입니다. 한 글자가 빠졌거나 하나를 더 친 것입니다. 여섯 자리 아니면 세 자리로 쓰세요. 복사해 붙일 때 맨 끝 한 글자를 놓치는 일이 자주 있습니다. 여기도 배경만 칠해졌습니다.

이런 실수를 합니다

실수 3 — hsl 에서 % 를 빠뜨림

```
color: hsl(210, 70, 45);
```

글자색만 안 먹습니다. 두 번째와 세 번째 값은 퍼센트라서 % 를 반드시 붙여야 합니다. 첫 번째 색상 값에는 붙이지 않습니다. 셋 다 숫자로 보아서 잘 헷갈립니다. 여기도 배경만 칠해졌습니다.

이런 실수를 합니다

실수 4 — 배경을 바꾸려고 color 를 씀

```
#colorForBg { color: #2d6cdf; }
```

글자만 파래지고 배경은 그대로입니다. 에러는 없습니다. “색을 바꾸려했더니 왜 글자가 바뀌지?” 하게 됩니다. color 는 글자색, background-color 는 배경색입니다. color 라는 이름이 짧아서 “색 전체” 로 오해하기 쉽습니다. 배경을 바꾸려고 했습니다.

이런 실수를 합니다

실수 5 — 없는 색 이름을 씀

```
color: 하늘색;
```

글자색만 안 먹습니다. 색 이름은 정해진 영어 이름만 됩니다. 한글은 안 됩니다. skyblue 처럼 영어로 쓰세요. 영어라도 lightblue2 같이 없는 이름이면 마찬가지로입니다. 한글 색 이름을 써 봤습니다.

정리

▸ 직접 해보기

7

정답

1) style 안의 #nameGold 를 이렇게 고칩니다.

```
#nameGold {  
  color: navy;  
  background-color: gold;  
}
```

→ 노란 배경 위의 글자가 검은색에서 짙은 남색으로 바뀝니다.

색 이름 두 개를 각각 다른 속성에 준 것입니다.

2) style 안의 #hex3 을 이렇게 고칩니다.

```
#hex3 {  
  color: #ffffff;  
  background-color: #ff6600;  
}
```

→ 화면이 하나도 안 바뀝니다.

#f60 은 #ff6600 을 줄여 쓴 것이라 완전히 같은 색이기 때문입니다.
F12 로 Computed 값을 보면 둘 다 rgb(255, 102, 0) 으로 나옵니다.

3) style 안의 #rgbaBg 를 이렇게 고칩니다.

```
#rgbaBg {
  background-color: rgba(45, 108, 223, 0.6);
}
```

→ 배경이 훨씬 진한 파란색이 됩니다. 글자가 조금 읽기 힘들어집니다.

0 으로 바꾸면 배경이 완전히 투명해져서 상자의 연한 회색만 보입니다.
확인했으면 0.15 로 되돌려 놓으세요.

4) style 안의 #hslMid 를 이렇게 고칩니다.

```
#hslMid {
  color: #ffffff;
  background-color: hsl(120, 70%, 45%);
}
```

→ 배경이 파란색에서 초록색으로 바뀝니다. 밝기는 그대로입니다.

0 으로 바꾸면 빨간색이 됩니다.
숫자 하나로 색 계열만 알아 끼울 수 있다는 것이 hsl 의 장점입니다.

5) F12 로 확인한 결과는 이렇습니다.

```
#sameHex 의 Computed color → rgb(45, 108, 223)
#sameRgb 의 Computed color → rgb(45, 108, 223)
```

→ 소스에는 #2d6cdf 와 rgb(45, 108, 223) 으로 다르게 적혀 있는데

브라우저 안에서는 완전히 같은 값이 됩니다.
F12 를 처음 열어 보는 것 자체가 이 문제의 목적입니다.

6) style 안의 #contrastBad 를 이렇게 고칩니다.

```
#contrastBad {
  color: #222222;
  background-color: #fff3a0;
}
```

→ 배경은 같은 연한 노란색인데 글자가 아주 잘 읽히게 됩니다.

문제는 배경이 아니라 글자색이었다는 뜻입니다.

```
#nameRed {
  color: red;
}
```

화면 빨간 글자 한 줄

```
#nameTomato {
  color: #ffffff;
  background-color: tomato;
}
```

화면 주황빛 붉은 배경 위에 흰 글자

```
#nameSeagreen {
  color: seagreen;
}
```

화면 짙은 초록 글자 한 줄

```
#nameGold {
  background-color: gold;
}
```

화면 노란 배경 위에 원래 색인 검은 글자

#으로 시작하는 16진수

```
#hex6 {
  color: #ffffff;
  background-color: #2d6cdf;
}
```

화면 파란 배경 위에 흰 글자

```
#hex6b {
  color: #c0392b;
}
```

화면 벽돌색 글자 한 줄

```
#hex3 {
  color: #ffffff;
  background-color: #f60;
}
```

화면 주황 배경 위에 흰 글자

```
#hex3b {
  color: #08a;
}
```

화면 청록빛 파란 글자 한 줄

rgb() 와 rgba()

섹션 3

```
#rgbDemo {
  color: #ffffff;
  background-color: rgb(39, 133, 63);
}
```

화면 초록 배경 위에 흰 글자

```
#rgbaText {
  color: rgba(192, 57, 43, 0.35);
}
```

화면 아주 연한 분홍빛 글자. 흐려서 읽기 힘듭니다

```
#rgbaBg {
  background-color: rgba(45, 108, 223, 0.15);
}
```

화면 아주 연한 하늘색 배경 위에 검은 글자. 글자가 잘 읽힙니다

```
#rgbaFull {
  color: #ffffff;
  background-color: rgba(45, 108, 223, 1);
}
```

화면 진한 파란 배경 위에 흰 글자. 투명도 1 은 안 비치는 것과 같습니다

```
#hslLight {
  background-color: hsl(210, 70%, 85%);
}
```

화면 연한 하늘색 배경 위에 검은 글자

```
#hslMid {
  color: #ffffff;
  background-color: hsl(210, 70%, 45%);
}
```

화면 진한 파란 배경 위에 흰 글자

```
#hslDark {
  color: #ffffff;
  background-color: hsl(210, 70%, 25%);
}
```

화면 아주 어두운 남색 배경 위에 흰 글자

표기는 달라도 같은 색

```
#sameHex {
  color: #2d6cdf;
}
```

화면 파란 글자 한 줄

```
#sameRgb {
  color: rgb(45, 108, 223);
}
```

화면 위와 완전히 같은 파란 글자 한 줄

대비

```
#contrastGood {
  color: #ffffff;
  background-color: #2d6cdf;
}
```


화면 진한 파란 배경 위에 흰 글자. 잘 읽힙니다

```
#contrastBad {
  color: #f5f5f5;
  background-color: #fff3a0;
}
```

화면 연한 노란 배경 위에 거의 흰 글자. 눈을 가까이 대야 겨우 읽힙니다

```
#contrastFix {
  color: #6b5323;
  background-color: #fff3a0;
}
```

화면 같은 노란 배경인데 짙은 갈색 글자라 잘 읽힙니다

자주 하는 실수

섹션 7

```
#missHash {
  color: 2d6cdf;
  background-color: #fff3a0;
}
```

화면 노란 배경 위에 검은 글자. 글자색 줄만 버려졌습니다

```
#hex5 {
  color: #2d6cd;
  background-color: #fff3a0;
}
```

화면 위와 같습니다. 노란 배경에 검은 글자

```
#hslNoPercent {
  color: hsl(210, 70, 45);
  background-color: #fff3a0;
}
```

화면 위와 같습니다. 노란 배경에 검은 글자

```
#colorForBg {
  color: #2d6cdf;
}
```

화면 파란 글자 한 줄. 배경은 하나도 안 바뀝니다

```
#badName {  
  color: 하늘색;  
  background-color: #fff3a0;  
}
```

화면 노란 배경 위에 검은 글자

상속과 기본값

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 파일 맨 위 <style> 을 함께 보세요.



실행하면 나오는 화면 — 상속과 기본값

지금까지는 꾸미고 싶은 요소를 하나하나 골라서 규칙을 줬습니다. 그런데 실제로 CSS 를 써 보면 이상한 일이 두 가지 생깁니다.

- 1 규칙을 안 준 요소인데 색이 바뀌어 있다
- 2 규칙을 하나도 안 썼는데 제목은 크고 굵게 나온다

1번의 원인이 상속이고, 2번의 원인이 브라우저 기본 스타일입니다. 이 파일에서 그 둘을 다룹니다.

상속(inheritance) 이란 이런 뜻입니다.

부모 요소에 준 스타일 중 일부가 자식 요소에게 저절로 내려가는 것

01단원 개념02 에서 부모·자식이라는 말을 배웠습니다. 감싼 쪽이 부모, 감싸인 쪽이 자식입니다.

<code><div class="family-box"></code>	← 부모. 여기에만 color 를 줍니다
<code><p>안녕하세요</p></code>	← 자식. 아무것도 안 줬는데 색이 바뀝니다
<code></div></code>	

모든 속성이 내려가는 것은 아닙니다. 글자와 관련된 것만 내려갑니다. 왜 그렇게 나뉘을까요? 그게 사람이 기대하는 방식이기 때문입니다.

글꼴을 정하면 안쪽 글자에도 다 적용되는 게 자연스럽습니다
테두리를 그렸는데 안쪽 요소마다 테두리가 또 생기면 이상합니다

이 성질을 알고 나면 CSS 를 훨씬 적게 쓰게 됩니다. 자식마다 규칙을 쓰지 않고 부모 한 곳에만 쓰면 되기 때문입니다.

이 파일에서는 내가 안 썼는데 이미 적용되어 있는 것들을 다룹니다. 부모에게서 내려온 것(상속) 과 브라우저가 원래 갖고 있던 것(기본 스타일) 두 가지입니다. 둘을 구분할 줄 알아야 “왜 이 색이 안 바뀌지?” 를 스스로 풀 수 있습니다.

부모에 준 것이 내려옵니다

섹션 1

color 는 상속되는 속성입니다. 부모 한 곳에만 주면 안쪽 글자가 전부 그 색이 됩니다.

style 에 적은 규칙은 이것 하나뿐입니다.

```
.family-box { color: #2d6cdf; }
```

그런데 아래 상자 안에는 문단도 있고 스패도 있고 굵은 글자도 있습니다. 그 요소들에는 규칙을 하나도 안 줬습니다. 그래도 전부 파랑입니다.

<code><div class="family-box"></code>	← 규칙을 받은 곳은 여기뿐입니다
<code><p>...</p></code>	← 내려받음
<code><p>... ...</p></code>	← 내려받고 그 안쪽까지 또 내려감
<code><p>...</p></code>	← 여기도 내려받음
<code></div></code>	

상속은 한 단계만 내려가는 게 아닙니다. 안쪽으로 계속 타고 들어갑니다. 위의 span 은 div 의 손자인데도 색을 받았습니.

더 큰 예가 지금 여러분 눈앞에 있습니다. 이 화면의 거의 모든 글자는 검은빛이 도는 진한 회색입니다. 그 색을 정한 규칙은 공통.css 안에 딱 한 줄뿐입니다.

```
body { color: #222; }
```

body 는 문서 전체를 감싸는 요소입니다. 모든 요소의 조상입니다. 그래서 body 에 색을 주면 문서 안 거의 모든 글자가 그 색이 됩니다. 글꼴도 마찬가지입니다. 공통.css 는 body 한 곳에만 글꼴을 정했습니다.

상속이 없다면 어떻게 될까요? 문단마다, 제목마다, 목록 항목마다 글꼴과 색을 다시 써야 합니다. 수백 줄이 됩니다. 상속은 그 수고를 없애 주는 장치입니다.

화면 상자 안의 글자가 전부 같은 파란색입니다.

굵은 글자만 굵기가 다르고 색은 똑같습니다

저는 자식 문단입니다. 규칙을 받은 적이 없습니다.
저 안에 스패 이 하나 들어 있습니다.
굵은 글자도 색을 받습니다.

상속되는 대표적인 속성은 color, font-family(글꼴), font-size(글자 크기), line-height(줄 간격), text-align(정렬) 입니다. 글자에 관한 것이라고 묶어서 기억하세요. 글꼴과 크기는 09단원에서 제대로 배웁니다.

▹ 직접 해보기 1

.family-box 의 색을 #27853f 로 바꿔 보세요. 규칙 한 줄만 고쳤는데 상자 안 세 문단이 한꺼번에 초록색이 됩니다.

안 내려오는 것도 있습니다

섹션 2

background-color 는 상속되지 않습니다. 부모에 준 배경색은 자식의 배경색이 되지 않습니다.

style 에는 이렇게 적었습니다.

```
.paint-box {
  color: #ffffff;
  background-color: #c0392b;
}
```

결과를 보면 상자 전체가 빨강고 글자는 흰색입니다. 여기서 헷갈리기 쉬운 점이 있습니다.

글자색 자식에게 내려갔습니다. 자식 문단의 color 는 흰색입니다
배경색 자식에게 안 내려갔습니다. 자식 문단의 배경색은 여전히 없습니다

“그런데 자식 자리도 빨간데요?” 라고 할 수 있습니다. 자식이 빨간 게 아니라, 자식이 투명해서 부모의 빨강이 비쳐 보이는 것입니다. 유리판을 빨간 종이 위에 올려 둔 것과 같습니다. 유리판이 빨개진 게 아닙니다.

지금은 겉보기가 같아서 구분할 실익이 없어 보이지만, 10단원에서 자식에게 여백이나 다른 배경을 주기 시작하면 바로 드러납니다.

상속되는 것과 안 되는 것을 나누는 기준은 이렇습니다.

내려감	글자에 관한 것	color · font-family · line-height · text-align
안 내려감	상자에 관한 것	background-color · border · padding · margin · width

border, padding, margin, width 는 10단원에서 배웁니다. 지금은 이름만 알아 두고, “상자에 관한 것은 안 내려간다” 만 기억하세요.

왜 이렇게 나왔을까요? 안 그러면 화면이 엉망이 되기 때문입니다. 테두리가 상속된다고 생각해 보세요. 바깥 상자에 테두리를 하나 그렸는데 안쪽의 모든 문단, 모든 단어에까지 테두리가 겹겹이 생깁니다.

화면 빨간 덩어리 안에 흰 글자 두 줄이 보입니다.

줄 사이에도 빨강이 이어져 있어서 한 덩어리로 보입니다

저는 자식 문단입니다. 글자색은 받았고 배경색은 못 받았습니다.
제 뒤에 보이는 빨강은 부모의 배경이 비치는 것입니다.

▣ 직접 해보기 2

F12 를 눌러 위 상자의 첫 번째 문단을 검사하고 Computed 탭에서 background-color 를 찾아보세요. rgba(0, 0, 0, 0) 이라고 나옵니다. 완전히 투명하다는 뜻입니다. color 는 rgb(255, 255, 255) 입니다.

inherit

섹션 3

안 내려오는 속성도 inherit 이라고 값을 적으면 부모의 값을 그대로 받아 옵니다.

inherit 는 색 이름이 아닙니다. “부모 것을 그대로 쓰겠다” 는 뜻의 특별한 값입니다. 거의 모든 속성에 쓸 수 있습니다.

```
.paint-box2 {
  color: #ffffff;
  background-color: #27853f;
}
.take-bg {
  background-color: inherit;  ← 부모의 초록을 그대로 받아 옵니다
}
```

아래 상자에서 두 번째 문단에만 take-bg 를 붙였습니다. 그런데 화면으로는 첫 번째 문단과 구별이 안 됩니다. 둘 다 초록이니까요.

그러면 왜 쓸까요? 겉보기가 같아도 속이 다르기 때문입니다.

첫 번째 문단 자기 배경은 투명. 부모 배경이 비쳐 보이는 것
두 번째 문단 자기 배경이 진짜로 초록. 부모와 상관없이 자기 것

F12 로 Computed 값을 보면 둘이 다릅니다.

```
첫 번째 → rgba(0, 0, 0, 0)
두 번째 → rgb(39, 133, 63)
```

실제로 쓰는 경우는 이렇습니다.

1. 부모 색이 나중에 바뀌어도 자식이 자동으로 따라가게 하고 싶을 때

- 색 값을 두 곳에 적어 두면 한쪽만 고치는 실수가 납니다
- 2. 브라우저가 자기 색을 정해 둔 요소를 부모 색에 맞추고 싶을 때
링크나 버튼이 그런 요소입니다. 섹션 6 에서 봅니다

주의할 점이 하나 있습니다. 부모가 그 속성을 정한 적이 없으면 inherit 로도 받을 게 없습니다. 부모의 값 (예를 들면 투명) 을 그대로 받아 오니 아무 일도 안 일어납니다. 섹션 7 의 실수 4 에서 그 경우를 봅니다.

화면 초록 덩어리 안에 흰 글자 두 줄. 눈으로는 두 줄을 구분할 수 없습니다

저는 배경이 투명합니다. 부모 초록이 비칩니다.
저는 제 배경이 진짜 초록입니다.

inherit 는 색이 아니라 값의 종류입니다. color: inherit; 처럼 다른 속성에도 똑같이 쓸 수 있습니다.

⇒ 직접 해보기 3

.paint-box2 의 배경색을 #8e44ad 로 바꿔 보세요. 두 문단 모두 보라색이 됩니다. .take-bg 는 한 글자도 안 고쳤는데 따라옵니다.

브라우저 기본 스타일

섹션 4

CSS 를 하나도 안 써도 제목은 크고 굵게 나옵니다. 브라우저가 원래 갖고 있는 CSS 파일 때문입니다.

01단원에서 “태그에는 뜻이 들어 있어서 브라우저가 알아서 그려 준다” 고 했습니다. 그 “알아서” 의 정체가 이것입니다.

모든 브라우저는 자기만의 CSS 파일을 하나 갖고 있습니다. 여러분이 만든 CSS 를 읽기 전에 그 파일을 먼저 읽습니다. 대략 이런 내용입니다.

```
h1      { font-size: 2em; font-weight: bold; margin: 0.67em 0; }
p       { margin: 1em 0; }
strong { font-weight: bold; }
a       { color: blue; text-decoration: underline; }
ul      { padding-left: 40px; list-style-type: disc; }
```

그래서 아무 CSS 없이도 이렇게 됩니다.

h1 이 크고 굵은 이유 → font-size 와 font-weight 가 이미 정해져 있어서
문단 사이가 벌어지는 이유 → p 에 위아래 margin 이 이미 있어서
링크가 파랗고 밑줄 있는 이유 → a 에 color 와 text-decoration 이 이미 있어서

아래 상자에는 CSS 를 하나도 안 준 요소들을 넣어 두었습니다. 그런데도 다 다르게 생겼습니다. 전부 브라우저 기본 스타일입니다.

브라우저마다 기본값이 조금씩 다릅니다. 그래서 실무에서는 기본값을 초기화하는 CSS 를 맨 앞에 깔기도 합니다. 그건 10단원 이후의 이야기입니다.

여기서 알아 둘 것은 하나입니다. 화면에 이상한 여백이나 크기가 보이면 세 가지 중 하나입니다.

1 내가 쓴 규칙

2 부모에게서 내려온 상속

3 브라우저 기본 스타일

F12 의 Styles 탭을 보면 어느 것 때문인지 알 수 있습니다. 브라우저 기본 스타일은 user agent stylesheet 라고 표시됩니다.

화면 맨 위 "저는 h1 입니다" 가 아주 크고 굵게 나옵니다.

문단 세 줄은 사이가 벌어져 있고, 굵은 글자와 보통 글자가 한 줄에 섞여 있고, 맨 아래 링크는 파란 글자에 밑줄이 그어져 있습니다

저는 h1 입니다
저는 문단입니다. 위아래에 여백이 이미 있습니다.
저는 그다음 문단입니다. 위 문단과 붙어 있지 않습니다.
저는 굵습니다 / 저는 보통입니다
저는 링크입니다

위 상자의 h1 이 이 페이지 맨 위의 제목보다 더 커 보이지요? 맨 위 제목도 h1 인데, 공통.css 가 거기에 다른 크기를 정해 뒀기 때문입니다. 기본값은 덮어쓸 수 있습니다.

▹ 직접 해보기

4

위 상자의 <p> 두 줄 사이에 문단을 하나 더 넣어 보세요. 여백을 지정한 적이 없는데도 새 문단 위아래가 똑같이 벌어집니다.

기본값 덮어쓰기

섹션 5

내가 쓴 규칙이 브라우저 기본값보다 셉니다. 같은 속성을 지정하면 내 값으로 바뀝니다.

방법은 특별할 게 없습니다. 그냥 그 속성을 다시 쓰면 됩니다.


```
#overrideTitle { color: #c0392b; }
```

아래 상자의 제목이 빨강게 나옵니다. 그런데 크기와 굵기는 그대로 큼니다. 왜 그럴까요?

내가 지정한 것 color → 내 값으로 덮어씀
내가 지정 안 한 것 font-size 등 → 브라우저 기본값이 그대로 남음

덮어쓰기는 속성 단위로 일어납니다. “내 규칙을 하나 썼으니 브라우저 기본값이 다 사라진다” 가 아닙니다. 내가 건드린 속성만 바뀌고 나머지는 그대로입니다. 이게 CSS 를 조금씩 고쳐 나갈 수 있는 이유입니다.

크기와 굵기를 바꾸는 방법은 09단원에서 배웁니다. 이 단원에서 덮어쓸 수 있는 것은 색 두 가지뿐입니다.

화면 두 제목이 똑같이 크고 굵은데, 위쪽만 빨간색이고 아래쪽은 검은색입니다

저는 색만 바뀐 제목입니다
저는 아무것도 안 준 제목입니다

▹ 직접 해보기 5

#overrideTitle 규칙에 background-color: #fff3a0; 을 한 줄 더 넣어 보세요. 제목 뒤가 노랑게 칠해집니다. 크기와 굵기는 여전히 그대로입니다.

상속을 이기는 것

섹션 6

부모에 색을 줬는데 링크만 파랗게 남는 일이 있습니다. 고장이 아니라 규칙대로 된 것입니다.

아래 상자에는 이렇게 줬습니다.

```
.link-box { color: #c0392b; }
```

상자 안 보통 글자는 빨개졌는데 링크는 여전히 파랑고, 버튼 글자는 검습니다. 이유는 브라우저 기본 스타일에 이런 줄이 있기 때문입니다.

```
a      { color: blue; }  
button { color: buttontext; }
```

여기서 중요한 원칙이 나옵니다.

상속으로 내려온 값은 가장 약합니다.
그 요소를 '직접 고른' 규칙이 있으면 무조건 그쪽이 이깁니다.
브라우저 기본 스타일도 a 를 직접 고른 규칙입니다.

정리하면 이렇습니다.

부모의 color → 자식을 직접 고른 게 아님 → 약함
브라우저의 a { ... } → a 를 직접 고름 → 이김

그래서 링크 색을 바꾸려면 a 를 직접 골라야 합니다.

```
#linkFixed { color: #c0392b; }
```

아래 상자의 두 번째 링크가 그렇게 고친 것입니다. 빨강게 바뀌었습니다. 밑줄은 그대로 남아 있는데, 그건 text-decoration 이라는 다른 속성이라 내가 건드리지 않았기 때문입니다. 09단원에서 지우는 법을 배웁니다.

규칙이 여러 개 겹쳤을 때 누가 이기는지 계산하는 방법을 명시도라고 합니다. 08단원에서 제대로 배웁니다. 지금은 이 한 줄만 기억하세요.

직접 고른 것이 상속보다 세다

화면 첫 줄은 빨간 글자, 둘째 줄 링크는 파란 글자에 밑줄,

셋째 줄 링크는 빨간 글자에 밑줄, 맨 아래 회색 버튼의 글자는 검은색입니다

저는 보통 글자입니다. 부모 색을 받았습니다.
저는 손대지 않은 링크입니다
저는 직접 색을 받은 링크입니다
저는 버튼입니다

버튼은 색뿐 아니라 글꼴까지 부모를 안 따릅니다. 위 버튼의 글자 모양이 주변과 다른 것이 그 때문입니다. 브라우저가 버튼에 자기 글꼴을 직접 지정해 두었기 때문입니다.

◁ 직접 해보기

6

#linkFixed 의 값을 inherit 로 바꿔 보세요(color: inherit;). 부모의 빨간색을 그대로 받아서 결과가 같아집니다. 색 값을 두 곳에 적지 않아도 되니 이쪽이 더 안전합니다.

자주 하는 실수

섹션 7

상속과 기본값은 보이지 않는 곳에서 움직입니다. 그래서 “분명히 규칙을 썼는데 안 먹는다” 의 원인이 되는 일이 많습니다.

이런 실수를 합니다

실수 1 — 부모 배경색이 자식에게 내려간 줄 앎

겉보기는 같지만 자식의 배경색은 여전히 없습니다. 부모 배경이 비쳐 보이는 것뿐입니다. 지금은 차이가 안 보이지만, 10단원에서 자식에게 여백을 주기 시작하면 “왜 여기만 색이 끓기지?” 하는 문제로 나타납니다. 섹션 2 와 섹션 3 의 차이를 F12 로 꼭 한 번 확인해 두세요.

이런 실수를 합니다

실수 2 — 자식이 자기 색을 갖고 있는데 부모만 계속 고침

```
.parent-blue { color: #2d6cdf; }
.kid-own     { color: #c0392b; }
```

그 자식만 안 바꿉니다. 부모 색을 아무리 고쳐도 소용없습니다. 자식을 직접 고른 규칙이 있기 때문입니다. 아래 상자에서 가운데 줄이 그 경우입니다. “한 줄만 색이 안 따라온다” 면 그 요소에 직접 붙은 규칙을 찾으세요. 저는 부모 색을 받았습시다. 저는 제 색을 갖고 있습니다. 저도 부모 색을 받았습시다.

이런 실수를 합니다

실수 3 — 링크와 버튼도 부모 색을 따를 거라고 생각

안 따릅니다. 섹션 6 에서 본 그대로입니다. 브라우저가 a 와 button 의 색을 직접 정해 뒀기 때문입니다. a 를 직접 골라서 색을 주거나 color: inherit; 를 쓰세요. “다 바꿨는데 링크만 파랗다” 는 초보자 단골 질문입니다.

이런 실수를 합니다

실수 4 — 부모가 정한 적 없는 값을 inherit 로 받으려 함

```
.take-nothing { background-color: inherit; }
```

아무 일도 안 일어납니다. 에러도 없습니다. inherit 는 “부모 것을 그대로 쓰겠다” 는 뜻인데, 부모가 배경색을 정한 적이 없으면 부모의 값인 투명을 받아 옵니다. 받아 오긴 했는데 받아 온 것이 투명이라 보이는 변화가 없습니다. 배경을 물려받으려 했습니다.

이런 실수를 합니다

실수 5 — 브라우저 기본 여백을 자기가 만든 줄 앎

문단 사이의 여백은 내가 만든 것이 아닙니다. p 에 원래 위아래 여백이 들어 있습니다. “여백을 준 적이 없는데 왜 벌어지지?” 하고 HTML 을 계속 뒤지게 됩니다. 여백을 없애는 방법은 10단원에서 배웁니다. 그리고 브라우저마다 기본값이 조금씩 달라서, 같은 파일이 크롬과 다른 브라우저에서 살짝 다르게 보이는 일도 있습니다.

정리

▹ 직접 해보기

7

정답

1) style 안의 .family-box 를 이렇게 고칩니다.

```
.family-box {
  color: #27853f;
}
```

→ 섹션 1 상자 안의 문단 세 줄과 스패, 굵은 글자가 전부 초록색이 됩니다.

규칙은 한 줄인데 다섯 곳이 바뀌었습니다. 이것이 상속의 힘입니다.
확인했으면 #2d6cdf 로 되돌려 놓으세요.

2) F12 로 확인한 결과는 이렇습니다.

```
background-color → rgba(0, 0, 0, 0)
color             → rgb(255, 255, 255)
```

→ 눈에는 빨강게 보이는데 자기 배경색은 투명입니다.

보이는 빨강은 부모의 배경이 비친 것입니다.
글자색은 부모에게서 제대로 내려받았습니다.

3) style 안의 .paint-box2 를 이렇게 고칩니다.

```
.paint-box2 {
  color: #ffffff;
  background-color: #8e44ad;
}
```

→ 섹션 3 상자가 통째로 보라색이 됩니다.

두 번째 문단은 .take-bg 로 inherit 를 받고 있어서 자동으로 따라옵니다.
색 값을 한 곳에만 적어 두면 이렇게 편합니다.
확인했으면 #27853f 로 되돌려 놓으세요.

4) 섹션 4 상자의 문단 두 줄 사이에 이렇게 넣습니다.

```
<p>저는 나중에 끼워 넣은 문단입니다.</p>
```

→ 세 문단이 똑같은 간격으로 벌어집니다.

여백을 준 적이 없는데 벌어지는 것은 브라우저 기본 스타일 때문입니다.

5) style 안의 #overrideTitle 을 이렇게 고칩니다.

```
#overrideTitle {
  color: #c0392b;
  background-color: #fff3a0;
}
```

→ 첫 번째 제목 뒤가 노랗게 칠해집니다.

글자 크기와 굵기는 그대로입니다. 내가 건드린 속성만 바뀌기 때문입니다.

6) style 안의 #linkFixed 를 이렇게 고칩니다.

```
#linkFixed {
  color: inherit;
}
```

→ 화면은 그대로 빨간 링크입니다. 부모인 .link-box 의 색을 받았기 때문입니다.

차이는 나중에 나타납니다. .link-box 의 색을 바꾸면 #c0392b 라고 적어 뒀을 때는 링크만 옛 색으로 남지만, inherit 로 적어 두면 링크도 같이 따라옵니다.

부모에만 색을 줬습니다

섹션 1

```
.family-box {
  color: #2d6cdf;
}
```

화면 이 상자 안의 문단·스팬·굵은 글자가 전부 파랗게 보입니다.

자식에게는 규칙을 하나도 안 줬는데 그렇습니다

배경색은 안 내려옵니다

섹션 2

```
.paint-box {
  color: #ffffff;
  background-color: #c0392b;
}
```

화면 빨간 상자 안의 글자가 전부 흰색입니다.

자식 문단 자체의 배경색은 여전히 없습니다

inherit 로 역지로 받아 오기

섹션 3

```
.paint-box2 {  
  color: #ffffff;  
  background-color: #27853f;  
}
```

화면 초록 상자 안의 글자가 전부 흰색입니다

```
.take-bg {  
  background-color: inherit;  
}
```

화면 이 클래스가 붙은 문단은 부모와 똑같은 초록 배경을 갖게 됩니다

브라우저 기본값 덮어쓰기

섹션 5

```
#overrideTitle {  
  color: #c0392b;  
}
```

화면 제목이 빨간색이 됩니다. 크고 굵은 것은 그대로 남습니다

상속을 이기는 것

섹션 6

```
.link-box {  
  color: #c0392b;  
}
```

화면 상자 안 보통 글자는 빨개지지만 링크와 버튼은 안 바뀝니다

```
#linkFixed {  
  color: #c0392b;  
}
```

화면 이 링크만 빨간 글자가 됩니다. 밑줄은 그대로 남습니다

자주 하는 실수

섹션 7

```
.parent-blue {  
  color: #2d6cdf;  
}
```

화면 상자 안 글자가 파래집니다. 단 자기 색을 직접 받은 자식은 예외입니다

```
.kid-own {  
  color: #c0392b;  
}
```

화면 부모가 파란색을 내려보내도 이 문단만 빨간 글자로 남습니다

```
.take-nothing {  
  background-color: inherit;  
}
```

화면 아무 일도 안 일어납니다. 부모가 배경색을 정한 적이 없기 때문입니다

CSS 시작하기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

문제는 이 파일 맨 위 <style> 안에 들어 있습니다.

VS Code 로도 같이 열어 두고 위에서부터 내려가며 푸세요.



실행하면 나오는 화면 — CSS 시작하기

문제는 이 파일 맨 위 <style> 안에 CSS 주석으로 들어 있습니다. VS Code 로 이 파일을 열고 위에서부터 내려가며 TODO 라고 적힌 자리에 규칙을 쓰세요. 저장(Ctrl+S) 한 뒤 브라우저에서 새로고침(F5) 하면 아래 상자들이 바뀝니다. 1번부터 11번까지는 기본, 12번과 13번은 응용, 14번은 도전, 15번은 확인 문제입니다.

문제 1

우리 동네 빵집

문제 2

오늘 구운 빵

문제 3

단팥빵
소금빵
크림빵

문제 4

오늘의 추천

문제 5

할인 중

문제 6

rgb 로 칠했습니다

문제 7 (개념02 섹션 2)

아래 문단에 인라인 style 속성을 붙여 글자색을 #8e44ad 로 만드세요. <p 와 id 사이에 한 칸 띄고 style="..." 을 씁니다. 중괄호도 선택자도 쓰지 않습니다. 속성과 값만 적습니다.

기대 출력 "인라인으로 칠해 보세요" 가 보라색 글자로 보입니다.
이렇게 되면 틀린 것: 아무 변화가 없다면 중괄호나 선택자를 같이 쓴 것입니다.

아래 p 태그에 style 속성을 붙이세요

인라인으로 칠해 보세요

문제 8

주석으로 꺼 봅니다

문제 9

첫 번째 줄입니다.
두 번째 줄입니다.
세 번째 줄입니다.

문제 10

저는 부모에게서 글자색만 받았습니다.
저는 안쪽 문단입니다.

문제 11

저는 부모 색을 받았습니다.
가게 위치 보러 가기

문제 12

연한 배경 위의 진한 글자

문제 13

공지 사항이 하나 있습니다.
공지 보러 가기

문제 14

새 글이 등록되었습니다.
저장하지 않은 내용이 있습니다.
처리가 모두 끝났습니다.

문제 15

파란 배경에 흰 글자가 되어야 합니다.
노란 배경이 되어야 합니다.
벽돌색 글자가 되어야 합니다.
초록 배경에 흰 글자가 되어야 합니다.

정리

07단원 연습문제 - CSS 시작하기

여기가 여러분의 작업 자리입니다.
각 문제의 설명을 읽고 바로 아래 TODO 자리에 규칙을 쓰세요.
저장(Ctrl+S) 한 뒤 브라우저에서 새로고침(F5) 하면 결과가 보입니다.

1~11은 기본, 12~13은 응용, 14는 도전, 15는 확인 문제입니다.

문제 1 (개념03 섹션 1)

id 가 q1Title 인 요소의 글자색을 #2d6cdf 로 만드세요.

기대 화면: "우리 동네 빵집" 이 파란 글자로 보입니다.
이렇게 되면 틀린 것: 검은 글자 그대로면 규칙이 적용되지 않은 것입니다.
id 선택자 앞에는 # 을 붙입니다.

여기에 코드를 쓰세요

문제 2 (개념03 섹션 2, 개념04 섹션 1)

class 가 q2-box 인 요소의 배경색을 색 이름 gold 로 만드세요.

기대 화면: "오늘 구운 빵" 줄의 뒷배경이 노랗게 칠해집니다. 글자는 검은색 그대로입니다.

이렇게 되면 틀린 것: 글자가 노래졌다면 background-color 가 아니라 color 를 쓴 것입니다.

여기에 코드를 쓰세요

문제 3 (개념03 섹션 2)

태그 이름 선택자를 써서 이 파일의 모든 li 의 글자색을 #27853f 로 만드세요. 선택자 앞에 점이나 # 을 붙이지 않습니다.

기대 화면: 문제 3 상자의 목록 세 줄이 전부 초록 글자가 됩니다.

이렇게 되면 틀린 것: 한 줄만 바뀌었다면 li 가 아니라 다른 것을 고른 것입니다.

여기에 코드를 쓰세요

문제 4 (개념04 섹션 2)

id 가 q4Line 인 요소를 글자색 #c0392b, 배경색 #fff3a0 으로 만드세요.
규칙 하나 안에 선언 두 개를 쓰면 됩니다. 선언마다 끝에 세미콜론을 찍으세요.

기대 화면: "오늘의 추천" 이 연한 노란 배경 위에 벽돌색 글자로 보입니다.
이렇게 되면 틀린 것: 둘 다 안 먹었다면 앞 선언의 세미콜론이 빠진 것입니다.

여기에 코드를 쓰세요

문제 5 (개념04 섹션 2)

class 가 q5-box 인 요소를 글자색 흰색, 배경색 주황색으로 만드세요.
두 색 모두 세 자리 축약형으로 쓰세요. 흰색은 #ffffff, 주황색은 #ff6600
입니다.

기대 화면: "할인 중" 이 주황 배경 위에 흰 글자로 보입니다.
이렇게 되면 틀린 것: 여섯 자리로 써도 화면은 같지만, 이 문제는 축약
연습입니다.

여기에 코드를 쓰세요

문제 6 (개념04 섹션 3)

id 가 q6Line 인 요소를 글자색 흰색, 배경색 파란색으로 만드세요.

이번에는 두 색 모두 rgb() 표기로 쓰세요.

흰색은 rgb(255, 255, 255), 파란색은 rgb(45, 108, 223) 입니다.

기대 화면: "rgb 로 칠했습니다" 가 파란 배경 위에 흰 글자로 보입니다.

이렇게 되면 틀린 것: 아무 변화가 없다면 rgb 라는 글자나 괄호를 빠뜨린 것입니다.

여기에 코드를 쓰세요

문제 7 (개념02 섹션 2)

이 문제만 style 이 아니라 body 안에서 풁니다. 문제 7 상자를 찾아가세요.

문제 8 (개념03 섹션 4)

아래 #q8Line 규칙에서 background-color 줄만 CSS 주석으로 감싸 꺾 두세요.

줄을 지우면 안 됩니다. 개념03 섹션 4 에서 배운 CSS 주석 기호로 감싸세요.

기대 화면: "주석으로 꺾 봅니다" 가 벽돌색 글자로 보이고 배경은 안 칠해집니다.

이렇게 되면 틀린 것: 글자색까지 사라졌다면 color 줄까지 같이 감싼 것입니다.

```
#q8Line {  
  color: #c0392b;  
  background-color: #fff3a0;  
}
```

문제 9 (개념05 섹션 1)

class 가 q9-card 인 바깥 div 에만 규칙을 하나 써서
그 안의 문단 세 줄이 모두 글자색 #8e44ad 가 되게 하세요.
문단마다 규칙을 쓰면 안 됩니다. 규칙은 딱 하나만 씁니다.

기대 화면: 문제 9 상자 안의 세 줄이 모두 보라색 글자가 됩니다.
이렇게 되면 틀린 것: 한 줄만 바뀌었다면 자식 하나를 고른 것입니다.

여기에 코드를 쓰세요

문제 10 (개념05 섹션 2)

아래 .q10-card 규칙은 이미 주어져 있습니다. 고치지 마세요.
배경색은 자식에게 상속되지 않습니다. 그래서 안쪽 문단은 자기 배경이 없습니다.
id 가 q10Kid 인 문단에 직접 규칙을 써서
배경색 #ffffff, 글자색 #2d6cdf 로 만드세요.

기대 화면: 파란 카드 안에 흰 띠가 하나 생기고 그 위에 파란 글자가 보입니다.
이렇게 되면 틀린 것: 흰 띠가 안 생기면 자식이 아니라 부모를 고친 것입니다.

```
.q10-card {
  color: #ffffff;
  background-color: #2d6cdf;
}
```

여기에 코드를 쓰세요

문제 11 (개념05 섹션 3, 섹션 6)

아래 .q11-card 규칙은 이미 주어져 있습니다. 고치지 마세요.
그런데 상자 안의 링크만 파란색으로 남아 있습니다.
브라우저가 a 의 색을 직접 정해 뒀기 때문입니다.
id 가 q11Link 인 링크에 규칙을 써서 부모 색을 그대로 받아 오게 하세요.
색 값을 직접 적지 말고 inherit 를 쓰세요.

기대 화면: 상자 안의 링크가 벽돌색 글자가 됩니다. 밑줄은 그대로 남습니다.
이렇게 되면 틀린 것: 링크가 계속 파랗다면 규칙이 a 를 못 고른 것입니다.

```
.q11-card {  
  color: #c0392b;  
}
```

여기에 코드를 쓰세요

문제 12 [응용] (개념04 섹션 3, 섹션 6)

id 가 q12Line 인 요소에 아래 두 가지를 주세요.

- 배경색: 벽돌색 rgb(192, 57, 43) 에 투명도 0.15 를 붙인 rgba 값
- 글자색: 그 연한 배경 위에서 잘 읽히도록 #c0392b

기대 화면: 아주 연한 분홍빛 배경 위에 진한 벽돌색 글자가 또렷하게 보입니다.
이렇게 되면 틀린 것: 배경이 진한 빨강이면 투명도를 안 붙인 것입니다.
글자가 잘 안 읽히면 글자색을 너무 연하게 고른 것입니다.

여기에 코드를 쓰세요

문제 13 [응용] (개념05 섹션 1, 섹션 6)

class 가 q13-panel 인 바깥 div 를 배경색 #8e44ad, 글자색 #ffffff 로 만드세요.
그런데 그것만으로는 안쪽 링크가 파란색으로 남습니다.
id 가 q13Link 인 링크도 흰 글자로 보이게 하세요.
링크 색은 inherit 를 써도 되고 #ffffff 를 직접 적어도 됩니다.

기대 화면: 보라색 상자 안에 흰 글자 한 줄과 흰 글자 링크(밑줄 있음)가
보입니다.

이렇게 되면 틀린 것: 링크만 파랗게 남았다면 a 를 직접 고르는 규칙을 안 쓴
것입니다.

여기에 코드를 쓰세요

문제 14 [도전] (개념01 ~ 개념05 종합)

알림 상자 세 종류를 만듭니다. 클래스는 이미 HTML 에 붙어 있습니다.
세 클래스 모두 글자색은 #ffffff 이고, 배경색만 다릅니다.
그리고 배경색은 세 가지를 일부러 서로 다른 표기법으로 쓰세요.

.alert-info	배경 파랑	16진수 여섯 자리로	#2d6cdf
.alert-warn	배경 빨강	rgb() 표기로	rgb(192, 57, 43)
.alert-done	배경 초록	색 이름으로	seagreen

기대 화면: 파랑·빨강·초록 띠 세 개가 위에서 아래로 놓이고 글자는 모두
흰색입니다.

이렇게 되면 틀린 것: 한 줄만 색이 없으면 그 줄의 표기법에서 오타가 난
것입니다.

표기법이 달라도 결과는 다 똑같이 잘 나와야 합니다.

여기에 코드를 쓰세요

문제 15 [확인] (개념03 섹션 5, 섹션 6 / 개념04 섹션 7)

아래 네 규칙은 전부 일부러 틀리게 써 두었습니다.
브라우저는 에러를 하나도 안 냅니다. 틀린 선언만 조용히 버릴 뿐입니다.
무엇이 틀렸는지 찾아 고치세요. 규칙마다 잘못된 곳은 딱 한 군데씩입니다.

넷 중 하나는 두 줄 중 한 줄만 죽어서 반쯤은 먹은 것처럼 보입니다.
어느 것인지도 찾아내세요.

기대 화면: 고치고 나면 네 줄이 이렇게 보입니다.

- q15a 파란 배경 위에 흰 글자
- q15b 노란 배경 위에 검은 글자
- q15c 벽돌색 글자
- q15d 초록 배경 위에 흰 글자

이렇게 되면 틀린 것: 한 줄이라도 그대로면 그 규칙의 잘못된 곳을 아직 못 찾은 것입니다.

힌트: 각각 개념03 섹션 5, 개념03 섹션 6, 개념04 섹션 7 에서 본 실수입니다.

```
#q15a {  
  color: #ffffff  
  background-color: #2d6cdf;  
}  
  
q15-tag {  
  background-color: #fff3a0;  
}  
  
#q15c {  
  color: c0392b;  
}  
  
#q15d {  
  color = #ffffff;  
  background-color: #27853f;  
}
```

07단원 마무리 CSS 시작하기

자주 넘어지는 자리

- 1 속성 이름을 일부러 틀려서 그 줄만 조용히 버려지는 것

선택자

OPEN 08_선택자

이 문단은 li 가 아니라서 원래 색 그대로입니다.
문 여는 시간이 바뀌었습니다.
이 문단에는 이름표를 붙이지 않았습니다.
재료가 떨어지면 일찍 닫습니다.

01 기본 선택자

02 여러 개 고르기

03 관계로 고르기

04 상태로 고르기

05 순서로 고르기

06 명시도

- 연습문제

기본 선택자

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 기본 선택자

07단원에서 CSS 규칙이 이렇게 생겼다는 것을 배웠습니다.

```
p { color: crimson; }
|   └──┬──┘
|       └─ 선언 : 무엇을 어떻게 바꿀지
└─ 선택자 : 누구를 바꿀지
```

07단원 개념03 에서 선택자 자리에 넣을 수 있는 것을 세 가지 봤습니다. 태그 이름, 클래스, id 였습니다. 딱 쓸 만큼만 맛본 셈입니다.

그런데 진짜 웹페이지에는 p 요소가 수십 개씩 있습니다. “그중에서 이 세 개만”, “상자 안의 것만”, “마우스를 올린 것만” 처럼 더 좁게 고르지 못하면 CSS 는 쓸모가 없습니다.

그래서 08단원 전체가 “고르는 법” 하나를 다룹니다. 이 파일에서는 07단원의 세 가지를 제대로 다시 정리하고, 전체 선택자를 하나 더해서 기본 네 가지를 봅니다.

p	태그 선택자	: 그 태그인 요소를 전부
.menu	클래스 선택자	: class="menu" 라고 붙여 둔 요소를 전부
#logo	id 선택자	: id="logo" 인 요소 하나
*	전체 선택자	: 페이지의 모든 요소

여기서 초보자가 가장 많이 헤매는 곳을 미리 말해 둡니다.

HTML 에는 점을 쓰지 않습니다. class="menu"
CSS 에만 점을 붙입니다. .menu { }

점은 “이건 클래스 이름이야” 라고 CSS 에게 알려 주는 표시입니다. HTML 은 class= 라고 이미 말했으니 표시가 필요 없습니다.

이 파일은 “누구를 꾸밀지 고르는 법” 네 가지를 다룹니다. 화면에서 색이 바뀐 곳을 먼저 찾고, VS Code 에서 그 색을 만든 규칙을 찾아보세요. “이 색은 어느 선택자가 골라서 칠한 걸까?” 를 맞추는 것이 오늘 할 일입니다.

태그 선택자

섹션 1

태그 이름을 그대로 쓰면 페이지에 있는 그 태그 전부가 골라집니다.

이 파일의 style 안에 이렇게 한 줄 썼습니다.

```
li {
  color: #8b3a62;
}
```

li 는 목록 항목 태그입니다(03단원). 이 규칙은 “이 페이지의 li 요소를 전부 찾아서 글자색을 #8b3a62 로 바꿔라” 라는 뜻입니다.

중요한 것은 ‘전부’ 입니다. 아래에 상자를 두 개 두었습니다. 서로 멀리 떨어져 있고 부모도 다릅니다. 그런데도 둘 다 자주색이 됩니다. 태그 이름만 맞으면 위치는 상관없기 때문입니다.

이것이 태그 선택자의 장점이자 위험입니다.

장점 : 규칙 한 줄로 페이지 전체를 정리할 수 있습니다.
위험 : 딱 하나만 바꾸고 싶었는데 백 군데가 같이 바뀝니다.

여기서 말하는 것은 ‘미치는 범위’ 이지 ‘규칙이 겹쳤을 때의 세기’ 가 아닙니다. 세기는 섹션 4 와 개념06 에서 따로 짚니다. 태그 선택자는 범위는 넓지만 세기는 가장 약한 축입니다.

그래서 실제 작업에서는 태그 선택자를 “이 페이지의 목록은 원래 다 이런 색” 같은 밑바탕을 깔 때만 쓰고, 개별 조정은 다음 섹션의 클래스로 합니다.

화면 두 항목의 글자가 자주색으로 보입니다

HTML	class="notice"	점 없음
CSS	.notice { }	점 있음

왜 CSS 에만 붙을까요? CSS 는 선택자 자리에 태그 이름도 오고 클래스 이름도 오고 id 도 옵니다. 그냥 notice 라고 쓰면 브라우저는 "notice 라는 태그" 를 찾습니다. 그래서 "이건 태그가 아니라 클래스야" 를 알려 주는 표시가 필요합니다. 그게 점입니다. 반대로 HTML 은 class= 라고 이미 말했으니 표시가 필요 없습니다.

클래스의 두 가지 성질을 기억하세요.

1 몇 번이든 다시 쓸 수 있습니다. 같은 이름표를 열 군데에 붙여도 됩니다.

2 태그 종류를 가리지 않습니다. p 에도 span 에도 li 에도 붙일 수 있습니다.

화면 1번째와 3번째 문단이 빨간 글자, 2번째 문단은 검은 글자입니다.

4번째 문단은 "봄내로 12" 여섯 글자만 빨강합니다

문 여는 시간이 바뀌었습니다.
이 문단에는 이름표를 붙이지 않았습니니다.
재료가 떨어지면 일찍 닫습니다.
주소는 봄내로 12 입니다.

클래스 이름은 내가 짓는 이름입니다. 정해진 목록이 없습니다. notice, menu-item, price 처럼 무엇인지를 나타내는 영어 소문자로 짓고, 단어 사이는 붙임표(-)로 잇습니다. red 처럼 색 이름으로 짓지 마세요. 나중에 파란색으로 바꾸면 이름이 거짓말이 됩니다.

◀ 직접 해보기

2

위 상자의 두 번째 문단 ("이 문단에는 이름표를 붙이지 않았습니니다") 에도 class="notice" 를 붙여 빨강게 만들어 보세요.

id 선택자

섹션 3

페이지에 딱 하나뿐인 것에 붙이는 이름표입니다. 기호는 # 입니다.

id 도 이름표입니다. 붙이는 방법은 클래스와 똑같이 생겼습니다.

```
HTML <p id="mainNotice">오늘은 재료 소진으로 3시에 닫습니다.</p>
CSS #mainNotice { background-color: #fff3a0; }
|
└─ 우물정자. "뒤에 오는 것은 id 다" 라는 표시입니다.
```

기호만 점에서 우물정자로 바뀌었습니다. 그런데 뜻이 하나 더 있습니다.

id 값은 한 페이지 안에서 딱 한 번만 써야 합니다.

주민등록번호 같은 것이라고 생각하세요. 클래스는 "3학년" 처럼 여러 명이 나눠 가질 수 있는 이름이고, id 는 "학번 20240137" 처럼 한 사람만 가지는 이름입니다.

아래 상자에는 id 를 하나만 붙였습니다. 그래서 문단 세 개 중 한 줄만 노란 형광펜이 칠해집니다.

화면 가운데 한 줄만 노란 배경에 갈색 글자이고, 위아래 두 줄은 그대로입니다

오늘의 원두는 에티오피아입니다.
오늘은 재료 소진으로 3시에 닫습니다.
내일은 평소대로 엽니다.

id 는 CSS 말고도 쓰임이 있습니다. 04단원에서 배운 페이지 안 링크 () 와 05단원에서 배운 <label for="..."> 가 id 를 찾아갑니다. 그래서 id 가 겹치면 링크와 label 도 엉뚱한 곳을 가리키게 됩니다.

직접 해보기

3

위 상자의 마지막 문단에 id="tomorrowNotice" 를 붙이고, <style> 안에 #tomorrowNotice { color: #1a7f5a; } 규칙을 추가해 초록 글자로 만들어 보세요.

클래스와 id, 언제 무엇을 쓰나

섹션 4

헛갈리면 이렇게 정하세요. 기본은 클래스입니다. id 는 정말 하나뿐일 때만 씁니다.

둘의 차이를 표로 정리하면 이렇습니다.

	클래스 .name	id #name
몇 번 쓰나	몇 번이든	페이지에 딱 한 번
무엇에 쓰나	같은 모양을 여러 곳에	그 페이지에 하나뿐인 자리
예	메뉴 항목, 가격, 딱지	페이지 맨 위 로고, 주요 안내문
힘(명시도)	보통	아주 썸 (개념06 에서 쥅니다)

판단 기준은 딱 하나입니다.

"이 모양을 다른 곳에도 또 쓸 일이 있을까?"
있다 → 클래스
없다 → 그래도 웬만하면 클래스

왜 클래스 쪽으로 기울까요? 오늘 하나뿐이던 것이 내일 두 개가 되는 일이 아주 흔하기 때문입니다. id 로 만들어 두면 그때 HTML 과 CSS 를 둘 다 고쳐야 합니다. 그리고 id 는 힘이 너무 세서, 나중에 다른 규칙으로 덮어쓰기가 어렵습니다. 이 '힘' 이야기가 개념06 의 명시도입니다.

한 요소에 둘 다 붙일 수도 있습니다.

```
<span class="badge" id="todayBadge">오늘</span>
```

이러면 .badge 규칙과 #todayBadge 규칙이 둘 다 적용됩니다. 아래 상자에서 딱지 세 개가 공통 모양 (.badge)을 가지고, 그중 하나만 id 로 더 진하게 칠해진 것을 확인하세요.

화면 딱지 세 개 중 "오늘" 만 진한 파란 배경에 흰 글자이고,

"주간" 과 "신메뉴" 는 연한 회색빛 파란 배경에 파란 글자입니다

오늘 아메리카노 3000원
주간 카페라떼 4000원
신메뉴 딸기라떼 5000원

딱지끼리 아직 간격이 없어서 글자가 배경색에 딱 붙어 보입니다. 안쪽 여백은 padding 으로 주는데, 그건 10단원에서 배웁니다. 지금은 색만으로 누가 골라졌는지 구분하면 충분합니다.

⇒ 직접 해보기 4

"신메뉴" 딱지에도 id="todayBadge" 를 붙여 보세요. 진한 파랑이 두 개가 됩니다. 브라우저는 아무 불평도 하지 않지만 이것은 잘못된 HTML 입니다. 확인했으면 지우세요.

전체 선택자

섹션 5

별표 하나면 페이지의 모든 요소가 골라집니다. 그래서 거의 안 씁니다.

전체 선택자는 별표 한 글자입니다.

```
* {  
  color: #1a7f5a;  
}
```

"이 페이지에 있는 요소를 하나도 빼놓지 말고 전부" 라는 뜻입니다. h1 도, p 도, span 도, div 도, body 도 전부입니다.

이 파일의 style 안에는 이 규칙을 넣어 두긴 했지만 주석으로 감싸서 꺼 두었습니다. 커면 이 설명 글까지 초록색이 되어 무엇을 읽고 있는지 알기 어려워지기 때문입니다.

⇒ 직접 해보기 5

에서 잠깐 켜 보고, 두 가지를 관찰하세요.

1 제목과 본문 글자가 초록색으로 바뀝니다.

2 그런데 전부 바뀌지는 않습니다.

회색 부제목, 검은 코드 상자, 빨간 딱지, 목록의 자주색은 그대로입니다.

2번이 이상하게 느껴져야 정상입니다. “전부 고르다면서 왜 안 바뀌지?” 라는 질문이 개념06 으로 이어집니다. 미리 한 줄만 말하면, 전체 선택자는 힘이 0 이라서 클래스나 태그 이름으로 정해 둔 색에 밀립니다.

실제로 전체 선택자를 쓰는 자리는 사실상 한 곳뿐입니다. 페이지 전체의 크기 계산 방식을 통일하는 * { box-sizing: border-box; } 입니다. 10단원에서 다시 만납니다. 지금은 이런 게 있다는 것만 알아 두세요.

화면 첫 문단은 검은 글자, 딱지는 연한 파란 배경에 파란 글자입니다.

전체 선택자를 켜면 첫 문단만 초록으로 바뀌고 딱지는 그대로입니다

이 문단은 아직 검은 글자입니다.
딱지 는 파란 글자입니다.

▹ 직접 해보기 5

<style> 맨 아래에 * { color: #1a7f5a; } 를 한 줄 그대로 써 넣고 새로고침해 보세요. 어디가 바뀌고 어디가 안 바뀌는지 세어 본 다음, 그 한 줄을 지우세요.

자주 하는 실수

섹션 6

CSS 도 HTML 처럼 틀려도 에러가 나지 않습니다. 틀린 규칙은 그냥 조용히 버려집니다. 아래 다섯 가지는 전부 “아무 일도 안 일어나는” 종류의 사고입니다.

이런 실수를 합니다

CSS 에서 점을 빠뜨림

이런 실수를 합니다

실수 1 — CSS 에서 점을 빠뜨림

```
dot-missing { color: crimson; }
```

점이 없으니 브라우저는 dot-missing 이라는 이름의 태그를 찾습니다. 그런 태그는 세상에 없으므로 아무것도 못 찾고, 규칙은 조용히 버려집니다. 에러도 경고도 없습니다. 아래 문단은 class=“dot-missing” 을 제대로 붙였는데도 빨개지지 않습니다. 저는 빨개지지 않습니다.

이렇게 쓰세요

```
.dot-missing { color: crimson; }
```

이런 실수를 합니다

HTML class 값에 점을 붙임

이런 실수를 합니다

실수 2 — HTML class 값에 점을 붙임

```
<p class=".dot-in-class">...</p>
```

실수 1 을 배우고 나서 정반대로 하는 사람이 많습니다. 이러면 클래스 이름 자체가 .dot-in-class (맨 앞의 점까지 이름에 포함된 열세 글자) 가 되어 버립니다. CSS 의 .dot-in-class 는 dot-in-class 라는 이름을 찾으니 서로 만나지 못합니다. 역시 아무 일도 안 일어납니다. 저도 빨개지지 않습니다.

이렇게 쓰세요

```
<p class="dot-in-class">...</p>
```

이런 실수를 합니다

점과 우물정자를 바꿔 씀

이런 실수를 합니다

실수 3 — 클래스를 # 로, id 를 . 으로 부름

```
<p class="wrongMark">...</p>
#wrongMark { color: crimson; }
```

HTML 은 클래스인데 CSS 는 id 를 찾고 있습니다. 이름은 똑같지만 기호가 다르면 서로 다른 것을 가리킵니다. 기호가 이름보다 먼저입니다. 점은 클래스, 우물정자는 id 입니다. 저도 그대로입니다.

이런 실수를 합니다

같은 id 를 여러 요소에 씀

이런 실수를 합니다

실수 4 — 같은 id 를 여러 요소에 씀

```
<p id="dupNotice">첫 번째</p>
<p id="dupNotice">두 번째</p>
```

이것이 가장 알아채기 어려운 실수입니다. 왜냐하면 겉보기에는 잘 되기 때문입니다. 아래 두 문단은 같은 id 를 가졌고, 브라우저는 둘 다 칠해 줍니다. 하지만 링크나 <label for="dupNotice"> 는 첫 번째 하나만 찾아갑니다. 나중에 자바스크립트를 배우면 그때도 첫 번째 하나만 잡힙니다. 첫 번째 문단입니다. 두 번째 문단입니다.

이렇게 쓰세요

```
<p class="dupNotice">첫 번째</p>
<p class="dupNotice">두 번째</p>
```

이런 실수를 합니다

태그 선택자를 너무 넓게 씀

이런 실수를 합니다

실수 5 — 태그 선택자가 페이지 전체에 걸림

```
li { color: #8b3a62; }
```

섹션 1에서 쓴 규칙입니다. 그때는 메뉴 상자만 생각하고 썼습니다. 그런데 아래 목록은 섹션 1과 아무 상관이 없는데도 자주색입니다. 이 파일의 li 전부가 걸렸기 때문입니다. “왜 여기까지 색이 변했지?” 싶으면 태그 선택자를 먼저 의심하세요. 여기는 섹션 6입니다. 그런데도 자주색입니다.

이렇게 쓰세요

```
.menu-list li { color: #8b3a62; }
```

정리

▸ 직접 해보기 7

정답

1) style 안의 규칙을 이렇게 바꿉니다.

```
ol {
  color: #8b3a62;
}
```

→ 첫 번째 상자(ul)의 두 항목은 검은색으로 돌아가고,

두 번째 상자(번호 목록 ol)의 두 항목만 자주색으로 남습니다.
단, 섹션 6의 목록도 ul이라 함께 검은색이 됩니다.
태그 이름을 바꾸면 고르는 대상이 통째로 바뀐다는 뜻입니다.
확인했으면 li로 되돌리세요.

2) 섹션 2 상자의 두 번째 문단을 이렇게 고칩니다.

```
<p class="notice">이 문단에는 이름표를 붙이지 않았습니다.</p>
```

→ 그 줄도 빨간 글자가 됩니다. 상자 안 빨간 줄이 두 개에서 세 개가 됩니다.

같은 클래스 이름을 몇 번이든 다시 쓸 수 있다는 것을 확인하는 문제입니다.

3) HTML 을 이렇게 고치고

```
<p id="tomorrowNotice">내일은 평소대로 엽니다.</p>
```

style 맨 아래에 이 규칙을 넣습니다.

```
#tomorrowNotice {
  color: #1a7f5a;
}
```

→ 상자의 마지막 줄이 초록 글자가 됩니다.

노란 형광펜 줄(#mainNotice)은 그대로입니다. 서로 다른 id 니까요.

4) 세 번째 딱지를 이렇게 고칩니다.

```
<span class="badge" id="todayBadge">신메뉴</span>
```

→ “신메뉴” 딱지도 진한 파란에 흰 글자가 됩니다.

브라우저는 경고 한 줄 없이 그려 줍니다. 그래서 더 위험합니다.
같은 id 가 두 개면 링크와 label 이 첫 번째만 찾아갑니다.
확인했으면 id="todayBadge" 를 지우세요.

5) style 맨 아래에 이 한 줄을 넣습니다.

```
* { color: #1a7f5a; }
```

→ 맨 위 큰 제목, 노란 안내 상자 글, 파란 섹션 제목, 각 섹션 설명,

맨 아래 정리 상자 글이 모두 초록색이 됩니다.

반면 아래 것들은 그대로 남습니다.

회색 부제목(08단원 · 선택자) ← .doc-sub 가 색을 정해 둬
"실제 결과" 같은 작은 회색 글 ← .doc-demo-label 이 색을 정해 둬
검은 코드 상자 글자 ← .doc-code 가 색을 정해 둬
빨간 실수 딱지 ← .doc-bad-tag 가 색을 정해 둬
목록의 자주색 항목 ← li 가 색을 정해 둬
섹션 2 의 빨간 notice 글자 ← .notice 가 색을 정해 둬

전체 선택자는 힘이 0 이라 클래스나 태그 이름에 전부 밀립니다.

이 '힘' 을 재는 법이 개념06 입니다. 확인했으면 그 한 줄을 지우세요.

태그 선택자

섹션 1

```
li {  
  color: #8b3a62;  
}
```

화면 이 파일 안에 있는 목록 항목 글자가 전부 자주색이 됩니다.

상자가 몇 개든 상관없이 li 라면 모두 바뀝니다

클래스 선택자

섹션 2

```
.notice {  
  color: crimson;  
}
```

화면 class="notice" 라고 적어 둔 글자만 빨간색이 됩니다.

p 든 span 이든 태그 종류는 상관없습니다

id 선택자

섹션 3

```
#mainNotice {  
  background-color: #fff3a0;  
  color: #7a4b00;  
}
```

화면 id="mainNotice" 인 문단 한 줄만 노란 형광펜을 칠한 것처럼 됩니다

클래스와 id 를 같이 쓰기

섹션 4

```
.badge {  
  background-color: #eef2f7;  
  color: #2d6cdf;  
}  
#todayBadge {  
  background-color: #2d6cdf;  
  color: white;  
}
```

화면 회색빛 파란 딱지가 세 개 보이고, 그중 "오늘" 딱지 하나만

진한 파란 배경에 흰 글자입니다

전체 선택자

섹션 5

아래 규칙은 일부러 꺼(주석 처리해) 두었습니다.
 페이지 전체 글자색을 바꿔 버려서 설명을 읽기 어려워지기 때문입니다.
 ✎ 직접 해보기 5 에서 잠깐 켜 봅니다.

```
* {
  color: #1a7f5a;
}
```

화면: 켜면 제목과 본문 글자가 초록색으로 바뀝니다.
 다만 회색 부제목·검은 코드 상자·빨간 딱지처럼
 자료 겹데기가 클래스로 색을 정해 둔 곳은 그대로 남습니다.
 목록 글자도 자주색 그대로입니다.

왜 어떤 곳은 안 바뀌는지는 개념06 에서 배웁니다.

자주 하는 실수 (일부러 틀리게 쓴 규칙들)

섹션 6

```
dot-missing {
  color: crimson;
}
```

화면 아무 일도 일어나지 않습니다. dot-missing 이라는 태그는 없습니다

```
.dot-in-class {
  color: crimson;
}
```

화면 아무 일도 일어나지 않습니다.

HTML 쪽에 class=".dot-in-class" 라고 점을 붙여 써 두었기 때문입니다

```
#wrongMark {
  color: crimson;
}
```

화면 아무 일도 일어나지 않습니다.

HTML 쪽은 class="wrongMark" 인데 CSS 는 # 로 id 를 찾고 있습니다

```
#dupNotice {
  background-color: #ffe9e9;
}
```

화면

같은 id 를 가진 문단 두 개가 모두 연한 분홍 배경이 됩니다

08단원 · 개념 02

여러 개 고르기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 여러 개 고르기

개념01에서는 선택자 하나로 한 종류만 골랐습니다. 실제로 페이지를 꾸미다 보면 이런 일이 생깁니다.

"제목이랑 가격이랑 버튼 글자, 이 셋을 다 같은 색으로 하고 싶다"
 "카드 중에서 '큰 카드' 만 배경을 다르게 하고 싶다"

이 파일은 그 두 가지를 다룹니다. 그리고 이 단원에서 가장 많이 틀리는 세 가지 표기가 여기 다 모여 있습니다. 지금 확실히 갈라 두세요.

| | |
|-------------|--------------------------------|
| .card, .big | 선표 : card 이거나 big 인 것 (둘 다 고름) |
| .card.big | 붙임 : card 와 big 을 둘 다 가진 것 하나 |
| .card .big | 공백 : card 안에 들어 있는 big |

눈으로 보면 선표 하나, 공백 한 칸 차이입니다. 그런데 고르는 대상이 완전히 다릅니다. 게다가 틀려도 에러가 안 나서, 화면이 안 바뀌는 것으로만 알 수 있습니다.

그래서 이 파일에서는 셋을 나란히 놓고 눈으로 비교합니다. 읽기만 하지 말고, 상자 안에서 실제로 어느 줄에 색이 칠해졌는지 하나씩 세어 보세요.

선표, · 붙여 쓰기 · 공백 한 칸. 이 세 가지 표기의 차이가 이 파일의 전부입니다. 상자마다 “몇 줄에 색이 칠해졌는지” 를 세면서 읽으세요.

그룹 선택자

섹션 1

선택자 사이에 선표를 찍으면 “이것 또는 저것” 이 됩니다.

같은 색을 여러 곳에 주고 싶을 때, 규칙을 여러 번 쓸 수도 있습니다.

```
.drink { background-color: #eef7ff; }  
.food { background-color: #eef7ff; }
```

이러면 색을 바꿀 때 두 군데를 고쳐야 합니다. 한 군데를 빠뜨리면 어긋납니다. 선표를 쓰면 한 규칙으로 묶을 수 있습니다.

```
.drink,  
.food {  
  background-color: #eef7ff;  
}
```

읽는 법은 “drink 이거나 food 인 요소를 전부” 입니다. 선표를 ‘또는’ 이라고 소리 내어 읽으면 헷갈리지 않습니다.

선표 뒤에 줄을 바꿔도 되고 한 줄에 이어 써도 됩니다. 결과는 같습니다.

```
.drink, .food { background-color: #eef7ff; }
```

보통은 줄을 바꿔 씁니다. 선택자가 늘어나도 한눈에 세기 좋기 때문입니다.

그리고 선표로 묶는 선택자는 종류가 달라도 됩니다. 아래 두 번째 규칙은 태그·id·클래스를 한꺼번에 묶었습니다.

```
h3,  
#shopTitle,  
.price {  
  color: #1a7f5a;  
}
```

화면 “봄내 카페” 와 “오늘의 메뉴” 가 초록 글자,

음료 줄과 케이크 줄은 연한 하늘색 배경, 그 안의 “3000원” “5500원” 도 초록 글자, 마지막 빨대 줄은 아무 색도 없습니다

봄내 카페
오늘의 메뉴
아메리카노 3000원
치즈케이크 5500원
종이 빨대는 무료입니다

선표로 묶는다고 해서 그 요소들이 한 덩어리가 되는 것은 아닙니다. 규칙을 여러 번 쓰는 수고를 줄여 줄 뿐입니다. 결과는 따로따로 쓴 것과 똑같습니다.

▸ 직접 해보기 1

<style>의 첫 규칙에 .etc-item 을 선표로 추가해서 빨대 줄까지 하늘색으로 만들어 보세요.

선표를 빼뜨리면

섹션 2

선표를 안 찍으면 다른 규칙이 되어 버립니다. 에러가 나는 게 아니라 엉뚱한 것을 고릅니다.

CSS 에서 선택자 사이의 공백은 그냥 빈칸이 아닙니다. 그 자체가 뜻을 가집니다.

| | |
|-------------|----------------------------------|
| .tip, .warn | 선표 있음 → tip 이거나 warn 인 것 (둘 다) |
| .tip .warn | 선표 없음 → tip 안에 들어 있는 warn 만 (하나) |

공백이 “안에 들어 있는” 이라는 뜻인데, 이것을 자손 선택자라고 부릅니다. 자세한 것은 개념03 에서 배웁니다. 지금은 “선표를 빼면 전혀 다른 뜻이 된다” 하나만 확인하면 됩니다.

아래 두 상자의 HTML 은 글자만 빼고 완전히 똑같은 모양입니다.

```
<p class="tip-a">...</p> <p class="tip-b">...</p>
<p class="warn-a">...</p> <p class="warn-b">...</p>
```

다른 것은 CSS 한 글자, 선표뿐입니다.

| | |
|--|-------|
| .tip-a, .warn-a { background-color: #fff3a0; } | 선표 있음 |
| .tip-b .warn-b { background-color: #fff3a0; } | 선표 없음 |

아래 상자를 보면 위 상자는 두 줄이 노랑고, 아래 상자는 한 줄도 노랑지 않습니다. 아래 상자에서 tip-b 와 warn-b 는 나란히 놓인 사이라서 “tip-b 안에 있는 warn-b” 가 하나도 없기 때문입니다.

이런 사고가 무서운 이유는, 화면이 안 바뀔 뿐 아무 신호가 없다는 것입니다. “색이 왜 안 들어가지?” 하고 색 이름을 백 번 고쳐 봐야 소용이 없습니다. 범인은 색이 아니라 선택자의 선표입니다.

화면 두 줄 모두 노란 배경입니다

따뜻하게 드시려면 미리 말씀해 주세요.
유리컵은 뜨거우니 조심하세요.

화면 두 줄 다 배경색이 없습니다. 아무 일도 일어나지 않았습니다

따뜻하게 드시려면 미리 말씀해 주세요.
유리컵은 뜨거우니 조심하세요.

같은 실수를 색이 아니라 글꼴이나 크기에서 하면 더 못 찾습니다. “규칙을 썼는데 아무 변화가 없다” 면 속성 이름보다 선택자를 먼저 의심하세요.

▫ 직접 해보기 2

아래 상자의 규칙 .tip-b .warn-b 에 심표를 찍어 .tip-b, .warn-b 로 고쳐 보세요. 두 줄이 노랑게 바뀝니다. 확인했으면 다시 심표를 지우세요.

한 요소에 클래스 여러 개

섹션 3

class 값에 공백을 두고 이름을 여러 개 쓸 수 있습니다. 붙인 이름표의 규칙이 전부 적용됩니다.

지금까지는 한 요소에 이름표를 하나만 붙였습니다. 여러 개도 됩니다.

```
<p class="card sold">딸기라떼</p>
```

└──┬──
 | └─ 두 번째 이름표
 └─ 첫 번째 이름표

공백으로 구분합니다. 심표가 아닙니다. 공백입니다. 이 요소는 이제 이름표를 두 개 가진 셈이고, 둘의 규칙이 모두 적용됩니다.

```
.card { background-color: #eef2f7; } → 배경이 회색빛 파랑  
.sold { color: #8a8a8a; } → 글자가 흐린 회색
```

순서는 상관없습니다. class="sold card" 라고 써도 결과가 똑같습니다. HTML 의 class 값은 “가진 이름표 목록” 일 뿐, 순서에 뜻이 없습니다.

이 방식이 왜 좋을까요? “카드 모양” 은 .card 한 곳에서만 관리하고, “품절 표시” 는 .sold 한 곳에서만 관리할 수 있기 때문입니다. 품절 상품이 생기면 HTML 에 sold 한 단어만 더 쓰면 됩니다.

이것을 부품처럼 조립한다고 해서 조합해 쓴다고 말합니다. 실제 웹사이트의 CSS 는 거의 이 방식으로 만들어져 있습니다.

화면 1번째 줄은 회색빛 파란 배경에 검은 글자,

2번째 줄은 회색빛 파란 배경에 흐린 회색 글자, 3번째 줄은 배경 없이 흐린 회색 글자입니다

아메리카노 – 이름표 하나 (card)
 딸기라떼 – 이름표 둘 (card sold)
 종이 빨대 – 이름표 하나 (sold)

class="card sold" 는 클래스가 두 개라는 뜻입니다. card sold 라는 이름 하나가 아닙니다. CSS 에서 부를 때도 .card 와 .sold 로 따로 부릅니다.

▸ 직접 해보기 3

위 상자의 첫 줄에도 sold 를 추가해 class="card sold" 로 만들어 보세요. 글자색이 어떻게 바뀌는지 보세요.

붙여 쓰기

섹션 4

.card.big 처럼 공백 없이 붙여 쓰면 두 이름표를 모두 가진 요소만 골라집니다.

섹션 3 에서 "카드 중 큰 카드만 배경을 다르게" 하고 싶다고 해 봅시다. .big 만 골라서는 안 됩니다. 카드가 아닌 것에도 big 이 붙어 있을 수 있으니까요. "card 이면서 동시에 big 인 것" 을 골라야 합니다.

```
.card.big {
  |
  ↳ 사이에 공백이 하나도 없습니다

  background-color: #ffe9e9;
}
```

읽는 법은 "card 이고 그리고 big 인 요소" 입니다. 섹션 1 의 심표가 '또는' 이었다면, 붙여 쓰기는 '그리고' 입니다.

| | | |
|-------------|--------------|--------------------|
| .card, .big | card 이거나 big | → 넓어집니다 (더 많이 골라짐) |
| .card.big | card 이고 big | → 좁아집니다 (더 적게 골라짐) |

심표는 대상을 늘리고, 붙여 쓰기는 대상을 줄입니다. 정반대입니다.

붙여 쓰기는 클래스끼리만 되는 것이 아닙니다.

p.card p 이면서 card 인 것
 .card#todayCard card 이면서 id 가 todayCard 인 것

태그 이름은 항상 맨 앞에 씁니다. .cardp 라고 쓰면 안 됩니다.

아래 상자에서 분홍 배경이 몇 줄인지 세어 보세요. 딱 한 줄입니다.

화면 1번째 줄은 회색빛 파란 배경에 검은 글자,

2번째 줄은 배경 없이 빨간 글자, 3번째 줄만 연한 분홍 배경에 빨간 글자입니다

```
card 만 가진 줄
big 만 가진 줄
card 와 big 을 둘 다 가진 줄
```

3번째 줄은 .card(회색빛 파랑) 와 .card.big(분홍) 이 둘 다 맞습니다. 그런데 분홍이 이겼습니다. 조건이 더 많이 붙은 선택자가 더 세기 때문입니다. 이 '세기' 를 재는 법이 개념06 입니다.

▣ 직접 해보기 4

두 번째 줄("big 만 가진 줄")에 card 를 추가해 class="big card" 로 만들어 보세요. 순서를 바꿔 써도 분홍이 되는지 확인하세요.

붙여 쓰기 vs 공백

섹션 5

.card.big 과 .card .big 는 공백 한 칸 차이인데 완전히 다른 것을 고릅니다.

세 표기를 한자리에 모읍니다. 이 표를 외우려 하지 말고, 아래 상자에서 어느 줄에 색이 칠해졌는지 눈으로 확인하세요.

| | | | |
|-------------|----|---------------------------|----------|
| .card, .big | 선표 | card 이거나 big 인 요소를 전부 | (둘 다 고름) |
| .card.big | 붙임 | card 와 big 을 둘 다 가진 요소 하나 | (좁힘) |
| .card .big | 공백 | card 안에 들어 있는 big | (다른 요소) |

가장 중요한 차이는 마지막 것입니다. 붙여 쓰기는 요소 '하나' 를 가리키지만, 공백은 요소가 '두 개' 있어야 합니다. 감싸는 쪽과 감싸이는 쪽입니다.

붙여 쓰기가 찾는 모양

```
<p class="card big">글자</p>
```

한 요소가 이름표를 둘 다 가짐

공백이 찾는 모양

```
<div class="card"> <p class="big">글자</p> </div>
```

바깥 요소가 card, 그 안의 다른 요소가 big

아래 상자에 두 모양을 나란히 두었습니다. 분홍 배경은 붙여 쓰기가, 연한 초록 배경은 공백이 칠한 것입니다. 서로 다른 줄이 칠해졌다는 점을 꼭 확인하세요.

읽는 요령을 하나 알려 드립니다. 선택자를 소리 내어 읽을 때 공백이 나오면 "안의" 를 넣어 읽으세요.

| | |
|------------|-----------------|
| .card .big | → "카드 안의 빅" |
| .card.big | → "카드빅" (한 덩어리) |

화면 첫 줄은 연한 분홍 배경에 빨간 글자입니다.

그 아래 회색빛 파란 상자가 있고, 그 안의 줄만 연한 초록 배경에 빨간 글자입니다. 두 줄의 배경색이 서로 다릅니다

한 요소가 card 와 big 을 둘 다 가진 경우

card 상자 안에 들어 있는 big

첫 줄은 .card .big 에 걸리지 않습니다. 자기 자신은 자기 자신의 '안' 이 아니기 때문입니다. 반대로 안쪽 줄은 .card.big 에 걸리지 않습니다. 그 줄은 big 만 가지고 있고 card 는 바깥 상자가 가졌기 때문입니다.

▣ 직접 해보기

5

<style> 에서 .card .big 의 공백을 지워 .card.big 로 만들어 보세요. 연한 초록 배경이 어디로 사라지는지 확인한 뒤 되돌려 놓으세요.

자주 하는 실수

섹션 6

아래 다섯 가지는 전부 에러가 나지 않습니다. 화면이 안 바뀌는 것으로만 알 수 있습니다.

이런 실수를 합니다

침표를 빠뜨림

이런 실수를 합니다

실수 1 — 침표를 빠뜨림

```
.tip-b .warn-b { background-color: #fff3a0; }
```

섹션 2 에서 본 사고입니다. "둘 다 노랑게" 하려고 썼는데 "tip-b 안의 warn-b" 를 찾게 됩니다. 그런 요소가 없으면 한 줄도 안 바뀝니다. 여러 선택자를 세로로 늘어놓고 쓸 때 마지막 줄 빼고 전부 침표가 있는지 세어 보세요.

이렇게 쓰세요

```
.tip-b,
.warn-b {
  background-color: #fff3a0;
}
```

이런 실수를 합니다

붙여 쓰기와 공백을 혼동

이런 실수를 합니다

실수 2 — 붙여 써야 할 곳에 공백을 넣음

```
<p class="zone mark">...</p>
.zone .mark { background-color: #fff3a0; }
```

HTML 은 한 요소에 이름표 두 개를 붙였습니다. 그런데 CSS 는 공백을 써서 “zone 안에 있는 mark” 를 찾습니다. 이런 요소는 없으니 아무 일도 안 일어납니다. 아래 줄이 그 상태입니다. 저는 노래지지 않습니다.

이렇게 쓰세요

```
.zone.mark { background-color: #fff3a0; }
```

이런 실수를 합니다

HTML class 값을 쉼표로 구분

이런 실수를 합니다

실수 3 — HTML class 값을 쉼표로 구분

```
<p class="comma-cls, other">...</p>
```

CSS 에서 쉼표를 쓰는 것을 배우고 나면 HTML 에서도 쉼표를 찍게 됩니다. class 값은 공백으로만 나눕니다. 쉼표를 찍으면 첫 번째 이름이 comma-cls, (쉼표까지 포함) 가 되어 .comma-cls 로는 찾을 수 없습니다. 저도 빨개지지 않습니다.

이렇게 쓰세요

```
<p class="comma-cls other">...</p>
```

이런 실수를 합니다

클래스 두 개를 붙여 씀

이런 실수를 합니다

실수 4 — HTML 에서 클래스 두 개를 붙여 씀

```
<p class="join-ajoin-b">...</p>
```

CSS 의 .card.big 을 보고 HTML 에서도 붙여 쓰는 경우입니다. CSS 는 붙여 쓰고 HTML 은 띄어 씁니다. 정반대입니다. 붙여 쓰면 join-ajoin-b 라는 이름표 하나가 됩니다. 저도 아무 변화가 없습니다.

이렇게 쓰세요

```
<p class="join-a join-b">...</p>
```

이런 실수를 합니다

선택자 목록 끝에 쉼표를 남김

이런 실수를 합니다

실수 5 — 마지막 선택자 뒤에 쉼표를 남김

```
.trail-a,  
.trail-b, {  
  color: crimson;  
}
```

선택자를 지우다가 쉼표만 남기는 일이 자주 있습니다. 이때는 규칙 전체가 통째로 버려집니다. 멀쩡한 `.trail-a` 까지 같이 죽습니다. “한 줄만 안 되는 게 아니라 여러 줄이 한꺼번에 안 될 때” 는 바로 위 선택자 줄의 쉼표를 보세요. 저는 빨개지지 않습니다. 저도 빨개지지 않습니다.

이렇게 쓰세요

```
.trail-a,  
.trail-b {  
  color: crimson;  
}
```

정리

◁ 직접 해보기 6

정답

1) style 의 첫 규칙을 이렇게 고칩니다.

```
.drink,  
.food,  
.etc-item {  
  background-color: #eef7ff;  
}
```

→ 상자의 마지막 줄(“종이 빨대는 무료입니다”)에도 연한 하늘색 배경이 생깁니다.

하늘색 줄이 두 줄에서 세 줄이 됩니다.

`.food` 뒤에 쉼표를 찍는 것을 잊지 마세요. 안 찍으면 섹션 2 의 사고가 납니다.

2) style 의 규칙을 이렇게 고칩니다.

```
.tip-b,
.warn-b {
  background-color: #fff3a0;
}
```

→ 아래 상자의 두 줄이 위 상자와 똑같이 노란 배경이 됩니다.

HTML 은 한 글자도 안 고쳤는데 결과가 달라졌다는 점이 중요합니다.
범인은 처음부터 CSS 의 심표였습니다. 확인했으면 심표를 다시 지우세요.

3) 섹션 3 상자의 첫 줄을 이렇게 고칩니다.

```
<p class="card sold">아메리카노 - 이름표 하나 (card)</p>
```

→ 배경은 그대로 회색빛 파랑이고, 글자만 흐린 회색으로 바뀝니다.

.card 규칙과 .sold 규칙이 서로 다른 속성을 건드리기 때문에
둘 다 살아서 함께 적용됩니다.

4) 섹션 4 상자의 두 번째 줄을 이렇게 고칩니다.

```
<p class="big card">big 만 가진 줄</p>
```

→ 그 줄도 연한 분홍 배경이 됩니다. 분홍 줄이 두 개가 됩니다.

class 값의 순서는 뜻이 없다는 것을 확인하는 문제입니다.
.card.big 은 "둘 다 가졌는가" 만 보고, 어느 쪽을 먼저 썼는지는 보지 않습니다.

5) style 의 규칙을 이렇게 고칩니다.

```
.card.big {
  background-color: #e2f4ea;
}
```

→ 섹션 5 상자의 안쪽 줄에서 연한 초록 배경이 사라집니다.

대신 그 위의 첫 줄이 분홍에서 연한 초록으로 바뀝니다.
(.card.big 규칙이 두 개 되었고, 나중에 쓴 초록이 이깁니다.
같은 세기일 때 나중 것이 이기는 이유는 개념06 에서 배웁니다)
섹션 4 상자의 셋째 줄도 함께 초록으로 바뀝니다.
확인했으면 공백을 다시 넣어 .card .big 으로 되돌리세요.

그룹 선택자 (선택표)

섹션 1

```
.drink,
.food {
  background-color: #eef7ff;
}
```

화면 class="drink" 인 줄과 class="food" 인 줄이 둘 다 연한 하늘색 배경이 됩니다

```
h3,
#shopTitle,
.price {
  color: #1a7f5a;
}
```

화면 h3 제목, id 가 shopTitle 인 줄, class="price" 인 글자가 모두 초록 글자입니다.

선택표 하나로 태그·id·클래스를 한꺼번에 고를 수 있습니다

선택표를 빠뜨리면

섹션 2

```
.tip-a,
.warn-a {
  background-color: #fff3a0;
}
```

화면 위 상자의 두 줄이 모두 노란 배경이 됩니다

```
.tip-b .warn-b {
  background-color: #fff3a0;
}
```

화면 아무 일도 일어나지 않습니다.

선택표가 없으니 "tip-b 안에 있는 warn-b" 를 찾는데, 둘은 나란히 있어서

그런 요소가 없습니다

```
.card {
  background-color: #eef2f7;
}
.sold {
  color: #8a8a8a;
}
```

화면 card 가 붙은 줄은 회색빛 파란 배경, sold 가 붙은 줄은 흐린 회색 글자.

둘 다 붙은 줄은 회색빛 파란 배경 + 흐린 회색 글자가 동시에 나옵니다

붙여 쓰기

```
.big {
  color: #c0392b;
}
.card.big {
  background-color: #ffe9e9;
}
```

화면 card 와 big 을 둘 다 가진 줄만 연한 분홍 배경이 됩니다.

한쪽만 가진 줄은 분홍이 되지 않습니다

붙여 쓰기 vs 공백

```
.card .big {
  background-color: #e2f4ea;
}
```

화면 card 상자 "안에 들어 있는" big 만 연한 초록 배경이 됩니다

자주 하는 실수 (일부러 틀리게 쓴 규칙들)

```
.zone .mark {
  background-color: #fff3a0;
}
```

화면 아무 일도 일어나지 않습니다.

HTML 은 class="zone mark" 로 한 요소에 둘 다 붙였는데

CSS 는 공백을 써서 "zone 안의 mark" 를 찾고 있습니다

```
.comma-cls {
  color: crimson;
}
```

화면 아무 일도 일어나지 않습니다.

HTML 에 class="comma-cls, other" 라고 써서

클래스 이름이 "comma-cls," 가 되어 버렸습니다

```
.join-a {
  color: crimson;
}
.join-b {
  background-color: #fff3a0;
}
```

화면 아무 일도 일어나지 않습니다.

HTML 에 class="join-ajoin-b" 라고 붙여 써서 이름이 하나가 되었습니다

```
.trail-a,
.trail-b, {
  color: crimson;
}
```

화면 아무 일도 일어나지 않습니다.

선표를 하나 더 남겨 두면 선택자 목록 전체가 잘못된 것이 되어

규칙이 통째로 버려집니다. 앞의 .trail-a 까지 같이 죽습니다

관계로 고르기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 관계로 고르기

지금까지는 요소를 “그 자체의 이름표” 로만 골랐습니다. 태그 이름, 클래스, id 전부 그 요소에 직접 붙어 있는 것입니다.

그런데 실제로는 이렇게 말하고 싶을 때가 훨씬 많습니다.

"머리말 안에 있는 링크만"
 "메뉴 목록의 항목만"
 "제목 바로 아래 문단만"

전부 위치, 즉 다른 요소와의 관계로 말하고 있습니다. 이렇게 관계로 고르는 선택자가 네 가지 있습니다.

| | | |
|-------|------|--------------------------|
| A B | 자손 | A 안에 있는 B 를 전부 (깊이 상관없음) |
| A > B | 자식 | A 의 바로 한 단계 아래 B 만 |
| A + B | 인접형제 | A 바로 다음에 오는 B 하나 |
| A ~ B | 이후형제 | A 뒤에 오는 B 를 전부 |

가운데의 공백, 부등호, 더하기, 물결을 결합자라고 부릅니다. 결합자는 “왼쪽과 오른쪽이 어떤 사이인가”를 말합니다. 그리고 색이 칠해지는 것은 언제나 맨 오른쪽 것입니다.

```
.notice-box p { color: crimson; }
  ↳ 빨개지는 것은 p 입니다. .notice-box 가 아닙니다.
```

이 점을 먼저 못 박아 두세요. 여기서 정말 많이 헛갈립니다.

그리고 이 네 가지를 쓰려면 부모·자식·형제·자손이라는 말을 정확히 알아야 합니다. 섹션 1 에서 그것부터 정리합니다.

이 파일은 요소와 요소의 사이로 고르는 법을 다룹니다. $A \cdot B \cdot A > B \cdot A + B \cdot A \sim B$ 네 가지입니다. 색이 칠해지는 것은 언제나 오른쪽이라는 점을 계속 확인하면서 읽으세요.

가족 용어

섹션 1

01단원과 06단원에서 태그 안에 태그를 넣는 것을 배웠습니다. 그 포개진 모양에 붙는 이름을 여기서 정리합니다.

아래 상자의 HTML 은 이렇게 생겼습니다.

```
<div class="shop"> <p class="shop-name">봄내 카페</p> <ul class="menu"> <li>
아메리카노</li> <li>카페라떼</li> </ul>
</div>
```

들여쓰기만 남기고 그리면 이런 모양입니다.

```
div.shop
  p.shop-name
  ul.menu
    li
    li
```

여기에 이름을 붙입니다.

부모(parent) 바로 바깥에서 감싸고 있는 요소 하나
→ li 의 부모는 ul.menu 입니다.

자식(child) 바로 한 단계 안에 있는 요소
→ div.shop 의 자식은 p.shop-name 과 ul.menu 두 개입니다.
li 는 div.shop 의 자식이 아닙니다. 한 단계 더 들어갔습니다.

자손(descendant) 안에 들어 있는 모든 요소. 깊이는 상관없습니다.
→ div.shop 의 자손은 p.shop-name, ul.menu, li, li 넷 다입니다.
자식은 자손에 포함됩니다. 자손이 더 넓은 말입니다.

형제(sibling) 부모가 같은 요소들
→ p.shop-name 과 ul.menu 는 형제입니다(부모가 둘 다 div.shop).
li 두 개도 서로 형제입니다(부모가 둘 다 ul.menu).
p.shop-name 과 li 는 형제가 아닙니다. 부모가 다릅니다.

사람 가족과 똑같습니다. 자식은 한 세대 아래, 자손은 그 아래 전부입니다. 형제는 같은 부모를 둔 사이입니다. 사촌은 형제가 아닙니다.

헛갈리면 이 한 문장만 기억하세요. "자식은 한 칸, 자손은 몇 칸이든, 형제는 옆칸"

화면 연한 회청색 상자가 하나 보이고, 그 안에 초록 글자로 "봄내 카페",

그 아래에 점이 붙은 목록 두 줄이 들어 있습니다

봄내 카페

아메리카노
카페라떼

위 상자를 들여쓰기만 남기고 그리면 이렇습니다.

| | |
|---------------------|-----------------------------|
| div class="shop" | ← 바깥 상자 |
| p class="shop-name" | ← shop 의 자식 |
| ul class="menu" | ← shop 의 자식, shop-name 과 형제 |
| li | ← menu 의 자식, shop 의 자손(손자) |
| li | ← menu 의 자식, shop 의 자손(손자) |

상자에 배경색을 준 이유는 "어디까지가 안쪽인지" 를 눈으로 보기 위해서입니다. 배경색이 칠해진 범위가 곧 div.shop 의 영역입니다. 상자에 테두리와 여백을 주면 더 잘 보이는데, 그건 10단원에서 배웁니다.

◦ 직접 해보기 1

위 상자의 ul class="menu" 안에 딸기라떼 를 하나 더 넣어 보세요. 새로 넣은 항목의 부모는 무엇이고, div.shop 과는 어떤 사이인지 말해 보세요.

자손 선택자

섹션 2

선택자 사이에 공백을 두면 “왼쪽 안에 있는 오른쪽” 입니다. 깊이는 따지지 않습니다.

개념02 에서 살짝 봤던 그 공백입니다. 이제 제대로 봅시다.

```
.notice-box p {
  ↳ 여기 공백 한 칸이 결합자입니다
  color: #c0392b;
}
```

읽는 법: “notice-box 안에 있는 p 를 전부”

여기서 ‘안에’ 는 아주 넓은 뜻입니다. 바로 안이든, 상자 두 겹 안이든, 열 겹 안이든 전부입니다. 아래 상자에서 두 번째 문단은 상자가 한 겹 더 있는데도 빨개집니다.

이게 왜 편할까요? 실제 웹페이지에서는 안쪽 구조가 자주 바뀌기 때문입니다. 나중에 div 로 한 번 더 감싸도 자손 선택자는 그대로 살아 있습니다.

반대로 위험한 점도 같습니다. 너무 넓게 걸립니다. “머리말 안의 모든 p” 라고 써 두면, 나중에 머리말 안에 새 문단을 넣을 때마다 생각지도 못한 색이 따라붙습니다.

그리고 한 번 더 강조합니다. .notice-box 자신은 빨개지지 않습니다. 색이 칠해지는 것은 오른쪽 p 입니다.

화면 위의 두 문단은 빨간 글자이고, 마지막 문단만 검은 글자입니다

알림 상자 안에 바로 있는 문단입니다.

상자 안의 또 다른 상자 안에 있는 문단입니다.

알림 상자 밖에 있는 문단입니다.

자손 선택자는 몇 단계든 이어 쓸 수 있습니다. .shop .menu li 처럼요. 다만 길어질수록 깨지기 쉽습니다. 두 단계 정도에서 끊고, 그 이상 필요하면 클래스를 하나 더 만드세요.

◀ 직접 해보기 2

위 상자의 마지막 문단(“알림 상자 밖에 있는 문단입니다”)을 <div class=“notice-box”> 안쪽 맨 아래로 옮겨 보세요. 빨개지는 것을 확인한 뒤 되돌려 놓으세요.

자식 선택자

섹션 3

> 를 쓰면 바로 한 단계 아래만 골라집니다. 손자는 걸리지 않습니다.

자손 선택자가 너무 넓어서 곤란할 때 쓰는 것이 자식 선택자입니다.

```
.crumb-box > p {  
    ↳ 부등호가 결합자입니다. 앞뒤 공백은 있어도 없어도 같습니다.  
    background-color: #fff3a0;  
}
```

읽는 법: “crumb-box 의 바로 아래 단계 p 만”

| | | |
|----------------|-------------|------|
| .crumb-box p | 안에 있으면 전부 | (자손) |
| .crumb-box > p | 바로 아래 한 단계만 | (자식) |

아래 상자를 보면, 한 겹 더 들어간 문단에는 노란 배경이 없습니다. 그 문단의 부모는 crumb-box 가 아니라 안쪽 상자이기 때문입니다.

언제 자식 선택자를 쓸까요? 목록 안에 목록이 들어가는 경우가 대표적입니다. “겉 목록의 항목만 굵게” 하고 싶는데 자손으로 쓰면 안쪽 목록까지 굵어집니다. 그때 > 로 한 단계만 잡습니다.

참고로 부등호 앞뒤의 공백은 뜻이 없습니다.

| | |
|----------------|-----------------------------|
| .crumb-box>p | 같은 뜻 |
| .crumb-box > p | 같은 뜻 (이렇게 띄어 쓰는 편이 읽기 좋습니다) |

여기서 조심할 것이 있습니다. 공백이 결합자라고 배웠는데, 부등호 옆의 공백은 결합자가 아닙니다. 부등호가 이미 결합자이기 때문입니다. 결합자는 두 선택자 사이에 하나만 옵니다.

화면 첫 문단만 노란 배경이고, 아래 문단은 배경색이 없습니다

상자의 바로 아래 단계 문단입니다.

한 겹 더 들어간 문단입니다.

▹ 직접 해보기

3

<style> 의 .crumb-box > p 에서 부등호를 지워 .crumb-box p 로 만들어 보세요. 노란 배경이 몇 줄이 되는지 세어 본 뒤 되돌려 놓으세요.

자손과 자식을 나란히

섹션 4

아래 두 상자의 HTML 구조는 완전히 같습니다. 다른 것은 CSS 의 부등호 하나뿐입니다.

말로 백 번 듣는 것보다 한 번 보는 것이 낫습니다. 아래 두 상자는 클래스 이름만 다르고 구조가 똑같습니다.

```
<div class="deep-a"> <div class="deep-b"> <p>자식 문단</p> <p>자식 문단
</p> <div> <div> <p>손자 문단</p> <p>손자 문단</p> </div> </div>
</div> </div>
```

CSS 만 다릅니다.

```
.deep-a p { background-color: #ffe9e9; }  자손 → 자식도 손자도
.deep-b > p { background-color: #e2f4ea; }  자식 → 자식만
```

결과를 세어 보세요.

위 상자 색칠된 문단 2개 (자식, 손자)
아래 상자 색칠된 문단 1개 (자식만)

손자 문단이 색칠되는가, 아닌가. 이 한 가지가 자손과 자식의 전부입니다.

왜 손자는 안 걸릴까요? 손자 문단의 부모는 deep-b 가 아니라 가운데 div 이기 때문입니다. > 는 “네 부모가 왼쪽이냐” 만 묻습니다. 할아버지는 묻지 않습니다.

화면 두 문단 모두 연한 분홍 배경입니다

자식 문단입니다.

손자 문단입니다.

화면 첫 문단만 연한 초록 배경이고, 손자 문단은 배경색이 없습니다

자식 문단입니다.

손자 문단입니다.

평소에는 자손 선택자를 기본으로 쓰고, “안쪽까지 번지면 곤란하다” 는 것을 알았을 때만 > 로 좁히세요. 처음부터 > 를 쓰면 나중에 구조를 한 겹 감쌀 때 조용히 죽습니다.

▫ 직접 해보기 4

아래 상자(deep-b)의 손자 문단을 가운데 <div> 밖으로 꺼내 자식으로 만들어 보세요. 연한 초록 배경이 두 줄이 됩니다.

인접 형제 선택자

섹션 5

A + B 는 A 바로 다음에 오는 형제 B 하나입니다. 안이 아니라 옆입니다.

지금까지 본 둘은 “안쪽” 을 봤습니다. 이번 둘은 “옆” 을 봅니다.

```
.q + p {
  ↳ 더하기가 결합자입니다
  background-color: #eef7ff;
}
```

읽는 법: “class 가 q 인 요소 바로 다음에 오는 형제 p”

1 부모가 같아야 합니다 (형제여야 합니다)

2 바로 다음이어야 합니다 (사이에 아무 요소도 없어야 합니다)

아래 상자에서 색칠된 줄을 세어 보세요. 질문 줄이 두 개니까 답 줄도 두 개가 칠해집니다. “추가 안내” 줄은 질문 바로 다음이 아니라서 안 칠해집니다.

어디에 쓸까요? “제목 바로 아래 문단만 조금 다르게” 같은 곳입니다. HTML 을 안 고치고도 위치만으로 첫 문단을 골라낼 수 있습니다.

그리고 이것도 마찬가지입니다. 색이 칠해지는 것은 오른쪽 p 입니다. 질문 줄(.q) 자체는 아무 변화가 없습니다.

화면 2번째 줄과 5번째 줄만 연한 하늘색 배경입니다.

질문 줄과 “주말에는” 줄은 배경색이 없습니다

주차할 수 있나요?
건물 뒤에 두 자리 있습니다.
주말에는 만차일 수 있습니다.
반려동물과 갈 수 있나요?
테라스 자리만 가능합니다.

더하기는 앞쪽으로는 절대 가지 않습니다. “바로 앞 형제” 를 고르는 방법은 이 자료의 범위 밖입니다. 앞을 꾸미고 싶으면 그 요소에 클래스를 하나 붙이는 것이 정답입니다.

▹ 직접 해보기 **5**

위 상자의 “주말에는 만차일 수 있습니다” 줄을 첫 번째 질문 줄 바로 위로 옮겨 보세요. 하늘색 줄이 어떻게 바뀌는지 확인하세요.

이후 형제 선택자

섹션 6

A ~ B 는 A 뒤에 오는 형제 B 를 전부 고릅니다. 바로 다음이 아니어도 됩니다.

```
.qq ~ p {
```

└ 물결이 결합자입니다

```
background-color: #e2f4ea;
}
```

읽는 법: “qq 뒤에 오는 형제 p 를 전부”
더하기와 물결의 차이는 딱 하나, 개수입니다.
A + B A 다음 첫 형제 하나만 A ~ B A 다음 형제 전부
둘 다 “형제여야 한다” 와 “뒤쪽만 본다” 는 똑같습니다.
아래 상자에서 기준 줄 뒤의 문단이 전부 초록으로 칠해집니다. 기준 줄 앞에 있는 문단은 칠해지지 않습니다.
뒤쪽만 보기 때문입니다.
물결은 자주 쓰지는 않습니다. 다만 05단원에서 배운 체크박스과 함께 쓰면 자바스크립트 없이도 “체크하면
아래가 바뀌는” 화면을 만들 수 있습니다. 그건 개념04 에서 봅니다.

화면 마지막 세 문단이 모두 연한 초록 배경입니다.

기준 줄과 그 앞 문단은 배경색이 없습니다

기준 줄보다 앞에 있는 문단입니다.
여기가 기준 줄입니다.
기준 줄 바로 다음 문단입니다.
그다음 문단입니다.
맨 마지막 문단입니다.

네 결합자를 한 문장으로 줄이면 이렇습니다. 공백은 안쪽 전부, 부등호는 안쪽 한 칸, 더하기는 옆쪽 한 칸,
물결은 옆쪽 전부입니다.

◦ 직접 해보기 6

<style> 의 .qq ~ p 를 .qq + p 로 바꿔 보세요. 초록 줄이 세 개에서 몇 개로 줄어드는지 세어 본 뒤
되돌려 놓으세요.

자주 하는 실수

섹션 7

관계 선택자의 실수는 “일부만 바뀌거나 하나도 안 바뀌는” 모양으로 나타납니다. 예러는 끝까지 안 납니다.

이런 실수를 합니다
자식으로 써서 손자를 놓침

이런 실수를 합니다

실수 1 — 자식으로 썼는데 손자까지 바꾸고 싶었음

```
.miss1 > p { color: crimson; }
```

“상자 안 문단을 전부 빨강게” 하려고 썼는데 부등호를 넣었습니다. 한 겹 더 들어간 문단은 그대로 남습니다. “왜 어떤 건 되고 어떤 건 안 되지?” 싶으면 부등호를 의심하세요. 고치려면 부등호를 지우고 공백만 남기면 됩니다. 자식 문단입니다. 빨개집니다. 손자 문단입니다. 안 빨개집니다.

이런 실수를 합니다

왼쪽도 함께 꾸며진다고 생각함

이런 실수를 합니다

실수 2 — 왼쪽 요소도 함께 바뀐다고 생각함

```
.miss2 p { color: crimson; }
```

이 규칙은 p 만 빨강게 합니다. .miss2 상자 자체나 상자에 바로 쓴 글자는 바뀌지 않습니다. 색은 언제나 맨 오른쪽 것에 칠해집니다. 상자도 같이 바꾸려면 .miss2, .miss2 p 처럼 선표로 묶어야 합니다. 상자에 바로 쓴 글자입니다. 상자 안의 문단입니다.

이런 실수를 합니다

더하기가 앞도 고른다고 생각함

이런 실수를 합니다

실수 3 — 더하기가 앞 형제도 고른다고 생각함

```
.key3 + p { background-color: #fff3a0; }
```

더하기와 물결은 뒤쪽만 봅니다. 앞은 절대 안 봅니다. 아래에서 기준 줄 앞의 문단은 아무리 기다려도 노래지지 않습니다. 앞 요소를 꾸미고 싶으면 그 요소에 클래스를 붙이세요. 기준 줄보다 앞에 있는 문단입니다. 기준 줄입니다. 기준 줄 다음 문단입니다.

이런 실수를 합니다

사이에 다른 요소가 끼어 있음

이런 실수를 합니다

실수 4 — 사이에 다른 요소가 끼어서 인접이 아니게 됨

```
.key4 + p { background-color: #fff3a0; }
```

더하기는 바로 다음만 봅니다. 아래에서는 기준 줄과 문단 사이에 가로줄(<hr />) 이 하나 있습니다. 그것 때문에 규칙이 통째로 안 걸립니다. 보이는 것이 거의 없는 요소도 순서에는 포함됩니다. 기준 줄입니다. 가로줄 때문에 안 걸리는 문단입니다.

이런 실수를 합니다

한 겹 감싸서 형제가 아니게 됨

이런 실수를 합니다

실수 5 — 한 겹 감쌌더니 형제가 아니게 됨

```
.key5 + p { background-color: #fff3a0; }
```

어제까지 잘 되던 규칙이 오늘 갑자기 안 되는 대표적인 이유입니다. 문단을 <div> 로 한 번 감쌌더니 기준 줄의 다음 형제가 p 가 아니라 div 가 되었습니다. 형제 선택자는 구조를 바꾸면 쉽게 깨집니다. 기준 줄입니다. 한 겹 감싸인 문단입니다.

이렇게 고칩니다

```
.key5 + div p { background-color: #fff3a0; }
```

정리

◦ 직접 해보기 7

정답

1) 섹션 1 상자의 ul 안에 이렇게 넣습니다.

```
<ul class="menu"> <li>아메리카노</li> <li>카페라떼</li> <li>딸기라떼</li>
</ul>
```

→ 목록이 세 줄이 됩니다.

새 li 의 부모는 ul.menu 입니다.
div.shop 과는 자손(손자) 사이입니다. 자식이 아닙니다.
그리고 나머지 li 두 개와는 형제입니다.

2) 섹션 2 상자의 마지막 문단을 notice-box 안쪽으로 옮깁니다.

```
<div class="notice-box"> <p>알림 상자 안에 바로 있는 문단입니다.
</p> <div class="notice-inner"> <p>상자 안의 또 다른 상자 안에 있는 문단입니다.
</p> </div> <p>알림 상자 밖에 있는 문단입니다.</p>
</div>
```

→ 그 문단도 빨간 글자가 됩니다. 상자 안 문단이 세 줄 모두 빨개집니다.

HTML 위치만 바꿨는데 색이 바뀌었습니다. 이것이 관계 선택자의 성질입니다.

3) style 의 규칙을 이렇게 고칩니다.

```
.crumb-box p {
    background-color: #fff3a0;
}
```

→ 노란 배경이 한 줄에서 두 줄이 됩니다.

한 겹 더 들어간 문단까지 걸리기 때문입니다.
부등호 하나로 "한 단계만" 과 "안쪽 전부" 가 걸립니다.

4) 섹션 4 아래 상자를 이렇게 고칩니다.

```
<div class="deep-b"> <p id="deepBChild">자식 문단입니다.</p> <p id="deepBGrand">손자
문단입니다.</p> <div></div>
</div>
```

→ 두 문단 모두 연한 초록 배경이 됩니다.

문단 자체는 한 글자도 안 고쳤습니다. 부모가 바뀌었을 뿐입니다.
자식 선택자는 "부모가 누구냐" 만 본다는 것을 확인하는 문제입니다.

5) 섹션 5 상자를 이렇게 고칩니다.

```
<p>주말에는 만차일 수 있습니다.</p>
<p class="q">주차할 수 있나요?</p>
<p>건물 뒤에 두 자리 있습니다.</p>
<p class="q">반려동물과 갈 수 있나요?</p>
<p>테라스 자리만 가능합니다.</p>
```

→ 하늘색 줄은 그대로 두 개입니다. 위치만 2번째와 5번째에서

3번째와 5번째로 바뀝니다.
웁긴 줄 자체는 질문 바로 다음이 아니므로 여전히 안 칠해집니다.

6) style 의 규칙을 이렇게 고칩니다.

```
.qq + p {
  background-color: #e2f4ea;
}
```

→ 초록 줄이 세 개에서 한 개로 줄어듭니다.

기준 줄 바로 다음 문단 하나만 남습니다.
더하기는 하나, 뺄셈은 전부입니다. 확인했으면 물결로 되돌리세요.

가족 용어

섹션 1

```
.shop {
  background-color: #f0f4f8;
}
.shop-name {
  color: #1a7f5a;
}
```

화면 바깥 상자가 연한 회청색으로 칠해져서 "어디까지가 상자 안인지" 보이고,

가게 이름 줄만 초록 글자입니다

자손 선택자 (공백)

섹션 2

```
.notice-box p {
  color: #c0392b;
}
```

화면 알림 상자 안에 있는 문단은 깊이에 상관없이 전부 빨간 글자가 되고,

상자 밖 문단은 검은 글자 그대로입니다

자식 선택자

섹션 3

```
.crumb-box > p {
  background-color: #fff3a0;
}
```

화면 상자의 바로 아래 단계 문단만 노란 배경이 되고,

한 겹 더 들어간 문단은 배경이 없습니다

자손과 자식을 나란히

섹션 4

```
.deep-a p {  
  background-color: #ffe9e9;  
}  
.deep-b > p {  
  background-color: #e2f4ea;  
}
```

화면 위 상자는 자식 문단과 손자 문단이 모두 분홍,

아래 상자는 자식 문단만 연한 초록이고 손자 문단은 색이 없습니다

인접 형제 선택자

섹션 5

```
.q + p {  
  background-color: #eef7ff;  
}
```

화면 질문 줄 바로 다음 줄만 연한 하늘색 배경이 됩니다

이후 형제 선택자

섹션 6

```
.qq ~ p {  
  background-color: #e2f4ea;  
}
```

화면 기준 줄 뒤에 오는 형제 문단이 전부 연한 초록 배경이 됩니다

자주 하는 실수

섹션 7

```
.miss1 > p {  
  color: crimson;  
}
```

화면 자식 문단만 빨개지고, 한 겹 더 안에 있는 손자 문단은 그대로입니다

```
.miss2 p {  
  color: crimson;  
}
```

화면 상자 안의 문단만 빨개지고, 상자에 바로 쓴 글자는 검은색 그대로입니다

```
.key3 + p {
  background-color: #fff3a0;
}
```

화면 기준 줄의 다음 줄만 노랑고, 앞 줄은 아무 변화가 없습니다

```
.key4 + p {
  background-color: #fff3a0;
}
```

화면 아무 일도 일어나지 않습니다. 기준 줄과 문단 사이에 가로줄이 끼어 있습니다

```
.key5 + p {
  background-color: #fff3a0;
}
```

화면 아무 일도 일어나지 않습니다. 다음 형제가 p 가 아니라 상자입니다

상태로 고르기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

이 파일은 읽기만 해서는 절반도 못 봅니다.

마우스를 올리고, Tab 키를 누르고, 체크박스를 눌러 보세요.

아메리카노 3000원 — 여기에 마우스를 올려 보세요

카페라떼 4000원 — 여기에도 올려 보세요

주문하기

이름 연락처

- [개념01 — 기본 선택자](#)
- [개념03 — 관계로 고르기](#)

☒ 열음을 추가합니다

☐ 시럽을 추가합니다

☒ 선물 포장을 원합니다

포장지 색은 계산대에서 골라 주세요.

실행하면 나오는 화면 — 상태로 고르기

지금까지 배운 선택자는 전부 HTML 에 적혀 있는 것을 봤습니다. 태그 이름, 클래스, id, 부모 자식 관계. 전부 파일에 고정되어 있습니다.

그런데 화면에는 파일에 안 적혀 있는 정보가 또 있습니다.

지금 마우스가 이 버튼 위에 있는가
 지금 이 입력칸에 글자를 쓰고 있는가
 이 체크박스가 지금 체크되어 있는가
 이 링크에 전에 가 본 적이 있는가

전부 “지금 어떤 상태인가” 입니다. 시시각각 바뀝니다. 이런 상태로 요소를 고르는 것이 가상 클래스입니다.

```
.btn:hover { background-color: #2d6cdf; }  
↳ 콜론 하나 뒤에 상태 이름을 씁니다
```

가상(假想) 클래스라고 부르는 이유는 이렇습니다. 마치 마우스를 올린 순간에만 class="hover" 가 붙었다가 치우면 떨어지는 것처럼 동작하기 때문입니다. 실제로 HTML 에 클래스가 붙는 것은 아니라서 '가상' 입니다.

이 파일에서 배울 상태는 일곱 가지입니다.

| | |
|----------------|-------------------------|
| :hover | 마우스가 위에 올라와 있는 동안 |
| :active | 마우스로 누르고 있는 동안 |
| :focus | 지금 키보드 입력을 받고 있는 칸 |
| :focus-visible | 키보드로 이동해서 focus 가 된 경우만 |
| :visited | 전에 가 본 적이 있는 링크 |
| :checked | 체크되어 있는 체크박스 |
| :disabled | 쓸 수 없게 막아 둔 입력칸이나 버튼 |

자바스크립트 없이 화면이 반응하게 만드는 방법이 바로 이것입니다.

이 파일은 눈으로 읽기만 하면 절반도 못 봅니다. 아래 상자들 위에 마우스를 올려 보세요. 입력칸은 Tab 키로 옮겨 다녀 보세요. 체크박스는 직접 눌러 보세요. 손을 움직여야 보이는 색이 이 파일의 내용 전부입니다.

:hover

섹션 1

아래 상자 위에 마우스를 올려 보세요. 올린 동안만 색이 바뀌고, 치우면 돌아옵니다.

가장 많이 쓰는 가상 클래스입니다. 규칙은 두 개를 짝으로 씁니다.

```
.hover-card {  
  background-color: #eef2f7;    평소 모습  
}  
.hover-card:hover {  
  background-color: #2d6cdf;    마우스가 올라와 있는 동안  
  color: white;  
}
```

읽는 법: "hover-card 인데, 마우스가 올라와 있는 상태인 것"

콜론 앞뒤에 공백을 넣으면 안 됩니다. 반드시 붙여 씁니다.

| | |
|--------------------|----------------------------|
| .hover-card:hover | 맞음 |
| .hover-card :hover | 틀림 (뜻이 완전히 달라집니다. 섹션 7 참고) |

왜 :hover 를 쓸까요? “이건 눌러도 되는 것이다” 를 알려 주기 위해서입니다. 마우스를 올렸을 때 아무 반응이 없으면 사람은 그것을 그냥 글자로 봅니다. 색이 살짝 바뀌면 “아, 여기를 누를 수 있구나” 하고 알아챱니다.

주의할 점이 하나 있습니다. 휴대폰과 태블릿에는 마우스가 없습니다. 손가락은 ‘올려 두는’ 동작이 없습니다. 그래서 :hover 에만 중요한 정보를 담으면 안 됩니다. :hover 는 거들 뿐이고, 없어도 쓸 수 있어야 합니다.

화면 평소에는 두 줄 다 연한 회청색 배경입니다.

마우스를 올린 줄만 진한 파란 배경에 흰 글자가 되고, 마우스를 치우면 즉시 연한 회청색으로 돌아옵니다

아메리카노 3000원 - 여기에 마우스를 올려 보세요
카페라떼 4000원 - 여기에도 올려 보세요

마우스를 올리는 순간과 치우는 순간에 색이 딱딱 끊어져서 바뀝니다. 부드럽게 넘어가게 하려면 transition 이 필요한데, 이 자료의 범위 밖입니다.

◦ 직접 해보기

1

<style> 의 .hover-card:hover 규칙에서 color: white; 줄을 지우고 마우스를 올려 보세요. 글자가 잘 보이는지 확인한 뒤 되돌려 놓으세요.

:active

섹션 2

아래 버튼을 누른 채로 잠깐 있어 보세요. 마우스 왼쪽 버튼을 떼면 색이 돌아옵니다.

:active 는 “지금 누르고 있는 중” 입니다. 마우스 버튼을 누른 순간부터 뗄 때까지의 짧은 시간입니다.

```
.press-btn:active {  
  background-color: #c0392b;  
  color: white;  
}
```

아래 버튼에는 규칙이 세 개 걸려 있습니다.

| | | |
|-------------------|-----------|-----------|
| .press-btn | 평소 | 연한 회청색 |
| .press-btn:hover | 올린 동안 | 조금 진한 회청색 |
| .press-btn:active | 누르고 있는 동안 | 빨강 + 흰 글자 |

순서가 중요합니다. :active 를 :hover 보다 먼저 쓰면 누르고 있어도 빨간색이 안 보입니다. 누르고 있는 동안에는 마우스가 위에 올라와 있기도 해서 두 규칙이 동시에 맞기 때문입니다. 같은 세기의 규칙이 동시에 맞으면 나중에 쓴 것이 이깁니다. 이 이야기가 개념06 의 명시도입니다.

그래서 :hover 를 먼저 쓰고 :active 를 나중에 씁니다.

화면 평소에는 연한 회청색 버튼입니다.

마우스를 올리면 살짝 진해지고, 누르고 있는 동안만 빨간 배경에 흰 글자가 됩니다. 떼면 곧바로 돌아옵니다

주문하기

:active 는 보이는 시간이 아주 짧습니다. 그래서 "눌렀다" 는 느낌을 주는 용도로만 씁니다. 여기에 중요한 정보를 담으면 아무도 못 읽습니다.

▸ 직접 해보기

2

<style> 에서 .press-btn:active 규칙을 .press-btn:hover 규칙보다 위로 옮겨 보세요. 눌러도 빨개지지 않습니다. 확인한 뒤 되돌려 놓으세요.

:focus 와 :focus-visible

섹션 3

아래 입력칸을 클릭하거나 Tab 키를 눌러 보세요. 지금 글자가 들어갈 칸 하나만 색이 바뀝니다.

키보드로 글자를 치면 그 글자는 어디로 갈까요? 지금 '초점' 이 맞춰진 곳으로 갑니다. 그것이 focus 입니다.

```
.focus-input:focus {
  background-color: #fff3a0;
}
```

한 화면에 focus 를 가진 요소는 많아야 하나입니다. 입력칸을 클릭하면 그리로 옮겨 가고, Tab 키를 누르면 다음 칸으로 넘어갑니다.

이것이 왜 중요할까요? 마우스를 못 쓰는 사람은 Tab 키만으로 화면 전체를 돌아다닙니다. 지금 어디에 있는지 안 보이면 길을 잃습니다. 그래서 브라우저는 기본으로 focus 된 곳에 테두리를 그려 줍니다. 그 테두리를 지우고 아무 표시도 안 남기는 것은 아주 나쁜 습관입니다.

:focus 와 :focus-visible 의 차이는 이렇습니다.

| | |
|----------------|---|
| :focus | 어떤 방법으로든 초점이 온 경우 (클릭 포함) |
| :focus-visible | 브라우저가 "이건 표시해 줘야 한다" 고 판단한 경우
대체로 키보드로 옮겨 왔을 때만 켜집니다 |

버튼을 마우스로 클릭하면 focus 는 오지만, 사용자는 자기가 어디를 눌렀는지 이미 압니다. 이때 붉은 테두리가 남으면 지저분해 보입니다. 그래서 요즘은 버튼에 :focus-visible 을 씁니다.

아래에서 직접 비교해 보세요.

| | |
|-----|--|
| 입력칸 | : 클릭해도 노랗고 Tab 으로 가도 노랗습니다 (:focus) |
| 버튼 | : Tab 으로 가야만 노랗습니다. 클릭해서는 안 노랗습니다 (:focus-visible) |

화면 평소에는 입력칸 두 개가 아주 연한 회색 배경입니다.

클릭하거나 Tab 으로 옮겨 간 칸 하나만 노란 배경이 됩니다. 버튼은 Tab 으로 도착했을 때만 노래지고, 마우스로 클릭했을 때는 노래지지 않습니다

이름

연락처

확인

입력칸을 클릭한 다음 Tab 을 계속 눌러 보세요. 노란 칸이 오른쪽으로 한 칸씩 옮겨 갑니다. Shift + Tab 을 누르면 거꾸로 갑니다.

▸ 직접 해보기 3

.focus-btn:focus-visible 을 .focus-btn:focus 로 바꾼 뒤, 버튼을 마우스로 클릭해 보세요. 아까와 무엇이 달라지는지 확인하고 되돌려 놓으세요.

링크의 상태

섹션 4

링크는 가 봤는지 안 가 봤는지로도 고를 수 있습니다. 아래 링크를 눌러 보고 뒤로 가기로 돌아와 보세요.

04단원에서 만든 a 요소에는 상태가 두 가지 더 있습니다.

a:link 아직 가 본 적 없는 링크
a:visited 전에 가 본 적이 있는 링크

브라우저가 기억하는 방문 기록을 보고 정합합니다. 가 본 링크가 보라색으로 변하는 그 동작이 바로 :visited 입니다. 기본값이 그렇게 정해져 있을 뿐, 우리가 바꿀 수 있습니다.

```
.link-list a:link      { color: #2d6cdf; }  
.link-list a:visited { color: #7b3fa0; }  
.link-list a:hover    { background-color: #fff3a0; }
```

아래 링크 두 개는 같은 폴더의 다른 수업 파일로 갑니다. 눌러서 다녀온 다음 브라우저의 뒤로 가기(Alt + ←)로 돌아오면 그 링크만 보라색으로 바뀌어 있습니다.

:visited 에는 큰 제약이 하나 있습니다. 바꿀 수 있는 것이 색 계열뿐입니다. 배경색이나 크기는 못 바꿉니다. 방문 기록은 사생활이기 때문입니다. 만약 크기를 바꿀 수 있다면, 웹사이트가 그 크기를 재서 “이 사람이 어느 사이트에 가 봤는지” 를 알아낼 수 있게 됩니다. 그래서 브라우저가 일부러 막아 두었습니다.

규칙을 쓰는 순서도 정해져 있습니다.

a:link → a:visited → a:hover → a:active

이 순서를 지키지 않으면 뒤에 쓴 규칙에 가려집니다. 전부 세기가 같아서 나중에 쓴 것이 이기기 때문입니다 (개념06).

화면 처음 열면 두 링크가 모두 파란 글자에 밑줄입니다.

마우스를 올리면 노란 형광펜이 칠해집니다. 다녀온 링크는 보라색 글자로 바뀝니다

개념01 - 기본 선택자
개념03 - 관계로 고르기

밑줄은 우리가 그린 것이 아니라 브라우저가 원래 링크에 그려 두는 것입니다. 밑줄을 없애는 text-decoration 은 09단원에서 배웁니다. 함부로 없애지 마세요. 색만으로는 링크인 줄 모르는 사람이 있습니다.

▫ 직접 해보기 4

.link-list a:visited 의 색을 #c0392b(빨강) 으로 바꾸고, 링크를 다녀와 보세요. 가 본 링크가 빨강게 바뀝니다.

:checked

섹션 5

아래 체크박스를 눌러 보세요. 자바스크립트 없이 화면이 바뀝니다.

05단원에서 만든 체크박스입니다. 체크된 상태를 :checked 로 고릅니다.

```
.opt:checked + label {
  background-color: #fff3a0;
  color: #7a4b00;
}
```

여기서 개념03의 형제 선택자가 등장합니다. 읽는 법: “체크된 opt 바로 다음에 오는 형제 label”

왜 label 을 고를까요? 체크박스 자체는 색을 칠해도 거의 안 보입니다. 브라우저가 그리는 작은 네모라서 배경색이 먹지 않습니다. 그래서 체크박스 옆의 글자(label)를 대신 꾸밉니다. 이 방법이 실무에서 쓰는 정석입니다.

아래 첫 상자에는 체크박스가 두 개 있습니다. 하나는 처음부터 체크되어 있고:checked 를 써 두었습니다) 하나는 아닙니다. 직접 눌러서 켜고 끄면 노란 배경이 따라옵니다.

label 의 for 값과 input 의 id 값이 같아야 한다는 것도 기억하세요(05단원). 그래야 글자를 눌러도 체크가 됩니다.

두 번째 상자에서는 물결 선택자를 씁니다.

```
.opt:checked ~ .gift-note { background-color: #e2f4ea; }
```

“체크된 opt 뒤에 오는 형제 중 gift-note” 라는 뜻입니다. 체크를 풀면 아래 안내문의 초록 배경이 사라집니다. 자바스크립트를 한 줄도 안 쓰고 화면이 반응합니다.

화면

처음 열면 위 칸만 체크되어 있고 "얼음을 추가합니다" 글자에

노란 배경과 갈색 글자가 칠해져 있습니다. 아래 칸을 누르면 "시럽을 추가합니다" 에도 노란 배경이 생깁니다

얼음을 추가합니다

시럽을 추가합니다

화면

처음 열면 체크가 되어 있어서 아래 안내문이 연한 초록 배경입니다.

체크를 풀면 초록 배경이 사라집니다

선물 포장을 원합니다

포장지 색은 계산대에서 골라 주세요.

:checked 는 type="radio" 에도 똑같이 씁니다. 고른 항목 하나만 표시하고 싶을 때 쓰면 됩니다.

▣ 직접 해보기 5

두 번째 상자의 <input> 에서 checked 를 지우고 새로고침해 보세요. 안내문의 초록 배경이 처음부터 없는 상태로 열립니다.

:disabled 와 속성으로 고르기

섹션 6

쓸 수 없게 막아 둔 칸은 :disabled 로 고릅니다. 속성 값으로 고르는 방법도 함께 봅니다.

05단원에서 input 과 button 에 disabled 를 붙이면 놀리지 않게 된다고 배웠습니다. 그 상태를 :disabled 로 고릅니다.

```
.form-box input:disabled,  
.form-box button:disabled {  
  background-color: #e6e6e6;  
  color: #8a8a8a;  
}
```

아래 상자에서 두 번째 입력칸과 두 번째 버튼을 눌러 보세요. 아무 반응이 없습니다. 글자도 못 씁니다. 회색으로 칠한 것은 "이건 지금 못 씁니다" 를 눈으로 알려 주기 위해서입니다. 반대말인 :enabled 도 있지만 거의 안 씁니다. 기본 상태니까요.

여기서 결다리로 하나 더 배웁니다. 속성 선택자입니다.

```
.attr-box input[type="text"] {  
  background-color: #eef7ff;  
}
```

대괄호 안에 “속성 이름=값” 을 씁니다. 읽는 법: “attr-box 안의 input 중에서 type 이 text 인 것”

input 은 태그 이름이 전부 input 이라서 태그 선택자로는 구분이 안 됩니다. 글자 칸도 input, 체크박스도 input, 버튼도 input 입니다. 그래서 type 속성 값으로 갈라 줘야 합니다. 속성 선택자가 꼭 필요한 자리입니다.

값을 안 쓰고 속성이 있는지만 볼 수도 있습니다.

disabled · disabled 라고 적혀 있거나 하면

input[type] type 속성이 있거나 하면

disabled · 와 :disabled 는 결과가 거의 같습니다.

다만 [disabled] 는 “HTML 에 그렇게 적혀 있는가” 를 보고, :disabled 는 “지금 실제로 못 쓰는 상태인가” 를 봅니다. 상태를 볼 때는 :disabled 쪽이 정확합니다.

화면 첫 번째 입력칸과 첫 번째 버튼은 평범합니다.

회원번호 칸과 품질 버튼만 회색 배경에 흐린 회색 글자이고, 눌러도 아무 반응이 없습니다

주문자
회원번호
주문하기
품질

화면 글자 입력칸만 연한 하늘색 배경이고, 체크박스는 원래 모습 그대로입니다

쿠폰 번호
약관에 동의합니다

속성 선택자는 [href], [required] 처럼 어디에나 쓸 수 있습니다. 값에는 따옴표를 붙이는 것이 안전합니다. [type=text] 도 동작하지만 값에 특수문자가 들어가면 깨집니다.

➤ 직접 해보기 6

아래 상자의 체크박스에도 disabled 를 붙여 보세요. (<input type=“checkbox” id=“agreeChk” disabled />) 눌러도 체크가 안 되는 것을 확인한 뒤 지우세요.

가상 클래스의 실수는 콜론 한 글자에서 거의 다 나옵니다. 역시 에러는 나지 않습니다.

이런 실수를 합니다

콜론을 빠뜨림

이런 실수를 합니다

실수 1 — 콜론을 빠뜨림

```
.no-colon hover { background-color: #fff3a0; }
```

콜론이 없으면 hover 가 태그 이름으로 읽힙니다. “no-colon 안에 있는 hover 태그” 를 찾는데 그런 태그는 없습니다. 마우스를 아무리 올려도 아무 일이 없습니다. 마우스를 올려도 아무 일이 없습니다.

이렇게 쓰세요

```
.no-colon: hover { background-color: #fff3a0; }
```

이런 실수를 합니다

콜론 앞에 공백을 넣음

이런 실수를 합니다

실수 2 — 콜론 앞에 공백을 넣음

```
.space-colon :checked + label { background-color: #fff3a0; }
```

공백은 개념03 에서 배운 자손 결합자입니다. 그래서 이 규칙은 “space-colon 안에 있는 체크된 요소” 를 찾습니다. 그런데 space-colon 은 체크박스 자신입니다. 체크박스 안에는 아무것도 없으니 한 개도 못 찾습니다. 아래 칸은 분명히 체크되어 있는데도 글자에 배경이 없습니다. 체크되어 있는데 배경이 없습니다.

이렇게 쓰세요

```
.space-colon:checked + label { background-color: #fff3a0; }
```

이런 실수를 합니다

콜론을 두 개 씀

이런 실수를 합니다

실수 3 — 콜론을 두 개 씀

```
.double-colon::checked + label { background-color: #fff3a0; }
```

콜론 두 개는 가상 요소라는 다른 문법입니다. ::checked 같은 가상 요소는 없으므로 이 선택자는 잘못된 것이 되고, 규칙이 통째로 버려집니다. 아래 칸도 체크되어 있지만 그대로입니다. 여기도 배경이 없습니다.

이런 실수를 합니다

링크 규칙의 순서를 어김

이런 실수를 합니다

실수 4 — 링크 규칙을 순서 없이 씀

```
a:hover { color: crimson; }
a:link { color: #2d6cdf; }
a:visited { color: #7b3fa0; }
```

링크의 상태 규칙은 a:link 부터 a:active 까지 네 개가 모두 세기가 같습니다. 그래서 나중에 쓴 것이 이깁니다. 위처럼 쓰면 마우스를 올려도 a:link 의 파란색에 가려집니다. “hover 가 안 먹어요” 의 절반은 이 순서 문제입니다. link · visited · hover · active 순서로 쓰세요.

이렇게 쓰세요

```
a:link { color: #2d6cdf; }
a:visited { color: #7b3fa0; }
a:hover { color: crimson; }
a:active { color: #c0392b; }
```

이런 실수를 합니다

:visited 로 배경색을 바꾸려 함

이런 실수를 합니다

실수 5 — :visited 로 배경색이나 크기를 바꾸려 함

```
a:visited { background-color: #fff3a0; }
```

이 규칙은 문법이 맞는데도 브라우저가 일부러 무시합니다. :visited 로 바꿀 수 있는 것은 글자색 계열뿐입니다. 방문 기록이 새어 나가는 것을 막기 위한 조치입니다. “왜 이것만 안 되지?” 하고 오래 헤매기 딱 좋은 자리입니다.

정답

1) style 의 규칙을 이렇게 고칩니다.

```
.hover-card:hover {
  background-color: #2d6cdf;
}
```

→ 마우스를 올리면 배경은 진한 파랑이 되는데 글자는 검은색 그대로라

잘 안 읽힙니다. 배경을 진하게 바꿀 때는 글자색도 같이 바뀌어야 한다는 것을 눈으로 확인하는 문제입니다. 확인했으면 `color: white;` 를 되살리세요.

2) style 에서 두 규칙의 순서를 이렇게 바꿉니다.

```
.press-btn:active { background-color: #c0392b; color: white; }
.press-btn:hover { background-color: #dbe6f7; }
```

→ 버튼을 눌러도 빨개지지 않습니다.

누르고 있는 동안에는 마우스가 위에 있기도 해서 `:hover` 규칙도 함께 맞는데, 세기가 같으면 나중에 쓴 것이 이기기 때문입니다.

한 가지 더 보입니다. 누르는 동안 글자가 안 보이게 됩니다.

배경색은 `:hover` 의 연한 파랑이 이겼는데, `:hover` 에는 `color` 가 없어서 `:active` 의 흰 글자만 그대로 살아남기 때문입니다.

연한 파랑 위의 흰 글자라 거의 안 읽힙니다.

승부는 규칙 통째로가 아니라 속성 하나하나마다 따로 난다는 뜻입니다.

확인했으면 `:hover` 를 위로, `:active` 를 아래로 되돌리세요.

3) style 의 규칙을 이렇게 고칩니다.

```
.focus-btn:focus {
  background-color: #fff3a0;
}
```

→ 이번에는 버튼을 마우스로 클릭해도 노란 배경이 남습니다.

다른 곳을 클릭해야 사라집니다.

`:focus` 는 클릭이든 키보드든 가리지 않고 켜지기 때문입니다.

버튼에는 대체로 `:focus-visible` 이 더 깔끔합니다.

4) style 의 규칙을 이렇게 고칩니다.

```
.link-list a:visited {
  color: #c0392b;
}
```

→ 링크를 눌러 다녀온 뒤 뒤로 가기로 돌아오면 그 링크만 빨간 글자입니다.

아직 안 가 본 링크는 파란색 그대로입니다.
같은 a 요소인데 상태에 따라 다른 규칙이 걸린다는 것을 확인하는 문제입니다.

5) 두 번째 상자의 input 을 이렇게 고칩니다.

```
<input type="checkbox" class="opt" id="giftCheck" />
```

→ 새로고침하면 체크가 풀린 상태로 열리고,

아래 안내문 "포장지 색은 계산대에서 골라 주세요." 에
연한 초록 배경이 없습니다.
체크박스를 직접 누르면 그 자리에서 초록 배경이 생깁니다.

6) 마지막 상자의 체크박스를 이렇게 고칩니다.

```
<input type="checkbox" id="agreeChk" disabled />
```

→ 체크박스가 흐릿해지고, 눌러도 체크가 되지 않습니다.

옆의 "약관에 동의합니다" 글자를 눌러도 마찬가지입니다.
다만 이 체크박스는 type 이 text 가 아니라서
하늘색 배경 규칙에는 걸리지 않습니다. 확인했으면 disabled 를 지우세요.

:hover

섹션 1

```
.hover-card {
  background-color: #eef2f7;
}
.hover-card:hover {
  background-color: #2d6cdf;
  color: white;
}
```

화면 평소에는 연한 회청색 상자입니다.

마우스를 올린 동안만 진한 파란 배경에 흰 글자로 바뀌고,

치우면 곧바로 원래대로 돌아옵니다

```
.press-btn {
  background-color: #eef2f7;
  color: #222;
}
.press-btn:hover {
  background-color: #dbe6f7;
}
.press-btn:active {
  background-color: #c0392b;
  color: white;
}
```

화면 평소에는 연한 회청색 버튼입니다.

마우스를 올리면 조금 진해지고,
누르고 있는 동안만 빨간 배경에 흰 글자가 됩니다.

손을 떼면 다시 마우스 올린 색으로 돌아옵니다

```
.focus-input {
  background-color: #f7f7f7;
}
.focus-input:focus {
  background-color: #fff3a0;
}
.focus-btn {
  background-color: #eef2f7;
  color: #222;
}
.focus-btn:focus-visible {
  background-color: #fff3a0;
}
```

화면 Tab 키를 눌러 입력칸으로 옮겨 가면 그 칸만 노란 배경이 됩니다.

버튼은 Tab 으로 갔을 때만 노래지고, 마우스로 클릭했을 때는 노래지지 않습니다

링크의 상태

섹션 4

```
.link-list a:link {
  color: #2d6cdf;
}
.link-list a:visited {
  color: #7b3fa0;
}
.link-list a:hover {
  background-color: #fff3a0;
}
```

화면 아직 안 가 본 링크는 파란 글자입니다.

한 번 다녀오면 보라색 글자로 바뀝니다.

마우스를 올리면 노란 형광펜이 칠해집니다

:checked

섹션 5

```
.opt:checked + label {
  background-color: #fff3a0;
  color: #7a4b00;
}
.opt:checked ~ .gift-note {
  background-color: #e2f4ea;
}
```

화면 체크된 칸의 글자만 노란 배경이 됩니다.

체크를 풀면 그 자리에서 노란 배경이 사라집니다

:disabled 와 속성 선택자

섹션 6

```
.form-box input:disabled,
.form-box button:disabled {
  background-color: #e6e6e6;
  color: #8a8a8a;
}
```

화면 쓸 수 없게 막아 둔 입력칸과 버튼만 회색 배경에 흐린 글자가 됩니다

```
.attr-box input[type="text"] {
  background-color: #eef7ff;
}
```

화면 글자 입력칸만 연한 하늘색 배경이 되고, 체크박스는 그대로입니다

자주 하는 실수 (일부러 틀리게 쓴 규칙들)

섹션 7

```
.no-colon hover {
  background-color: #fff3a0;
}
```

화면 아무 일도 일어나지 않습니다.

콜론이 없어서 "no-colon 안의 hover 라는 태그" 를 찾고 있습니다

```
.space-colon :checked + label {
  background-color: #fff3a0;
}
```

화면 아무 일도 일어나지 않습니다.

콜론 앞의 공백 때문에 "space-colon 안에 있는 체크된 것" 이 되었습니다

```
.double-colon::checked + label {
  background-color: #fff3a0;
}
```

화면 아무 일도 일어나지 않습니다.

콜론을 두 개 쓰면 다른 문법이 되어 규칙이 통째로 버려집니다

순서로 고르기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 순서로 고르기

목록을 만들다 보면 이런 요구가 나옵니다.

"표에 한 줄씩 건너뛰며 배경색을 갈아 주세요"
"메뉴의 마지막 항목만 구분선을 빼 주세요"
"첫 문단만 조금 다르게 보이게 해 주세요"

전부 순서 이야기입니다. 그런데 여기에 클래스를 일일이 붙이면 항목이 하나 늘어날 때마다 HTML 을 전부 고쳐야 합니다. 그래서 CSS 가 대신 세어 줍니다.

| | |
|-----------------|-------------------|
| :first-child | 부모의 첫 번째 자식 |
| :last-child | 부모의 마지막 자식 |
| :nth-child(n) | 부모의 n 번째 자식 |
| :nth-of-type(n) | 같은 태그끼리만 세어서 n 번째 |
| :not(...) | 괄호 안에 해당하지 않는 것 |

전부 개념04 와 같은 가상 클래스입니다. 콜론 하나로 씁니다.

이 파일에는 CSS 전체에서 가장 유명한 함정이 들어 있습니다.

:first-child 는 "그 태그 중 첫 번째" 가 아닙니다.
"부모의 첫 자식인데, 마침 그 태그이기도 한 것" 입니다.

이 둘의 차이 때문에 "규칙을 제대로 썼는데 아무 일도 안 일어나는" 일이 수도 없이 일어납니다. 섹션 2 에서 반례를 직접 봅시다.

이 파일은 몇 번째인지로 고르는 법을 다룹니다. 상자마다 색칠된 줄이 몇 번째인지 세면서 읽으세요. 섹션 2 는 이 단원에서 가장 유명한 함정이니 천천히 보세요.

첫째와 막내

섹션 1

부모의 첫 자식과 마지막 자식을 고릅니다.

```
.rank-list li:first-child { background-color: #fff3a0; }  
.rank-list li:last-child { background-color: #eef7ff; }
```

읽는 법: "rank-list 안의 li 중에서, 부모의 첫 자식인 것"

중요한 것은 기준이 부모라는 점입니다. "목록의 첫 줄" 이 아니라 "부모의 첫 자식" 입니다. 지금은 li 의 부모가 ul 이고 ul 안에는 li 만 있으니 결과가 같습니다. 하지만 섹션 2 에서 이 차이가 사고로 돌아옵니다.

이런 것을 왜 쓸까요? 항목이 늘고 줄어도 CSS 를 안 고쳐도 되기 때문입니다. 직접 첫 줄에 class="first" 를 붙이면, 나중에 항목을 위에 하나 추가할 때 그 클래스를 옮겨 달아야 합니다. 잊으면 그대로 어긋납니다. :first-child 는 브라우저가 매번 다시 세어 주니 그럴 일이 없습니다.

화면 첫 줄은 노란 배경, 마지막 줄은 연한 하늘색 배경이고,

가운데 두 줄은 배경색이 없습니다

1등 - 아메리카노
2등 - 카페라떼
3등 - 딸기라떼
4등 - 자몽차

항목이 하나뿐이면 그 하나가 첫 자식이자 마지막 자식입니다. 두 규칙이 다 걸립니다. 그때는 나중에 쓴 규칙의 색이 보입니다(개념06).

▸ 직접 해보기 1

위 목록 맨 아래에 5등 — 유자차 를 추가해 보세요. CSS 를 한 글자도 안 고쳤는데 하늘색 줄이 어디로 옮겨 가는지 확인하세요.

:first-child의 함정

섹션 2

p:first-child는 “p 중에서 첫 번째”가 아닙니다. “첫 자식인데 그게 p인 경우”입니다.

선택자는 왼쪽부터 오른쪽으로 조건을 덧붙여 읽습니다.

```
p:first-child
|  ↳ 조건 2: 부모의 첫 자식이어야 한다
|  ↳ 조건 1: p 여야 한다
```

두 조건을 동시에 만족해야 합니다. 순서가 없습니다. “p만 모아 놓고 그중 첫 번째”가 아니라 “첫 자식을 데려왔더니 마침 p였다”입니다.

그래서 이런 HTML에서는 한 개도 안 걸립니다.

```
<div class="trap-box">
  <h3>오늘의 메뉴</h3>      ← 이게 첫 자식입니다. 그런데 p가 아닙니다.
  <p>아메리카노</p>        ← p이지만 첫 자식이 아닙니다.
  <p>카페라떼</p>
</div>
```

첫 자식은 h3이고 p가 아닙니다. p들은 p이지만 첫 자식이 아닙니다. 둘 다 만족하는 요소가 하나도 없으니 규칙은 아무 일도 안 합니다.

아래에 상자를 세 개 두었습니다. HTML이 조금씩 다릅니다.

- 1 h3가 있는 상자 + p:first-child → 한 줄도 안 칠해짐
- 2 h3가 없는 상자 + p:first-child → 첫 문단이 칠해짐
- 3 h3가 있는 상자 + p:first-of-type → 첫 문단이 칠해짐

1번과 2번의 차이는 CSS가 아니라 HTML입니다. 규칙은 똑같습니다. 제목 한 줄을 넣었다는 이유로 결과가 완전히 달라졌습니다.

3번이 우리가 원하던 것입니다. :first-of-type은 “같은 태그끼리만 세어서 첫 번째”라는 뜻입니다. 형제 중 p만 골라 놓고 그중 첫 번째를 잡습니다.

정리하면 이렇습니다.

| | | |
|-----------------|------------------------------------|--------------|
| p:first-child | 첫 자식인데 그게 p인 경우 | (제목이 있으면 실패) |
| p:first-of-type | p들 중 첫 번째 | (제목이 있어도 성공) |
| 화면 | 세 줄 모두 배경색이 없습니다. 규칙이 한 개도 안 걸렸습니다 | |

08

오늘의 메뉴
아메리카노
카페라떼

화면 첫 문단만 노란 배경입니다

아메리카노
카페라떼

화면 제목이 있는데도 첫 문단이 연한 초록 배경입니다

오늘의 메뉴
아메리카노
카페라떼

1번 상자를 보고 “CSS 를 잘못 썼나?” 하며 색 이름을 고치고 속성 이름을 고치다가 한 시간을 보내는 일이 정말 많습니다. 범인은 CSS 가 아니라 HTML 의 h3 한 줄입니다. :first-child 가 안 먹으면 바로 위 형제를 보세요.

⇒ 직접 해보기

2

1번 상자에서 <h3>오늘의 메뉴</h3> 줄을 지워 보세요. CSS 는 그대로인데 첫 문단이 노랗게 바뀝니다. 확인했으면 h3 를 되살리세요.

:nth-child

섹션 3

괄호 안에 숫자·키워드·수식을 넣어 원하는 순서를 고릅니다.

괄호 안에 넣을 수 있는 것은 세 종류입니다.

| | | |
|-----|------------------|-------------------------|
| 숫자 | :nth-child(2) | 두 번째 하나 |
| 키워드 | :nth-child(odd) | 홀수 번째 전부 (1, 3, 5, ...) |
| | :nth-child(even) | 짝수 번째 전부 (2, 4, 6, ...) |
| 수식 | :nth-child(3n+1) | 1, 4, 7, ... (세 칸마다) |

먼저 꼭 기억할 것. 순서는 1 부터 쉰다. 0 이 아닙니다. 프로그래밍을 조금 해 본 사람이 가장 많이 틀리는 곳입니다.

수식 읽는 법을 봅니다. n 자리에 0, 1, 2, 3, ... 을 차례로 넣어 계산합니다.

| | | |
|------|--------------------|-----------------------------|
| 2n | → 0, 2, 4, 6, ... | 0 은 없는 순서라 무시됩니다. 결국 짝수입니다. |
| 2n+1 | → 1, 3, 5, 7, ... | 홀수입니다. |
| 3n+1 | → 1, 4, 7, 10, ... | 세 칸마다 하나씩입니다. |
| n+3 | → 3, 4, 5, 6, ... | 세 번째부터 끝까지 전부입니다. |

마지막 $n+3$ 을 조심하세요. “세 칸마다” 가 아니라 “세 번째부터” 입니다. n 앞에 숫자가 없으면 $1n$ 이라는 뜻이라서 한 칸씩 다 걸립니다.

odd 는 $2n+1$ 과 같고, even 은 $2n$ 과 같습니다. 읽기 쉬우니 홀짝은 키워드를 쓰세요.

표에 줄무늬를 까는 것이 `:nth-child` 의 대표 용도입니다. 줄이 많은 표는 눈이 옆줄로 새기 쉬운데, 한 줄씩 색을 깔면 훨씬 잘 읽힙니다.

| | |
|--|-------------------------------------|
| 화면 | 두 번째 줄만 노란 배경입니다 |
| 첫 번째 줄
두 번째 줄
세 번째 줄 | |
| 화면 | 1, 3, 5번 줄에 연한 회청색 배경이 깔려 줄무늬로 보입니다 |
| 1번 줄
2번 줄
3번 줄
4번 줄
5번 줄 | |
| 화면 | 1, 4, 7번 줄에 연한 초록 배경이 깔립니다 |
| 1번 줄
2번 줄
3번 줄
4번 줄
5번 줄
6번 줄
7번 줄 | |

`:nth-last-child(1)` 처럼 뒤에서부터 세는 것도 있습니다. `:last-child` 와 같은 뜻입니다. 뒤에서 두 번째가 필요할 때 씁니다.

▢ 직접 해보기 3

세 번째 상자의 $3n+1$ 을 $3n$ 으로 바꿔 보세요. 초록 줄이 어느 줄로 옮겨 가는지 세어 본 뒤 되돌려 놓으세요.

:nth-child 와 :nth-of-type

섹션 4

세는 대상이 다릅니다. 하나는 형제 전부를 세고, 하나는 같은 태그끼리만 셉니다.

아래 두 상자의 HTML 은 완전히 같습니다. CSS 만 다릅니다.

```

<div>
  <h3>오늘의 메뉴</h3>      ← 자식 1번, p 로 세면 해당 없음
  <p>아메리카노</p>         ← 자식 2번, p 중에서는 1번
  <p>카페라떼</p>           ← 자식 3번, p 중에서는 2번
  <p>딸기라떼</p>           ← 자식 4번, p 중에서는 3번
</div>

```

번호가 두 줄로 매겨지는 것이 핵심입니다.

```

:nth-child(2)    형제 전부를 세어 2번 → 아메리카노
:nth-of-type(2)  p 끼리만 세어 2번   → 카페라떼

```

한 줄씩 어긋납니다. 제목이 하나 끼어 있기 때문입니다.

어느 쪽을 써야 할까요? “문단 중에서 두 번째” 처럼 태그를 기준으로 말하고 있다면 :nth-of-type 입니다.
 “위에서 두 번째 자리” 처럼 자리를 말하고 있다면 :nth-child 입니다.

표나 목록처럼 자식이 전부 같은 태그면 둘의 결과가 같습니다. 그래서 목록에서는 아무거나 써도 됩니다.
 섞여 있을 때만 갈립니다.

마찬가지로 :first-of-type, :last-of-type 도 있습니다. 섹션 2 에서 함정을 피하는 데 쓴 것이 바로 :first-of-type 입니다.

화면 "아메리카노" 줄이 연한 분홍 배경입니다

오늘의 메뉴
아메리카노
카페라떼
딸기라떼

화면 "카페라떼" 줄이 연한 초록 배경입니다. 한 줄 아래입니다

오늘의 메뉴
아메리카노
카페라떼
딸기라떼

헛갈리면 이렇게 외우지 말고 세어 보세요. child 는 형제를 전부 세고, of-type 은 같은 태그만 셉니다.
 자식이 전부 같은 태그라면 둘은 같은 결과입니다.

▹ 직접 해보기

4

두 번째 상자의 <h3> 를 지워 보세요. 연한 초록 배경이 옮겨 가는지, 그대로인지 확인하세요.

:not()

섹션 5

괄호 안에 쓴 것만 빼고 고릅니다.

```
.stock-list li:not(.sold) {
  background-color: #e2f4ea;
}
```

읽는 법: “stock-list 안의 li 중에서 sold 가 아닌 것”

이게 왜 편할까요? “품질 아닌 것” 을 클래스로 표시하려면 모든 항목에 class=“available” 을 붙여야 합니다. 항목이 백 개면 백 번 씁니다. :not() 을 쓰면 예외인 것에만 클래스를 붙이면 됩니다.

괄호 안에는 선택자를 넣습니다. 클래스, 태그, id, 가상 클래스 다 됩니다.

li:not(.sold) sold 가 아닌 li p:not(:first-child) 첫 자식이 아닌 p input:not([type=“checkbox”]) 체크박스가 아닌 input

괄호 안에 심표로 여러 개를 넣을 수도 있습니다.

li:not(.sold, .hidden) 둘 중 어느 것도 아닌 li

조심할 것이 하나 있습니다. :not() 은 대상을 넓히는 도구입니다. 자칫하면 생각보다 훨씬 많이 걸립니다.

“이 목록 안의 li 중에서” 처럼 앞에서 범위를 좁혀 놓고 쓰세요.

화면 품질이 아닌 1, 3번째 줄만 연한 초록 배경이고,

품질 줄 두 개는 배경 없이 흐린 회색 글자입니다

```
아메리카노
카페라떼 (품질)
딸기라떼
자몽차 (품질)
```

:not() 은 괄호 안의 선택자 세기를 그대로 가져옵니다. li:not(.sold) 는 li.sold 와 세기가 같습니다. 이 이야기는 개념06 에서 다시 다룹니다.

▹ 직접 해보기 5

위 목록의 “딸기라떼” 에도 class=“sold” 를 붙여 보세요. 초록 줄이 몇 개로 줄어드는지 확인하세요.

자주 하는 실수

섹션 6

순서 선택자의 실수는 “한 개도 안 걸리거나 너무 많이 걸리는” 모양으로 나타납니다.

이런 실수를 합니다

:first-child 를 “그 태그 중 첫째” 로 읽음

이런 실수를 합니다

실수 1 — **:first-child** 를 “p 중 첫째” 로 읽음

```
.m1 p:first-child { color: crimson; }
```

섹션 2 의 그 함정입니다. 아래 상자의 첫 자식은 h3 입니다. 그래서 한 개도 안 걸립니다. “p 중 첫째” 를 원했다면 p:first-of-type 이라고 써야 합니다. 제목입니다 첫 문단인데 빨개지지 않습니다.

이렇게 쓰세요

```
.m1 p:first-of-type { color: crimson; }
```

이런 실수를 합니다

0 부터 센다고 생각함

이런 실수를 합니다

실수 2 — 순서를 0 부터 센다고 생각함

```
.m2 li:nth-child(0) { background-color: #fff3a0; }
```

CSS 의 순서는 1 부터 셉니다. 0 번째 자식은 세상에 없으므로 아무 일도 안 일어납니다. 첫 줄을 고르려면 nth-child(1) 입니다. 첫 줄인데 노래지지 않습니다. 둘째 줄

이런 실수를 합니다

2n 과 n+2 를 혼동

이런 실수를 합니다

실수 3 — n 앞뒤의 숫자를 바꿔 씀

```
.m3 li:nth-child(n+2) { background-color: #fff3a0; }
```

“두 칸마다 하나씩” 을 원하고 썼는데 두 번째부터 끝까지 전부 칠해집니다. 숫자를 n 앞에 쓰면 간격이고, n 뒤에 쓰면 시작점입니다. 두 칸마다는 2n 입니다. 1번 줄 2번 줄 3번 줄 4번 줄

이런 실수를 합니다

부모가 여러 개면 첫째도 여러 개

이런 실수를 합니다

실수 4 — 첫째가 하나뿐일 거라고 생각함

```
.m4 li:first-child { background-color: #fff3a0; }
```

:first-child 는 부모마다 셉니다. 목록이 두 개면 첫 줄도 두 개입니다. “한 개만 칠하고 싶다” 면 부모를 먼저 좁혀야 합니다. 첫 번째 목록의 첫 줄 첫 번째 목록의 둘째 줄 두 번째 목록의 첫 줄 두 번째 목록의 둘째 줄

이런 실수를 합니다

콜론 앞에 공백

이런 실수를 합니다

실수 5 — 콜론 앞에 공백을 넣음

```
.m5 li :first-child { background-color: #fff3a0; }
```

공백이 자손 결합자라서 “li 안에 있는 첫 자식” 을 찾습니다. 아래 li 안에는 글자만 있고 요소가 없으니 한 개도 못 찾습니다. 개념04 에서 본 것과 똑같은 사고입니다. 콜론은 항상 붙여 씁니다. 첫 줄인데 노래지지 않습니다. 둘째 줄

이렇게 쓰세요

```
.m5 li:first-child { background-color: #fff3a0; }
```

정리

▹ 직접 해보기 6

정답

1) 섹션 1 목록 맨 아래에 이렇게 넣습니다.

```
<li>5등 - 유자차</li>
```

→ 하늘색 배경이 “4등 — 자몽차” 에서 “5등 — 유자차” 로 옮겨 갑니다.

노란 배경은 여전히 첫 줄에 있습니다.
CSS 를 한 글자도 안 고쳤는데 결과가 따라 움직입니다.
이것이 순서 선택자를 쓰는 이유입니다.

2) 섹션 2 의 1번 상자에서 h3 줄을 지웁니다.

```
<div class="trap-box"> <p id="trapP1">아메리카노</p> <p>카페라떼</p>
</div>
```

→ “아메리카노” 줄이 노란 배경으로 바뀝니다.

CSS 는 손도 안 댔습니다. 제목 한 줄이 있느냐 없느냐가 전부였습니다.
확인했으면 h3 를 되살리세요.

3) style 의 규칙을 이렇게 고칩니다.

```
.tr i li:nth-child(3n) {  
    background-color: #e2f4ea;  
}
```

→ 초록 줄이 1, 4, 7번에서 3, 6번으로 바뀝니다. 세 줄에서 두 줄로 줄어듭니다.

3n 은 3, 6, 9 이고 3n+1 은 1, 4, 7 입니다.
뒤의 +1 이 시작점을 한 칸 당긴다는 뜻입니다.

4) 섹션 4 의 두 번째 상자에서 h3 를 지웁니다.

```
<div class="type-b"> <p id="typeBp1">아메리카노</p> <p id="typeBp2">카페라떼</p> <p>  
딸기라떼</p>  
</div>
```

→ 연한 초록 배경은 “카페라떼” 줄에 그대로 있습니다. 옮겨 가지 않습니다.

:nth-of-type 은 처음부터 p 끼리만 세고 있어서
h3 가 있든 없든 결과가 같기 때문입니다.
같은 일을 첫 번째 상자(:nth-child)에서 하면 분홍 배경이
“아메리카노” 에서 “카페라떼” 로 한 줄 내려갑니다. 그것도 해 보세요.

5) 섹션 5 목록의 “딸기라떼” 줄을 이렇게 고칩니다.

```
<li class="sold">딸기라떼</li>
```

→ 초록 줄이 두 개에서 한 개(“아메리카노”)로 줄어듭니다.

딸기라떼 줄은 초록 배경이 사라지고 흐린 회색 글자가 됩니다.
:not(.sold) 가 그 줄을 더 이상 안 고르기 때문입니다.

첫째와 막내

섹션 1

```
.rank-list li:first-child {  
    background-color: #fff3a0;  
}  
.rank-list li:last-child {  
    background-color: #eef7ff;  
}
```

화면 목록의 첫 줄은 노란 배경, 마지막 줄은 연한 하늘색 배경입니다.

가운데 줄들은 배경색이 없습니다

:first-child의 함정

섹션 2

```
.trap-box p:first-child {
  background-color: #fff3a0;
}
```

화면 아무 일도 일어나지 않습니다.

상자의 첫 자식이 h3 라서, "첫 자식이면서 p 인 것" 이 하나도 없습니다

```
.trap-plain p:first-child {
  background-color: #fff3a0;
}
```

화면 첫 문단만 노란 배경이 됩니다. 이 상자에는 h3 가 없습니다

```
.trap-fix p:first-of-type {
  background-color: #e2f4ea;
}
```

화면 h3 가 있어도 첫 번째 문단이 연한 초록 배경이 됩니다

:nth-child

섹션 3

```
.pick2 li:nth-child(2) {
  background-color: #fff3a0;
}
```

화면 목록의 두 번째 줄만 노란 배경입니다

```
.stripe li:nth-child(odd) {
  background-color: #eef2f7;
}
```

화면 홀수 번째 줄에 연한 회청색 배경이 깔려 줄무늬가 됩니다.

아래 상자는 항목이 다섯 개라 1, 3, 5번 줄이 칠해집니다

```
.tri li:nth-child(3n+1) {
  background-color: #e2f4ea;
}
```

화면 1, 4, 7번째 줄에 연한 초록 배경이 깔립니다

:nth-child 와 :nth-of-type

섹션 4

```
.type-a p:nth-child(2) {
  background-color: #ffe9e9;
}
```

화면 제목 다음의 첫 문단이 연한 분홍 배경입니다.

그 문단이 상자 전체에서 두 번째 자식이기 때문입니다

```
.type-b p:nth-of-type(2) {
  background-color: #e2f4ea;
}
```

화면 문단 중에서 두 번째인 것이 연한 초록 배경입니다.

제목은 세는 데 끼지 않습니다

:not()

섹션 5

```
.stock-list li:not(.sold) {
  background-color: #e2f4ea;
}
.stock-list li.sold {
  color: #8a8a8a;
}
```

화면 품절이 아닌 줄만 연한 초록 배경이고,

품절 줄은 배경 없이 흐린 회색 글자입니다

자주 하는 실수 (일부러 틀리게 쓴 규칙들)

섹션 6

```
.m1 p:first-child {
  color: crimson;
}
```


화면 아무 일도 일어나지 않습니다. 첫 자식이 h3 입니다

```
.m2 li:nth-child(0) {
  background-color: #fff3a0;
}
```

화면 아무 일도 일어나지 않습니다. 순서는 0 이 아니라 1 부터 셉니다

```
.m3 li:nth-child(n+2) {
  background-color: #fff3a0;
}
```

화면 두 번째 줄부터 끝까지 전부 노란 배경이 됩니다.

“두 칸마다” 가 아니라 “두 번째부터” 라는 뜻입니다

```
.m4 li:first-child {
  background-color: #fff3a0;
}
```

화면 목록이 두 개라서 노란 줄도 두 개가 됩니다

```
.m5 li :first-child {
  background-color: #fff3a0;
}
```

화면 아무 일도 일어나지 않습니다.

콜론 앞의 공백 때문에 "li 안에 있는 첫 자식" 을 찾고 있는데

li 안에는 글자만 있고 요소가 없습니다

명시도

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 명시도

08단원의 마지막이자 정점입니다.

지금까지 고르는 법을 여덟 가지쯤 배웠습니다. 그런데 실제로 CSS 를 쓰다 보면 이런 일이 반드시 생깁니다.

한 요소에 규칙이 두 개, 세 개, 네 개가 동시에 맞습니다.
게다가 서로 다른 색을 시킵니다.

브라우저는 그중 하나를 골라야 합니다. 어떻게 고를까요? 마음대로 고르지 않습니다. 정해진 순서가 있습니다.

- 1 먼저 !important 가 붙었는지 봅니다.
- 2 그다음 HTML 의 style 속성인지 봅니다.
- 3 그다음 선택자의 세기를 켭니다. ← 이것이 명시도입니다
- 4 세기까지 같으면 나중에 쓴 것을 씁니다.

명시도(specificity) 는 “이 선택자가 얼마나 콕 집어 말했는가” 입니다.

| | | |
|----------|----------------|-------------|
| li | "목록 항목 전부" | 두루뭉술합니다 |
| .hot | "hot 이라고 붙인 것" | 조금 콕 집었습니다 |
| #hotItem | "그 한 개" | 완전히 콕 집었습니다 |

콕 집어 말한 쪽이 이깁니다. 상식적입니다. “우리 반 전부 파란 옷” 과 “김민준은 빨간 옷” 이 같이 있으면 김민준은 빨간 옷입니다. 더 구체적인 지시가 이기는 것입니다.

이 파일이 왜 중요할까요? CSS 를 배운 사람이 가장 많이 겪는 좌절이 이것이기 때문입니다.

“규칙을 분명히 썼는데 색이 안 바뀌어요.”

이때 사람들은 색 이름을 고치고, 속성 이름을 고치고, 브라우저를 꺾다 컵니다. 전부 소용없습니다. 진짜 이유는 다른 규칙이 이기고 있어서입니다. 명시도를 셀 줄 알면 이 좌절이 통째로 사라집니다.

한 요소에 규칙이 여럿 맞을 때 누가 이기는지를 다룹니다. “규칙을 썼는데 왜 안 바뀌지?” 의 답이 전부 여기 있습니다. 상자마다 어느 규칙이 이겼는지를 맞춰 보면서 읽으세요.

규칙이 겹치면

섹션 1

아래 첫 줄에는 배경색을 시키는 규칙이 네 개 걸려 있습니다. 전부 다른 색입니다.

아래 상자의 HTML 은 이렇습니다.

```
<ul class="menu" id="menuBox"> <li class="hot" id="hotItem">아메리카노 (오늘의 추천)
</li> <li>카페라떼</li>
</ul>
```

첫 줄(hotItem)에 맞는 규칙을 세어 봅니다.

| | | |
|------------|--------------------------------|-----------------------|
| li | { background-color: #f2f2f2; } | li 니까 맞습니다 |
| .hot | { background-color: #ffe9e9; } | hot 이니까 맞습니다 |
| .menu .hot | { background-color: #fff3a0; } | menu 안의 hot 이니까 맞습니다 |
| #hotItem | { background-color: #d7ecff; } | id 가 hotItem 이니까 맞습니다 |

네 개 전부 맞습니다. 그런데 배경색은 하나만 보입니다. 실제 화면을 보면 첫 줄은 연한 하늘색입니다. #d7ecff 입니다. 즉 #hotItem 이 이겼습니다.

둘째 줄에는 li 규칙 하나만 맞습니다. 그래서 연한 회색입니다.

왜 #hotItem 이 이겼을까요? 그것이 섹션 2 의 계산법입니다. 지금은 “규칙이 여러 개 맞을 수 있다” 와 “그중 하나만 화면에 나타난다” 두 가지만 확인하면 됩니다.

한 가지 더 중요한 사실이 있습니다. 진 규칙이 통째로 버려지는 것이 아닙니다. 속성 하나하나마다 따로 싸웁니다. 만약 .hot 이 색깔 대신 글자색을 시켰다면 그건 그대로 적용됩니다. 같은 속성을 두고 부딪힐 때만 승부가 납니다.

화면 첫 줄은 연한 하늘색 배경, 둘째 줄은 연한 회색 배경입니다

아메리카노 (오늘의 추천)
카페라떼

진 규칙은 사라지지 않고 그대로 파일에 남아 있습니다. 같은 속성을 두고 부딪힐 때만 승부가 납니다. 서로 다른 속성이면 둘 다 적용됩니다.

⇒ 직접 해보기 1

<style> 에서 #hotItem 규칙을 통째로 주석 처리해 보세요. (/ * 와 * / 로 감쌉니다) 첫 줄이 무슨 색이 되는지 확인한 뒤 되돌려 놓으세요.

세는 법

섹션 2

선택자에 들어 있는 것을 세 칸에 나눠 셉니다. 왼쪽 칸부터 비교합니다.

세는 방법은 아주 단순합니다. 선택자를 보고 개수를 셉니다.

| | | |
|------|-----------------------|------|
| 첫째 칸 | id 선택자 (#로 시작) | 의 개수 |
| 둘째 칸 | 클래스 · 속성 선택자 · 가상 클래스 | 의 개수 |
| 셋째 칸 | 태그 선택자 · 가상 요소 | 의 개수 |

예를 들어 봅시다.

| | | |
|-----------------|-------|--------------------|
| li | 0-0-1 | |
| .hot | 0-1-0 | |
| li.hot | 0-1-1 | |
| .menu .hot | 0-2-0 | |
| .menu li:hover | 0-2-1 | :hover 는 둘째 칸입니다 |
| #hotItem | 1-0-0 | |
| #menuBox li.hot | 1-1-1 | |
| * | 0-0-0 | 전체 선택자는 아무것도 안 셉니다 |

비교는 왼쪽 칸부터 합니다. 왼쪽에서 승부가 나면 오른쪽은 안 봅니다.

1-0-0 vs 0-2-0 → 왼쪽 칸에서 1 대 0. 앞이 이깁니다.
 0-2-0 vs 0-1-1 → 첫 칸 동점, 둘째 칸 2 대 1. 앞이 이깁니다.
 0-1-1 vs 0-1-0 → 두 칸 동점, 셋째 칸 1 대 0. 앞이 이깁니다.

많은 책이 "id 는 100점, 클래스는 10점, 태그는 1점" 이라고 설명합니다. 더하기 쉬워서 편하지만, 사실 정확한 설명은 아닙니다. 점수를 더하는 게 아니라 자릿수를 따로따로 비교하기 때문입니다.

차이가 나는 경우를 보여 드립니다.

클래스 11개 → 점수로 세면 110점, 자릿수로 세면 0-11-0
 id 1개 → 점수로 세면 100점, 자릿수로 세면 1-0-0

점수 방식이 맞다면 클래스 11개(110점)가 id(100점)를 이겨야 합니다. 아래 두 번째 상자에서 실제로 재 보면 id 가 이깁니다. 첫째 칸에서 1 대 0 으로 이미 끝나 버리기 때문입니다.

그래서 이렇게 외우세요. "클래스를 아무리 많이 붙여도 id 하나를 못 이깁니다."

실제로 이런 극단적인 선택자를 쓸 일은 없습니다. 다만 "왜 id 를 함부로 쓰면 안 되는가" 의 이유가 여기 있습니다. id 로 색을 정해 두면 나중에 클래스로는 절대 못 바꿉니다.

세는 법을 표로 보면 이렇습니다.

| 선택자 | id | 클래스 | 태그 | 읽는 값 |
|--------------------|----|-----|----|-------|
| li | 0 | 0 | 1 | 0-0-1 |
| .hot | 0 | 1 | 0 | 0-1-0 |
| li.hot | 0 | 1 | 1 | 0-1-1 |
| .menu .hot | 0 | 2 | 0 | 0-2-0 |
| .menu li:hover | 0 | 2 | 1 | 0-2-1 |
| input[type="text"] | 0 | 1 | 1 | 0-1-1 |
| #hotItem | 1 | 0 | 0 | 1-0-0 |
| #menuBox li.hot | 1 | 1 | 1 | 1-1-1 |
| * | 0 | 0 | 0 | 0-0-0 |

화면 이 줄은 연한 하늘색 배경입니다. 노란색이 아닙니다

클래스를 열한 개 붙였는데도 id 하나에 졌습니다.

:not() 은 그 자체로는 세지 않고 괄호 안의 선택자를 셉니다. li:not(.sold) 는 li.sold 와 같은 0-1-1 입니다.

▣ 직접 해보기 2

아래 선택자들의 명시도를 종이에 적어 보세요. p / .card p / #box .card / a:hover / ul li.hot — 답은 파일 맨 아래에 있습니다.

명시도가 완전히 같으면 CSS 파일에서 아래쪽에 쓴 규칙이 이깁니다.

.later-a { background-color: #fff3a0; } 노랑 .later-b { background-color: #e2f4ea; } 연한 초록
둘 다 0-1-0 입니다. 완전히 같습니다. 이럴 때는 나중에 쓴 .later-b 가 이깁니다. 그래서 초록이 보입니다.
여기서 반드시 짚고 갈 것이 있습니다. 많은 사람이 "HTML 의 class 순서" 가 영향을 준다고 생각합니다.

```
<p class="later-a later-b">...</p>
<p class="later-b later-a">...</p>
```

이 둘의 결과가 다를 것 같지요? 같습니다. 둘 다 초록입니다. 아래 상자에서 직접 확인하세요.

HTML 의 class 값은 "가진 이름표 목록" 일 뿐 순서에 뜻이 없습니다. 승부는 오로지 CSS 파일 안의 순서로 납니다.

이 규칙이 실무에서 어떻게 쓰일까요? 남이 만든 CSS 를 덮어써야 할 때, 굳이 선택자를 세게 만들지 않고 같은 세기로 그냥 아래쪽에 쓰면 됩니다. 그래서 CSS 파일을 연결하는 순서가 중요합니다(07단원). 공통 스타일을 먼저, 개별 스타일을 나중에 연결합니다.

화면 두 줄 모두 연한 초록 배경입니다. 색이 서로 다르지 않습니다

```
class="later-a later-b" 라고 썼습니다.
class="later-b later-a" 라고 썼습니다.
```

이 성질 때문에 CSS 는 위에서 아래로 읽으며 쌓아 가는 것이 됩니다. 위쪽에 넓은 규칙(태그·기본값)을, 아래쪽에 좁은 규칙(예외·변형)을 쓰세요.

▶ 직접 해보기

3

<style> 에서 .later-a 규칙과 .later-b 규칙의 순서를 바꿔 보세요. 두 줄이 무슨 색이 되는지 확인한 뒤 되돌려 놓으세요.

인라인 style

07단원에서 배운 인라인 방식은 명시도 계산 자체를 건너뛸니다. 어떤 선택자보다 셉니다.

07단원에서 CSS 를 붙이는 방법 세 가지를 배웠습니다. 그중 하나가 HTML 태그에 직접 쓰는 방법이었습니다.

```
<p style="background-color: #d7ecff;">...</p>
```

이것을 인라인 스타일이라고 부릅니다. 인라인 스타일에는 선택자가 없습니다. 이미 그 요소를 손으로 짚었으니까요. 그래서 명시도를 짤 것이 없고, 아예 한 단계 위에서 이깁니다.

아래 상자의 줄에는 이런 두 가지가 걸려 있습니다.

| | |
|---|---------------|
| #inlineP { background-color: #ffe9e9; } | 1-0-0, 아주 썩니다 |
| style="background-color: #d7ecff;" | 인라인, 더 썩니다 |

결과는 연한 하늘색입니다. 인라인이 이겼습니다. id 선택자로도 인라인 스타일을 덮을 수 없습니다.

그래서 인라인 스타일은 되도록 쓰지 마세요. 이유가 셋 있습니다.

1 나중에 CSS 로 못 바꿉니다. 이 파일에서 방금 본 그대로입니다.

2 같은 모양을 다른 곳에 다시 쓸 수 없습니다. 그 요소에만 붙어 있습니다.

3 HTML 과 CSS 가 섞여서 둘 다 읽기 어려워집니다.

“그럼 언제 쓰나요?” 자바스크립트로 그때그때 값을 바꿀 때 정도입니다. 수업에서 손으로 쓸 일은 거의 없습니다.

화면 이 줄은 연한 하늘색 배경입니다. 분홍이 아닙니다

style 속성으로 하늘색을, CSS 의 id 규칙으로 분홍색을 시켰습니다.

▫ 직접 해보기

4

위 줄의 style="background-color: #d7ecff;" 를 통째로 지워 보세요. 무슨 색이 나타나는지 확인한 뒤 되돌려 놓으세요.

!important

섹션 5

값 뒤에 !important 를 붙이면 모든 것을 이깁니다. 그래서 쓰면 안 됩니다.

```
.imp-cls {
  color: #1a7f5a !important;
```

↳ 값과 세미콜론 사이에 붙입니다

```
}
```

아래 상자의 줄에는 세 가지가 걸려 있습니다.

| | |
|-------------------------|----------|
| style="color: #2d6cdf;" | 인라인 (파랑) |
|-------------------------|----------|

#impId { color: #c0392b; } id 선택자 (빨강) .imp-cla{ color: #1a7f5a !important; } 클래스 + important (초록)

결과는 초록입니다. 가장 약한 클래스 선택자가 인라인까지 이겼습니다. !important 는 명시도 계산 바깥에 있기 때문입니다.

여기까지 보면 “그럼 이걸 쓰면 편하겠네” 싶습니다. 정반대입니다. !important 는 CSS 에서 가장 큰 골칫거리로 꼽힙니다. 왜일까요?

안 되면 !important 를 붙입니다. 잘 됩니다. 다음에 그걸 바꾸려니 안 바뀝니다. 그래서 또 !important 를 붙입니다. 이제 두 개가 됐습니다. 어느 쪽이 이기는지 알 수 없습니다. 결국 파일 전체가 !important 로 덮이고, 아무것도 못 고치게 됩니다.

!important 를 쓴다는 것은 “명시도 규칙을 포기한다” 는 뜻입니다. 규칙이 있으니까 예측이 되는 것인데, 그 규칙을 스스로 버리는 셈입니다.

안 되는 진짜 이유는 거의 항상 명시도입니다. !important 를 붙이기 전에 이렇게 해 보세요.

1 브라우저에서 F12 를 눌러 그 요소를 봅니다. 이긴 규칙과 진 규칙(줄이 그어져 있습니다)이 다 보입니다.

2 진 규칙의 선택자를 한 단계만 더 구체적으로 만듭니다. .hi-text 가 졌다면 .warnbox .hi-text 로 바꿉니다.

3 또는 같은 세기로 만들고 아래쪽으로 옮깁니다.

그럼 !important 를 언제 쓸까요? 고칠 수 없는 남의 CSS 를 덮어써야 할 때 정도입니다. 내가 쓴 CSS 를 내가 덮으려고 쓰는 것은 거의 항상 잘못된 신호입니다.

화면 이 줄은 초록 글자입니다. 파랑도 빨강도 아닙니다

가장 약한 클래스 규칙이 이겼습니다.

이기는 순서를 한 줄로 줄이면 이렇게 됩니다. !important > 인라인 style > 명시도 > 나중에 쓴 것. 위에서 승부가 나면 아래는 보지 않습니다.

➡ 직접 해보기 **5**

.imp-cla 규칙에서 !important 를 지워 보세요. 이번에는 어느 색이 이기는지 확인한 뒤 되돌려 놓으세요.

상속은 싸움에 끼지 않습니다

섹션 6

07단원의 상속과 명시도는 다른 이야기입니다. 직접 지정한 값이 항상 이깁니다.

07단원에서 색은 자식에게 물려 내려간다고 배웠습니다. 그럼 이런 질문이 생깁니다.

"부모에 1-0-0 짜리 id 규칙으로 색을 줬으면
자식은 0-1-1 규칙보다 그 색을 더 세게 물려받나요?"

아닙니다. 물려받은 값은 명시도 싸움에 아예 참가하지 못합니다. 상속은 "아무도 안 정해 줬을 때 부모 것을 쓴다" 는 뜻일 뿐입니다. 자기한테 직접 걸린 규칙이 하나라도 있으면 그것이 이깁니다.

아래 상자에는 이런 두 규칙이 있습니다.

```
#inheritBox      { color: #c0392b; }   1-0-0
.inherit-box li { color: #1a7f5a; }   0-1-1
```

상자 안 문단에는 자기 규칙이 없으니 상자 색인 빨강을 물려받습니다. 상자 안 목록 항목에는 자기 규칙이 있으니 초록입니다. 1-0-0 이 0-1-1 보다 세지만, 물려받은 값이라 아예 상대가 되지 않습니다.

이 사실을 알아 두면 자주 겪는 사고 하나가 풀립니다.

"부모에 색을 줬는데 링크만 색이 안 바뀝니다"

브라우저가 a 요소에 기본 파란색을 직접 지정해 두었기 때문입니다. 물려받은 색은 그 기본값을 못 이깁니다. 그럴 때는 a 를 직접 골라야 합니다. 07단원 개념05 섹션6 에서 본 "링크와 버튼은 왜 색이 안 바뀔까" 의 답이 바로 이것입니다.

화면 문단은 빨간 글자, 목록 항목은 초록 글자입니다.

목록 항목의 배경이 연한 회색인 것은 섹션 1 의 li 규칙 때문입니다

이 문단은 자기 규칙이 없어서 상자 색을 물려받았습니다.

이 항목은 자기 규칙이 있어서 물려받지 않습니다.

정리하면 이렇습니다. 직접 걸린 규칙이 하나라도 있으면 그것들끼리 명시도로 싸웁니다. 하나도 없을 때만 부모 값을 물려받습니다.

⇒ 직접 해보기 6

<style> 에서 .inherit-box li 규칙을 주석 처리해 보세요. 목록 항목이 무슨 색이 되는지 확인한 뒤 되돌려 놓으세요.

자주 하는 실수

섹션 7

명시도 사고의 겉모습은 언제나 똑같습니다. "규칙을 썼는데 화면이 안 바뀐다" 입니다.

이런 실수를 합니다

자손 선택자가 클래스를 이겨 버림

이런 실수를 합니다

실수 1 — 자손 선택자가 강조 클래스를 이겨 버림

```
.warnbox p { color: #8a8a8a; } /* 0-1-1 */
.hi-text   { color: #c0392b; } /* 0-1-0 */
```

가장 흔한 명시도 사고입니다. 상자 안 글자를 회색으로 깔아 둔 상태에서 한 줄만 빨강게 강조하려고 클래스를 붙였는데 0-1-1 이 0-1-0 을 이겨서 회색 그대로입니다. 아래 줄은 class="hi-text" 를 제대로 붙였는데도 흐린 회색입니다. 강조하려고 했는데 회색입니다.

이렇게 고칩니다

```
.warnbox p.hi-text { color: #c0392b; } /* 0-2-1 로 만들어 이깁니다 */
```

이런 실수를 합니다

!important 를 붙였다가 자기 발등을 찍음

이런 실수를 합니다

실수 2 — !important 를 붙여 두고 나중에 못 고침

```
.imp-bad      { color: #c0392b !important; } /* 0-1-0 */
.imp-bad.imp-fix { color: #1a7f5a; }         /* 0-2-0 인데도 집니다 */
```

예전에 안 바뀌길래 !important 를 붙여 뒀습니다. 오늘 그걸 고치려고 더 센 선택자를 만들었는데도 안 바뀝니다. !important 는 명시도 바깥에 있어서 아무리 세게 만들어도 소용이 없습니다. 더 센 규칙을 썼는데도 빨간 글자입니다.

이렇게 고칩니다

```
.imp-bad { color: #c0392b; } /* 먼저 !important 를 떼어 냅니다 */
```

이런 실수를 합니다

id 로 정해 놓고 클래스로 덮으려 함

이런 실수를 합니다

실수 3 — id 로 정해 둔 것을 클래스로 덮으려 함

```
#idLock { color: #c0392b; } /* 1-0-0 */
.class-try { color: #1a7f5a; } /* 0-1-0 */
```

클래스를 몇 개를 붙여도 id 하나를 못 이깁니다(섹션 2). 그래서 id 로 색을 정하는 습관이 위험합니다. 개념01 에서 “웬만하면 클래스를 쓰라” 고 한 이유가 이것입니다. 클래스를 붙였는데도 빨간 글자입니다.

이런 실수를 합니다

인라인 style 을 잊고 CSS 만 고침

이런 실수를 합니다

실수 4 — HTML 의 style 속성을 잊고 CSS 만 고침

```
<p class="inline-trap" style="color: #c0392b;">...</p>
.inline-trap { color: #1a7f5a; }
```

CSS 파일만 열심히 고치는데 화면이 꿈쩍도 안 합니다. 범인은 HTML 에 남아 있는 style 속성입니다. CSS 를 고쳐도 안 바뀌면 HTML 쪽을 먼저 보세요. CSS 에서 초록을 시켰는데 빨간 글자입니다.

이런 실수를 합니다

같은 선택자를 두 번 쓰고 위쪽을 고침

이런 실수를 합니다

실수 5 — 같은 선택자를 두 번 쓰고 위쪽을 고침

```
.dup-rule { color: #c0392b; } /* 위쪽 - 아무리 고쳐도 안 보입니다 */
.dup-rule { color: #1a7f5a; } /* 아래쪽 - 이쪽이 이깁니다 */
```

파일이 길어지면 같은 선택자를 두 번 쓰고도 모릅니다. 위쪽 규칙을 아무리 고쳐도 화면은 그대로입니다. 세기가 같으면 나중 것이 이기기 때문입니다. Ctrl+F 로 선택자를 검색해서 몇 개나 있는지 세어 보세요. 위쪽 규칙은 빨강인데 초록 글자입니다.

정리

▣ 직접 해보기

7

정답

1) style 의 #hotItem 규칙을 이렇게 감쌉니다.

```
/* #hotItem {
  background-color: #d7ecff;
} */
```

→ 첫 줄이 하늘색에서 노란색(#fff3a0)으로 바뀝니다.

남은 규칙 셋 중에서 .menu .hot 이 0-2-0 으로 가장 세기 때문입니다.
.hot(0-1-0)도 li(0-0-1)도 여기에 밀립니다.
확인했으면 주석을 풀어 되돌리세요.

2) 명시도는 이렇습니다.

| | | |
|------------|-------|-------------------|
| p | 0-0-1 | 태그 1개 |
| .card p | 0-1-1 | 클래스 1개 + 태그 1개 |
| #box .card | 1-1-0 | id 1개 + 클래스 1개 |
| a:hover | 0-1-1 | 태그 1개 + 가상 클래스 1개 |
| ul li.hot | 0-1-2 | 클래스 1개 + 태그 2개 |

→ 센 순서로 늘어놓으면

#box .card → ul li.hot → .card p 와 a:hover (동점) → p 입니다.
.card p 와 a:hover 가 같은 0-1-1 이라는 점을 확인하세요.
가상 클래스는 클래스와 같은 칸에서 씩니다.

3) style 에서 두 규칙의 순서를 이렇게 바꿉니다.

```
.later-b { background-color: #e2f4ea; }  
.later-a { background-color: #fff3a0; }
```

→ 두 줄이 모두 노란색(#fff3a0)으로 바뀝니다.

이번에는 .later-a 가 아래쪽에 있으니 이깁니다.
HTML 은 한 글자도 안 고쳤는데 결과가 바뀌었다는 점이 중요합니다.
확인했으면 순서를 되돌리세요.

4) 섹션 4 의 문단을 이렇게 고칩니다.

```
<p id="inlineP">
```

→ 줄의 배경이 연한 하늘색에서 연한 분홍색(#ffe9e9)으로 바뀝니다.

인라인 style 이 사라지자 그제야 #inlineP 규칙이 보이게 된 것입니다.
규칙은 처음부터 그 자리에 있었습니다. 가려져 있었을 뿐입니다.

5) style 의 규칙을 이렇게 고칩니다.

```
.imp-cls {  
  color: #1a7f5a;  
}
```

→ 글자가 초록에서 파랑(#2d6cdf)으로 바뀝니다.

!important 가 사라지자 인라인 style 이 이깁니다.
빨강(#c0392b)이 아니라는 점을 확인하세요.
id 규칙도 인라인에는 못 이깁니다.

6) style 의 규칙을 이렇게 감쌉니다.

```
/* .inherit-box li {
  color: #1a7f5a;
} */
```

→ 목록 항목이 초록에서 빨강으로 바뀝니다. 위 문단과 같은 색이 됩니다.

자기한테 걸린 규칙이 없어지자 그제야 상자 색을 물려받은 것입니다.
확인했으면 주석을 풀어 되돌리세요.

섹션 1~2 공용 예제 — 한 요소에 규칙 네 개가 동시에 맞습니다 —

```
li {
  background-color: #f2f2f2;
}
```

0-0-1 — 이 파일의 모든 li 에 깔리는 밑바탕입니다

```
.hot {
  background-color: #ffe9e9;
}
```

0-1-0

```
.menu .hot {
  background-color: #fff3a0;
}
```

0-2-0

```
#hotItem {
  background-color: #d7ecff;
}
```

1-0-0 — 넷 중 이 규칙이 이깁니다

화면 첫 줄만 연한 하늘색 배경이고, 둘째 줄은 연한 회색 배경입니다

자릿수 비교를 증명하는 예제

섹션 2

```
.c1.c2.c3.c4.c5.c6.c7.c8.c9.c10.c11 {
  background-color: #fff3a0;
}
```

0-11-0 — 클래스를 열한 개나 붙였습니다

```
#digitP {  
  background-color: #d7ecff;  
}
```

1-0-0 — 그런데도 id 하나가 이깁니다

화면 그 줄은 연한 하늘색 배경입니다. 노란색이 아닙니다

세기가 같으면 나중에 쓴 것

섹션 3

```
.later-a {  
  background-color: #fff3a0;  
}  
.later-b {  
  background-color: #e2f4ea;  
}
```

둘 다 0-1-0 으로 같습니다. 나중에 쓴 .later-b 가 이깁니다

화면 두 줄 모두 연한 초록 배경입니다. HTML 의 class 순서는 상관없습니다

인라인 style

섹션 4

```
#inlineP {  
  background-color: #ffe9e9;  
}
```

1-0-0 인데도 집니다. HTML 의 style 속성이 더 세기 때문입니다

화면 그 줄은 연한 하늘색 배경입니다

!important

섹션 5

```
#impId {  
  color: #c0392b;  
}  
.imp-cls {  
  color: #1a7f5a !important;  
}
```

화면 그 줄은 초록 글자입니다. id 도 인라인 style 도 다 졌습니다

상속은 싸움에 끼지 않습니다

섹션 6

```
#inheritBox {
  color: #c0392b;
}
.inherit-box li {
  color: #1a7f5a;
}
```

화면 상자 안 문단은 빨간 글자(물려받음),

목록 항목은 초록 글자(자기 규칙이 있음)입니다

자주 하는 실수

섹션 7

```
.warnbox p {
  color: #8a8a8a;
}
.hi-text {
  color: #c0392b;
}
```

화면 강조하려던 글자가 흐린 회색입니다. 0-1-1 이 0-1-0 을 이겼습니다

```
.imp-bad {
  color: #c0392b !important;
}
.imp-bad.imp-fix {
  color: #1a7f5a;
}
```

화면 더 센 선택자를 썼는데도 빨간 글자 그대로입니다

```
#idLock {
  color: #c0392b;
}
.class-try {
  color: #1a7f5a;
}
```

화면 빨간 글자입니다. 클래스로는 id 를 덮을 수 없습니다

```
.inline-trap {  
  color: #1a7f5a;  
}
```

화면 빨간 글자입니다. HTML 의 style 속성이 이겼습니다

```
.dup-rule {  
  color: #c0392b;  
}  
.dup-rule {  
  color: #1a7f5a;  
}
```

화면 초록 글자입니다. 같은 선택자를 두 번 쓰면 나중 것이 이깁니다

연습문제

제목 두 줄을 빨간 글자로 만드세요.

오늘의 메뉴

아메리카노, 카페라떼

내일의 메뉴

딸기라떼, 자몽차

notice 를 붙인 두 줄에 노란 배경을 넣으세요.

문 여는 시간이 바뀌었습니다.

평일은 8시에 엽니다.

일요일은 쉽니다.

id 가 mainTitle 인 줄만 초록 글자로 만드세요.

봄내 카페

강원도 춘천시 봄내로 12

실행하면 나오는 화면 — 연습문제

이 파일 위쪽 `<style>` 안의 TODO 라고 적힌 자리에 CSS 규칙을 쓰고 저장한 뒤 새로고침(F5) 하세요. 각 문제 위의 설명에 “이렇게 보이면 맞습니다” 가 적혀 있으니 아래 상자와 비교하면 혼자서도 정답을 확인할 수 있습니다. 1~12번은 기본, 13~14번은 응용, 15번은 도전, 16번은 잘못된 코드 고치기입니다. body 의 HTML 은 고치지 마세요.

문제 1 (개념01 섹션1)

h3 제목 두 줄을 빨간 글자로 만드세요.

오늘의 메뉴

아메리카노, 카페라떼

내일의 메뉴

딸기라떼, 자몽차

문제 2 (개념01 섹션2)

notice 를 붙인 두 줄에 노란 배경을 넣으세요.
문 여는 시간이 바뀌었습니다.
평일은 8시에 엽니다.
일요일은 쉽니다.

문제 3 (개념01 섹션3)

id 가 mainTitle 인 줄만 초록 글자로 만드세요.
봄내 카페
강원도 춘천시 봄내로 12

문제 4 (개념02 섹션1)

차와 케이크 두 줄만 규칙 하나로 하늘색 배경을 넣으세요.
캐모마일차
아메리카노
치즈케이크

문제 5 (개념02 섹션4)

card 와 big 을 둘 다 가진 줄만 분홍 배경으로 만드세요.
card 만 가진 줄
big 만 가진 줄
card 와 big 을 둘 다 가진 줄

문제 6 (개념03 섹션2)

notice-box 안의 문단을 깊이 상관없이 전부 빨강게 만드세요.

상자 안에 바로 있는 문단

한 겹 더 안에 있는 문단

상자 밖 문단

문제 7 (개념03 섹션3)

crumb 의 바로 아래 단계 문단만 노란 배경으로 만드세요.

바로 아래 단계 문단

한 겹 더 들어간 문단

문제 8 (개념03 섹션5)

질문 줄 바로 다음 문단 하나만 하늘색 배경으로 만드세요.

주차할 수 있나요?

건물 뒤에 두 자리 있습니다.

주말에는 만차일 수 있습니다.

문제 9 (개념04 섹션1)

마우스를 올린 동안만 파란 배경에 흰 글자가 되게 하세요.

여기에 마우스를 올려 보세요

문제 10 (개념04 섹션5)

체크된 칸의 글자에만 노란 배경을 넣으세요.

얼음을 추가합니다

시럽을 추가합니다

문제 11 (개념05 섹션1)

첫 줄은 노란 배경, 마지막 줄은 하늘색 배경으로 만드세요.

1등 아메리카노

2등 카페라떼

3등 딸기라떼

문제 12 (개념05 섹션3)

짝수 번째 줄에만 연한 회청색 배경을 깔아 주세요.

- 1번 줄
- 2번 줄
- 3번 줄
- 4번 줄
- 5번 줄
- 6번 줄

문제 13 [응용] (개념05 섹션5)

품절이 아닌 항목만 연한 초록 배경으로 만드세요.

- 아메리카노
- 카페라떼 (품절)
- 딸기라떼
- 자몽차 (품절)

문제 14 [응용] (개념05 섹션2)

제목이 먼저 있는 상자에서 첫 번째 문단만 노란 배경으로 만드세요.

- 가게 소개
- 2019년에 문을 열었습니다.
- 원두는 매주 볶습니다.

문제 15 [도전] (개념06)

아래 두 줄은 이미 써 둔 규칙 때문에 흐린 회색입니다.
strong-tip 줄만 빨강게 바꾸세요. !important 와 id 는 금지입니다.

- 보통 안내 문단입니다.
- 이 줄만 빨간 글자로 만드세요.

문제 16 [확인]

`<style>` 맨 아래 세 규칙은 전부 아무 일도 하지 않습니다.
각각 한 군데씩 고치세요.

- (가) 이 줄에 노란 배경이 깔려야 합니다.
- (나) 이 줄이 빨간 글자가 되어야 합니다.
- (나) 이 줄도 빨간 글자가 되어야 합니다.
- (다) 이 글자에 노란 배경이 깔려야 합니다.

정리

08단원 연습문제 - 선택자

쓰는 법

- 1) 아래 각 문제의 "TODO" 자리에 CSS 규칙을 씁니다.
- 2) 저장하고(Ctrl+S) 브라우저에서 새로고침(F5)합니다.
- 3) 브라우저에 보이는 상자와 "기대 화면" 을 비교합니다.

HTML 은 고치지 않습니다. body 는 손대지 말고 이 style 안에서만 푸세요.
(문제 16 만 예외로, 이미 써 둔 잘못된 규칙을 고칩니다)

08

문제 1 (개념01 섹션1)

문제 1 상자 안에는 h3 제목이 두 개 있습니다.
태그 선택자로 h3 의 글자색을 crimson 으로 만드세요.

기대 화면: 상자 안의 제목 두 줄("오늘의 메뉴", "내일의 메뉴")이
모두 빨간 글자가 됩니다.

이렇게 되면 틀린 것: 한 줄만 빨개지면 태그가 아니라 다른 것으로 고른 것입니다.

여기에 규칙을 쓰세요

문제 2 (개념01 섹션2)

문제 2 상자에는 class="notice" 인 문단이 두 개 있습니다.
클래스 선택자로 그 두 줄의 배경색을 #fff3a0 으로 만드세요.

기대 화면: 첫째 줄과 셋째 줄에 노란 배경이 깔리고,
가운데 "평일은 8시에 엽니다." 줄은 그대로입니다.

이렇게 되면 틀린 것: 아무 변화가 없으면 점을 빠뜨렸는지 보세요.

여기에 규칙을 쓰세요

문제 3 (개념01 섹션3)

문제 3 상자의 id 가 mainTitle 인 줄의 글자색을 #1a7f5a 로 만드세요.

기대 화면: "봄내 카페" 한 줄만 초록 글자가 됩니다.
이렇게 되면 틀린 것: 주소 줄까지 초록이 되면 id 가 아니라

더 넓은 것으로 고른 것입니다.

여기에 규칙을 쓰세요

문제 4 (개념02 섹션1)

문제 4 상자에는 class 가 tea, coffee, cake 인 문단이 하나씩 있습니다.
선표를 써서 tea 와 cake 두 줄만 배경색 #eef7ff 로 만드세요.
규칙은 한 개만 씁니다.

기대 화면: "캐모마일차" 와 "치즈케이크" 두 줄이 연한 하늘색 배경이고,
가운데 "아메리카노" 줄은 그대로입니다.

이렇게 되면 틀린 것: 한 줄도 안 바뀌면 심표 자리에 공백만 쓴 것입니다.

여기에 규칙을 쓰세요

문제 5 (개념02 섹션4)

문제 5 상자에는 문단이 세 개 있습니다.
card 와 big 을 둘 다 가진 줄만 배경색 #ffe9e9 로 만드세요.

기대 화면: 셋째 줄만 연한 분홍 배경입니다. 위의 두 줄은 그대로입니다.
이렇게 되면 틀린 것: 세 줄이 다 분홍이면 심표를 쓴 것이고,

한 줄도 안 바뀌면 사이에 공백을 넣은 것입니다.

여기에 규칙을 쓰세요

08

문제 6 (개념03 섹션2)

문제 6 상자에서 class="notice-box" 안에 있는 문단을
깊이에 상관없이 전부 글자색 #c0392b 로 만드세요.

기대 화면: 위의 두 줄이 빨간 글자가 되고,
맨 아래 "상자 밖 문단" 은 검은 글자 그대로입니다.

이렇게 되면 틀린 것: 첫 줄만 빨개지면 부등호를 쓴 것입니다.

여기에 규칙을 쓰세요

문제 7 (개념03 섹션3)

문제 7 상자에서 class="crumb" 의 바로 아래 단계 문단만
배경색 #fff3a0 으로 만드세요.

기대 화면: 첫 줄만 노란 배경이고, 한 겹 더 들어간 줄은 그대로입니다.

이렇게 되면 틀린 것: 두 줄 다 노래지면 부등호를 빠뜨린 것입니다.

여기에 규칙을 쓰세요

문제 8 (개념03 섹션5)

문제 8 상자에서 class="q" 인 줄 바로 다음 문단만
배경색 #eef7ff 로 만드세요.

기대 화면: "건물 뒤에 두 자리 있습니다." 한 줄만 연한 하늘색 배경입니다.
질문 줄과 "주말에는" 줄은 그대로입니다.

이렇게 되면 틀린 것: 아래 두 줄이 다 바뀌면 물결을 쓴 것입니다.

여기에 규칙을 쓰세요

문제 9 (개념04 섹션1)

문제 9 상자의 class="hover-item" 인 줄에
마우스를 올린 동안만 배경색 #2d6cdf, 글자색 white 가 되게 하세요.

기대 화면: 평소에는 아무 색도 없습니다.
 마우스를 올린 동안만 파란 배경에 흰 글자가 되고,
 치우면 곧바로 원래대로 돌아옵니다.
이렇게 되면 틀린 것: 마우스를 안 올렸는데도 파랗다면

:hover 를 빠뜨린 것입니다.

여기에 규칙을 쓰세요

문제 10 (개념04 섹션5)

문제 10 상자에는 class="opt" 인 체크박스가 두 개 있습니다.
체크된 체크박스 바로 다음 label 의 배경색을 #fff3a0 으로 만드세요.

기대 화면: 처음 열면 "얼음을 추가합니다" 글자만 노란 배경입니다.
 아래 체크박스를 누르면 "시럽을 추가합니다" 에도 노란 배경이
 생기고,
 체크를 풀면 사라집니다.

이렇게 되면 틀린 것: 두 줄 다 노래지면 :checked 를 빠뜨린 것입니다.

여기에 규칙을 쓰세요

문제 11 (개념05 섹션1)

문제 11 상자의 class="rank" 목록에서
첫 줄은 배경색 #fff3a0, 마지막 줄은 배경색 #eef7ff 로 만드세요.
규칙 두 개가 필요합니다.

기대 화면: "1등" 줄은 노란 배경, "3등" 줄은 연한 하늘색 배경이고,
"2등" 줄은 배경색이 없습니다.

이렇게 되면 틀린 것: 세 줄이 다 같은 색이면 가상 클래스를 안 붙인 것입니다.

여기에 규칙을 쓰세요

문제 12 (개념05 섹션3)

문제 12 상자의 class="stripe" 목록에서
짝수 번째 줄에만 배경색 #eef2f7 을 깔아 줄무늬로 만드세요.

기대 화면: 2, 4, 6번 줄에만 연한 회청색 배경이 깔립니다.

이렇게 되면 틀린 것: 1, 3, 5번이 칠해졌다면 odd 를 쓴 것입니다.

여기에 규칙을 쓰세요

문제 13 [응용] (개념05 섹션5)

문제 13 상자의 class="stock" 목록에서
 품질(class="sold")이 아닌 항목만 배경색 #e2f4ea 로 만드세요.
 품질 항목에는 클래스를 새로 붙이지 말고 :not() 을 쓰세요.

기대 화면: 1번째와 3번째 줄만 연한 초록 배경이고,
 품질 줄 두 개는 배경색이 없습니다.

이렇게 되면 틀린 것: 네 줄이 다 초록이면 :not() 이 빠진 것입니다.

여기에 규칙을 쓰세요

문제 14 [응용] (개념05 섹션2)

문제 14 상자의 class="intro" 안에서 첫 번째 문단만
 배경색 #fff3a0 으로 만드세요.
 상자의 첫 자식은 문단이 아니라 h4 제목이라는 점에 주의하세요.

기대 화면: "2019년에 문을 열었습니다." 줄만 노란 배경입니다.
 이렇게 되면 틀린 것: 아무 변화가 없으면 :first-child 를 쓴 것입니다.

첫 자식이 h4 라서 한 개도 안 걸립니다.

여기에 규칙을 쓰세요

문제 15 [도전] (개념06)

아래 규칙은 이미 써 두었습니다. 지우지 마세요.

```
.tip-area p {
  color: #8a8a8a;
}
```

이 규칙 때문에 문제 15 상자의 문단이 둘 다 흐린 회색입니다.

class="strong-tip" 인 줄만 글자색 #c0392b 로 바꾸세요.

조건 두 가지를 지켜야 합니다.

- !important 를 쓰지 않습니다.
- id 선택자를 쓰지 않습니다.

명시도를 세어 보고, 위 규칙보다 세게 만드세요.

기대 화면: 아래 줄만 빨간 글자가 되고, 위 줄은 흐린 회색 그대로입니다.

이렇게 되면 틀린 것: 아무 변화가 없으면 .strong-tip 만 쓴 것입니다.

그것은 0-1-0 이라 0-1-1 인 위 규칙에 집니다.

여기에 규칙을 쓰세요

문제 16 [확인] (개념01 · 개념02 · 개념04)

아래 세 규칙은 전부 "아무 일도 일어나지 않습니다".
에러도 안 납니다. 각각 무엇이 잘못됐는지 찾아 고치세요.
한 규칙에 한 군데씩만 고치면 됩니다.

기대 화면 (다 고친 뒤):

- (가) 줄에 노란 배경이 깔립니다.
- (나) 두 줄이 모두 빨간 글자가 됩니다.
- (다) 체크박스 옆 글자에 노란 배경이 깔립니다.

이렇게 되면 틀린 것:

- (가) 가 그대로면 선택자 앞에 클래스 기호(.)를 아직 안 붙인 것입니다.
- (나) 가 한 줄만 빨개졌다면 두 선택자 사이에 쉼표가 아니라 공백을 쓴 것입니다.
- (다) 가 그대로면 콜론 앞에 공백이 남아 있는지 확인하세요.

가상 클래스는 앞 선택자에 붙여 씁니다. (개념04 섹션7 실수 2)

```
menu-item {  
  background-color: #fff3a0;  
}  
.list-a .list-b {  
  color: crimson;  
}  
.chk :checked + label {  
  background-color: #fff3a0;  
}
```

08단원 마무리 선택자

자주 넘어지는 자리

- 1 **F12 → Styles** 에서 취소선 그어진 규칙. 명시도를 눈으로 보는 유일한 방법입니다

텍스트와 폰트

OPEN 09_텍스트와_폰트

아무것도 지정하지 않았습니다.

font-size 를 16px 로 직접 지정했습니다.

12px – 작은 안내 글씨 크기입니다.

16px – 본문 크기입니다.

01 글자 크기와 단위

02 글꼴

03 줄과 간격

04 정렬과 꾸미기

05 넘치는 글자

– 연습문제

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

font-size 를 16px 로 직접 지정했습니다.

16px — 본문 크기입니다.

40px — 큰 제목 크기입니다.

자식 — 150% 입니다.

07단원에서 CSS 규칙이 어떻게 생겼는지, 08단원에서 선택자로 무엇을 고르는지 배웠습니다. 이제부터는 “골라서 무엇을 바꿀 것인가”, 즉 속성을 배웁니다. 09단원은 그중 글자에 관한 속성을 모아서 다룹니다.

```
p {
  font-size: 20px;
}
```

└──┬──┘ └──┬──┘
 | └─ 값 : 숫자 20 과 꼬리표 px 가 붙어 한 덩어리입니다
 └─ 속성 이름 : 글자 크기

단위 = 숫자 뒤에 붙여서 "이 숫자를 무엇으로 재는가" 를 알려 주는 꼬리표

길이를 말할 때 “20” 만 말하면 20cm 인지 20m 인지 알 수 없는 것과 같습니다. CSS 도 마찬가지로 숫자만 쓰면 그 줄을 통째로 버립니다. 이 파일에서 배울 단위는 네 가지입니다.

| | |
|-----|-----------------------------------|
| px | 화면의 점 개수로 재는 고정된 자 |
| % | 부모의 글자 크기를 100 으로 놓고 재는 자 |
| em | 부모의 글자 크기를 1 로 놓고 재는 자 (% 와 같은 말) |
| rem | 문서의 뿌리(html) 글자 크기를 1 로 놓고 재는 자 |

이 중 em 에는 초보자가 거의 반드시 한 번은 빠지는 함정이 있습니다. 섹션 4 에서 그 함정을 실제로 눈으로 보겠습니다.

이 파일은 글자 크기 하나만 다룹니다. 아래 상자들의 글자 크기를 눈으로 비교하면서 읽으세요. 특히 섹션 4 의 노란 상자는 이 단원에서 가장 유명한 함정입니다. 글자가 왜 저렇게 커지는지 스스로 설명할 수 있게 되는 것이 오늘의 목표입니다.

기준이 되는 크기

섹션 1

아무것도 지정하지 않은 글자는 16px 입니다. 모든 계산이 여기서 출발합니다.

브라우저는 아무 지시가 없어도 글자를 어떤 크기로든 그려야 합니다. 그래서 미리 정해 둔 기본값이 있습니다. 크롬·엣지·파이어폭스 모두 16px 입니다.

px 은 pixel(픽셀) 의 줄임말입니다. 화면을 이루는 아주 작은 점 하나를 말합니다. 16px 은 “글자 한 줄의 높이를 점 16개만큼으로 잡아라” 라는 뜻입니다.

아래 상자에는 이렇게 썼습니다.

```
<p>아무것도 지정하지 않았습니다.</p>
<p id="base-16">font size 를 16px 로 직접 지정했습니다.</p>

#base-16 { font-size: 16px; }
```

결과를 보면 두 줄의 크기가 똑같습니다. 16px 을 직접 쓴 것과 안 쓴 것이 같다는 뜻이고, 이것이 “기본이 16px” 이라는 말의 실제 의미입니다.

중요한 점이 하나 더 있습니다. 이 16px 은 브라우저 설정에서 사용자가 바꿀 수 있습니다. 눈이 불편한 사람은 이 값을 20px 로 올려 두고 웹을 봅니다. 섹션 5 에서 이 이야기가 다시 나옵니다. 기억해 두세요.

화면 두 줄의 글자 크기가 완전히 똑같아 보입니다

```
아무것도 지정하지 않았습니다.
font-size 를 16px 로 직접 지정했습니다.
```

▫ 직접 해보기 1

<style> 의 #base-16 을 font-size: 32px; 으로 바꾸고 저장(Ctrl+S) 한 뒤 새로고침(F5) 하세요. 두 줄의 크기가 달라지는 것을 확인하고 다시 16px 로 되돌려 놓으세요.

px

섹션 2

px 로 쓴 크기는 주변이 어떻든 그 숫자 그대로입니다.

px 의 장점은 예측이 쉽다는 것입니다. 24px 이라고 쓰면 어디에 놓든 24px 입니다. 부모가 무엇이든 상관없습니다.

```
#px-12 { font-size: 12px; }  
#px-16 { font-size: 16px; }  
#px-24 { font-size: 24px; }  
#px-40 { font-size: 40px; }
```

단점은 섹션 5 에서 이야기합니다. 미리 한 줄로 말하면 “사용자가 브라우저 글자 크기를 키워도 안 커진다” 입니다.

화면 네 줄이 위에서 아래로 갈수록 뚜렷하게 커집니다

12px – 작은 안내 글씨 크기입니다.
16px – 본문 크기입니다.
24px – 작은 제목 크기입니다.
40px – 큰 제목 크기입니다.

px 은 정수로만 쓸 필요가 없습니다. 15.5px 처럼 소수도 됩니다. 다만 사람이 읽기 좋으라고 보통 짝수나 4의 배수로 맞춰 씁니다.

▫ 직접 해보기 2

#px-40 을 80px 으로 바꿔 보세요. 글자가 상자를 뚫고 나가지 않고 줄이 바뀌면서 상자가 세로로 늘어납니다.

% 와 em

섹션 3

% 와 em 은 부모 요소의 글자 크기를 기준 삼습니다.

01단원 개념02 에서 부모·자식이라는 말을 처음 썼습니다. 감싼 쪽이 부모, 감싸인 쪽이 자식입니다.

```
<div id="unit-parent">      ← 부모. font-size: 20px
  <p id="pct-child">...</p> ← 자식. font-size: 150%
  <p id="em-child">...</p>  ← 자식. font-size: 1.5em
</div>
```

자식에 쓴 150% 와 1.5em 은 둘 다 “부모의 1.5배” 라는 뜻입니다. 부모가 20px 이니 자식은 둘 다 30px 이 됩니다.

$$20\text{px} \times 1.5 = 30\text{px}$$

즉 % 와 em 은 표기만 다르고 계산 방법이 똑같습니다.

```
150% = 1.5em
100% = 1em
80%  = 0.8em
```

한 가지만 미리 못 박아 둡니다. “부모 기준” 은 em 을 font-size 에 쓸 때의 이야기입니다. font-size 가 아닌 다른 속성(개념03 의 text-indent 같은 것) 에 em 을 쓰면 부모가 아니라 그 요소 자신의 글자 크기가 기준이 됩니다. 이 단원의 이 파일에서는 font-size 만 다루니 지금은 “부모 기준” 으로 보면 됩니다. 자세한 이야기는 개념03 섹션5 에서 다시 합니다.

여기서 왜 이런 단위가 필요한지 짚고 갑시다. px 은 “24px 로 해 줘” 라고 콕 집어 말하는 방식이고, em 은 “지금 글자보다 1.5배만 크게 해 줘” 라고 관계로 말하는 방식입니다. 나중에 부모 크기 하나만 바꾸면 자식들이 알아서 따라오기 때문에 편리합니다.

편리한 대신 함정이 있습니다. 바로 다음 섹션입니다.

화면 회색 상자 안에서 아래 두 줄이 첫 줄보다 크고, 두 줄끼리는 크기가 같습니다

```
부모입니다. 이 줄이 20px 입니다.
자식 - 150% 입니다.
자식 - 1.5em 입니다.
```

▫ 직접 해보기

3

#unit-parent 의 font-size 를 40px 으로 바꿔 보세요. 자식 두 줄은 손대지 않았는데도 같이 커집니다. 얼마가 되었을지 먼저 계산해 보고 확인하세요.

em 의 함정

섹션 4

같은 em 값을 겹쳐 쓰면 계속 곱해져서 글자가 폭발합니다.

아래 상자에는 똑같은 클래스를 네 겹으로 겹쳐 썼습니다.

```
.em-step { font-size: 1.2em; }

<div class="em-step">1단계
  <div class="em-step">2단계
    <div class="em-step">3단계
      <div class="em-step">4단계</div> </div> </div>
</div>
```

똑같이 1.2em 이라고 썼는데 왜 크기가 다 다를까요. em 의 기준이 “부모” 이기 때문입니다. 그리고 여기서는 부모도 1.2em 입니다.

```
1단계 : 16px × 1.2 = 19.2px
2단계 : 19.2px × 1.2 = 23.04px    ← 부모가 이미 19.2px 이라서
3단계 : 23.04px × 1.2 = 27.648px
4단계 : 27.648px × 1.2 = 33.1776px
```

곱셈이 계속 이어집니다. 이자가 붙는 것과 똑같습니다. 메뉴 안의 메뉴, 목록 안의 목록처럼 같은 클래스가 겹칠 수 있는 곳에서 이 일이 벌어지면 “왜 어떤 글자만 유난히 크지?” 하고 한참 헤매게 됩니다.

더 나쁜 것은 이것이 에러가 아니라는 점입니다. 브라우저는 시킨 대로 정확히 계산했을 뿐이고, 아무 경고도 하지 않습니다.

화면 안쪽으로 들어갈수록 글자가 한 단계씩 커집니다.

1단계 19.2px, 2단계 23.04px, 3단계 27.648px, 4단계 33.1776px 입니다

```
1단계 (1.2em)

2단계 (1.2em)

3단계 (1.2em)
4단계 (1.2em)
```

여기서 얻을 교훈은 “em 을 쓰지 마라” 가 아닙니다. em 은 겹칠 수 있는 곳에 쓰면 위험하다는 것입니다. 겹칠 걱정 없이 쓰고 싶으면 다음 섹션의 rem 을 씁니다.

▣ 직접 해보기

4

.em-step 안에 5단계를 하나 더 넣어 보세요. 넣기 전에 몇 px 이 될지 먼저 계산해 보세요. (힌트: 33.1776×1.2)

rem

섹션 5

rem 은 부모를 보지 않고 문서의 뿌리만 봅니다. 그래서 안 겹칩니다.

rem 은 root em 의 줄임말입니다. root 는 뿌리라는 뜻입니다. 문서의 뿌리는 <html> 요소입니다. 모든 태그를 감싸는 가장 바깥 태그였습니다(01단원).

```
1rem = html 요소의 글자 크기 = 보통 16px
```

아래 상자는 섹션 4 와 똑같은 모양으로 네 겹을 겹쳤습니다. 다른 것은 단위 하나뿐입니다.

```
.rem-step { font-size: 1.2rem; }
```

네 겹 모두 19.2px 로 똑같습니다. 몇 겹으로 겹쳐도 기준이 늘 <html> 하나라서 곱셈이 일어나지 않습니다.

```
1단계 : 16px × 1.2 = 19.2px
2단계 : 16px × 1.2 = 19.2px    ← 부모를 보지 않습니다
3단계 : 16px × 1.2 = 19.2px
4단계 : 16px × 1.2 = 19.2px
```

그러면 px 을 쓰면 되지 왜 rem 이냐고 물을 수 있습니다. 섹션 1 에서 “사용자가 브라우저 기본 글자 크기를 바꿀 수 있다” 고 했습니다. 그 사람이 기본을 20px 로 올려 두면

```
px 으로 쓴 글자 : 그대로 16px. 사용자의 설정을 무시합니다
rem 으로 쓴 글자 : 20px × 1.2 = 24px 로 같이 커집니다
```

눈이 불편한 사람을 위해서라도 글자 크기는 rem 쪽이 낫습니다.

화면 네 겹 모두 글자 크기가 똑같습니다. 전부 19.2px 입니다

```
1단계 (1.2rem)
2단계 (1.2rem)
3단계 (1.2rem)
4단계 (1.2rem)
```

⇒ 직접 해보기 5

.rem-step 의 1.2rem 을 1.2em 으로 바꿔 보세요. 위 섹션 4 와 똑같이 커집니다. 확인했으면 다시 rem 으로 되돌려 놓으세요.

언제 무엇을 쓸까

섹션 6

고민되면 글자 크기에는 rem 을 쓰세요. 그것만 지켜도 대부분 해결됩니다.

실무에서 쓰는 기준을 정리하면 이렇습니다.

rem 글자 크기에 기본으로 씁니다. 겹쳐도 안전하고 사용자 설정도 따릅니다
 px 테두리 두께처럼 절대 안 변해야 하는 값에 씁니다 (10단원에서 다시 만납니다)
 em 지금 글자 크기에 따라 같이 커져야 하는 값에 씁니다. 겹치지 않는 곳에서만
 % 글자 크기에는 em 과 같습니다. 굳이 둘 다 알 필요는 없습니다

rem 은 16 으로 나눠서 계산합니다. 자주 쓰는 값을 외워 두면 편합니다.

| | | |
|----------|--------|----------|
| 0.75rem | = 12px | 작은 안내 글씨 |
| 0.875rem | = 14px | 보조 설명 |
| 1rem | = 16px | 본문 |
| 1.25rem | = 20px | 작은 제목 |
| 1.5rem | = 24px | 제목 |
| 2rem | = 32px | 큰 제목 |

그리고 한 가지 더. 본문 크기는 되도록 건드리지 마세요. 16px 은 오랜 시간에 걸쳐 “가장 읽기 편한 크기”로 자리 잡은 값입니다. 본문을 12px 로 줄이면 예뻐 보일 수는 있어도 읽기가 힘들어집니다.

화면 세 줄이 위에서 아래로 갈수록 조금씩 작아집니다

1.5rem – 제목 (24px)
 1rem – 본문 (16px). 가장 많이 쓰는 크기입니다.
 0.875rem – 작은 글씨 (14px). 날짜나 안내에 씁니다.

➡ 직접 해보기

6

#guide-title 을 2rem 으로 바꾸면 몇 px 이 될까요? 먼저 계산해서 적어 보고, 바꿔서 확인하세요.

자주 하는 실수

섹션 7

07단원에서 배운 대로 CSS 는 틀려도 에러가 나지 않습니다. 틀린 줄만 조용히 버리고 나머지는 그대로 그립니다. 아래는 전부 살아 있는 예입니다.

이런 실수를 합니다

단위를 빠뜨림

이런 실수를 합니다

실수 1 — 단위를 빠뜨림

```
font-size: 20;
```

이 줄에는 font-size: 20; 이 걸려 있습니다. 커지지 않았습니다. 아무 일도 일어나지 않습니다. 숫자만 있고 단위가 없으니 브라우저가 이 줄을 읽지 못하고 버립니다. 가장 많이 나는 실수이고, 에러가 안 나서 가장 오래 헤매는 실수입니다.

이렇게 쓰세요

```
font-size: 20px;
```

이런 실수를 합니다

숫자와 단위 사이를 띄움

이런 실수를 합니다

실수 2 — 숫자와 단위 사이를 띄움

```
font-size: 20 px;
```

이 줄에는 font-size: 20 px; 이 걸려 있습니다. 역시 안 커집니다. 숫자와 단위는 붙여서 한 덩어리로 써야 합니다. 20px 가 하나의 값이지, 20 과 px 두 개가 아닙니다.

이런 실수를 합니다

em 을 겹쳐 써서 글자가 폭발함

이런 실수를 합니다

실수 3 — em 을 겹쳐 써서 글자가 폭발함

```
.card { font-size: 1.2em; } ← .card 안에 또 .card 가 들어가는 구조
```

섹션 4 에서 본 그대로입니다. 안쪽으로 들어갈수록 글자가 계속 커집니다. 화면 한쪽만 글자가 유난히 크다면 em 이 겹친 것은 아닌지 먼저 의심하세요. 해결은 간단합니다. 그 자리의 em 을 rem 으로 바꾸면 됩니다.

이런 실수를 합니다

속성 이름을 잘못 씀

이런 실수를 합니다

실수 4 — 속성 이름에서 하이픈을 빠뜨림

```
fontsize: 20px;
```

이 줄에는 fontsize: 20px; 이 걸려 있습니다. fontsize 라는 속성은 없습니다. 없는 속성은 그냥 무시됩니다. 이름을 틀려도 빨간 글씨 하나 안 뜹니다. VS Code 에서 속성 이름이 색으로 강조되지 않으면 오타를 의심하세요.

이런 실수를 합니다

세미콜론을 빠뜨림

이런 실수를 합니다

실수 5 — 세미콜론을 빠뜨림

```
font-size: 20px color: #c0392b;
```

이 줄에는 위 코드가 걸려 있습니다. 크기도 안 커지고 색도 안 바뀝니다. 두 선언이 같이 죽습니다. 세미콜론이 없으니 브라우저는 이 전체를 한 줄로 읽고, 뜻을 알 수 없어 통째로 버립니다. “위쪽 한 줄만 안 먹는 게 아니라 아래 줄까지 같이 안 먹는다” 면 거의 항상 그 위 어딘가에 세미콜론이 빠져 있습니다.

이런 실수를 합니다

대문자 단위 — 사실은 괜찮습니다

이런 실수를 합니다

실수 6 — 단위를 대문자로 씀

```
font-size: 20PX;
```

이 줄에는 font-size: 20PX; 이 걸려 있습니다. 이건 실제로 잘 동작합니다. 커진 것이 보이지요. 단위 이름은 대소문자를 가리지 않습니다. 그런데도 소문자로 쓰는 이유는 모두가 소문자로 쓰기로 약속했기 때문입니다. 항상 소문자로 쓰세요.

정리

▸ 직접 해보기 7

정답

1) <style> 안의 #base-16 을 이렇게 바꿉니다.

```
#base-16 { font-size: 32px; }
```

→ 아래 줄만 두 배로 커집니다. 위 줄은 그대로 16px 입니다.

두 줄이 같아 보였던 이유가 "기본이 16px 이라서" 였음을 확인할 수 있습니다.

2) #px-40 을 이렇게 바꿉니다.

```
#px-40 { font-size: 80px; }
```

→ 글자가 아주 커지면서 한 줄에 다 안 들어가 두 줄이 되고,

점선 상자가 세로로 길어집니다. 상자를 뚫고 나가지는 않습니다.

60px 까지는 아직 한 줄에 들어가서 줄이 안 바뀝니다.

이 상자에서는 70px 부터 두 줄이 됩니다. 폭이 좁아지면 그 기준도 달라집니다.

3) #unit-parent 를 이렇게 바꿉니다.

```
#unit-parent { font-size: 40px; background-color: #eef2f7; }
```

→ 자식 두 줄은 손대지 않았는데 60px 이 됩니다. $40 \times 1.5 = 60$ 이기 때문입니다.

% 와 em 이 "부모 기준" 이라는 말의 뜻입니다.

4) 4단계 안에 이렇게 한 겹 더 넣습니다.

```
<div class="em-step" id="em-step4">4단계 (1.2em)
  <div class="em-step">5단계 (1.2em)</div>
</div>
```

→ $33.1776 \times 1.2 = 39.81312\text{px}$ 이 됩니다. 계속 곱해진다는 것을 확인하세요.

5) .rem-step 을 이렇게 바꿉니다.

```
.rem-step { font-size: 1.2em; background-color: #f0f7ff; }
```

→ 파란 상자가 노란 상자와 똑같이 계단처럼 커집니다.

바뀐 것은 글자 세 개(rem → em) 뿐인데 결과가 완전히 달라집니다.

6) 2rem 은 $16 \times 2 = 32\text{px}$ 입니다.

```
#guide-title { font-size: 2rem; }
```

→ 첫 줄이 눈에 띄게 커집니다. rem 계산은 항상 16 을 곱하거나 나누면 됩니다.

이 파일에서 '가르치는' CSS 만 여기에 씁니다.

기준이 되는 크기

섹션 1

```
#base-16 {
  font-size: 16px;
}
```

화면 바로 위의 아무것도 지정하지 않은 줄과 글자 크기가 똑같아 보입니다

```
#px-12 {
  font-size: 12px;
}
#px-16 {
  font-size: 16px;
}
#px-24 {
  font-size: 24px;
}
#px-40 {
  font-size: 40px;
}
```

화면 위에서 아래로 갈수록 글자가 눈에 띄게 커집니다

% 와 em 은 부모를 기준으로 잡니다

```
#unit-parent {
  font-size: 20px;
  background-color: #eef2f7;
}
#pct-child {
  font-size: 150%;
}
#em-child {
  font-size: 1.5em;
}
```

화면 두 자식 줄의 글자 크기가 서로 똑같습니다 (둘 다 20px 의 1.5배 = 30px)

em 이 겹치면 곱해집니다

```
.em-step {
  font-size: 1.2em;
  background-color: #fff8dc;
}
```

화면 안으로 한 겹 들어갈 때마다 글자가 계속 커집니다

rem 은 뿌리만 봅니다

섹션 5

```
.rem-step {
  font-size: 1.2rem;
  background-color: #f0f7ff;
}
```

화면 네 겹 모두 글자 크기가 똑같습니다

실제로 쓰는 크기

섹션 6

```
#guide-title {
  font-size: 1.5rem;
}
#guide-body {
  font-size: 1rem;
}
#guide-small {
  font-size: 0.875rem;
}
```

화면 제목 / 본문 / 작은 글씨 세 줄의 크기가 단계적으로 작아집니다

자주 하는 실수 (일부러 틀리게 써 둔 것들)

섹션 7

```
#miss-unit {
  font-size: 20;
}
```

화면 아무 일도 일어나지 않습니다. 다른 줄과 크기가 같습니다

```
#miss-space {
  font-size: 20 px;
}
```

화면 역시 아무 일도 일어나지 않습니다

```
#miss-name {
  fontsize: 20px;
}
```

화면 역시 아무 일도 일어나지 않습니다

```
#miss-semi {  
  font-size: 20px color: #c0392b;  
}
```

화면 크기도 색도 둘 다 안 바뀝니다. 두 선언이 같이 죽습니다

```
#ok-upper {  
  font-size: 20PX;  
}
```

화면 이건 정상으로 커집니다. 단위는 대문자로 써도 됩니다

글꼴

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 글꼴

앞 파일에서 글자의 '크기'를 정했습니다. 이번에는 글자의 '모양'입니다.

```
p {
  font-family: "맑은 고딕", sans-serif;
}
```

속성 이름 : 글꼴 값 : 글꼴 이름을 쉼표로 여러 개 적습니다

여기서 초보자가 가장 먼저 놀라는 것이 "왜 이름을 여러 개 적지?" 입니다.

이유는 간단합니다. 웹은 남의 컴퓨터에서 열립니다. 내 컴퓨터에 있는 글꼴이 그 사람 컴퓨터에도 있으리라는 보장이 없습니다. 브라우저는 목록을 앞에서부터 훑어서 그 컴퓨터에 설치된 첫 글꼴을 씁니다.

```
font-family: "없는글꼴", "맑은 고딕", sans-serif;
```

① ② ③

① 을 찾아봅니다. 없습니다 → ② 로 넘어갑니다

② 를 찾아봅니다. 있습니다 → 이걸로 그림니다. ③ 은 보지도 않습니다

이 목록을 폰트 스택(font stack) 이라고 부릅니다. 쌓아 둔 더미라는 뜻입니다.

참고로 이 자료는 '글꼴' 과 '폰트' 를 같은 뜻으로 씁니다. 우리말로는 글꼴, 영어에서 온 말로는 폰트입니다. font-family 처럼 코드에 붙은 이름이나 '폰트 스택' 같은 굳어진 말에서는 폰트를, 설명 문장에서는 글꼴을 주로 씁니다. 다른 것을 가리키는 것이 아닙니다.

그리고 아주 중요한 성질이 하나 있습니다.

글꼴 이름을 틀리게 적어도 에러가 나지 않습니다.
브라우저는 조용히 다음 이름으로 넘어갈 뿐입니다.

그래서 "글꼴이 안 바뀌어요" 의 원인 대부분은 이름 오타입니다. 이 파일에서는 그 오타를 눈으로 알아채는 방법까지 함께 봅니다.

참고로 이 자료의 공통.css 는 body 에 이미 이렇게 걸어 두었습니다.

```
body { font-family: "맑은 고딕", "Malgun Gothic", sans-serif; }
```

font-family 는 상속되는 속성입니다(07단원 개념05). 그래서 지금 이 페이지의 모든 글자는 따로 지정하지 않아도 맑은 고딕으로 그려지고 있습니다.

이 파일은 글꼴 이름을 어떻게 적는가를 다룹니다. 아래 상자들의 글자 모양을 눈으로 비교하면서 읽으세요. 크기가 아니라 획의 생김새를 보는 것입니다. 마지막에는 인터넷에서 글꼴을 받아 쓰는 웹폰트도 이름만 알아 둡니다.

font-family

섹션 1

font-family 는 글자의 모양을 정하는 속성입니다.

family 는 '가족' 이라는 뜻입니다. 글꼴 하나에는 보통·굵게·기울임 같은 식구가 여럿 딸려 있어서 이런 이름이 붙었습니다.

아래 상자에는 이렇게 썼습니다.

```
#ff-serif { font-family: serif; }
```

serif(세리프) 는 글자 획 끝에 붙은 작은 빠침을 가리키는 말입니다. 한글로는 명조체·바탕체 계열이 여기에 해당합니다.

비교 대상인 첫 줄에는 아무것도 지정하지 않았습니다. 공통.css 가 body 에 맑은 고딕을 걸어 두었고 font-family 는 상속되므로, 첫 줄은 맑은 고딕으로 그려집니다.

화면 첫 줄은 획이 곧은 고딕체, 둘째 줄은 획 끝에 뾰침이 있는 명조체입니다

아무것도 지정하지 않았습니다. Hello 123
font-family: serif 를 지정했습니다. Hello 123

크기는 그대로이고 모양만 바뀐 것을 확인하세요. font-size 와 font-family 는 서로 아무 상관이 없습니다.

▹ 직접 해보기

1

#ff-serif 의 값을 monospace 로 바꿔 보세요. 글자 하나하나의 가로 폭이 전부 같아지는 것을 확인하세요.

폰트 스택

섹션 2

브라우저는 목록을 앞에서부터 훑어서 처음 찾은 글꼴을 씁니다.

아래 세 줄을 나란히 보면 규칙이 한눈에 들어옵니다.

```
#ff-only-serif { font-family: serif; }
#ff-fallback   { font-family: 없는글꼴, serif; }
#ff-first      { font-family: "맑은 고딕", serif; }
```

두 번째 줄에는 없는글꼴이라는 이름을 일부를 앞에 두었습니다. 그런 글꼴은 어느 컴퓨터에도 없습니다. 그런데도 에러는 나지 않습니다. 브라우저는 그냥 다음 칸인 serif 로 넘어가서 명조체로 그립니다. 그래서 첫 줄과 둘째 줄이 똑같아 보입니다.

세 번째 줄은 반대입니다. 맑은 고딕이 이 컴퓨터에 있으므로 거기서 멈춥니다. 뒤에 적어 둔 serif 는 읽지도 않습니다.

정리하면 이렇습니다.

```
font-family: A, B, C;
A 가 있으면 A 로 끝. 없으면 B. B 도 없으면 C.
```

이 목록을 폰트 스택이라고 부릅니다. 보통 이런 순서로 씁니다.

1 정말 쓰고 싶은 글꼴

2 그게 없을 때 비슷한 대체 글꼴

3 맨 뒤에 총칭 글꼴 (다음 섹션)

화면 첫 줄과 둘째 줄이 똑같은 명조체이고, 셋째 줄만 고딕체입니다

serif – 명조체입니다. Hello 123
없는글꼴, serif – 앞이 없어서 명조체가 됩니다. Hello 123
"맑은 고딕", serif – 앞이 있어서 고딕체가 됩니다. Hello 123

둘째 줄에서 아무 경고도 없었다는 점이 중요합니다. 없는 이름을 적어도 브라우저는 말없이 다음으로 넘어갑니다. 이것이 다음 섹션들과 실수 섹션의 뿌리가 되는 성질입니다.

▫ 직접 해보기

2

#ff-first의 "맑은 고딕"을 "맑은 고딕2"로 바꿔 보세요. 셋째 줄도 명조체가 됩니다. 확인했으면 되돌려 놓으세요.

총칭 글꼴

섹션 3

총칭 글꼴은 어느 컴퓨터에나 반드시 있는 마지막 안전망입니다.

총칭 글꼴(generic family)은 특정 글꼴 이름이 아니라 '분류 이름'입니다. 브라우저가 "그 분류에 해당하는 글꼴 중 아무거나"를 알아서 골라 줍니다. 따라서 절대 실패하지 않습니다.

| | |
|------------|--------------------------------|
| serif | 획 끝에 뾰침이 있는 글꼴 (한글 명조체·바탕체 계열) |
| sans-serif | 뾰침이 없는 글꼴 (한글 고딕체 계열) |
| monospace | 글자 폭이 전부 같은 글꼴 (코드를 보여 줄 때) |
| cursive | 손글씨 느낌 |
| fantasy | 장식용 |

실제로 쓰는 것은 앞의 셋뿐입니다. 뒤의 둘은 컴퓨터마다 너무 달라 잘 안 씁니다.

총칭 글꼴은 이름이 아니라 정해진 낱말이므로 따옴표를 붙이지 않습니다. 따옴표를 붙이면 "sans-serif라는 이름의 글꼴"을 찾게 되어 실패합니다.

아래 마지막 줄은 총칭 글꼴을 안 적었을 때입니다.

```
#gen-none { font-family: 없는글꼴; }
```

찾을 글꼴이 하나도 없으니 브라우저가 자기 기본 글꼴로 그립니다. 그 기본 글꼴이 무엇인지는 컴퓨터와 브라우저 설정마다 다릅니다. 내 화면에서는 멀쩡해 보여도 남의 화면에서는 엉뚱하게 보일 수 있다는 뜻입니다. 그래서 스택 맨 뒤에는 항상 총칭 글꼴을 하나 붙여 둡니다.

참고로 아래 세 줄에는 font-size: 1rem을 같이 적어 두었습니다. 세 줄의 크기를 똑같이 맞춰서 모양만 비교하려는 것입니다.

화면 위 세 줄의 글자 모양이 서로 뚜렷이 다릅니다.

마지막 줄은 이 컴퓨터에서는 고딕체로 보이지만, 다른 컴퓨터에서는 다르게 보일 수 있습니다

serif – 빠침이 있습니다. Hamburg 123
 sans-serif – 빠침이 없습니다. Hamburg 123
 monospace – 글자 폭이 같습니다. Hamburg 123
 총칭 글꼴을 안 적었습니다. Hamburg 123

▫ 직접 해보기 3

#gen-mono 의 monospace 를 “monospace” 처럼 따옴표로 감싸 보세요. 글자 폭이 같아지는 효과가 사라집니다. 총칭 글꼴에는 따옴표를 붙이면 안 된다는 뜻입니다. 확인했으면 되돌려 놓으세요.

따옴표

섹션 4

공백이나 숫자·점이 들어간 이름은 따옴표로 감싸는 것이 안전합니다.

한글 글꼴 이름에는 공백이 자주 들어갑니다. 맑은 고딕, 나눔 고딕처럼요. 이럴 때 따옴표를 씌우는 것이 규칙입니다.

| | |
|-----------------------------------|-------------|
| font-family: "맑은 고딕", sans-serif; | 권장 |
| font-family: 맑은 고딕, sans-serif; | 크롬에서는 동작합니다 |

둘째 줄도 실제로는 잘 동작합니다. 아래 두 줄을 보면 결과가 같습니다. 그래서 “따옴표 없어도 되네” 하고 넘어가기 쉽습니다.

그런데 이런 이름에서는 이야기가 달라집니다.

| | |
|----------------------------|-----------------|
| font-family: 2026체, serif; | ← 이름이 숫자로 시작합니다 |
|----------------------------|-----------------|

CSS 는 따옴표 없는 이름이 숫자로 시작하는 것을 허용하지 않습니다. 그래서 이 줄을 통째로 버립니다. 결과가 어떻게 되는지 보세요.

serif 조차 적용되지 않습니다.

“없는 글꼴이라 serif 로 넘어갔겠지” 가 아닙니다. 선언 자체가 사라져서 물려받은 맑은 고딕이 그대로 남습니다. 따옴표를 씌우면 줄이 살아나고, 그때는 정상적으로 serif 로 넘어갑니다.

구분해서 기억해 두면 좋습니다.

| | |
|--------|--------------------------------|
| 이름이 틀림 | → 그 이름만 건너뛴니다. 뒤의 글꼴은 적용됩니다 |
| 문법이 틀림 | → 줄 전체가 죽습니다. 뒤의 글꼴도 적용되지 않습니다 |

헛갈릴 일을 아예 만들지 않는 방법은 하나입니다. 총칭 글꼴이 아닌 실제 글꼴 이름에는 그냥 항상 따옴표를 씌우세요.

영어 이름도 마찬가지입니다.

```
font-family: "Times New Roman", serif;  
font-family: "Courier New", monospace;
```

화면 위 세 줄은 모두 고딕체이고, 마지막 줄만 명조체입니다.

셋째 줄이 명조체가 아닌 것이 핵심입니다

```
따옴표를 씌웠습니다. 고딕체입니다. Hello  
따옴표가 없습니다. 이것도 고딕체입니다. Hello  
2026체, serif – 따옴표 없음. 명조체가 되지 않습니다. Hello  
"2026체", serif – 따옴표 있음. 명조체가 됩니다. Hello
```

▹ 직접 해보기

4

#q-digit의 값을 "2026체", serif로 고쳐 보세요. 셋째 줄도 명조체가 됩니다. 확인했으면 되돌려 놓으세요.

font-weight

섹션 5

굵기는 숫자로도, 낱말로도 적을 수 있습니다.

쓸 수 있는 값은 두 가지 방식입니다.

```
숫자 : 100 200 300 400 500 600 700 800 900 (100 단위로만)  
낱말 : normal(= 400) bold(= 700)
```

normal과 400은 완전히 같은 말이고, bold와 700도 완전히 같은 말입니다. 02단원에서 배운 `<strong>` 태그가 굵게 보였던 이유가 바로 이것입니다. 브라우저가 strong 요소에 `font-weight: bold`를 미리 걸어 두었기 때문입니다.

여기서 초보자가 꼭 겪는 일이 하나 있습니다.

숫자를 잘게 나눠 써도 화면이 안 바뀔 수 있습니다.

굵기 100부터 900까지 아홉 단계를 다 가진 글꼴은 흔하지 않습니다. 맑은 고딕은 크게 보통과 굵게 두 가지로 되어 있습니다. 없는 굵기를 요청하면 브라우저가 가진 것 중 가까운 쪽으로 대신 그립니다.

그래서 아래 결과에서 400과 normal이 완전히 같고, 700과 bold와 900도 서로 완전히 같습니다. 아홉 단계를 다 적어 봐도 실제로는 두세 가지 모습밖에 안 나옵니다.

100 줄만 눈여겨보세요. 윈도우에는 맑은 고딕의 가는 짝(Semilight)도 같이 깔려 있어서, 100~300은 400보다 아주 살짝 가늘게 그려지기도 합니다. 요청한 만큼 "아주 가늘게"는 아닙니다. 본문 크기에서는 알아채기 어려울 정도입니다.

어느 쪽이든 여러분이 잘못 쓴 것이 아니라, 그 글꼴에 그 굵기가 없는 것입니다. 이 결과는 컴퓨터에 설치된 글꼴에 따라 달라질 수 있습니다.

화면 위 세 줄은 굵기 차이가 거의 없고(100 이 아주 살짝 가늘 수 있습니다),

아래 세 줄은 뚜렷하게 굵습니다. strong 태그도 아래 세 줄과 같은 굵기입니다

100 - 아주 가늘게 (요청은 했습니다)
 normal - 보통
 400 - 보통 (normal 과 같은 말)
 700 - 굵게
 bold - 굵게 (700 과 같은 말)
 900 - 아주 굵게 (요청은 했습니다)
 02단원에서 배운 strong 태그도 굵게 보입니다.

굵기를 되돌릴 때는 font-weight: normal; 을 씁니다. 예를 들어 h1 은 원래 굵은데, 굵지 않은 제목을 만들고 싶을 때 이렇게 씁니다.

⇒ 직접 해보기

5

#fw-900 을 font-weight: 500; 으로 바꿔 보세요. 굵어질까요, 안 굵어질까요? 예상하고 확인해 보세요.

font-style

섹션 6

font-style: italic; 이면 기울고, normal 이면 똑바로 섭니다.

값은 사실상 두 개만 쓰면 됩니다.

normal 똑바로 (기본값)
 italic 기울임

02단원의 태그가 기울어 보였던 이유도 같습니다. 브라우저가 em 요소에 font-style: italic 을 미리 걸어 두었습니다.

한글에서는 주의할 점이 하나 있습니다. 영문 글꼴에는 기울임 전용 모양이 따로 만들어져 있지만, 한글 글꼴에는 대개 그런 것이 없습니다. 그래서 브라우저가 글자를 억지로 비스듬히 눕혀서 보여 줍니다. 모양이 어색해지기 쉬우므로 한글 본문에는 잘 쓰지 않습니다.

강조를 하고 싶다면 09단원 안에서는 이런 선택지가 더 낫습니다.

굵게 하기 font-weight: bold;
 색 바꾸기 color: #c0392b; (07단원)

화면 첫 줄이 오른쪽으로 비스듬히 눕고, 둘째 줄은 똑바릅니다.

한글보다 영어 쪽 기울기가 더 자연스러워 보입니다

italic – 기울어집니다. Hello World
normal – 똑바로 섭니다. Hello World
02단원에서 배운 em 태그도 기울어 보입니다.

▣ 직접 해보기 6

#fs-normal 에 font-style: italic; 과 font-weight: bold; 을 함께 걸어 보세요. 두 속성은 서로 방해하지 않습니다.

웹폰트

섹션 7

웹폰트는 글꼴 파일을 인터넷에서 받아 와서 쓰는 방법입니다.

지금까지 배운 방식에는 한계가 있습니다. 아무리 예쁜 글꼴을 적어도 보는 사람 컴퓨터에 없으면 소용이 없습니다.

그래서 나온 것이 웹폰트입니다. “이 주소에 글꼴 파일이 있으니 받아서 쓰세요” 라고 브라우저에게 알려 주는 것입니다.

방법은 두 가지입니다.

- 1 글꼴 파일을 직접 준비하고 @font-face 로 이름을 붙여 줍니다
- 2 구글 폰트 같은 서비스가 만들어 준 주소를 <link> 로 붙여 옵니다

★ 이 자료에서는 실제로 불러오지 않습니다. 웹폰트는 인터넷 연결이 있어야 동작하기 때문입니다. 인터넷이 끊긴 교실에서 열면 글꼴이 적용되지 않고 다음 글꼴로 넘어갑니다. 어떻게 생겼는지만 눈에 익혀 두고, 실제 사용은 인터넷이 되는 곳에서 해 보세요.

그리고 주의할 점이 하나 더 있습니다. 한글 웹폰트는 글자 수가 많아서 파일이 큼니다(수 MB 인 경우도 있습니다). 생각 없이 여러 개를 불러오면 페이지가 눈에 띄게 느려집니다.

화면 검은 코드 상자 안에 위 코드가 글자 그대로 보입니다

```
@font-face {  
  font-family: "내글꼴";           /* 내가 붙이는 이름 */  
  src: url("../fonts/my-font.woff2"); /* 글꼴 파일이 있는 곳 */  
}  
  
body {  
  font-family: "내글꼴", "맑은 고딕", sans-serif;  
}
```

화면 검은 코드 상자 안에 위 코드가 글자 그대로 보입니다

```
<!-- head 안에 붙입니다 (인터넷 연결 필요) -->
<link rel="stylesheet" href="https://fonts.googleapis.com/..." />

/* 그러면 CSS 에서 이름으로 부를 수 있게 됩니다 */
body {
  font-family: "Noto Sans KR", "맑은 고딕", sans-serif;
}
```

어느 방법이든 맨 뒤의 폰트 스택은 그대로 필요합니다. 인터넷이 느리거나 끊기면 웹폰트가 안 올 수 있고, 그때 뒤의 글꼴이 대신 나옵니다.

▫ 직접 해보기 7

위 두 코드 상자에서 글꼴 이름이 적힌 자리를 모두 찾아보세요. 방법 1 에는 두 곳, 방법 2 에는 한 곳 있습니다. 이름을 바꾸려면 어디를 고쳐야 하는지 짚어 보는 연습입니다.

자주 하는 실수

섹션 8

글꼴은 특히 틀려도 티가 안 나는 속성입니다. “적용이 안 된 것” 과 “다음 글꼴로 넘어간 것” 을 구분하는 눈이 필요합니다.

이런 실수를 합니다

글꼴 이름 오타

이런 실수를 합니다

실수 1 — 글꼴 이름 오타로 조용히 무시됨

```
font-family: 맑은고딕, serif; /* 공백을 빠뜨렸습니다 */
```

이 줄에는 위 코드가 걸려 있습니다. 명조체가 되었습니다. 맑은고딕 은 맑은 고딕 과 다른 이름이라 그런 글꼴이 없고, 그래서 뒤의 serif 로 넘어간 것입니다. 경고는 어디에도 없습니다. “원하는 글꼴이 아니라 뒤의 글꼴이 나왔다” 는 것 자체가 오타의 신호입니다.

이렇게 쓰세요

```
font-family: "맑은 고딕", serif;
```

이런 실수를 합니다

한글 글꼴 이름에 따옴표를 안 씀

이런 실수를 합니다

실수 2 — 한글 글꼴 이름에 따옴표를 안 씀

```
font-family: 2026체, serif;
```

섹션 4 에서 본 그대로입니다. 이름이 숫자로 시작하는데 따옴표가 없으면 그 줄 전체가 버려집니다. 뒤에 적어 둔 serif 도 함께 사라집니다. 공백이 든 이름도 원칙은 같습니다. 크롬에서는 넘어가 주지만 기대지 마세요. 실제 글꼴 이름에는 항상 따옴표를 씁주세요.

이런 실수를 합니다

총칭 글꼴을 맨 앞에 둠

이런 실수를 합니다

실수 3 — 총칭 글꼴을 맨 앞에 둠

```
font-family: sans-serif, "맑은 고딕";
```

이 줄에는 위 코드가 걸려 있습니다. sans-serif 는 절대 실패하지 않는 값입니다. 그러니 여기서 찾기가 끝나고 뒤에 적은 맑은 고딕은 읽히지도 않습니다. 쓰고 싶은 글꼴은 앞에, 안전망인 총칭 글꼴은 맨 뒤에 둡니다.

이런 실수를 합니다

총칭 글꼴을 아예 안 씀

이런 실수를 합니다

실수 4 — 총칭 글꼴을 아예 안 씀

```
font-family: "내가 만든 글꼴";
```

내 컴퓨터에 그 글꼴이 깔려 있으면 잘 보입니다. 그래서 다 됐다고 생각하게 됩니다. 하지만 그 글꼴이 없는 컴퓨터에서는 브라우저 기본 글꼴로 그려집니다. 어떤 글꼴이 나올지는 컴퓨터마다 다릅니다. 맨 뒤에 총칭 글꼴을 한 칸 붙여 두세요.

이런 실수를 합니다

font-weight 값에 없는 낱말을 씀

이런 실수를 합니다

실수 5 — 굵기 값에 없는 낱말을 씀

```
font-weight: 굵게;
```

이 줄에는 font-weight: 굵게; 가 걸려 있습니다. 굵어지지 않았습니니다. 아무 일도 일어나지 않습니다. CSS 가 아는 낱말은 normal 과 bold 뿐이고, 나머지는 100 부터 900 까지의 숫자입니다. 한글 낱말은 알아듣지 못합니다.

이런 실수를 합니다

내 컴퓨터에만 있는 글꼴을 씀

이런 실수를 합니다

실수 6 — 내 컴퓨터에만 있는 글꼴을 믿음

```
font-family: "내가 예전에 설치한 폰트", sans-serif;
```

이 경우는 코드가 틀린 것이 아니라 확인 방법이 틀린 것입니다. 내 화면만 보고 "잘 나오네" 하면 안 됩니다. 웹은 남의 컴퓨터에서 열립니다. 남에게도 있는 글꼴인지 먼저 따지고, 꼭 써야 한다면 섹션 7 의 웹폰트를 씁니다.

정리

▹ 직접 해보기 8

정답

1) #ff-serif 를 이렇게 바꿉니다.

```
#ff-serif { font-family: monospace; }
```

→ 둘째 줄의 글자 폭이 전부 같아집니다. 특히 Hello 123 부분에서

i 와 m 이 같은 칸을 차지하는 것이 눈에 띕니다.

2) #ff-first 를 이렇게 바꿉니다.

```
#ff-first { font-family: "맑은 고딕2", serif; }
```

→ 셋째 줄도 명조체가 됩니다.

"맑은 고딕2" 라는 글꼴이 없어서 뒤의 serif 로 넘어갔기 때문입니다. 이름 한 글자만 틀려도 이렇게 된다는 것을 확인하세요.

3) #gen-mono 를 이렇게 바꿉니다.

```
#gen-mono { font-family: "monospace"; font-size: 1rem; }
```

→ 글자 폭이 같아지는 효과가 사라지고 고딕체로 보입니다.

따옴표를 씌우는 순간 "monospace 라는 이름의 글꼴" 을 찾게 되고,
그런 이름의 글꼴은 없으니 브라우저 기본 글꼴로 그려집니다.

4) #q-digit 을 이렇게 바꿉니다.

```
#q-digit { font-family: "2026체", serif; }
```

→ 셋째 줄이 명조체가 됩니다.

따옴표를 씌우자 줄이 살아나서, 없는 글꼴을 건너뛰고 serif 가 쓰인 것입니다.

5) #fw-900 을 이렇게 바꿉니다.

```
#fw-900 { font-weight: 500; }
```

→ 굵어지지 않습니다. 보통 굵기로 보입니다.

맑은 고딕에 500 짜리 굵기가 없어서 가까운 400 쪽으로 그려지기 때문입니다.
값이 틀린 것이 아니라 글꼴에 그 굵기가 없는 것입니다.

6) #fs-normal 을 이렇게 바꿉니다.

```
#fs-normal { font-style: italic; font-weight: bold; }
```

→ 둘째 줄이 기울면서 동시에 굵어집니다.

굵기와 기울임은 서로 다른 속성이라 함께 걸 수 있습니다.

7) 글꼴 이름이 적힌 자리는 이렇습니다.

방법 1 : @font-face 안의 font-family: "내글꼴" (이름을 짓는 곳)
body 의 font-family: "내글꼴" (그 이름을 쓰는 곳)
방법 2 : body 의 font-family: "Noto Sans KR"

→ 방법 1 은 두 곳의 이름이 같아야 합니다. 한쪽만 고치면 연결이 끊겨서

글꼴이 적용되지 않고 다음 글꼴로 넘어갑니다.

이 파일에서 '가르치는' CSS 만 여기에 씁니다.

font-family 하나만 적어 보기

섹션 1

```
#ff-serif {
  font-family: serif;
}
```

화면 획 끝에 빠침이 달린 글꼴로 바뀝니다. 한글은 명조체처럼 보입니다

폰트 스택

섹션 2

```
#ff-only-serif {
  font-family: serif;
}
#ff-fallback {
  font-family: 없는글꼴, serif;
}
#ff-first {
  font-family: "맑은 고딕", serif;
}
```

화면 위 두 줄은 명조체, 마지막 줄만 고딕체입니다.

앞에서부터 찾다가 처음 만난 글꼴을 쓴다는 뜻입니다

총칭 글꼴

섹션 3

```
#gen-serif {
  font-family: serif;
  font-size: 1rem;
}
#gen-sans {
  font-family: sans-serif;
  font-size: 1rem;
}
#gen-mono {
  font-family: monospace;
  font-size: 1rem;
}
#gen-none {
  font-family: 없는글꼴;
}
```

화면 명조체 / 고딕체 / 글자폭이 모두 같은 글꼴, 세 가지가 확실히 다르게 보입니다

따옴표

섹션 4

```
#q-noquote {
  font-family: 맑은 고딕, sans-serif;
}
#q-quote {
  font-family: "맑은 고딕", sans-serif;
}
#q-digit {
  font-family: 2026체, serif;
}
```

화면 이름이 숫자로 시작하는데 따옴표가 없어서 이 줄 전체가 버려집니다.

serif 도 적용되지 않고 물려받은 맑은 고딕 그대로 보입니다

```
#q-digitq {
  font-family: "2026체", serif;
}
```

화면 따옴표를 씌우니 줄이 살아나서, 없는 글꼴을 건너뛰고 serif 가 적용됩니다

font-weight

섹션 5

```
#fw-100 {
  font-weight: 100;
}
#fw-normal {
  font-weight: normal;
}
#fw-400 {
  font-weight: 400;
}
#fw-700 {
  font-weight: 700;
}
#fw-bold {
  font-weight: bold;
}
#fw-900 {
```

```
font-weight: 900;
}
```

화면 100·normal·400 세 줄은 굵기 차이가 거의 없고,

700·bold·900 세 줄은 뚜렷하게 굵습니다

font-style

섹션 6

```
#fs-italic {
  font-style: italic;
}
#fs-normal {
  font-style: normal;
}
```

화면 첫 줄은 오른쪽으로 기울고 둘째 줄은 똑바로 서 있습니다

자주 하는 실수

섹션 8

```
#miss-typo {
  font-family: 맑은고딕, serif;
}
```

화면 '맑은고딕' 을 붙여 쓴 이름은 없는 이름이라 건너뛰고 serif 가 적용됩니다

```
#miss-generic-first {
  font-family: sans-serif, "맑은 고딕";
}
```

화면 고딕체로 보입니다. sans-serif 에서 이미 끝나서 뒤는 읽지도 않습니다

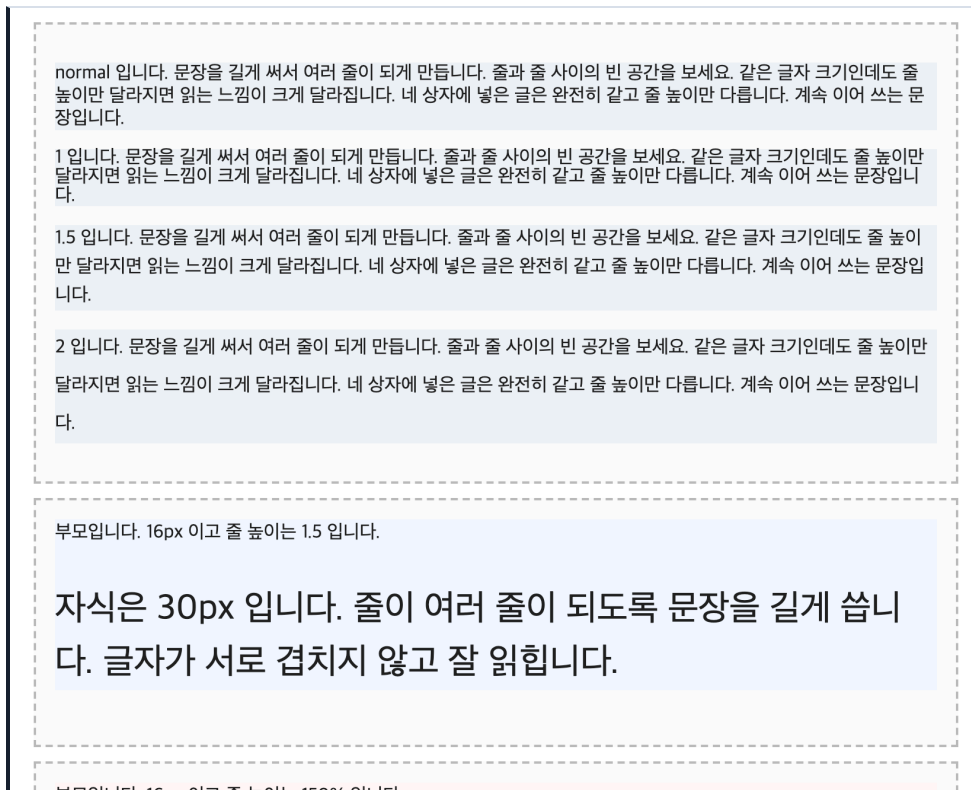
```
#miss-weight {
  font-weight: 굵게;
}
```

화면 굵어지지 않습니다. 아무 일도 일어나지 않습니다

줄과 간격

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 줄과 간격

앞의 두 파일에서 글자의 크기와 모양을 정했습니다. 이번에는 글자와 글자 '사이', 줄과 줄 '사이' 입니다.

읽기 편한 글의 조건은 사실 글꼴이 아니라 간격입니다. 같은 글이라도 줄 간격만 바꾸면 읽는 느낌이 완전히 달라집니다.

이 파일에서 배울 속성은 다섯 개입니다.

| | |
|----------------|--------------------|
| line-height | 줄과 줄 사이 (가장 중요합니다) |
| letter-spacing | 글자와 글자 사이 (자간) |
| word-spacing | 단어와 단어 사이 |
| text-indent | 문단 첫 줄 들여쓰기 |
| white-space | 공백과 줄바꿈을 어떻게 다룰지 |

먼저 line-height 가 무엇인지 그림으로 봅시다. 한 줄이 차지하는 세로 공간 전체를 줄 높이라고 합니다.



글자 크기가 16px 인데 줄 높이를 24px 로 주면, 글자가 차지하는 높이보다 줄이 높아집니다. 남는 공간을 브라우저가 위아래로 반씩 나눠 붙입니다. 그래서 줄 높이를 키우면 줄 사이가 벌어져 보이는 것입니다.

“글자가 차지하는 높이” 는 글자 크기와 딱 같지 않습니다. 글꼴마다 위아래로 여유를 조금씩 더 갖고 있어서, 맑은 고딕 16px 은 21px 을 씁니다. 그래서 줄 높이를 16px(= 1배) 로 줄이면 남는 공간이 없는 정도가 아니라 줄끼리 살짝 파고들게 됩니다. 섹션 1 의 두 번째 상자(line-height: 1) 가 유난히 답답해 보이는 이유입니다.

한 가지 미리 알아 둘 것이 있습니다. 이 자료의 공통.css 는 body 에 이미 이렇게 걸어 두었습니다.

```
body { line-height: 1.7; }
```

line-height 는 상속되는 속성입니다. 그래서 이 페이지의 모든 글자는 따로 지정하지 않아도 1.7 을 물려받고 있습니다. 즉 이 페이지에서 “아무것도 안 준 상태” 는 normal 이 아니라 1.7 입니다.

이 파일은 글자 사이의 빈 공간을 다룹니다. 아래 상자들을 볼 때는 글자가 아니라 글자 사이의 여백을 보세요. 특히 섹션 2 는 “왜 line-height 에는 단위를 안 붙이는가” 라는 실무에서 매우 자주 나오는 질문의 답입니다.

line-height

섹션 1

line-height 는 한 줄이 차지하는 세로 높이입니다.

값을 주는 방법은 세 가지입니다.

| | |
|----------------------|---------------------------------|
| line-height: normal; | 브라우저가 알아서 (글자 크기의 1.2 ~ 1.3 배쯤) |
| line-height: 1.5; | 글자 크기의 1.5배 ← 이 방법을 씁니다 |
| line-height: 24px; | 무조건 24px |

아래 상자에는 이렇게 썼습니다. 글자 크기는 모두 16px 로 같습니다.

```
#lh-normal { line-height: normal; }
#lh-1      { line-height: 1; }
#lh-15     { line-height: 1.5; }
#lh-2      { line-height: 2; }
```

결과를 보면 줄 사이만 달라집니다. 글자 크기는 넷 다 같습니다. line-height 는 글자를 키우는 속성이 아니라 줄이 차지하는 자리를 키우는 속성입니다.

line-height: 1 은 “글자 크기와 줄 높이가 같다” 는 뜻입니다. 남은 공간이 없다 못해 줄끼리 살짝 파고들어서 답답해 보입니다(도입부 설명 참고).

첫 상자의 normal 이 둘째 상자(1) 보다 넓다는 점도 같이 보세요. 맑은 고딕에서 normal 은 16px 글자에 21px 을 줍니다. 1 과 1.5 사이쯤입니다. 그래서 네 상자의 세로 길이는 1(48px) < normal(63px) < 1.5(72px) < 2(96px) 순입니다. (세 줄짜리 문단이라 한 줄 높이에 3 을 곱한 값입니다)

회색 배경을 깔아 둔 이유는 상자가 세로로 얼마나 길어지는지 보라는 것입니다. 줄 높이를 키우면 그만큼 상자도 길어집니다.

화면 네 상자 모두 글자 크기가 같고 글도 같으며, 모두 세 줄로 접힙니다.

1 짜리 상자는 줄이 서로 붙어서 답답하고, 1.5 → 2 로 갈수록 줄 사이가 넓어집니다. 첫 상자(normal) 는 1 보다 넓고 1.5 보다 좁습니다

normal 입니다. 문장을 길게 써서 여러 줄이 되게 만듭니다. 줄과 줄 사이의 빈 공간을 보세요.

같은 글자 크기인데도 줄 높이만 달라지면 읽는 느낌이 크게 달라집니다.

네 상자에 넣은 글은 완전히 같고 줄 높이만 다릅니다. 계속 이어 쓰는 문장입니다.

1 입니다. 문장을 길게 써서 여러 줄이 되게 만듭니다. 줄과 줄 사이의 빈 공간을 보세요.

같은 글자 크기인데도 줄 높이만 달라지면 읽는 느낌이 크게 달라집니다.

네 상자에 넣은 글은 완전히 같고 줄 높이만 다릅니다. 계속 이어 쓰는 문장입니다.

1.5 입니다. 문장을 길게 써서 여러 줄이 되게 만듭니다. 줄과 줄 사이의 빈 공간을 보세요.

같은 글자 크기인데도 줄 높이만 달라지면 읽는 느낌이 크게 달라집니다.

네 상자에 넣은 글은 완전히 같고 줄 높이만 다릅니다. 계속 이어 쓰는 문장입니다.

2 입니다. 문장을 길게 써서 여러 줄이 되게 만듭니다. 줄과 줄 사이의 빈 공간을 보세요.

같은 글자 크기인데도 줄 높이만 달라지면 읽는 느낌이 크게 달라집니다.

네 상자에 넣은 글은 완전히 같고 줄 높이만 다릅니다. 계속 이어 쓰는 문장입니다.

normal 은 대략 1.2 ~ 1.3 배쯤이지만 정확한 값은 글꼴이 정합니다. 이 페이지의 맑은 고딕에서는 16px 글자에 21px, 곧 1.3배쯤입니다. 글꼴을 바꾸면 normal 의 실제 높이도 달라집니다. 그래서 읽는 느낌을 내가 정하고 싶으면 숫자를 직접 적는 편이 좋습니다.

▣ 직접 해보기 1

#lh-2 를 line-height: 4; 로 바꿔 보세요. 글자 크기는 그대로인데 상자만 훨씬 길어지는 것을 확인하세요.

단위 없는 숫자

섹션 2

단위를 붙이면 계산된 결과가 상속되고, 안 붙이면 비율이 상속됩니다.

이것이 이 파일에서 가장 중요한 내용입니다.

line-height 는 상속되는 속성입니다. 그런데 무엇이 상속되는지가 값의 형태에 따라 다릅니다.

```
line-height: 1.5;    ← 1.5 라는 '비율' 이 자식에게 그대로 갑니다
line-height: 150%;   ← 부모에서 계산이 끝난 '결과 px' 이 자식에게 갑니다
line-height: 24px;   ← 마찬가지로 계산된 24px 이 자식에게 갑니다
```

말로만 보면 사소해 보입니다. 실제로 무슨 일이 벌어지는지 봅시다.

— 비율을 물려주는 경우

```
부모 : font-size 16px, line-height 1.5
자식 : font-size 30px
자식이 물려받는 것은 숫자 1.5 입니다.
자식의 줄 높이 = 30px × 1.5 = 45px    ← 자식 글자에 맞게 커집니다
```

— 계산된 결과를 물려주는 경우

```
부모 : font-size 16px, line-height 150%
부모의 줄 높이 = 16px × 1.5 = 24px
자식 : font-size 30px
자식이 물려받는 것은 24px 이라는 결과입니다.
자식의 줄 높이 = 24px    ← 글자(30px)보다 줄이 낮습니다
```

줄 높이가 글자 크기보다 작으면 어떻게 될까요. 글자가 서로 겹쳐서 읽을 수 없게 됩니다. 아래 붉은 상자가 바로 그 결과입니다.

그래서 규칙은 하나입니다.

line-height 에는 단위 없는 숫자를 씁니다.

px 이나 % 로 써야만 하는 특별한 이유가 없다면 언제나 이쪽입니다.

화면 파란 상자 안의 큰 글자가 겹치지 않고 편하게 읽힙니다

부모입니다. 16px 이고 줄 높이는 1.5 입니다.

자식은 30px 입니다. 줄이 여러 줄이 되도록 문장을 길게 씁니다.
글자가 서로 겹치지 않고 잘 읽힙니다.

윗줄의 받침과 아랫줄의 윗부분이 맞닿아 읽기가 힘듭니다

부모입니다. 16px 이고 줄 높이는 150% 입니다.

자식은 30px 입니다. 줄이 여러 줄이 되도록 문장을 길게 씁니다.
글자가 위아래로 서로 닿아서 읽기가 힘듭니다.

두 상자에서 부모의 설정은 1.5 와 150% 로 사실상 같은 값입니다. 자식은 아예 건드리지도 않았습니니다. 그런데 결과가 이렇게 다릅니다. 물려주는 것이 비율인가, 계산이 끝난 px 인가의 차이뿐입니다.

▣ 직접 해보기 2

#lh-pct-parent 의 150% 를 1.5 로 바꿔 보세요. 자식은 손대지 않았는데 붉은 상자의 글자가 겹치지 않게 됩니다.

읽기 좋은 줄 간격

섹션 3

본문 줄 간격은 1.5 ~ 1.8 사이가 무난합니다. 한글은 조금 넉넉한 편이 좋습니다.

아래 네 상자는 글자와 문장이 완전히 같고 줄 높이만 다릅니다. 직접 읽어 보고 어느 것이 편한지 느껴 보세요. 이건 계산이 아니라 감각입니다.

- 1 줄이 붙어서 어디까지가 한 줄인지 눈이 헛갈립니다
- 1.5 편합니다
- 1.8 더 편합니다. 한글 본문에서 특히 좋습니다
- 3 줄끼리 너무 멀어서 다음 줄을 찾느라 눈이 헤맵니다

한글이 영문보다 넉넉한 줄 간격을 좋아하는 이유가 있습니다. 한글은 네모 칸을 꽉 채우는 글자라서 위아래 여백이 적습니다. 그래서 같은 1.4 라도 영문보다 답답하게 느껴집니다.

참고로 이 자료의 공통.css 는 1.7 을 쓰고 있습니다. 이 범위 안입니다. 지금 읽고 있는 이 설명 글이 바로 1.7 입니다.

제목처럼 짧고 큰 글자는 오히려 좁게 잡습니다. 큰 글자는 그 자체로 이미 세로 공간을 많이 차지하기 때문입니다.

| | |
|-------|------------------------|
| 본문 제목 | line-height: 1.2 ~ 1.3 |
| 본문 | line-height: 1.5 ~ 1.8 |

마지막 상자는 줄끼리 너무 멀어 하나의 글로 안 보입니다

줄 높이 1 입니다. 웹에서 글을 읽는 사람은 한 줄을 다 읽고 나면 눈을 왼쪽으로 되돌려 다음 줄의 시작을 찾습니다. 이때 줄 간격이 적당해야 눈이 헤매지 않습니다.

줄 높이 1.5 입니다. 웹에서 글을 읽는 사람은 한 줄을 다 읽고 나면 눈을 왼쪽으로 되돌려 다음 줄의 시작을 찾습니다. 이때 줄 간격이 적당해야 눈이 헤매지 않습니다.

줄 높이 1.8 입니다. 웹에서 글을 읽는 사람은 한 줄을 다 읽고 나면 눈을 왼쪽으로 되돌려 다음 줄의 시작을 찾습니다. 이때 줄 간격이 적당해야 눈이 헤매지 않습니다.

줄 높이 3 입니다. 웹에서 글을 읽는 사람은 한 줄을 다 읽고 나면 눈을 왼쪽으로 되돌려 다음 줄의 시작을 찾습니다. 이때 줄 간격이 적당해야 눈이 헤매지 않습니다.

▫ 직접 해보기 3

#read-15 와 #read-18 사이에 여러분이 가장 편하다고 느끼는 값을 하나 골라 #read-30 에 넣어 보세요. 1.6 이나 1.65 처럼 소수점을 자유롭게 써도 됩니다.

자간과 단어 간격

섹션 4

letter-spacing 은 글자 사이, word-spacing 은 단어 사이를 벌립니다.

```
letter-spacing: 2px;    글자마다 오른쪽에 2px 씩 더 붙입니다
letter-spacing: -1px;  음수도 됩니다. 글자를 서로 당깁니다
```

word-spacing: 10px; 띄어쓰기 자리에 10px 을 더 붙입니다

둘 다 단위가 필요합니다. 숫자만 쓰면 그 줄은 버려집니다. 기본값은 0 이 아니라 normal 입니다.

크롬에서는 0 이라고 써도 normal 로 돌아옵니다.

쓸 때 주의할 점이 있습니다.

한글 본문에 letter-spacing 을 크게 주면 오히려 읽기 어려워집니다. 한글은 이미 글자마다 네모 칸이 딱 차 있어서 여백이 필요 없습니다.

그래서 자간은 보통 이런 곳에만 씁니다.

큰 제목을 시원하게 벌릴 때 letter-spacing: 4px ~ 8px 아주 작은 안내 글씨를 또렷하게 letter-spacing: 0.5px ~ 1px

참고로 공통.css 의 .doc-demo-label(위의 '실제 결과' 라는 작은 회색 글씨) 에도 letter-spacing: 1px 이 들어가 있습니다. 작은 글씨를 또렷하게 만드는 용도입니다.

word-spacing 은 영어처럼 단어 사이가 뚜렷한 글에서 쓸모가 있습니다. 한글에서는 띄어쓰기 자리가 그대로 벌어져서 문장이 흩어져 보이기 쉽습니다.

화면 둘째 줄은 글자마다 사이가 벌어지고, 셋째 줄은 글자가 서로 붙습니다.

넷째 줄은 큰 제목이 시원하게 벌어져 보입니다. 마지막 줄은 글자끼리는 붙어 있고 띄어쓰기 자리만 넓어집니다

기본입니다. 글자 사이가 보통입니다. Spacing Test
letter-spacing: 2px 입니다. 글자 사이가 벌어집니다. Spacing Test
letter-spacing: -1px 입니다. 글자가 서로 붙습니다. Spacing Test
큰 제목에 자간 6px
word-spacing: 10px 입니다. 단어 사이만 벌어집니다. Spacing Test

letter-spacing 과 word-spacing 의 차이를 헛갈리면 마지막 두 줄을 비교해 보세요. 하나는 모든 글자 사이가 벌어지고, 다른 하나는 띄어쓰기 자리만 벌어집니다.

▣ 직접 해보기 4

#ls-title 의 자간을 1px 로 줄여 보세요. 6px 일 때와 어느 쪽이 제목답게 보이는지 비교해 보세요. 정답은 없습니다.

text-indent

섹션 5

text-indent 는 문단의 첫 줄만 안쪽으로 밀니다.

종이책에서 문단이 바뀔 때 첫 칸을 비우는 그 들여쓰기입니다.

```
#ti-2em { text-indent: 2em; }
```

여기서 em 을 쓴 이유가 있습니다. 다만 한 가지를 정확히 해 두어야 합니다.

개념01 에서 em 을 “부모의 글자 크기를 1 로 놓고 재는 자” 라고 배웠습니다. 그건 font-size 에 em 을 쓸 때의 이야기입니다. font-size 가 아닌 다른 속성에 쓴 em 은 기준이 하나 다릅니다.

| | |
|------------------|--------------------|
| font-size 에 쓴 em | 부모의 글자 크기가 기준 |
| 그 밖의 속성에 쓴 em | 그 요소 자신의 글자 크기가 기준 |

왜 이렇게 나뉘냐면, font-size 는 아직 자기 크기가 정해지기 전이라 자기를 기준으로 삼을 수가 없기 때문입니다. 그래서 부모를 봅니다. text-indent 를 계산할 때는 자기 글자 크기가 이미 정해져 있으니 그것을 씁니다.

아래 문단은 글자 크기가 16px 이라 2em 이 32px 이 됩니다. 이 문단의 글자를 32px 로 키우면 들여쓰기도 64px 로 같이 커집니다. “글자 두 칸만큼” 이라는 뜻이 유지되는 것입니다. px 로 쓰면 이 관계가 깨집니다.

이름을 잘 기억해 두세요. 첫 줄만입니다. 모든 줄을 밀고 싶다면 이 속성이 아니라 10단원의 padding 을 씁니다.

음수도 쓸 수 있습니다. 첫 줄만 왼쪽으로 튀어나오게 됩니다. 목록에서 번호를 바깥으로 빼는 모양을 만들 때 씁니다.

화면 둘째 상자의 첫 줄만 오른쪽으로 밀려 있고,

둘째 줄부터는 첫 상자와 똑같이 왼쪽 끝에 붙습니다

들여쓰기가 없는 문단입니다. 문장을 길게 써서 두 줄 이상이 되도록 만듭니다.
모든 줄이 왼쪽 끝에서 시작합니다. 계속 이어 쓰는 문장입니다.

들여쓰기가 2em 인 문단입니다. 문장을 길게 써서 두 줄 이상이 되도록 만듭니다.
첫 줄만 안으로 들어가 있습니다. 계속 이어 쓰는 문장입니다.

▫ 직접 해보기 5

#ti-2em 의 값을 4em 으로 바꿔 보세요. 몇 px 이 될지 먼저 계산해 보고 확인하세요. (힌트: 글자 크기가 16px 입니다)

white-space

섹션 6

HTML 은 원래 공백을 여러 칸 써도 한 칸으로 줄입니다. 그 규칙을 바꾸는 속성입니다.

02단원에서 “스페이스를 여러 번 눌러도 한 칸으로 보인다” 는 것을 배웠습니다. 그래서 라는 특수문자를 써야 했습니다. 그 규칙을 통째로 바꾸는 것이 white-space 입니다.

| | |
|----------|---|
| normal | (기본) 여러 칸 공백은 한 칸으로, 줄바꿈도 공백 한 칸으로.
줄은 상자 폭에 맞춰 알아서 바뀝니다 |
| pre | 공백도 줄바꿈도 코드에 쓴 그대로. 줄은 저절로 안 바뀝니다 |
| pre-line | 여러 칸 공백은 한 칸으로 줄이되, 줄바꿈은 그대로 지킵니다 |
| pre-wrap | 공백은 그대로 지키고, 줄은 상자 폭에 맞춰 바뀌기도 합니다 |
| nowrap | 줄을 절대 안 바꿉니다 (개념05 에서 자세히 봅니다) |

pre 는 preformatted(미리 서식이 정해진) 의 줄임말입니다. 시나 주소처럼 줄바꿈 자체가 내용인 글에 씁니다.

아래 세 상자에는 완전히 똑같은 글자를 넣었습니다. 코드에서는 낱말 사이를 여러 칸 띄우고 중간에 줄바꿈을 한 번 넣었습니다.

화면 공백이 전부 한 칸으로 줄고, 줄바꿈도 무시되어 한 줄로 이어집니다

공백 네 칸 그리고 아래에서 줄바꿈
한 번 했습니다

화면 공백 네 칸이 그대로 보이고, 줄도 코드에 쓴 자리에서 바뀝니다

공백 네 칸 그리고 아래에서 줄바꿈
한 번 했습니다

화면 공백은 한 칸으로 줄었지만 줄바꿈은 그대로 살아 있어 두 줄로 보입니다

공백 네 칸 그리고 아래에서 줄바꿈
한 번 했습니다

nowrap 은 줄을 아예 안 바꾸는 값이라 글자가 상자를 뚫고 나갑니다. 그 이야기는 개념05 에서 상자 폭과 함께 제대로 다룹니다.

▫ 직접 해보기

6

#ws-preline 을 pre-wrap 으로 바꿔 보세요. 공백 네 칸이 다시 살아나는 것을 확인하세요.

자주 하는 실수

섹션 7

간격은 틀렸는지 아닌지 눈으로만 알 수 있는 영역입니다. 아래 상자들은 전부 실제로 그렇게 그려진 결과입니다.

이런 실수를 합니다

line-height 에 단위를 붙임

이런 실수를 합니다

실수 1 — line-height 에 px 을 붙여서 글자가 겹침

```
#miss-lh-parent { font-size: 16px; line-height: 24px; }  
#miss-lh-child { font-size: 30px; }
```

부모입니다. 16px, 줄 높이 24px. 자식은 30px 입니다. 줄이 여러 줄이 되도록 길게 씁니다. 글자보다 줄 높이가 낮아서 위아래가 서로 닿습니다. 자식은 아무것도 안 건드렸는데 글자가 겹칩니다. 계산이 끝난 24px 이 그대로 상속되었기 때문입니다. 섹션 2 에서 본 그대로입니다. 고치는 방법은 단위를 빼는 것 하나입니다.

이렇게 쓰세요

```
#miss-lh-parent { font-size: 16px; line-height: 1.5; }
```

이런 실수를 합니다

소수점을 쉼표로 씀

이런 실수를 합니다

실수 2 — 소수점을 쉼표로 씀

```
line-height: 1,5;
```

이 줄에는 line-height: 1,5; 가 걸려 있습니다. 아무 일도 일어나지 않습니다. 물려받은 1.7 그대로입니다. CSS 에서 소수점은 마침표입니다. 쉼표는 값을 나누는 기호라서 브라우저가 이 줄을 이해하지 못하고 버립니다.

이런 실수를 합니다

letter-spacing 에 단위를 빠뜨림

이런 실수를 합니다

실수 3 — 자간에 단위를 빠뜨림

```
letter-spacing: 3;
```

이 줄에는 letter-spacing: 3; 이 걸려 있습니다. 아무 일도 일어나지 않습니다. 자간은 길이라서 단위가 꼭 필요합니다. line-height 만 단위 없는 숫자를 허락하는 예외입니다. 이 둘을 헷갈려서 letter-spacing: 3; 이라고 쓰는 일이 아주 잦습니다.

이런 실수를 합니다

text-indent 로 모든 줄을 밀려고 함

이런 실수를 합니다

실수 4 — text-indent 로 문단 전체를 밀려고 함

```
text-indent: 30px;
```

모든 줄을 안으로 밀고 싶어서 text-indent 를 걸었습니다. 그런데 문장을 길게 써서 두 줄이 되면 이렇게 됩니다. 둘째 줄부터는 왼쪽 끝에 그대로 붙습니다. 첫 줄만 들어가고 나머지는 그대로입니다. 속성이 원래 그런 것입니다. 문단 전체를 밀고 싶다면 10단원에서 배울 padding 을 씁니다.

이런 실수를 합니다

white-space: pre 를 본문에 걸음

이런 실수를 합니다

실수 5 — 본문에 white-space: pre 를 걸음

```
white-space: pre;
```

코드에서 줄을 바꾸고 들여쓰기까지 한 글입니다. 코드의 줄바꿈과 들여쓰기 공백이 그대로 화면에 나옵니다. 코드를 보기 좋게 정리하려고 넣은 들여쓰기까지 내용으로 취급되기 때문입니다. 게다가 pre 는 줄을 저절로 바꾸지 않아서 긴 문장을 넣으면 상자 밖으로 뚫고 나갑니다(개념05). 줄바꿈만 살리고 싶으면 pre-line 을 쓰세요.

이런 실수를 합니다

줄 간격을 너무 좁게 줌

이런 실수를 합니다

실수 6 — 예뻐 보이려고 줄 간격을 너무 좁게 줌

```
line-height: 1;
```

디자인 시안에서 글이 짧을 때는 좁은 줄 간격이 깔끔해 보입니다. 그런데 실제 본문처럼 여러 줄이 되면 어디까지가 한 줄인지 눈이 헷갈립니다. 섹션 1 의 첫 상자를 다시 읽어 보세요. 제목에는 좁게, 본문에는 1.5 이상으로 나눠 주는 것이 안전합니다.

정리

▹ 직접 해보기

7

정답

1) #lh-2 를 이렇게 바꿉니다.

```
#lh-2 { line-height: 4; background-color: #eef2f7; }
```

→ 글자 크기는 그대로인데 회색 상자만 세로로 크게 늘어납니다.

한 줄이 $16 \times 4 = 64\text{px}$ 을 차지하게 되었기 때문입니다.
세 줄짜리 문단이라 상자는 96px 에서 $64 \times 3 = 192\text{px}$ 이 됩니다.

2) #lh-pct-parent 를 이렇게 바꿉니다.

```
#lh-pct-parent { font-size: 16px; line-height: 1.5; ... }
```

→ 자식은 손대지 않았는데 붉은 상자의 큰 글자가 겹치지 않습니다.

자식이 물려받는 것이 24px 이라는 결과에서 1.5 라는 비율로 바뀌었고, 자식의 줄 높이가 $30 \times 1.5 = 45\text{px}$ 로 다시 계산되었기 때문입니다.

3) 예를 들어 1.65 를 골랐다면 이렇게 씁니다.

```
#read-30 { line-height: 1.65; background-color: #f4f4f4; }
```

→ 마지막 상자가 위의 두 상자와 비슷한 느낌이 됩니다.

값에 정답은 없지만 본문에서는 1.5 아래로 내려가지 않는 편이 좋습니다.

4) #ls-title 을 이렇게 바꿉니다.

```
#ls-title { font-size: 24px; font-weight: bold; letter-spacing: 1px; }
```

→ 제목이 훨씬 촘촘해 보입니다. 6px 쪽이 시원하고 1px 쪽이 단정합니다.

한글 제목에서는 2px ~ 4px 정도가 무난합니다.

5) #ti-2em 을 이렇게 바꿉니다.

```
#ti-2em { text-indent: 4em; background-color: #fff8dc; }
```

→ $16 \times 4 = 64\text{px}$ 이 됩니다. 첫 줄이 더 깊게 들어갑니다.

6) #ws-preline 을 이렇게 바꿉니다.

```
#ws-preline { white-space: pre-wrap; background-color: #f4f4f4; }
```

→ 공백 네 칸이 다시 살아나고 줄바꿈도 그대로입니다.

pre 와 다른 점은 상자 폭이 좁아지면 줄이 알아서 바뀐다는 것입니다.

이 파일에서 '가르치는' CSS 만 여기에 씁니다.

line-height 기본

섹션 1

```
#lh-normal {
  line-height: normal;
  background-color: #eef2f7;
}
#lh-1 {
  line-height: 1;
  background-color: #eef2f7;
}
```

```

line-height: 1.5;
    background-color: #eef2f7;
}
#lh-2 {
    line-height: 2;
    background-color: #eef2f7;
}

```

화면 1 → 1.5 → 2 로 갈수록 줄 사이가 넓어지고 상자도 세로로 길어집니다.

normal 은 1 보다 넓고 1.5 보다 좁습니다

단위 없는 숫자를 쓰는 이유

섹션 2

```

#lh-num-parent {
    font-size: 16px;
    line-height: 1.5;
    background-color: #f0f7ff;
}
#lh-num-child {
    font-size: 30px;
}

```

화면 자식의 줄 간격이 자식 글자 크기에 맞춰 함께 넓어집니다

```

#lh-pct-parent {
    font-size: 16px;
    line-height: 150%;
    background-color: #fff6f6;
}
#lh-pct-child {
    font-size: 30px;
}

```

화면 자식의 줄 간격이 부모 기준으로 계산된 24px 에 묶여서 글자끼리 겹칩니다

읽기 좋은 줄 간격

섹션 3

```

#read-10 {
    line-height: 1;
    background-color: #f4f4f4;
}
#read-15 {
    line-height: 1.5;
}

```



```

}

#read-18 {
  line-height: 1.8;
  background-color: #f4f4f4;
}
#read-30 {
  line-height: 3;
  background-color: #f4f4f4;
}

```

화면 1 은 답답하고, 1.5 와 1.8 은 편하고, 3 은 줄끼리 따로 노는 느낌입니다

자간과 단어 간격

섹션 4

```

#ls-2 {
  letter-spacing: 2px;
}
#ls-neg {
  letter-spacing: -1px;
}
#ls-title {
  font-size: 24px;
  font-weight: bold;
  letter-spacing: 6px;
}
#wsp-10 {
  word-spacing: 10px;
}

```

화면 글자 사이 또는 단어 사이가 눈에 띄게 벌어지거나 좁아집니다

첫 줄 들여쓰기

섹션 5

```

#ti-2em {
  text-indent: 2em;
  background-color: #fff8dc;
}
#ti-0 {
  background-color: #fff8dc;
}

```

화면 첫 줄만 오른쪽으로 밀리고 둘째 줄부터는 왼쪽 끝에 붙습니다

```
#ws-normal {
  background-color: #f4f4f4;
}
#ws-pre {
  white-space: pre;
  background-color: #f4f4f4;
}
#ws-preline {
  white-space: pre-line;
  background-color: #f4f4f4;
}
```

화면 기본은 여러 칸이 한 칸으로 줄고, pre 는 공백과 줄바꿈을 그대로 지킵니다

자주 하는 실수

```
#miss-lh-parent {
  font-size: 16px;
  line-height: 24px;
}
#miss-lh-child {
  font-size: 30px;
}
```

화면 자식 글자가 커졌는데 줄 높이는 24px 에 묶여 글자끼리 겹칩니다

```
#miss-comma {
  line-height: 1,5;
}
```

화면 아무 일도 일어나지 않습니다. 물려받은 1.7 그대로입니다

```
#miss-ls {
  letter-spacing: 3;
}
```

화면 아무 일도 일어나지 않습니다

```
#miss-indent {
  text-indent: 30px;
  background-color: #fff6f6;
}
```

화면 첫 줄만 들어가고 나머지 줄은 그대로입니다

```
#miss-pre {  
  white-space: pre;  
  background-color: #fff6f6;  
}
```

화면 코드에 쓴 줄바꿈과 들여쓰기 공백이 화면에 그대로 나옵니다

정렬과 꾸미기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 정렬과 꾸미기

이제 글자를 ‘어디에 붙일지’와 ‘어떻게 꾸밀지’를 배웁니다.

| | |
|------------------------------|--------------------------|
| <code>text-align</code> | 글자를 왼쪽·가운데·오른쪽 중 어디에 붙일까 |
| <code>text-decoration</code> | 밑줄·취소선을 그을까 |
| <code>text-transform</code> | 영어를 대문자로 바꿀까 |
| <code>text-shadow</code> | 글자에 그림자를 넣을까 |
| <code>color</code> | 글자색 (07단원 복습) |

이 파일에는 이 단원 전체에서 가장 자주 오해받는 내용이 하나 들어 있습니다.

`text-align: center` 는 글자를 가운데로 보냅니다.
상자를 가운데로 보내지 않습니다.

이 둘은 완전히 다른 일입니다. 그림으로 보면 이렇습니다.

— text-align: center 를 걸었을 때

안녕하세요

← 상자는 그대로 넓습니다

← 글자만 가운데로 갔습니다

— 사람들이 기대한 것

안녕하세요

← 상자 자체가 가운데로

아래쪽을 하려면 상자에 폭을 정하고 좌우 여백을 auto 로 줘야 합니다. 그것은 10단원(margin: auto) 의 내용입니다. 여기서는 하지 않습니다. 지금은 "정렬 속성으로는 상자가 안 움직인다" 만 확실히 해 두면 됩니다.

이 파일은 글자를 어디에 붙이고 어떻게 꾸미는지를 다룹니다. 상자마다 배경색을 깔아 두었습니다. 색칠된 넓이를 함께 보세요. 섹션 2 에서 "상자는 그대로인데 글자만 움직인다" 는 것을 눈으로 확인하게 됩니다.

text-align

섹션 1

값은 네 가지입니다. left center right justify.

text-align: left; 왼쪽에 붙입니다 (한국어·영어의 기본) text-align: center; 가운데에 놓습니다 text-align: right; 오른쪽에 붙입니다 text-align: justify; 양쪽 끝을 다 맞춥니다

아래 첫 상자 세 개는 같은 글을 서로 다르게 붙인 것입니다. 배경색을 보면 상자 자체는 셋 다 똑같은 넓이라는 것을 알 수 있습니다.

justify(양쪽 맞춤) 는 조금 다릅니다. 낱말 사이를 미세하게 늘려서 오른쪽 끝을 자로 그은 듯 맞춥니다. 신문이나 책에서 많이 보던 모양입니다.

한 가지 알아 둘 것이 있습니다.

마지막 줄은 늘리지 않습니다.

마지막 줄까지 늘리면 낱말 사이가 이상하게 벌어지기 때문입니다. 그래서 양쪽 맞춤을 걸어도 마지막 줄은 왼쪽에 붙은 채로 끝납니다.

★ 아래 비교용 문단을 영어로 쓴 이유가 있습니다. 한글은 글자 하나하나가 줄을 바꿀 수 있는 자리라서, 아무것도 안 걸어도 줄이 오른쪽 끝까지 거의 꼭 찹니다. 그래서 양쪽 맞춤을 걸어도 달라지는 것이 거의 없습니다. 영어는 낱말 단위로만 줄이 바뀌어서 줄 끝에 빈칸이 크게 남고, 그 빈칸을 나눠 넣는 양쪽 맞춤의 효과가 눈에 확 들어옵니다. 한글 본문에 justify 를 걸어도 대개 티가 안 난다는 뜻이기도 합니다.

참고로 text-align 은 상속되는 속성입니다. 부모에 걸면 그 안의 글자가 전부 따라갑니다.

화면 회색 상자 세 개의 넓이는 똑같고, 그 안의 글자만 좌·중앙·우로 옮겨 다닙니다

left – 왼쪽에 붙습니다.
center – 가운데에 놓입니다.
right – 오른쪽에 붙습니다.

화면 두 문단 모두 첫 줄이 just like a 에서, 둘째 줄이 huge gaps 에서 끝납니다.

위 문단은 그 두 줄의 오른쪽 끝이 서로 조금씩 어긋나 있고, 아래 문단은 두 줄 다 회색 상자 오른쪽 끝에 딱 맞습니다. 대신 아래 문단은 낱말 사이가 눈에 띄게 벌어져 있습니다. 마지막 줄은 두 문단이 똑같습니다

Justify stretches the spaces between words so that both edges of the paragraph line up neatly, just like a newspaper column does. The very last line is never stretched, because stretching it would leave huge gaps between the words. Compare the right edge of this paragraph with the one below it.

Justify stretches the spaces between words so that both edges of the paragraph line up neatly, just like a newspaper column does. The very last line is never stretched, because stretching it would leave huge gaps between the words. Compare the right edge of this paragraph with the one below it.

비교용 문단만 영어인 이유가 있습니다. 한글은 글자마다 줄을 바꿀 수 있어서 아무것도 안 걸어도 오른쪽 끝이 거의 딱 잡니다. 그래서 한글에 양쪽 맞춤을 걸면 티가 거의 안 납니다. 영어는 낱말 단위로만 줄이 바뀌어 빈칸이 크게 남고, 그 빈칸을 나눠 넣는 양쪽 맞춤의 효과가 뚜렷하게 보입니다.

◦ 직접 해보기 1

#ta-left 를 justify 로 바꿔 보세요. 한 줄짜리 짧은 글에서는 화면이 하나도 안 바뀝니다. 왜 그런지 생각해 보세요. (#ta-right 로 해 보면 글자가 오른쪽에서 왼쪽으로 옮겨 갑니다. 그건 낱말 사이가 늘어난 것이 아니라 오른쪽 맞춤이 사라진 것입니다)

가운데 정렬은 글자만 옮깁니다

섹션 2

이 단원에서 가장 자주 오해받는 내용입니다. 배경색을 잘 보세요.

아래 두 상자에는 이렇게 썼습니다.

```
#box-plain { background-color: #eef2f7; }  
#box-center { text-align: center; background-color: #ffe8e8; }
```

p 요소는 가로를 꽉 채우는 요소입니다(블록 요소, 11단원에서 이름을 제대로 배웁니다). 배경색을 깔면 그 요소가 실제로 차지한 넓이가 눈에 보입니다.

결과를 보세요. 두 상자의 색칠된 넓이가 똑같습니다. 시작 위치도 똑같습니다. 달라진 것은 그 안에서 글자가 앉은 자리뿐입니다.

`text-align` 은 '상자 안에서 글자를 어디에 놓을까' 를 정합니다.
'상자를 화면 어디에 놓을까' 는 정하지 않습니다.

이 둘을 헛갈리면 이런 일이 벌어집니다. 카드나 버튼을 화면 가운데에 놓고 싶어서 `text-align: center` 를 걸어 보고, 글자만 움직이는 것을 보고 “왜 안 되지?” 하며 몇 시간을 쓰게 됩니다.

상자 자체를 가운데로 옮기려면 두 가지가 필요합니다.

1 상자에 폭을 정해 줍니다 `width`

2 좌우 여백을 `auto` 로 줍니다 `margin: 0 auto`

둘 다 10단원 내용입니다. 지금은 이름만 알아 두세요.

그리고 한 가지 더. 아래 세 번째 줄은 `span` 에 가운데 정렬을 건 것입니다. `span` 은 글자만큼만 차지하는 요소라서(인라인 요소) 안에 남는 공간이 없습니다. 그래서 가운데로 보낼 자리 자체가 없고, 아무 일도 일어나지 않습니다.

화면 회색 상자와 분홍 상자의 가로 길이가 똑같고, 왼쪽 시작 위치도 같습니다.

분홍 상자 안의 글자만 가운데로 옮겨져 있습니다

정렬을 안 건 문단입니다.
가운데 정렬을 건 문단입니다.

화면 노란 배경이 글자 길이만큼만 칠해져 있고, 글자는 그대로 제자리입니다

앞 글자 가운데로 보내려 한 `span` 뒤 글자

정리하면 이렇습니다. 글자를 옮기는 것은 `text-align`, 상자를 옮기는 것은 10단원의 여백입니다. “가운데로 안 가요” 라는 질문의 절반은 이 구분에서 나옵니다.

◦ 직접 해보기 **2**

`#box-center` 에서 `text-align: center;` 를 잠시 지워 보세요. 분홍 상자의 넓이가 바뀌는지, 글자 위치만 바뀌는지 확인하고 되돌려 놓으세요.

text-decoration

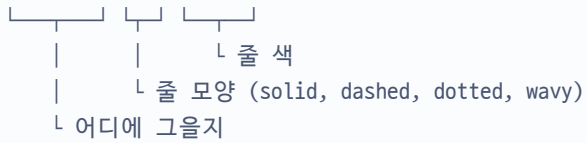
섹션 3

밑줄·취소선·윗줄을 긋는 속성입니다. 한 줄로 여러 값을 함께 쓸 수도 있습니다.

`text-decoration: none;` 줄을 없앱니다 `text-decoration: underline;` 밑줄 `text-decoration: line-through;` 가운데 취소선 `text-decoration: overline;` 윗줄

줄의 모양과 색까지 한 줄에 같이 적을 수 있습니다.

```
text-decoration: underline wavy #c0392b;
```



순서는 바뀌어도 됩니다. 브라우저가 알아서 구분합니다.

실무에서 가장 많이 쓰는 값은 사실 none 입니다. 링크에 자동으로 붙는 밑줄을 없애는 용도입니다. 다음 섹션에서 봅니다.

line-through 는 “품질”, “할인 전 가격” 처럼 취소된 값을 보여 줄 때 씁니다.

화면 위에서부터 밑줄, 가운데 취소선, 윗줄, 빨간 물결 밑줄이 보입니다

underline – 밑줄이 그어집니다.
line-through – 가운데에 줄이 그어집니다. 12,000원
overline – 글자 위에 줄이 그어집니다.
underline wavy 빨강 – 물결 모양 밑줄입니다.

▫ 직접 해보기 3

#td-over 를 text-decoration: line-through dashed #2d6cdf; 로 바꿔 보세요. 점선 파란 취소선이 나옵니다.

링크 밑줄 없애기

섹션 4

a 태그에는 브라우저가 미리 밑줄과 파란색을 걸어 둡니다.

04단원에서 만든 링크는 아무것도 안 했는데 파랑고 밑줄이 있었습니다. 브라우저가 a 요소에 이런 규칙을 기본으로 갖고 있기 때문입니다.

```
a { color: rgb(0, 0, 238); text-decoration: underline; }
```

웹사이트 메뉴처럼 링크가 여러 개 나란히 있으면 밑줄이 지저분해 보입니다. 그래서 이렇게 끕니다.

```
#link-clean { text-decoration: none; color: #2d6cdf; font-weight: bold; }
```

여기서 중요한 원칙이 하나 있습니다.

밑줄을 없앨 거면, 대신 링크임을 알려 줄 다른 표시를 남기세요.

밑줄도 없고 색도 본문과 같으면 사용자는 그것이 놀리는 글자인지 알 수 없습니다. 색을 다르게 하거나 굵게 하는 것으로 충분합니다.

그리고 08단원에서 배운 `:hover` 를 함께 쓰면 더 좋습니다. 평소에는 깔끔하게 두고, 마우스를 올렸을 때만 밑줄을 보여 주는 방식입니다.

```
#link-clean:hover { text-decoration: underline; }
```

아래 두 링크는 진짜로 놀리는 링크입니다. 같은 폴더의 다른 파일로 갑니다. 눌러 보고 브라우저의 뒤로 가기로 돌아오세요.

화면 첫 줄은 파란 밑줄 글씨, 둘째 줄은 밑줄 없는 진한 파란 굵은 글씨입니다.

둘째 줄에 마우스를 올리면 밑줄이 나타납니다

기본 링크 - 개념01 로 갑니다

밑줄을 없앤 링크 - 개념03 으로 갑니다

▹ 직접 해보기 4

`#link-clean:hover` 안에 `color: #c0392b;` 를 한 줄 더 넣어 보세요. 마우스를 올렸을 때 밑줄과 함께 색까지 바뀝니다.

text-transform

섹션 5

영어의 대소문자를 화면에서만 바꿔 주는 속성입니다.

`text-transform: uppercase;` 모두 대문자 `text-transform: lowercase;` 모두 소문자 `text-transform: capitalize;` 단어마다 첫 글자만 대문자 `text-transform: none;` 그대로 (기본)

중요한 것은 이 부분입니다.

바뀌는 것은 '보이는 모습' 뿐이고, HTML 에 쓴 글자는 그대로입니다.

화면에서는 HELLO 로 보여도 소스에는 hello 라고 적혀 있습니다. 그래서 나중에 그 글자를 검색하거나 복사할 때 원래 글자가 나옵니다.

한글에는 대문자·소문자라는 구분이 아예 없습니다. 그래서 한글에 걸면 아무 변화도 없습니다. 고장이 아닙니다.

어디에 쓸까요. 버튼이나 메뉴의 영어 라벨을 항상 대문자로 보이게 할 때 씁니다. HTML 을 고치지 않고도 화면 표기 규칙을 CSS 한 줄로 바꿀 수 있어서 편리합니다.

화면 차례로 hello world / HELLO WORLD / hello world / Hello World 로 보입니다

```
hello world
hello world
hello world
hello world
```

두 번째 줄을 마우스로 클릭해서 복사한 뒤 메모장에 붙여 보세요. HELLO WORLD 가 아니라 hello world 가 붙습니다. 화면 표시만 바뀐 것이라는 뜻입니다.

▫ 직접 해보기 5

#tt-cap 의 글자를 hello world of css 로 늘려 보세요. 몇 개의 첫 글자가 대문자가 될까요?

▫ 직접 해보기 5

#tt-cap 의 글자를 hello world of css 로 늘려 보세요. 몇 개의 첫 글자가 대문자가 될까요?

text-shadow 와 color 섹션 6

text-shadow 는 글자 뒤에 그림자를 깁니다. 값이 네 덩어리입니다.

```
text-shadow: 2px 2px 3px #999999;
```

┌ ┌ ┌ ┌
 │ │ │ └─ 그림자 색
 │ │ └─ 번짐 정도 (클수록 흐려집니다)
 │ └─ 아래로 얼마나
 └─ 오른쪽으로 얼마나

가로와 세로는 음수도 됩니다. 음수면 왼쪽.위로 갑니다. 번짐을 0 으로 두면 흐려지지 않고 또렷한 그림자가 생깁니다.

주의할 점이 있습니다. 본문 글자에 그림자를 넣으면 읽기가 나빠집니다. 큰 제목처럼 짧고 굵은 글자에만 살짝 쓰는 것이 좋습니다.

color 는 07단원에서 배운 그대로입니다. 글자색을 정합니다. 여기서 다시 꺼내는 이유는, 꾸미기의 시작이자 끝이 결국 색이기 때문입니다. 그림자를 넣기 전에 색부터 바꿔 보세요. 대부분은 그것으로 충분합니다.

하면 첫 줄은 글자 오른쪽 아래에 흐린 회색 그림자가 있고,

둘째 줄은 회색 글자가 살짝 파인 것처럼 보이며, 셋째 줄은 붉은색 글자입니다

오른쪽 아래로 흐린 그림자
회색 글자에 흰 그림자
그림자 없이 색만 바꾼 글자입니다. 대부분은 이걸로 충분합니다.

▣ 직접 해보기 6

#sh-1의 그림자를 `text-shadow: -2px -2px 3px #999999;`로 바꿔 보세요. 그림자가 어느 쪽으로 옮겨 가는지 확인하세요.

자주 하는 실수

섹션 7

여기 있는 여섯 가지는 모두 에러 없이 조용히 벌어지는 일들입니다.

이런 실수를 합니다

text-align으로 상자를 가운데 보내려 함

이런 실수를 합니다

실수 1 — text-align: center로 상자를 가운데 보내려 함

```
#card { text-align: center; } /* 상자가 가운데로 갈 것이라 기대함 */
```

이 상자에 `text-align: center`를 걸었습니다. 글자만 가운데로 갔고 상자는 그대로 가로로 꽉 채웁니다. 배경색이 왼쪽 끝부터 오른쪽 끝까지 칠해진 것을 보세요. 상자 자체를 옮기려면 10단원의 `width`와 `margin: 0 auto`가 필요합니다.

이런 실수를 합니다

없는 값 **middle**을 씀

이런 실수를 합니다

실수 2 — text-align: middle이라고 씀

```
text-align: middle;
```

이 상자에는 `text-align: middle`이 걸려 있습니다. 아무 일도 일어나지 않습니다. 글자가 왼쪽에 그대로 있습니다. 가로 가운데는 `center`입니다. `middle`이라는 낱말은 세로 정렬에서 쓰는 다른 속성의 값이라 여기서는 통하지 않습니다.

이런 실수를 합니다

인라인 요소에 정렬을 검

이런 실수를 합니다

실수 3 — span 에 text-align 을 검

```
span { text-align: center; }
```

앞 글자 가운데로 가라 뒤 글자 아무 일도 일어나지 않습니다. span 은 글자 길이만큼만 차지해서 안에 남는 공간이 없습니다. 남는 공간이 없으면 가운데로 보낼 자리도 없습니다. 정렬은 가로를 채우는 요소에 걸어야 뜻이 있습니다.

이런 실수를 합니다

값 이름 오타

이런 실수를 합니다

실수 4 — text-decoration: underlined 라고 씀

```
text-decoration: underlined;
```

이 줄에는 text-decoration: underlined; 가 걸려 있습니다. 밑줄이 없습니다. 올바른 값은 underline 입니다. 뒤에 d 가 없습니다. 값 이름을 틀리면 그 선언만 조용히 버려집니다. 영어 낱말을 자연스럽게 쓰다 보면 자꾸 d 를 붙이게 되는 실수입니다.

이런 실수를 합니다

한글에 uppercase 를 검

이런 실수를 합니다

실수 5 — 한글에 uppercase 를 걸고 왜 안 되냐고 함

```
text-transform: uppercase;
```

한글은 그대로이고 english 만 바뀝니다 한글 부분은 하나도 안 바뀝니다. 고장이 아닙니다. 한글에는 대문자·소문자라는 구분 자체가 없습니다. 바뀌는 것은 알파벳뿐입니다.

이런 실수를 합니다

링크 표시를 다 지움

이런 실수를 합니다

실수 6 — 링크에서 밑줄과 색을 둘 다 없앴

```
a { text-decoration: none; color: #222222; }
```

아래 문장 안에 링크가 하나 숨어 있습니다. 어디인지 찾아보세요. 공지사항은 여기 를 눌러 확인하실 수 있습니다. 놀리는 글자인지 전혀 알 수 없습니다. 코드는 멀쩡합니다. 잘못된 것은 사람이 못 알아본다는 점입니다. 밑줄을 없앨 거면 색이나 굵기 중 하나는 반드시 남겨 두세요.

정리

▣ 직접 해보기 7

정답

1) #ta-left 를 이렇게 바꿉니다.

```
#ta-left { text-align: justify; background-color: #eef2f7; }
```

→ 화면이 하나도 안 바뀝니다. 글자가 왼쪽에 그대로 있습니다.

양쪽 맞춤은 '날말 사이를 늘려서 오른쪽 끝을 맞추는' 일인데,
한 줄짜리 글은 그 한 줄이 곧 마지막 줄이라 늘리지 않기 때문입니다.
마지막 줄은 늘리지 않고 왼쪽에 붙이므로 결과가 left 와 똑같아집니다.

#ta-right 로 같은 실험을 하면 글자가 오른쪽에서 왼쪽으로 훌쩍 옮겨 갑니다.
날말 사이가 벌어져서가 아니라, 오른쪽 맞춤이 없어지고 마지막 줄이
왼쪽에 붙었기 때문입니다. 늘어난 날말 사이는 여전히 하나도 없습니다.

2) #box-center 에서 text-align: center; 를 지우면 이렇게 됩니다.

```
#box-center { background-color: #ffe8e8; }
```

→ 분홍 상자의 가로 길이는 조금도 바뀌지 않고, 글자만 왼쪽으로 돌아옵니다.

이것이 "상자는 안 움직인다" 의 증거입니다.

3) #td-over 를 이렇게 바꿉니다.

```
#td-over { text-decoration: line-through dashed #2d6cdf; }
```

→ 글자 가운데에 파란 점선이 그어집니다.

어디에(line-through), 어떤 모양으로(dashed), 무슨 색으로(#2d6cdf)
세 가지를 한 줄에 적은 것입니다.

4) #link-clean: hover 를 이렇게 바꿉니다.

```
#link-clean: hover { text-decoration: underline; color: #c0392b; }
```

→ 마우스를 올리면 밑줄이 생기면서 글자색도 붉게 바뀝니다.

마우스를 치우면 원래대로 돌아옵니다.

5) #tt-cap 의 글자를 hello world of css 로 바꾸면

Hello World Of Css

→ 네 단어 전부의 첫 글자가 대문자가 됩니다.

capitalize 는 띄어쓰기로 나뉜 모든 낱말에 적용되기 때문에
of 같은 짧은 낱말까지 대문자가 됩니다.

6) #sh-1 을 이렇게 바꿉니다.

```
#sh-1 { font-size: 28px; font-weight: bold;  
        text-shadow: -2px -2px 3px #999999; }
```

→ 그림자가 글자의 왼쪽 위로 옮겨 갑니다.

첫 번째 숫자가 가로, 두 번째 숫자가 세로이고, 음수면 반대 방향입니다.

이 파일에서 '가르치는' CSS 만 여기에 씁니다.

text-align

섹션 1

```
#ta-left {  
  text-align: left;  
  background-color: #eef2f7;  
}  
#ta-center {  
  text-align: center;  
  background-color: #eef2f7;  
}  
#ta-right {  
  text-align: right;  
  background-color: #eef2f7;  
}
```

화면 같은 회색 상자 안에서 글자만 왼쪽·가운데·오른쪽으로 옮겨 볼습니다

```
#ta-normal {
  background-color: #f4f4f4;
}
#ta-justify {
  text-align: justify;
  background-color: #f4f4f4;
}
```

화면 아래 상자는 오른쪽 끝이 자로 그은 듯 가지런합니다. 마지막 줄만 예외입니다

가운데 정렬은 글자만 옮깁니다

섹션 2

```
#box-plain {
  background-color: #eef2f7;
}
#box-center {
  text-align: center;
  background-color: #ffe8e8;
}
```

화면 두 상자의 색칠된 넓이와 시작 위치가 똑같습니다. 글자 위치만 다릅니다

```
#inline-center {
  text-align: center;
  background-color: #fff3cd;
}
```

화면 아무 일도 일어나지 않습니다. span 은 글자만큼만 차지하는 요소입니다

text-decoration

섹션 3

```
#td-underline {
  text-decoration: underline;
}
#td-line {
  text-decoration: line-through;
}
#td-over {
  text-decoration: overline;
}
#td-wavy {
  text-decoration: underline wavy #c0392b;
}
```

화면 밑줄 / 가운데 취소선 / 윗줄 / 빨간 물결 밑줄이 차례로 보입니다

링크 밑줄 없애기

섹션 4

```
#link-clean {
  text-decoration: none;
  color: #2d6cdf;
  font-weight: bold;
}
#link-clean:hover {
  text-decoration: underline;
}
```

화면 밑줄이 없는 파란 굵은 글씨이고, 마우스를 올리면 밑줄이 생깁니다

text-transform

섹션 5

```
#tt-upper {
  text-transform: uppercase;
}
#tt-lower {
  text-transform: lowercase;
}
#tt-cap {
  text-transform: capitalize;
}
```

화면 모두 대문자 / 모두 소문자 / 단어 첫 글자만 대문자로 보입니다

text-shadow 와 color

섹션 6

```
#sh-1 {
  font-size: 28px;
  font-weight: bold;
  text-shadow: 2px 2px 3px #999999;
}
#sh-2 {
  font-size: 28px;
  font-weight: bold;
  color: #888888;
  text-shadow: 1px 1px 0 #ffffff;
}
```



```
color: #c0392b;
}
```

화면 첫 줄 글자 오른쪽 아래에 흐린 회색 그림자가 깔립니다

자주 하는 실수

섹션 7

```
#miss-center-box {
  text-align: center;
  background-color: #fff6f6;
}
```

화면 글자만 가운데로 갑니다. 상자는 여전히 가로를 꽉 채웁니다

```
#miss-middle {
  text-align: middle;
  background-color: #fff6f6;
}
```

화면 아무 일도 일어나지 않습니다. 글자가 왼쪽에 그대로 있습니다

```
#miss-span {
  text-align: center;
  background-color: #fff3cd;
}
```

화면 아무 일도 일어나지 않습니다

```
#miss-td {
  text-decoration: underlined;
}
```

화면 밑줄이 생기지 않습니다

```
#miss-han {
  text-transform: uppercase;
}
```

화면 한글 부분은 그대로이고 영어 부분만 대문자가 됩니다

```
#miss-link {
  text-decoration: none;
  color: #222222;
}
```

화면

밑줄도 없고 색도 본문과 같아서 링크인지 알아볼 수 없습니다

넘치는 글자

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 넘치는 글자

지금까지는 글자를 예쁘게 만드는 이야기였습니다. 이번에는 글자가 말썽을 부릴 때 어떻게 하느냐는 이야기입니다.

상자는 좁은데 글자가 길면 무슨 일이 일어날까요. 브라우저는 보통 알아서 줄을 바꿔서 해결합니다.

| 오늘 날씨가 참 |
| 좋습니다 |

그런데 줄을 바꿀 자리가 없으면 그러지 못합니다. 띄어쓰기 없이 이어진 긴 영문 주소 같은 것이 그렇습니다.

```
https://www.example.com/...
```

↳ 상자를 뚫고 나갑니다

브라우저는 이때도 에러를 내지 않습니다. 그냥 빠져나온 채로 그림니다. 실제 서비스에서 화면이 깨지는 원인 중 아주 흔한 것이 이것입니다.

이 파일에서 배울 도구는 네 개입니다.

| | |
|---|---------------------|
| <code>white-space: nowrap;</code> | 일부러 줄을 안 바꿉니다 |
| <code>overflow-wrap: break-word;</code> | 안 들어가는 낱말만 잘라서 넘깁니다 |
| <code>word-break: break-all;</code> | 줄 끝에 오면 무조건 잘라 넘깁니다 |
| <code>text-overflow: ellipsis;</code> | 넘친 부분을 점 세 개로 대신합니다 |

★ 미리 알려 드립니다. 이 파일에는 `width` 와 `overflow` 라는 속성이 나옵니다. 둘 다 10단원에서 제대로 배웁니다. 지금은 `width` 는 ‘상자 폭을 정하는 것’, `overflow` 는 ‘상자 밖으로 나간 부분을 어떻게 할지 정하는 것’ 이라고만 알아 두세요. 좁은 상자가 없으면 넘치는 모습을 볼 수 없어서 미리 빌려 쓰는 것입니다.

이 파일의 상자들은 모두 폭이 좁게 고정되어 있습니다. 노란색과 파란색으로 칠해 둔 곳이 상자의 진짜 넓이입니다. 글자가 그 색 밖으로 나가는지를 계속 확인하면서 읽으세요. 마지막 섹션에서는 점 세 개(...)를 만드는 방법을 배웁니다.

글자가 상자를 뚫고 나갑니다

섹션 1

브라우저는 띄어쓰기나 하이픈처럼 정해진 자리에서만 줄을 바꿉니다. 그런 자리가 제때 안 나오면 그냥 넘칩니다.

먼저 상자를 좁게 만들어야 합니다.

```
.box { width: 260px; background-color: #fff8dc; }
```

`width` 는 10단원 내용이지만, 폭이 없으면 넘치는 모습을 볼 수 없어서 미리 씁니다. 지금은 “상자 가로 길이를 260px 로 못 박았다” 정도로만 알아 두세요.

아래 두 상자에는 완전히 같은 CSS 가 걸려 있고 글자만 다릅니다.

| | |
|-------|--------------------------------|
| 첫 상자 | : 보통 한국어 문장 (띄어쓰기가 여러 군데 있습니다) |
| 둘째 상자 | : 긴 영문 주소 (띄어쓰기가 하나도 없습니다) |

첫 상자는 멀쩡합니다. 브라우저가 띄어쓰기 자리에서 줄을 바꿔 줬습니다. 둘째 상자는 첫 줄이 노란 배경 밖으로 튀어나갑니다.

자세히 보면 둘째 상자도 두 줄이기는 합니다. 왜 그럴까요? 브라우저는 띄어쓰기 말고 하이픈(-) 자리에서도 줄을 바꿔 줍니다. 그런데 이 주소에서 처음 나오는 하이픈은 `very-` 의 하이픈입니다. 그 앞쪽이 통째로 한

덩어리가 되어 첫 줄에 들어가는데, 그 덩어리가 260px 보다 길어서 상자 밖으로 튀어나가는 것입니다. 띄어쓰기도 하이픈도 제때 안 나오면 브라우저는 손을 쓸 수 없습니다.

여기서 꼭 짚고 갈 것이 있습니다.

에러가 나지 않습니다. 콘솔에도 아무것도 안 뜹니다.
화면이 깨진 것을 사람이 눈으로 발견해야 합니다.

참고로 한글은 이 문제가 잘 안 생깁니다. 한글은 글자 하나하나가 줄을 바꿀 수 있는 자리로 취급되기 때문입니다. 문제가 되는 것은 대개 긴 영문 주소, 이메일, 주문번호 같은 것들입니다.

화면 첫 상자는 두 줄로 암전히 접혀서 노란 배경 안에 들어옵니다.

둘째 상자는 첫 줄이 노란 배경 오른쪽 밖으로 길게 튀어나가고, 둘째 줄만 배경 안에 들어옵니다

오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요
<https://www.example.com/blog/2026/01/very-long-article-name>

width 와 뒤에 나올 overflow 는 10단원에서 제대로 배웁니다. 지금은 width 를 “상자 폭을 정하는 것” 이라고만 알아 두세요.

▸ 직접 해보기 1

.box 의 width 를 400px 로 늘려 보세요. 둘째 상자의 글자가 안으로 들어오는지 확인하고 다시 260px 로 되돌려 놓으세요.

white-space: nowrap

섹션 2

nowrap 은 줄을 절대 바꾸지 말라는 뜻입니다.

개념03 섹션 6 에서 white-space 를 잠깐 봤습니다. 그때 미뤄 둔 값입니다.

```
white-space: nowrap;
```

no + wrap 입니다. wrap 은 ‘줄을 접다’ 라는 뜻이니, 접지 말라는 말입니다.

아래 두 상자는 글자도 같고 폭도 같습니다. 둘째 상자에만 nowrap 을 걸었습니다. 첫 상자는 두 줄로 접히고, 둘째 상자는 한 줄로 쭉 뻗어 나갑니다.

일부러 이렇게 하는 이유가 있습니다.

버튼 글자가 두 줄로 접히면 보기 싫습니다
표의 날짜나 금액이 중간에서 끊기면 읽기 어렵습니다
메뉴 이름이 두 줄이 되면 줄이 어긋납니다

이런 곳에 nowrap 을 씁니다.

다만 이대로 두면 빠져나간 글자가 옆의 내용을 덮습니다. 그래서 nowrap 은 거의 항상 섹션 5 의 점 세 개와 함께 씁니다.

화면 첫 상자는 두 줄로 겹쳐 노란 배경 안에 들어옵니다.

둘째 상자는 한 줄짜리 노란 띠가 되고, 글자가 그 오른쪽 밖으로 한참 더 이어집니다

오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요
오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요

▫ 직접 해보기

2

#nw-off 에도 white-space: nowrap; 을 걸어서 두 상자를 똑같이 만들어 보세요. 확인했으면 되돌려 놓으세요.

overflow-wrap: break-word

섹션 3

한 줄에 도저히 안 들어가는 낱말만 잘라서 다음 줄로 넘깁니다.

섹션 1 의 문제를 푸는 가장 무난한 방법입니다.

```
overflow-wrap: break-word;
```

이름 그대로입니다. 넘칠 것 같으면(overflow) 줄을 접되(wrap), 필요하면 낱말(word) 을 잘라도(break) 좋다는 허락입니다.

핵심은 '필요하면' 입니다.

그 낱말이 한 줄에 통째로 들어갈 수 있으면 자르지 않습니다.
어떻게 해도 안 들어갈 때만 자릅니다.

그래서 보통 문장에는 아무 영향이 없고, 문제가 되는 긴 낱말만 처리됩니다. 본문에 걸어 두어도 부작용이 거의 없는 이유입니다.

게시판 제목, 댓글, 사용자가 직접 입력하는 칸처럼 무엇이 들어올지 모르는 자리에 미리 걸어 두면 화면이 깨지지 않습니다.

화면 첫 상자는 첫 줄이 노란 배경 밖으로 튀어나갑니다.

둘째 상자는 두 줄 모두 노란 배경 안에 완전히 들어옵니다

```
https://www.example.com/blog/2026/01/very-long-article-name  
https://www.example.com/blog/2026/01/very-long-article-name
```

▹ 직접 해보기 3

#wrap-on 의 글자를 여러분 이메일 주소처럼 긴 영문으로 바꿔 보세요. 여전히 상자 안에 들어오는지 확인하세요.

word-break: break-all

섹션 4

break-all 은 줄 끝에 오면 무조건 낱말 중간을 자릅니다.

두 속성이 비슷해 보여서 자주 헷갈립니다. 차이는 딱 하나입니다.

```
overflow-wrap: break-word;   한 줄에 도저히 안 들어갈 때만 자릅니다
word-break: break-all;       줄 끝에 오면 들어가든 말든 자릅니다
```

아래 세 상자로 확인해 봅시다. 폭은 180px 이고 글자는 셋 다 같습니다.

주문번호 orderNumber20260811 을 확인하세요

첫 상자(아무것도 안 걸음) 와 둘째 상자(break-word) 는 결과가 같습니다. orderNumber20260811 이 한 줄에 통째로 들어가기 때문에 break-word 가 자를 이유를 찾지 못한 것입니다. 그래서 세 줄이 됩니다.

주문번호
orderNumber20260811
을 확인하세요

셋째 상자(break-all) 는 다릅니다. 줄 끝에 오자마자 잘라 버립니다.

주문번호 orderNumber2
0260811 을 확인하세요

두 줄로 줄어들어 공간은 아꼈지만, 주문번호가 중간에서 두 동강 났습니다. 읽는 사람 입장에서는 이쪽이 더 불편할 수 있습니다.

그래서 기본은 이렇습니다.

평소에는 overflow-wrap: break-word 를 씁니다.
빈틈 없이 꽉 채워야 하는 특별한 경우에만 word-break: break-all 을 씁니다.

화면 첫 상자와 둘째 상자는 완전히 같은 세 줄 모양입니다.

셋째 상자만 두 줄이 되고, 주문번호가 orderNumber2 와 0260811 로 잘려 있습니다

주문번호 orderNumber20260811 을 확인하세요
주문번호 orderNumber20260811 을 확인하세요
주문번호 orderNumber20260811 을 확인하세요

첫 상자와 둘째 상자가 같아 보이는 것이 실수가 아닙니다. break-word 는 자를 필요가 없으면 자르지 않는다는 뜻이고, 그 성질을 보여 주려고 일부러 이렇게 만든 예입니다.

▫ 직접 해보기

4

.box-narrow 의 폭을 120px 로 줄여 보세요. 이제 주문번호가 한 줄에 안 들어가므로 둘째 상자(break-word) 도 낱말을 자릅니다. 확인했으면 180px 로 되돌려 놓으세요.

점 세 개 3종 세트

섹션 5

overflow: hidden + white-space: nowrap + text-overflow: ellipsis — 셋 다 필요합니다.

긴 제목을 “오늘의 공지사항입니다 자세한 내...” 처럼 줄여 보여 주는 그 모양입니다. ellipsis 는 ‘말줄임표’ 라는 뜻입니다.

많은 사람이 text-overflow: ellipsis 한 줄만 쓰고 안 된다고 합니다. 왜 셋이 다 필요한지 하나씩 빼 가며 확인해 봅시다.

| | |
|----------------------------|--------------------|
| [1] text-overflow 만 | → 아무 일도 안 일어납니다 |
| [2] nowrap + text-overflow | → 한 줄은 되지만 점이 없습니다 |
| [3] overflow + nowrap | → 잘리기는 하는데 점이 없습니다 |
| [4] 셋 다 | → 드디어 점 세 개가 나옵니다 |

각각이 맡은 일이 다르기 때문입니다.

| | |
|-------------------------|-----------------------------------|
| white-space: nowrap | 한 줄로 만듭니다. 넘칠 상황을 만드는 역할 |
| overflow: hidden | 상자 밖으로 나간 부분을 감춥니다. 자를 자리를 만드는 역할 |
| text-overflow: ellipsis | 잘린 자리에 점 세 개를 대신 넣습니다 |

[1] 은 줄이 두 줄로 접혀서 애초에 넘치지 않습니다. 넘친 게 없으니 줄일 것도 없습니다. [2] 는 넘치기는 하는데 빠져나온 채로 다 보입니다. 감춘 게 없으니 줄일 것도 없습니다. [3] 은 감추기는 했는데 점을 넣으라고 시키지 않았습니니다. 그래서 똑 잘립니다. [4] 에서야 세 조건이 다 맞습니다.

★ overflow 는 10단원 내용입니다. 지금은 hidden 을 “상자 밖으로 나간 부분은 안 보이게 하라” 로만 알아 두세요.

마지막으로 한 가지. 이 방법은 한 줄에서만 됩니다. 두 줄까지 보여 주고 세 번째 줄부터 점으로 줄이는 것은 이 세 속성만으로는 안 됩니다. 다른 방법이 필요합니다.

화면 그냥 두 줄로 접힙니다. 점은 없습니다

오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요

화면 한 줄이 되었지만 글자가 노란 배경 밖으로 그대로 다 보입니다. 점은 없습니다

오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요

화면 노란 배경 끝에서 글자가 뚝 잘려 끝납니다. "내용" 의 오른쪽이

글자 중간에서 끊겨 보이고, 점은 없습니다

오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요

화면 "오늘의 공지사항입니다 자세한 내..." 까지만 보이고 점 세 개가 붙습니다

오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요

외워 둘 세 줄

```
overflow: hidden;
white-space: nowrap;
text-overflow: ellipsis;
```

▹ 직접 해보기 5

#ell-4 에서 overflow: hidden; 한 줄만 지워 보세요. 어떤 상자와 똑같아지는지 위 1~3 번과 비교해서 말해 보세요. 확인했으면 되돌려 놓으세요.

자주 하는 실수

섹션 6

넘침 문제는 내 화면에서는 멀쩡해 보이다가 긴 글자가 들어오는 순간 터집니다. 그래서 미리 대비해 두는 것이 중요합니다.

이런 실수를 합니다

ellipsis 만 쓰고 점이 안 나온다고 함

이런 실수를 합니다

실수 1 — text-overflow 만 쓰고 점이 안 나온다고 함

```
text-overflow: ellipsis;
```

오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요 아무 일도 일어나지 않습니다. 그냥 두 줄로 접힙니다. 줄이 접혀 버리면 넘치는 글자가 없고, 넘친 것이 없으면 줄일 것도 없습니다. white-space: nowrap 과 overflow: hidden 을 같이 써야 합니다.

이렇게 쓰세요

```
overflow: hidden; white-space: nowrap; text-overflow: ellipsis;
```

이런 실수를 합니다

overflow 만 써서 똑 잘림

이런 실수를 합니다

실수 2 — text-overflow 를 빠뜨려서 글자가 똑 잘림

```
overflow: hidden; white-space: nowrap;
```

오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요 글자가 상자 끝에서 그냥 끊깁니다. 마지막 글자가 반쯤 잘려서 이상해 보입니다. 읽는 사람은 글이 더 있는지 여기서 끝인지 알 수 없습니다. 점 세 개는 “뒤에 더 있습니다” 를 알려 주는 신호이기도 합니다.

이런 실수를 합니다

본문에 break-all 을 검

이런 실수를 합니다

실수 3 — 본문 전체에 word-break: break-all 을 검

```
word-break: break-all;
```

우리 회사 홈페이지는 wonderful beautiful design 으로 유명합니다 넘침은 확실히 막힙니다. 대신 멀쩡한 영어 낱말이 줄 끝에서 잘립니다. beautiful 이 be 와 autiful 로 두 동강 났습니다. 읽기가 눈에 띄게 나빠집니다.

이렇게 쓰세요

```
overflow-wrap: break-word;
```

이런 실수를 합니다

nowrap 을 걸어 놓고 잊음

이런 실수를 합니다

실수 4 — nowrap 을 걸어 놓고 왜 줄이 안 바뀌냐고 함

```
white-space: nowrap;
```

이 글은 아무리 길어져도 줄이 바뀌지 않고 계속 옆으로 이어집니다 버튼 하나 때문에 걸어 둔 nowrap 이 상속되어 안쪽 글자까지 전부 한 줄이 되는 경우가 많습니다. white-space 는 상속되는 속성입니다. “왜 갑자기 줄이 안 바뀌지?” 싶으면 부모 쪽을 먼저 확인하세요.

이런 실수를 합니다

여러 줄에 ellipsis 를 기대함

이런 실수를 합니다

실수 5 — 두 줄까지 보여 주고 점을 붙이려 함

```
overflow: hidden; text-overflow: ellipsis; /* nowrap 없이 */
```

오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요 점 세 개가 나오지 않습니다. 그냥 두 줄로 접힐 뿐입니다. text-overflow 는 한 줄짜리 글에서만 동작합니다. 여러 줄을 보여 주고 그 아래를 줄이는 것은 이 세 속성만으로는 안 되고, 다른 방법이 필요합니다. 이 자료의 범위를 넘어갑니다.

이런 실수를 합니다

폭을 안 정하고 넘침을 걱정함

이런 실수를 합니다

실수 6 — 폭을 정하지 않고 왜 안 잘리냐고 함

```
/* width 가 없는 상자에 */
overflow: hidden; white-space: nowrap; text-overflow: ellipsis;
```

폭이 정해져 있지 않으면 상자는 부모가 주는 만큼 넓어집니다. 넓으면 넘칠 일이 없고, 넘칠 일이 없으면 점도 안 나옵니다. 창을 아주 좁게 줄이면 그때서야 점이 나옵니다. “왜 안 되지?” 하기 전에 상자에 폭이 있는지부터 확인하세요. 폭 이야기는 10단원에서 이어집니다.

정리

▢ 직접 해보기

6

정답

1) .box 를 이렇게 바꿉니다.

```
.box { width: 400px; background-color: #fff8dc; }
```

→ 노란 상자들이 넓어집니다. 주소는 여전히 두 줄이지만,

이번에는 두 줄 모두 노란 배경 안에 들어와서 밖으로 튀어나가지 않습니다. 아이폰 앞의 그 덩어리가 400px 안에는 들어가기 때문입니다. 문제가 사라진 것처럼 보이지만, 주소가 더 길어지면 다시 넘칩니다. 폭을 넓히는 것은 근본 해결이 아니라는 점을 확인하세요.

2) 섹션 2 의 style 에 이렇게 한 덩어리를 더 넣습니다.

```
#nw-off { white-space: nowrap; }
```

→ 두 상자가 똑같이 한 줄짜리가 되고 둘 다 오른쪽으로 빠져나갑니다.

비교할 대상이 없으므로 확인 뒤에는 다시 지우세요.

3) 예를 들어 이렇게 바꿉니다.

```
<p class="box" id="wrap-on">mylongemailaddress2026@example.com</p>
```

→ 여전히 노란 상자 안에서 두 줄로 접힙니다.

break-word 가 걸려 있어서 어떤 긴 글자가 와도 상자를 넘지 않습니다.

4) .box-narrow 를 이렇게 바꿉니다.

```
.box-narrow { width: 120px; background-color: #f0f7ff; }
```

→ 이제 orderNumber20260811 이 120px 한 줄에 들어가지 못합니다.

그래서 둘째 상자(break-word) 도 낱말을 자르기 시작합니다.
첫 상자(아무것도 안 건 것) 만 여전히 밖으로 빠져나갑니다.
break-word 가 "필요할 때만 자른다" 는 말의 뜻이 여기서 분명해집니다.

5) #ell-4 에서 overflow: hidden; 을 지우면

```
#ell-4 { white-space: nowrap; text-overflow: ellipsis; }
```

→ 위의 2번 상자와 똑같아집니다.

한 줄이 되기는 하지만 넘친 글자가 그대로 다 보이고 점은 없습니다.
감추는 역할을 하는 overflow 가 빠졌기 때문입니다.

이 파일에서 '가르치는' CSS 만 여기에 씁니다.

★ width 는 10단원에서 제대로 배웁니다.

지금은 '상자 폭을 정하는 것' 이라고만 알아 두세요.

글자가 넘치는 모습을 보려면 좁은 상자가 필요해서 미리 씁니다.

```
.box {
  width: 260px;
  background-color: #fff8dc;
}
.box-narrow {
  width: 180px;
  background-color: #f0f7ff;
}
```

nowrap

섹션 2

```
#nw-on {
  white-space: nowrap;
}
```

화면 줄이 바뀌지 않아서 글자가 노란 상자 오른쪽 밖으로 길게 빠져나갑니다

overflow-wrap

섹션 3

```
#wrap-on {
  overflow-wrap: break-word;
}
```

화면 긴 주소가 상자 폭에 맞춰 잘려서 두 줄 모두 상자 안에 들어옵니다

word-break

섹션 4

```
#ba-word {
  overflow-wrap: break-word;
}
#ba-all {
  word-break: break-all;
}
```

화면 break-word 는 긴 낱말을 통째로 다음 줄로 내리고,

break-all 은 줄 끝에서 낱말 중간을 그냥 잘라 버립니다

★ overflow 도 10단원에서 제대로 배웁니다. 지금은 '상자 밖으로 나간 부분을 어떻게 할지 정하는 것' 이라고만 알아 두세요.

```
#ell-1 {
  text-overflow: ellipsis;
}
```

화면 아무 일도 일어나지 않습니다. 그냥 두 줄로 접힙니다

```
#ell-2 {
  white-space: nowrap;
  text-overflow: ellipsis;
}
```

화면 한 줄이 되었지만 점은 없고 글자가 상자 밖으로 빠져나갑니다

```
#ell-3 {
  overflow: hidden;
  white-space: nowrap;
}
```

화면 상자 밖으로 나간 부분이 안 보이지만, 글자가 똑 잘려서 끝납니다

```
#ell-4 {
  overflow: hidden;
  white-space: nowrap;
  text-overflow: ellipsis;
}
```

화면 상자 끝에서 글자가 끝나고 점 세 개가 붙습니다

자주 하는 실수

```
#miss-ell {
  text-overflow: ellipsis;
}
```

화면 점이 안 생깁니다

```
#miss-hidden {
  overflow: hidden;
  white-space: nowrap;
}
```

화면 글자가 상자 끝에서 뚝 끊깁니다

```
#miss-breakall {
  word-break: break-all;
}
```

화면 beautiful 이 be 와 autiful 로 두 동강 나서 읽기 불편합니다

```
#miss-breakword {
  overflow-wrap: break-word;
}
```

화면 beautiful 이 통째로 다음 줄로 내려가서 읽기 편합니다

```
#miss-nowrap {
  white-space: nowrap;
}
```

화면 줄이 아예 안 바뀝니다

```
#miss-multiline {
  overflow: hidden;
  text-overflow: ellipsis;
}
```

화면 두 줄로 접히고 점 세 개는 나오지 않습니다

연습문제

기본 줄입니다. 아무것도 지정하지 않았습니다.

이 줄을 24px 로 만드세요.

이 줄을 rem 으로 32px 로 만드세요.

이 줄을 명조체로 만드세요. Hello 123

이 줄을 굵고 기울게 만드세요. Hello

이 문단의 줄 높이를 2.4 로 만드세요. 줄이 여러 개가 되도록 문장을 길게 씁니다. 줄과 줄 사이가 벌어지는 것을 확인하세요. 계속 이어 쓰는 문장입니다.

이 줄의 글자 크기를 1.2배로 만드세요.

실행하면 나오는 화면 — 연습문제

이 파일을 VS Code 로 열고, 위쪽 `<style>` 안의 빈 자리에 CSS 를 쓰세요. 문제 설명도 전부 그 안에 있습니다. body 는 고치지 마세요. 저장(Ctrl+S) 하고 브라우저에서 새로고침(F5) 하면 결과가 바뀝니다. 1~11 은 기본, 12~13 은 응용, 14~15 는 도전, 16 은 확인 문제입니다.

기준 줄입니다. 아무것도 지정하지 않았습니니다.

이 줄을 24px 로 만드세요.

이 줄을 rem 으로 32px 로 만드세요.

이 줄을 명조체로 만드세요. Hello 123

이 줄을 굵고 기울게 만드세요. Hello

이 문단의 줄 높이를 2.4 로 만드세요. 줄이 여러 개가 되도록 문장을 길게 씁니다.
줄과 줄 사이가 벌어지는 것을 확인하세요. 계속 이어 쓰는 문장입니다.

이 줄의 글자 사이를 벌리세요.

이 줄을 가운데로 보내세요.

12,000원

new arrival today

개념04 로 가는 링크입니다

<https://www.example.com/blog/2026/01/very-long-article-name>

부모입니다. 20px 입니다.

자식입니다. 24px 로 만드세요.

부모입니다.

자식은 28px 입니다. 줄이 여러 줄이 되도록 문장을 길게 씁니다.
글자가 위아래로 겹치지 않게 만들어야 합니다.

오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요

이번 주 신상품 안내

세 군데가 틀렸습니다. 찾아 보세요.

09단원 연습문제 - 텍스트와 폰트

푸는 방법

- 1) 이 파일을 VS Code 로 엽니다.
- 2) 바로 여기 style 안의 TODO 자리에 CSS 를 씁니다.
- 3) 저장(Ctrl+S) 하고 브라우저에서 새로고침(F5) 합니다.
- 4) 각 문제의 '기대 화면' 과 같아졌는지 눈으로 확인합니다.

HTML 은 이미 다 만들어 두었습니다. body 는 건드리지 마세요.
고칠 곳은 이 style 안뿐입니다.

1~11 은 기본, 12~13 은 응용, 14~15 는 도전, 16 은 확인 문제입니다.

아래 세 덩어리는 문제를 풀 준비물입니다. 고치지 마세요.

width 는 10단원에서 제대로 배웁니다.

지금은 '상자 폭을 정하는 것' 이라고만 알아 두세요.

```
.q-box {  
  width: 260px;  
  background-color: #fff8dc;  
}  
#q12-parent {  
  font-size: 20px;  
  background-color: #eef2f7;  
}  
#q13-child {  
  font-size: 28px;  
}
```

문제 1 (개념01 섹션2)

q1 문단의 글자 크기를 24px 로 만드세요.

기대 화면: 그 줄이 바로 위 기준 줄보다 눈에 띄게 커집니다.

이렇게 되면 틀린 것: 크기가 그대로면 단위(px) 를 빠뜨린 것입니다.

```
#q1 {
```

여기에 코드를 쓰세요

```
}
```

문제 2 (개념01 섹션5)

q2 문단의 글자 크기를 32px 로 만드세요.
단, px 이 아니라 rem 을 써야 합니다. 몇 rem 인지 계산해 보세요.
(힌트: 1rem 은 16px 입니다)

기대 화면: 문제 1 의 줄보다 더 큰 글자가 됩니다.
이렇게 되면 틀린 것: 32rem 이라고 쓰면 글자가 화면을 뚫고 나갈 만큼 커집니다.

```
#q2 {
```

여기에 코드를 쓰세요

```
}
```

문제 3 (개념02 섹션2)

q3 문단에 글꼴을 지정하세요. 폰트 스택을 두 칸으로 만듭니다.
첫 번째 : "없는글꼴" (일부러 없는 이름을 넣습니다)
두 번째 : serif

기대 화면: 첫 번째 이름은 그런 글꼴이 없으니 건너뛰고,
명조체(획 끝에 빠침이 있는 글꼴) 로 보입니다.
이렇게 되면 틀린 것: 고딕체 그대로면 두 이름의 순서를 바꿔 썼거나
심표를 빠뜨린 것입니다.

```
#q3 {
```

여기에 코드를 쓰세요

```
}
```

문제 4 (개념02 섹션5·6)

q4 문단을 굵게, 그리고 기울임으로 만드세요. 두 줄이 필요합니다.

기대 화면: 글자가 굵어지면서 동시에 오른쪽으로 기울입니다.

이렇게 되면 틀린 것: 하나만 적용했다면 두 속성 중 하나의 이름이나 값이 틀린 것입니다.

```
#q4 {
```

여기에 코드를 쓰세요

```
}
```

문제 5 (개념03 섹션1)

q5 문단의 줄 높이를 2.4 로 만드세요. 단위는 붙이지 않습니다.

기대 화면: 두 줄 사이가 눈에 띄게 벌어지고

그 문단의 세로가 54px 에서 77px 로 커집니다.

(문단을 감싼 점선 상자는 143px 에서 165px 이 됩니다)

이렇게 되면 틀린 것: 2,4 처럼 심표를 쓰면 아무 일도 일어나지 않습니다.

```
#q5 {
```

여기에 코드를 쓰세요

```
}
```

문제 6 (개념03 섹션4)

q6 문단의 글자 사이(자간) 를 3px 로 벌리세요.

기대 화면: 글자 하나하나 사이가 눈에 띄게 벌어집니다.

이렇게 되면 틀린 것: 아무 변화가 없으면 단위(px) 를 빠뜨린 것입니다.

자간은 line-height 와 달리 단위가 꼭 필요합니다.

```
#q6 {
```

여기에 코드를 쓰세요

```
}
```

문제 7 (개념04 섹션1)

q7 문단의 글자를 가운데로 보내세요.

기대 화면: 글자만 점선 상자의 가로 가운데로 옮겨집니다. 상자는 움직이지 않습니다.

이렇게 되면 틀린 것: middle 이라고 쓰면 아무 일도 일어나지 않습니다.

```
#q7 {
```

여기에 코드를 쓰세요

```
}
```

문제 8 (개념04 섹션3)

q8 문단의 글자 가운데에 취소선을 그으세요.

기대 화면: "12,000원" 이 적힌 줄 한가운데에 가로줄이 그어집니다.
이렇게 되면 틀린 것: 밑줄이 그어졌다면 underline 을 쓴 것입니다.

```
#q8 {
```

여기에 코드를 쓰세요

```
}
```

문제 9 (개념04 섹션5)

q9 문단의 영어를 모두 대문자로 보이게 하세요.
HTML 의 글자는 고치지 않습니다. CSS 로만 해결하세요.

기대 화면: new arrival today 가 NEW ARRIVAL TODAY 로 보입니다.
이렇게 되면 틀린 것: 첫 글자만 대문자가 됐다면 capitalize 를 쓴 것입니다.

```
#q9 {
```

여기에 코드를 쓰세요

```
}
```

문제 10 (개념04 섹션4)

q10 링크의 밑줄을 없애고, 글자색을 #c0392b 로 바꾸세요. 두 줄이 필요합니다.

기대 화면: 밑줄이 사라지고 링크가 붉은색 글자가 됩니다.

이렇게 되면 틀린 것: 파란색 그대로면 color 를 안 쓴 것이고,
밑줄이 남아 있으면 값을 none 이 아닌 다른 것으로 쓴 것입니다.

```
#q10 {
```

여기에 코드를 쓰세요

```
}
```

문제 11 (개념05 섹션3)

q11 상자 안의 긴 주소가 노란 배경 밖으로 튀어나와 있습니다.

긴 낱말을 잘라서 상자 안으로 들어오게 하세요. 한 줄이면 됩니다.

기대 화면: 주소가 두 줄로 잘려서 노란 배경 안에 모두 들어옵니다.

이렇게 되면 틀린 것: 여전히 빠져나가면 속성 이름이 틀린 것입니다.
overflow 와 wrap 사이에 하이픈이 필요합니다.

```
#q11 {
```

여기에 코드를 쓰세요

```
}
```

문제 12 [응용] (개념01 섹션4·5)

q12-parent 는 글자 크기가 20px 로 정해져 있습니다(위쪽 준비물).
그 안의 q12-child 를 정확히 24px 로 만드세요.

조건: px 을 쓰면 안 됩니다. 부모가 20px 이든 40px 이든
항상 24px 이 나오는 단위를 골라 쓰세요.

기대 화면: 회색 상자 안의 둘째 줄이 첫 줄보다 조금 큼니다.
이렇게 되면 틀린 것: em 을 쓰면 부모(20px) 기준이 되어 24px 이 나오지
않습니다.

```
#q12-child {
```

여기에 코드를 쓰세요

```
}
```

문제 13 [응용] (개념03 섹션2)

q13-child 는 글자 크기가 28px 로 정해져 있습니다(위쪽 준비물).
지금 q13-parent 에는 line-height: 26px 이 걸려 있습니다.
26px 이라는 '계산이 끝난 값' 이 그대로 자식에게 내려가는 바람에,
28px 짜리 글자가 26px 짜리 줄에 담겨 위아래가 서로 닿습니다.

그 줄을 단위 없는 1.6 으로 고쳐서 겹침을 없애세요.

조건: 자식(#q13-child) 은 고치지 마세요. 부모의 그 한 줄만 바꿉니다.

기대 화면: 고치기 전에는 파란 상자 안의 큰 글자 두 줄이 서로 닿아 있다가,
고치면 줄 사이가 확 벌어집니다.
(자식의 줄 높이가 26px 에서 44.8px 로 바뀝니다)
이렇게 되면 틀린 것: 여전히 닿아 있으면 단위를 못 댔 것입니다.
단위를 붙이면 계산이 끝난 px 이 내려가고,
단위를 떼면 '비율' 이 내려가서 자식이 자기 글자 크기로 다시
계산합니다.

```
#q13-parent {
  background-color: #f0f7ff;
```

line-height: 26px; /* TODO: 이 줄을 고치세요

```
}
```

문제 14 [도전] (개념05 섹션5)

q14 상자의 긴 제목을 한 줄로 줄이고, 끝에 점 세 개(...) 가 나오게 하세요.
세 줄이 필요합니다. 하나라도 빠지면 점이 안 나옵니다.

기대 화면: 노란 상자 안에 한 줄만 보이고 끝이 ... 로 끝납니다.

이렇게 되면 틀린 것:

두 줄로 접힌다 → white-space 가 빠졌습니다
글자가 밖으로 나간다 → overflow 가 빠졌습니다
뚝 잘리고 점이 없다 → text-overflow 가 빠졌습니다

```
#q14 {
```

여기에 코드를 쓰세요

```
}
```

문제 15 [도전] (개념01·02·03·04 종합)

q15 문단을 '카드 제목' 처럼 꾸미세요. 여섯 줄이 필요합니다.

| | |
|-------|---------|
| 글자 크기 | 1.5rem |
| 굵기 | 굵게 |
| 자간 | 2px |
| 정렬 | 가운데 |
| 글자색 | #2d6cdf |
| 줄 높이 | 1.3 |

기대 화면: 파란색 굵은 큰 글자가 가운데에 놓이고, 글자 사이가 살짝 벌어집니다.

이렇게 되면 틀린 것: 하나라도 안 먹으면 그 줄의 세미콜론이 빠졌는지 먼저 보세요.

세미콜론이 빠지면 그 줄과 다음 줄이 같이 죽습니다.

```
#q15 {
```

여기에 코드를 쓰세요

```
}
```

문제 16 [확인] (개념01 섹션7 · 개념04 섹션7)

아래 q16 규칙은 이미 다 쓰여 있는데, 화면에서는 색만 바뀌고 크기도 굵기도 정렬도 하나도 안 바뀝니다.

틀린 곳이 세 군데 있습니다. 찾아서 고치세요.

지우지 말고, 무엇이 왜 틀렸는지 말로 설명해 보고 나서 고치세요.

기대 화면: 고치고 나면 붉은 글자가 28px 굵기로 커지고 가운데로 갑니다.

이렇게 되면 틀린 것: 크기만 커지고 가운데로 안 가면 text-align 값을 아직 안 고친 것입니다.

굵어지지 않으면 속성 이름의 하이픈이 아직 빠져 있습니다.

크기가 그대로면 값에 단위(px) 를 아직 안 붙인 것입니다.

틀린 곳 1: _____
틀린 곳 2: _____
틀린 곳 3: _____

```
#q16 {  
  font-size: 28;  
  fontweight: bold;  
  text-align: middle;  
  color: #c0392b;  
}
```

09단원 마무리 텍스트와 폰트

자주 넘어지는 자리

1 `em` 을 중첩하면 글자가 곱해져서 커지는 것

박스모델

OPEN 10_박스모델

저는 p 요소입니다. 한 줄을 통째로 차지합니다.

이 줄에서 이 부분 만 노랗습니다.

1 (가장 안)

글자와 그림이 그려지는 자리

01 모든 것은 상자다

02 padding (안쪽 여백)

03 border (테두리)

04 margin (바깥 여백)

05 width 와 box-sizing

06 배경과 넘침

- 연습문제

모든 것은 상자다

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 모든 것은 상자다

09단원까지는 ‘글자’를 꾸몄습니다. 색, 크기, 굵기, 줄 간격 같은 것들입니다. 이제부터는 ‘자리’를 다룹니다. 무엇이 어디에 놓이고, 얼마나 넓고, 옆것과 얼마나 떨어져 있는지를 정하는 일입니다.

그 일을 하려면 먼저 하나를 받아들여야 합니다.

웹페이지에 있는 모든 요소는 사각형 상자입니다.

제목도 상자, 문단도 상자, 그림도 상자, 버튼도 상자입니다. 동그란 프로필 사진도 사실은 네모난 상자를 둥글게 깎은 것입니다. 상자가 눈에 안 보이는 이유는 단지 배경색도 테두리도 없기 때문입니다.

그리고 그 상자는 한 겹이 아니라 네 겹입니다.

콘텐츠 → padding → border → margin
(내용) (안쪽 여백) (테두리) (바깥 여백)

이 네 겹을 통틀어 박스 모델(box model) 이라고 부릅니다. CSS 로 화면을 배치한다는 것은 결국 이 네 겹의 두께를 정하는 일입니다.

이 파일에서는 상자를 ‘보이게’ 만드는 것까지만 합니다. 네 겹을 각각 어떻게 조절하는지는 개념02 부터 하나씩 배웁니다.

이 파일은 보이지 않는 상자를 눈에 보이게 만드는 것 하나만 다룹니다. 배경색을 깔아서 상자의 경계를 드러내고, 그 상자가 네 겹으로 되어 있다는 것을 확인합니다. 마지막 섹션에서 F12 개발자도구 로 상자를 직접 재는 법을 익힙니다. 이 단원은 개발자도구 없이는 배우기 어렵습니다. 섹션 5 를 꼭 따라 해 보세요.

눈에 안 보여도 전부 상자다

섹션 1

배경색을 깔면 그 요소가 차지한 자리가 그대로 드러납니다. 상자를 보는 가장 쉬운 방법입니다.

아래 상자 안에 이렇게 썼습니다.

```
<p class="tint-blue">저는 p 요소입니다. ...</p>
<p class="tint-pink">저도 p 요소입니다.</p>
```

그리고 CSS 에 이렇게 적어 두었습니다(파일 위쪽 style 안에 있습니다).

```
.tint-blue { background-color: #dbe7fb; }
.tint-pink { background-color: #fbe0e0; }
```

결과를 보면 색칠된 부분이 글자 길이와 상관없이 ‘한 줄 전체’입니다. 글자가 짧아도 오른쪽 끝까지 색이 칠해집니다.

p 나 div 같은 태그는 한 줄을 통째로 차지하기 때문입니다. (02단원 개념04 에서 본 ‘블록’ 이야기입니다. 왜 그런지는 11단원에서 배웁니다. 지금은 “p, div 같은 태그는 한 줄을 통째로 차지한다” 만 기억하세요)

반대로 span 은 글자를 감싼 만큼만 차지합니다. 세 번째 줄에서 “이 부분” 네 글자에만 노란색이 칠해진 것을 보세요. span 도 상자입니다. 다만 한 줄을 다 먹지 않는 상자입니다.

화면 파란 줄, 분홍 줄, 파란 줄이 차례로 보입니다.

분홍 줄은 글자가 짧은데도 오른쪽 끝까지 분홍색입니다. 세 번째 줄에서는 “이 부분” 글자 자리에만 노란색이 칠해집니다

저는 p 요소입니다. 한 줄을 통째로 차지합니다.
짧습니다.

이 줄에서 이 부분 만 노랗습니다.

여기서 배경색은 진짜 색을 칠하려고 쓴 것이 아닙니다. 상자의 경계를 눈으로 확인하려고 잠깐 깔아 둔 것입니다. 배치가 뜻대로 안 될 때 background-color 를 임시로 깔아 보는 것은 실무에서도 쓰는 방법입니다.

▹ 직접 해보기 1

위 상자 안의 `<p id="boxP2" class="tint-pink">` 에서 `class="tint-pink"` 를 지우고 새로고침(F5) 해 보세요. 상자가 사라지는 것이 아니라 색만 사라진다는 것을 확인하세요.

상자는 네 겹이다

섹션 2

안쪽부터 콘텐츠 → padding → border → margin 순서입니다. 이 순서는 이 단원 내내 쓰입니다.

상자를 옆에서 잘라 보면 이렇게 생겼습니다.



네 겹의 뜻을 한 줄씩 정리합니다. 용어는 지금 확실히 익혀 두세요.

| | |
|---------|--|
| 콘텐츠 영역 | 글자나 그림이 실제로 그려지는 자리입니다.
width, height 로 크기를 정합니다 (개념05). |
| padding | 콘텐츠와 테두리 사이의 빈 자리입니다 (개념02).
상자 '안쪽' 여백이라서 배경색이 여기까지 칠해집니다. |
| border | 상자의 테두리 선입니다 (개념03).
두께가 있으므로 상자 크기에 포함됩니다. |
| margin | 테두리 바깥, 옆 상자와의 거리입니다 (개념04).
항상 투명합니다. 배경색을 줘도 여기는 안 칠해집니다. |

비유하자면 액자에 넣은 사진입니다.

사진 = 콘텐츠
 여백지 = padding (사진과 액자 사이의 흰 종이)
 액자를 = border
 벽과의 거리 = margin (옆 액자와 얼마나 떨어뜨려 걸 것인가)

비유는 비유일 뿐이니, 다음 섹션에서 진짜로 눈으로 확인합니다.

화면 네 줄짜리 표가 보입니다. 테두리는 없고 칸만 나뉘어 있습니다

순서
 이름
 무엇인가
 배우는 곳

1 (가장 안)
 콘텐츠
 글자와 그림이 그려지는 자리
 개념05

2
 padding
 콘텐츠와 테두리 사이 안쪽 여백
 개념02

3
 border
 상자의 테두리 선
 개념03

4 (가장 밖)
 margin
 옆 상자와 떨어지는 바깥 여백
 개념04

▹ 직접 해보기 2

지금 보고 있는 브라우저 화면에서 노란 안내 상자(맨 위 “이 파일은 ...” 부분)를 손가락으로 짚고, 그 상자의 네 겹이 각각 어디쯤인지 말로 설명해 보세요. 코드는 고치지 않습니다.

바깥 상자에 색을 깔면, 안쪽 상자의 margin 자리에 바깥 색이 비쳐 보입니다.

아래 상자에는 이렇게 썼습니다.

```
<div class="layer-stage"> <div class="layer-box">콘텐츠 영역입니다</div>
</div>
```

CSS 는 이렇습니다.

```
.layer-stage {
  background-color: #efe6ff;      연보라
  border: 2px dashed #9b7fd4;
}
.layer-box {
  margin: 20px;                  바깥 여백
  border: 6px solid #6b3fc0;     테두리
  padding: 24px;                 안쪽 여백
  background-color: #ffffff;     흰색
}
```

이제 색으로 네 겹을 구분할 수 있습니다.

| | | |
|------------|-----------|--------------------------------|
| 연보라 띠 20px | → margin | (안쪽 상자의 배경이 아니라 바깥 상자의 배경입니다) |
| 진보라 선 6px | → border | |
| 흰색 24px | → padding | (흰색인 이유는 안쪽 상자의 배경색이 흰색이라서입니다) |
| 글자 자리 | → 콘텐츠 | |

여기서 중요한 사실이 하나 나옵니다. margin 자리에는 연보라가 보입니다. 그런데 layer-box 의 배경색은 흰색입니다. 즉 margin 에는 자기 배경색이 칠해지지 않습니다. margin 은 언제나 투명하고, 뒤에 있는 것이 그대로 비칩니다. 이 이야기는 다음 섹션에서 한 번 더 확인합니다.

화면 연보라 점선 상자 안에, 진한 보라 테두리를 두른 흰 상자가 들어 있습니다.

두 상자 사이에 연보라 띠가 사방으로 20px 씩 둘러져 있고, 흰 상자 안에서는 글자가 테두리에서 24px 씩 떨어져 있습니다

콘텐츠 영역입니다

바깥 상자(layer-stage)의 가로 길이는 824px, 안쪽 상자(layer-box)의 가로 길이는 780px 입니다. 차이인 44px 은 왼쪽 margin 20px + 오른쪽 margin 20px + 바깥 상자 테두리 2px + 2px 입니다. 숫자를 손으로 세어 볼 필요는 없습니다. 섹션 5 에서 개발자도구로 읽는 법을 배웁니다.

▣ 직접 해보기 3

파일 위쪽 <style> 에서 .layer-box 의 margin: 20px; 을 margin: 60px; 로 바꾸고 새로고침해 보세요. 어떤 색 띠가 두꺼워지는지 확인하고 되돌려 놓으세요.

배경색은 어디까지 칠해지나

섹션 4

배경색은 콘텐츠 + padding + 테두리 아래까지 칠해집니다. margin 에는 칠해지지 않습니다.

이것을 확인하려면 테두리를 점선(dashed) 으로 그으면 됩니다. 점선은 중간중간 끊겨 있으므로, 그 틈으로 배경이 보이는지 안 보이는지를 눈으로 확인할 수 있습니다.

```
.bg-box {
  margin: 24px;
  border: 10px dashed #2d6cdf;   굵은 파란 점선
  padding: 20px;
  background-color: #cfe3ff;     연파랑
}
```

결과를 보면 파란 점선의 ‘끊긴 틈’이 노란색이 아니라 연파랑입니다. 배경이 테두리 아래까지 깔려 있다는 뜻입니다.

반대로 상자 바깥 24px(margin) 은 노란색 그대로입니다. 연파랑이 거기까지는 안 갔습니다.

정리하면 배경이 칠해지는 범위는 이렇습니다.

| | | |
|---------|---------------------|----------------------------------|
| 콘텐츠 | 칠해짐 | |
| padding | 칠해짐 | ← 그래서 padding 을 주면 색칠된 면적이 넓어집니다 |
| border | 칠해짐(테두리 선 아래에 깔립니다) | |
| margin | 안 칠해짐 | ← 언제나 투명합니다 |

“여백을 줬는데 색이 안 칠해졌어요” 라는 질문의 답이 여기에 있습니다. 색을 같이 넓히고 싶으면 margin 이 아니라 padding 을 써야 합니다.

화면 노란 상자 안에 연파랑 상자가 들어 있습니다.

굵은 파란 점선의 끊긴 틈으로 연파랑이 비쳐 보입니다. 두 상자 사이 24px 띠는 노란색 그대로입니다

배경은 padding 과 테두리 아래까지 칠해집니다

▣ 직접 해보기 4

.bg-box 의 padding: 20px; 을 padding: 0; 으로 바꿔 보세요. 연파랑 면적이 어떻게 줄어드는지 보고 되돌려 놓으세요.

브라우저에는 상자를 대신 재 주는 자가 들어 있습니다. 이 단원에서는 이것을 쓸 줄 알아야 합니다.

지금까지는 색을 깔아서 상자를 봤습니다. 하지만 실제 작업에서는 색을 깔 수 없는 경우가 훨씬 많습니다. 그래서 브라우저가 상자를 직접 그려 주고 숫자로 알려 주는 도구를 씁니다. 그것이 개발자도구(DevTools)입니다.

아래 순서대로 딱 한 번만 해 보세요. 이 단원의 나머지가 훨씬 쉬워집니다.

1) 이 파일을 크롬으로 열어 둔 채 F12 를 누릅니다.

(F12 가 안 먹으면 Ctrl + Shift + I 를 누르거나,
화면에서 오른쪽 클릭 → 메뉴에서 '검사' 를 고릅니다)

2) 창 오른쪽이나 아래에 새 패널이 열립니다.

위쪽 탭 줄에서 Elements 를 고릅니다.
왼쪽에는 HTML 코드가, 오른쪽에는 CSS 가 보입니다.

3) 개발자도구 왼쪽 위의 '화살표가 든 네모' 아이콘을 누릅니다.

그 상태로 아래 초록 상자 위에 마우스를 올리면
화면에 색색의 테두리가 겹쳐 보입니다. 그대로 클릭합니다.

4) 오른쪽 패널에서 Styles 탭을 맨 아래까지 스크롤합니다.

(또는 Computed 탭을 고르면 맨 위에 있습니다)
네모가 네 겹으로 겹쳐진 그림이 나옵니다. 이것이 박스 모델 그림입니다.

5) 그림은 안에서 밖으로 이렇게 읽습니다.

| | | |
|---------|---------|-------------------|
| 가장 안쪽 칸 | 콘텐츠 | 가로 x 세로 가 적혀 있습니다 |
| 그 바깥 칸 | padding | |
| 그 바깥 칸 | border | |
| 가장 바깥 칸 | margin | |

칸마다 위·오른쪽·아래·왼쪽 네 개의 숫자가 적혀 있습니다.
값이 0 인 자리에는 숫자 대신 짧은 줄표가 보입니다.
(칸 색은 크롬 버전에 따라 조금씩 다릅니다. 보통 콘텐츠가 파랑,
padding 이 초록, margin 이 주황 계열입니다)

6) 숫자 위에 마우스를 올리면 화면의 그 부분이 색으로 반짝입니다.

숫자를 더블클릭하면 그 자리에서 값을 바꿔 볼 수도 있습니다.
여기서 바꾼 값은 새로고침하면 사라집니다. 파일은 안 망가집니다.
마음껏 만져 보세요.

아래 초록 상자를 조사하면 이런 숫자가 나옵니다.

```
margin  16
border  4
padding 12
콘텐츠  760 x 27      ← 세로는 글꼴에 따라 조금 다를 수 있습니다
```

$16 + 4 + 12 + 760 + 12 + 4 + 16 = 824$ 이 824 가 초록 상자가 실제로 차지하는 가로 자리입니다. 상자 자체(테두리까지)의 가로 길이는 margin 을 뺀 792 입니다.

화면 연초록 바탕에 진초록 테두리를 두른 상자가 하나 보입니다.

위아래 좌우로 16px 씩 떨어져 있습니다

저를 오른쪽 클릭하고 '검사' 를 눌러 보세요

개발자도구는 고장 난 곳을 찾을 때 씁니다. “여백이 왜 이렇게 넓지?” 싶으면 그 상자를 찍어서 margin 칸을 보면 됩니다. 숫자가 눈에 보이면 추측할 일이 없어집니다.

⇒ 직접 해보기 5

위 순서대로 초록 상자를 조사해서 padding 값을 찾아보세요. 그다음 그 숫자를 더블클릭해 40 으로 바꿔 보고, 새로고침해서 원래대로 돌아오는 것까지 확인하세요.

자주 하는 실수

섹션 6

CSS 도 HTML 처럼 틀려도 에러가 나지 않습니다. 브라우저는 못 알아듣는 줄을 조용히 버리고 나머지만 적용합니다.

이런 실수를 합니다

상자가 없는 줄 알고 헤맸

이런 실수를 합니다

실수 1 — 상자가 있다는 것을 잊음

“글자만 오른쪽으로 옮기고 싶은데 왜 안 되지?” 하고 한참 헤맷니다. 움직이는 것은 글자가 아니라 상자입니다. 배치가 뜻대로 안 되면 먼저 그 요소에 배경색을 깔아 상자를 눈으로 확인하세요.

이런 실수를 합니다

margin 에 배경색이 칠해질 거라고 생각함

이런 실수를 합니다

실수 2 — margin 에도 색이 칠해질 거라고 생각함

```
margin: 20px; /* 여기는 색이 안 칠해집니다 */
```

색칠된 면적을 넓히려고 margin 을 줬는데 색은 그대로입니다. 아무 일도 안 일어난 것처럼 보이지만, 사실 여백은 제대로 들어갔습니다. margin 은 언제나 투명하기 때문입니다. 색까지 같이 넓히려면 padding 을 쓰세요.

이런 실수를 합니다

세미콜론 빠뜨림

이런 실수를 합니다

실수 3 — 세미콜론을 빠뜨림

```
.box {  
background-color: #dbe7fb  
padding: 20px;  
}
```

첫 줄 끝에 세미콜론이 없습니다. 브라우저는 두 줄을 한 줄로 붙여 읽고 둘 다 버립니다. 배경색도 안 나오고 padding 도 안 먹습니다. “한 줄만 틀렸는데 두 줄이 사라진다” 는 것이 이 실수의 무서운 점입니다.

이렇게 쓰세요

```
.box {  
background-color: #dbe7fb;  
padding: 20px;  
}
```

이런 실수를 합니다

단위 빠뜨림

이런 실수를 합니다

실수 4 — 단위를 빠뜨림

```
padding: 20;
```

20 은 CSS 가 아는 값이 아닙니다. 이 줄만 버려집니다. 아무 일도 안 일어납니다. 예러 표시도 없습니다. 숫자에는 반드시 px 같은 단위를 붙이세요. 단 0 만은 단위 없이 써도 됩니다.

이런 실수를 합니다

개발자도구에서 고친 것이 저장될 거라고 생각함

이런 실수를 합니다

실수 5 — 개발자도구에서 고친 것이 저장될 거라고 생각함

개발자도구에서 숫자를 바꾸면 화면이 즉시 바뀝니다. 그래서 다 됐다고 생각하고 새로고침하면 원래대로 돌아옵니다. 개발자도구는 시험해 보는 곳이고, 진짜 저장은 VS Code 에서 파일을 고쳐야 합니다. 개발자도구에서 값을 찾은 다음, 그 값을 파일에 옮겨 적는 순서로 쓰세요.

정리

▢ 직접 해보기 6

정답

1) 이렇게 바꿉니다.

```
<p id="boxP2">짧습니다.</p>
```

→ 분홍색이 사라지고 글자만 남습니다.

하지만 그 줄이 차지하는 자리는 그대로 한 줄 전체입니다. 색이 없어졌을 뿐 상자는 그대로 있습니다. 확인했으면 class="tint-pink" 를 다시 붙여 두세요.

2) 노란 안내 상자의 네 겹은 이렇습니다.

| | |
|---------|------------------------------|
| 콘텐츠 | "이 파일은 ..." 글자가 그려진 자리 |
| padding | 글자와 노란 배경의 끝 사이 빈 자리 |
| border | 왼쪽의 굵은 노란 세로 선 |
| margin | 노란 상자와 위쪽 제목·아래쪽 섹션 사이의 흰 공간 |

→ 왼쪽 노란 세로 선이 border 라는 점이 핵심입니다.

네 방향 중 왼쪽에만 테두리를 준 것입니다(개념03 섹션4 에서 배웁니다).

3) style 안의 .layer-box 를 이렇게 바꿉니다.

```
.layer-box {  
  margin: 60px;  
  ...  
}
```

→ 연보라 띠가 20px 에서 60px 로 두꺼워집니다.

흰 상자는 그만큼 작아지고, 안쪽 흰 여백(padding 24px) 은 그대로입니다.
연보라가 두꺼워지는 것으로 "margin 자리에는 바깥 상자 색이 보인다" 를
한 번 더 확인할 수 있습니다. 확인했으면 20px 로 되돌리세요.

4) style 안의 .bg-box 를 이렇게 바꿉니다.

```
.bg-box {  
  margin: 24px;  
  border: 10px dashed #2d6cdf;  
  padding: 0;  
  background-color: #cfe3ff;  
}
```

→ 글자와 파란 점선 사이의 연파랑 여백 20px 이 사라져서

글자가 테두리에 딱 붙습니다. 상자의 세로 길이도 40px 만큼 줄어듭니다.
배경이 padding 까지 칠해진다는 것을 반대로 확인한 셈입니다.
확인했으면 20px 로 되돌리세요.

5) 개발자도구 Styles 탭 맨 아래 박스 모델 그림에서 padding 칸의 네 숫자가 모두 12 로 적혀 있습니다.

→ 그 12 를 더블클릭해 40 으로 고치면 초록 상자가 즉시 세로로 길어지고

글자가 테두리에서 더 멀어집니다.
F5 로 새로고침하면 다시 12 로 돌아옵니다.
개발자도구는 파일을 고치지 않는다는 것을 확인하는 것이 목적입니다.

상자를 눈으로 보려고 배경색을 깎니다

섹션 1

```
.tint-blue {  
  background-color: #dbe7fb;  
}
```

화면 이 클래스가 붙은 요소가 연한 파란색 바탕이 됩니다


```
.tint-pink {
  background-color: #fbe0e0;
}
.tint-yellow {
  background-color: #ffe9a8;
}
```

네 겹을 눈으로 보기

섹션 3

```
.layer-stage {
  background-color: #efe6ff;
  border: 2px dashed #9b7fd4;
}
```

화면 연보라 바탕에 보라 점선 테두리인 바깥 상자입니다

```
.layer-box {
  margin: 20px;
  border: 6px solid #6b3fc0;
  padding: 24px;
  background-color: #ffffff;
}
```

화면 바깥 상자 안에 흰 상자가 놓이고,

둘 사이에 연보라 띠(margin) 가 20px 씩 보입니다

배경색이 칠해지는 범위

섹션 4

```
.bg-stage {
  background-color: #ffe9a8;
  border: 2px dashed #d9a13a;
}
.bg-box {
  margin: 24px;
  border: 10px dashed #2d6cdf;
  padding: 20px;
  background-color: #cfe3ff;
}
```

화면 파란 점선 테두리의 '틈' 사이로 연파랑 배경이 비쳐 보입니다.

반대로 상자 바깥 24px 은 노란색(바깥 상자 색) 그대로입니다

```
.inspect-box {  
  margin: 16px;  
  border: 4px solid #27853f;  
  padding: 12px;  
  background-color: #eaf7ee;  
}
```

padding (안쪽 여백)

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

F12 개발자도구도 함께 열어 두세요 (개념01 섹션5).



실행하면 나오는 화면 — padding (안쪽 여백)

개념01 에서 상자가 네 겹이라고 했습니다.

콘텐츠 → padding → border → margin

이번에는 안쪽에서 두 번째 겹인 padding 을 다룹니다.

padding = 콘텐츠와 테두리 사이의 빈 자리
= 상자 '안쪽' 여백

왜 필요한지는 글자를 상자에 넣어 보면 바로 압니다. 배경색을 준 상자에 글자를 그냥 넣으면 글자가 모서리에 딱 붙어서 숨이 막힌 것처럼 보입니다. 읽기도 불편합니다. padding 을 주면 글자와 상자 벽 사이에 숨 쉴 자리가 생깁니다.

버튼, 카드, 안내 상자 같은 것은 전부 padding 으로 모양이 잡힙니다. 실무에서 가장 자주 쓰는 속성 중 하나입니다.

이 파일에서 배우는 것은 세 가지입니다.

- 1 padding: 20px; 네 방향을 한꺼번에
- 2 padding: 10px 40px; 값을 여러 개 쓰기
- 3 padding-left: 60px; 한 방향만

그리고 마지막에 아주 중요한 사실 하나를 확인합니다. padding 을 주면 상자가 그만큼 커집니다. 이 이야기는 개념05 로 이어집니다.

이 파일은 상자 안쪽 여백 하나만 다룹니다. 값을 한 개부터 네 개까지 쓰는 방법과, 그 순서가 왜 그렇게 정해졌는지를 봅니다. 마지막 섹션은 이 단원 전체에서 가장 중요한 내용입니다. padding 을 주면 상자가 커집니다. 꼭 직접 재 보세요.

padding 은 안쪽 여백이다

섹션 1

padding: 20px; 이라고 쓰면 네 방향 모두 20px 씩 안쪽 여백이 생깁니다.

아래 상자에는 이렇게 썼습니다.

```
<p class="pad-none">padding 이 없습니다. ...</p>
<p class="pad-on">padding: 20px 입니다. ...</p>
```

CSS 는 이렇습니다.

```
.pad-none { background-color: #dbe7fb; }
.pad-on   { background-color: #dbe7fb; padding: 20px; }
```

두 줄의 배경색은 똑같습니다. 다른 것은 padding 뿐입니다.

결과를 보면 아래쪽 상자가 위아래로 두꺼워졌습니다. 위쪽 상자의 세로 길이는 27px, 아래쪽은 67px 입니다. $67 = 27 + 20(\text{위}) + 20(\text{아래})$ 입니다. 딱 padding 만큼 늘어났습니다.

가로도 마찬가지로 안쪽으로 20px 씩 밀립니다. 다만 두 상자 모두 한 줄을 통째로 차지하므로 겹보기 가로 길이는 824px 로 같습니다. 글자가 시작하는 위치만 오른쪽으로 20px 옮겨집니다.

여기서 padding 의 정의를 한 번 더 확인하세요.

padding 은 '상자를 키우는' 것이 아니라
'콘텐츠를 상자 벽에서 떼어 놓는' 것입니다.
그 결과로 상자가 커집니다.

화면 연파랑 줄 두 개가 보입니다.

아래쪽 줄이 위아래로 훨씬 두껍고, 글자도 왼쪽 벽에서 떨어져 있습니다

padding 이 없습니다. 글자가 벽에 붙어 있습니다.
padding: 20px 입니다. 사방으로 여백이 생겼습니다.

padding 은 영어로 '속을 채워 넣는 것' 이라는 뜻입니다. 택배 상자 안에 물건이 흔들리지 않게 채워 넣는 완충재를 떠올리면 됩니다.

➤ 직접 해보기 1

파일 위쪽 <style> 에서 .pad-on 의 padding: 20px; 을 padding: 50px; 로 바꾸고 새로고침(F5) 해 보세요. 상자가 얼마나 더 두꺼워지는지 확인하고 되돌리세요.

값 네 개 — 위 오른쪽 아래 왼쪽

섹션 2

값을 네 개 쓰면 방향마다 다르게 줄 수 있습니다. 순서는 시계 방향입니다.

값을 네 개 쓰면 이런 뜻입니다.

padding: 10px 40px 30px 80px;

원쪽

아래

오른쪽

위

왜 하필 이 순서일까요? 외울 필요가 없습니다. 시계를 떠올리면 됩니다.

시계 바늘은 12시(위) 에서 출발해 3시(오른쪽), 6시(아래), 9시(왼쪽) 순으로 돕니다.
CSS 의 네 값도 똑같이 12시에서 출발해 시계 방향으로 돕니다.

영어 앞 글자를 따서 T(top) R(right) B(bottom) L(left) 라고 외우는 사람도 많습니다. '트리블 (TRouBLe)' 로 기억하면 순서가 안 헛갈립니다.

이 순서는 padding 만의 규칙이 아닙니다. margin(개념04), border 두께 등 네 방향을 갖는 모든 속성이 같은 순서를 씁니다. 여기서 한 번 익혀 두면 이 단원 내내 다시 안 헛갈립니다.

아래 상자를 보면 왼쪽 여백이 압도적으로 넓습니다(80px). 그다음에 아래(30px), 오른쪽(40px), 위(10px)입니다.

화면 노란 상자에서 글자가 왼쪽 벽에서 멀찍이 떨어져 시작하고,

위쪽은 거의 붙어 있으며 아래쪽에는 넉넉한 여백이 있습니다

위 10, 오른쪽 40, 아래 30, 왼쪽 80 입니다. 왼쪽이 가장 많이 밀려 있습니다.

.pad-four 의 값을 padding: 60px 10px 10px 10px; 로 바꿔 보세요. 이번에는 위쪽만 넓어지는 것을 확인하고 되돌리세요.

값 한 개 · 두 개 · 세 개

섹션 3

값을 네 개 다 쓰지 않아도 됩니다. 빠진 값은 맞은편에서 빌려 옵니다.

규칙은 하나뿐입니다. "안 적은 쪽은 맞은편 값을 그대로 쓴다." 이것만 알면 아래 세 가지가 전부 설명됩니다.

```
padding: 20px;
→ 네 방향 모두 20px
(오른쪽·아래·왼쪽이 다 비었으니 전부 위 값을 따라갑니다)

padding: 10px 40px;
→ 위 10, 오른쪽 40, 아래 10(위의 맞은편), 왼쪽 40(오른쪽의 맞은편)
즉 "위아래 10, 좌우 40" 입니다. 가장 많이 쓰는 형태입니다.

padding: 10px 40px 30px;
→ 위 10, 오른쪽 40, 아래 30, 왼쪽 40(오른쪽의 맞은편)
"위 / 좌우 / 아래" 를 따로 주고 싶을 때 씁니다.
```

실무에서는 두 개짜리를 압도적으로 많이 씁니다. 버튼이나 카드는 보통 위아래보다 좌우 여백을 넉넉히 주기 때문입니다.

```
padding: 10px 20px; ← 버튼에서 흔히 보는 값
```

아래 세 상자를 위에서 아래로 비교해 보세요. 두 번째 상자는 위아래가 얇고 좌우가 넓습니다. 세 번째 상자는 아래쪽만 더 두껍습니다.

화면 연초록 상자 세 개가 세로로 놓입니다.

첫째는 사방이 고르고, 둘째는 좌우가 넓고 위아래가 얇으며, 셋째는 둘째와 비슷하지만 아래쪽 여백만 더 두껍습니다

```
padding: 20px; 네 방향 모두 20px 입니다.
padding: 10px 40px; 위아래 10, 좌우 40 입니다.
padding: 10px 40px 30px; 아래만 30 입니다.
```

헛갈리면 값을 네 개 다 쓰세요. 줄이는 것은 편의일 뿐이고, 네 개를 다 쓴 코드가 틀린 코드가 되지는 않습니다. 익숙해진 다음에 줄여도 늦지 않습니다.

▹ 직접 해보기 3

.pad-two 의 padding: 10px 40px; 를 값 네 개짜리로 똑같이 바꿔 써 보세요. 화면이 하나도 안 바뀌면 성공입니다.

한쪽만 정하기

섹션 4

방향 이름을 붙인 padding-top, padding-right, padding-bottom, padding-left 도 있습니다.

한 방향만 건드리고 싶을 때는 방향 이름이 붙은 속성을 씁니다.

```
padding-top: 40px;
padding-right: 40px;
padding-bottom: 40px;
padding-left: 60px;
```

아래 첫 번째 상자는 padding-left 하나만 줬습니다. 그래서 왼쪽만 60px 밀리고 나머지 세 방향은 0입니다.

두 방향 이상을 섞어 쓸 수도 있습니다. 이때 순서가 중요합니다.

```
.pad-mixed {
  padding: 10px;      먼저 네 방향을 10px 로 정하고
  padding-left: 60px; 그중 왼쪽만 60px 로 덮어씹니다
}
```

CSS 는 위에서 아래로 읽으면서, 같은 자리를 건드리는 규칙이 나오면 나중에 나온 것이 이깁니다 (08단원에서 배운 내용입니다). 그래서 왼쪽은 60px, 나머지는 10px 이 됩니다.

순서를 거꾸로 쓰면 어떻게 될까요?

```
padding-left: 60px;
padding: 10px;      ← 이 줄이 네 방향을 전부 다시 10px 로 되돌립니다
```

이번에는 padding 이 나중에 나왔으므로 왼쪽 60px 이 지워집니다. 네 방향을 한꺼번에 정하는 padding 을 '단축 속성' 이라고 부르는데, 단축 속성은 언제나 네 방향을 통째로 다시 씁니다.

규칙: 단축을 먼저 쓰고, 개별 방향을 나중에 씁니다.

화면 연보라 상자 두 개가 보입니다. 둘 다 글자가 왼쪽에서 멀리 떨어져 시작합니다.

첫째는 위아래 여백이 없어 얇고, 둘째는 위아래로 10px 씩 더 두껍습니다

padding-left: 60px; 왼쪽만 밀렸습니다.

padding: 10px 뒤에 padding-left: 60px 을 덮어썼습니다.

▣ 직접 해보기 4

.pad-mixed 안에서 두 줄의 순서를 바꿔 padding-left: 60px; 를 위로, padding: 10px; 을 아래로 놓아 보세요. 왼쪽 여백이 사라지는 것을 확인하고 되돌리세요.

padding 을 주면 상자가 커진다

섹션 5

이 섹션이 이 파일에서 가장 중요합니다. width: 300px 인 상자에 padding 을 주면 상자는 300px 이 아니게 됩니다.

여기서 width 라는 속성을 잠깐 씁니다.

width: 300px; 상자의 가로 길이를 300px 로 못 박습니다

width 는 개념05 에서 제대로 배웁니다. 지금은 “가로 길이를 숫자로 정하는 속성” 정도로만 보세요.

아래 두 상자에 이렇게 썼습니다.

```
.size-base { width: 300px; background-color: #cfe3ff; }  
.size-pad  { width: 300px; padding: 20px; background-color: #ffd9a8; }
```

둘 다 width 가 300px 입니다. 그런데 화면에서 재 보면 이렇습니다.

| | | |
|------------|-------------|---------------|
| .size-base | 실제 가로 300px | |
| .size-pad | 실제 가로 340px | ← 40px 더 넓습니다 |

40px 은 왼쪽 padding 20 + 오른쪽 padding 20 입니다.

왜 이럴까요? CSS 에서 width 는 기본적으로 ‘콘텐츠 영역의 가로 길이’ 입니다. 상자 전체의 가로 길이가 아닙니다.

실제 상자 가로 = width + 왼쪽 padding + 오른쪽 padding
+ 왼쪽 border + 오른쪽 border

그래서 padding 을 더할 때마다 상자가 그만큼 부풀어 오릅니다. border 를 더해도 마찬가지입니다 (개념03).

이것 때문에 초보자는 “300px 로 맞춰 놔는데 왜 빠져나오지?” 로 며칠을 씁니다. 해결책이 있습니다. box-sizing 이라는 속성인데, 개념05 에서 배웁니다. 지금은 이 현상 자체를 눈과 손으로 확인해 두는 것이 목적입니다.

아래 두 상자의 오른쪽 끝을 비교해 보세요. 주황 상자가 확실히 더 튀어나옵니다.

화면 파란 상자와 주황 상자가 위아래로 놓입니다.

주황 상자의 오른쪽 끝이 파란 상자보다 40px 만큼 더 오른쪽에 있습니다

```
width: 300px (padding 없음)
width: 300px + padding: 20px
```

개발자도구로 직접 확인해 보세요. 주황 상자를 클릭하고 박스 모델 그림을 보면 콘텐츠 칸에는 300 이, padding 칸에는 20 이 적혀 있습니다. 위쪽 Elements 패널에서 그 줄에 마우스를 올리면 상자 옆에 340 x 67 같은 크기가 뜹니다.

▸ 직접 해보기 5

.size-pad 의 padding: 20px; 을 padding: 50px; 로 바꾸고, 실제 가로 길이가 몇 px 이 되는지 먼저 계산한 뒤 개발자도구로 확인해 보세요.

자주 하는 실수

섹션 6

아래 실수들은 전부 에러가 나지 않습니다. 그냥 여백이 안 생기거나, 엉뚱한 쪽에 생길 뿐입니다.

이런 실수를 합니다
단위 빠뜨림

이런 실수를 합니다

실수 1 — 단위를 빠뜨림

```
padding: 20;
```

아래 분홍 상자에 진짜로 padding: 20; 을 줬습니다. 여백이 하나도 안 생깁니다. 브라우저가 이 줄을 통째로 버렸기 때문입니다. “분명히 padding 을 줬는데 아무 일도 안 일어난다” 면 제일 먼저 단위를 보세요. padding: 20; 이라고 썼습니다. 여백이 없습니다.

이렇게 쓰세요

```
padding: 20px;
```

이런 실수를 합니다
값 사이에 쉼표를 씀

이런 실수를 합니다

실수 2 — 값 사이에 쉼표를 찍음

```
padding: 10px, 40px;
```

값 여러 개는 공백으로만 나눕니다. 쉼표를 찍으면 CSS 가 못 알아듣고 이 줄을 버립니다. 결과는 실수 1 과 같습니다. 여백이 아예 안 생깁니다. 다른 언어를 배운 사람이 특히 자주 하는 실수입니다.

이런 실수를 합니다

순서를 착각

이런 실수를 합니다

실수 3 — 값 두 개의 뜻을 착각함

```
padding: 10px 40px; /* 위 10, 오른쪽 40 이라고 생각함 */
```

값이 두 개일 때는 위아래 10, 좌우 40 입니다. “위 10, 오른쪽 40” 이 아닙니다. 이렇게 착각하면 왼쪽 여백이 왜 생겼는지 몰라 엉뚱한 곳을 고치게 됩니다. 빠진 값은 맞은편에서 빌려 온다는 규칙을 기억하세요.

이런 실수를 합니다

단축을 나중에 써서 개별 지정이 지워짐

이런 실수를 합니다

실수 4 — 단축 속성을 개별 지정 뒤에 씀

```
padding-left: 60px;  
padding: 10px;
```

padding-left 가 조용히 사라집니다. 뒤에 온 padding 이 네 방향을 전부 다시 정하기 때문입니다. 지운 적이 없는데 없어지니 원인을 찾기 어렵습니다. 단축을 먼저, 개별을 나중에 쓰세요.

이런 실수를 합니다

padding 으로 상자끼리 떨어뜨리려 함

이런 실수를 합니다

실수 5 — padding 으로 상자끼리 떼어 놓으려 함

두 상자 사이를 벌리려고 padding 을 주면, 상자가 서로 떨어지는 게 아니라 각 상자가 뚱뚱해집니다. 배경색이 있으면 색칠된 면적이 커져서 바로 티가 납니다. 상자끼리 떼어 놓는 것은 margin 의 일입니다 (개념04).

이런 실수를 합니다

width 를 맞춰 놓고 **padding** 을 더함

이런 실수를 합니다

실수 6 — width 를 맞춰 놓고 **padding** 을 더함

```
width: 300px;
padding: 20px; /* 실제로는 340px 이 됩니다 */
```

숫자를 300 으로 맞춰 놔는데 화면에서는 340 입니다. 줄이 빼돌어지거나 옆으로 빠져나옵니다. 지금은 이 현상을 알아채는 것으로 충분합니다. 해결은 개념05 에서 합니다.

정리

◦ 직접 해보기 6

정답

1) style 안의 .pad-on 을 이렇게 바꿉니다.

```
.pad-on {
  background-color: #dbe7fb;
  padding: 50px;
}
```

→ 상자의 세로 길이가 67px 에서 127px 로 늘어납니다.

27(글자 한 줄) + 50 + 50 = 127 입니다.
글자도 왼쪽 벽에서 50px 떨어진 곳에서 시작합니다.
확인했으면 20px 로 되돌리세요.

2) style 안의 .pad-four 를 이렇게 바꿉니다.

```
.pad-four {
  background-color: #ffe9a8;
  padding: 60px 10px 10px 10px;
}
```

→ 글자 위쪽에 넓은 노란 여백이 생기고, 왼쪽 여백은 거의 없어집니다.

첫 번째 값이 '위' 라는 것을 눈으로 확인하는 것이 목적입니다.
확인했으면 10px 40px 30px 80px 으로 되돌리세요.

3) 이렇게 쓰면 됩니다.

```
.pad-two {
  background-color: #d7f0d7;
  padding: 10px 40px 10px 40px;
}
```

→ 화면이 하나도 바뀌지 않습니다.

아래(10px) 는 위의 맞은편, 왼쪽(40px) 은 오른쪽의 맞은편이므로 두 개짜리와 네 개짜리가 완전히 같은 뜻이기 때문입니다.

4) style 안의 .pad-mixed 를 이렇게 바꿉니다.

```
.pad-mixed {
  background-color: #f3dcf3;
  padding-left: 60px;
  padding: 10px;
}
```

→ 왼쪽 여백 60px 이 사라지고 네 방향이 전부 10px 이 됩니다.

나중에 나온 단축 padding 이 네 방향을 통째로 다시 정했기 때문입니다. 확인했으면 원래 순서로 되돌리세요.

5) 계산은 이렇습니다.

$$300(\text{width}) + 50(\text{왼쪽}) + 50(\text{오른쪽}) = 400\text{px}$$

→ 개발자도구에서 주황 상자를 클릭하면 콘텐츠 칸에 300,

padding 칸에 50 이 적혀 있고 실제 가로는 400px 입니다.
width 는 그대로 300 인데 상자는 400 이 되었다는 점이 핵심입니다.
확인했으면 20px 로 되돌리세요.

padding 이 있을 때와 없을 때

섹션 1

```
.pad-none {
  background-color: #dbe7fb;
}
```

화면 글자가 연파랑 바탕에 딱 붙어 보입니다

```
.pad-on {
  background-color: #dbe7fb;
  padding: 20px;
}
```

화면 글자 둘레로 연파랑 여백이 20px 씩 생겨 상자가 두꺼워집니다

값 네 개 (위 → 오른쪽 → 아래 → 왼쪽)

섹션 2

```
.pad-four {
  background-color: #ffe9a8;
  padding: 10px 40px 30px 80px;
}
```

화면 왼쪽이 가장 넓고(80px), 아래(30px), 오른쪽(40px), 위(10px) 순으로 좁습니다

값 한 개 · 두 개 · 세 개

섹션 3

```
.pad-one {
  background-color: #d7f0d7;
  padding: 20px;
}
.pad-two {
  background-color: #d7f0d7;
  padding: 10px 40px;
}
.pad-three {
  background-color: #d7f0d7;
  padding: 10px 40px 30px;
}
```

한쪽만 정하기

섹션 4

```
.pad-left-only {
  background-color: #f3dcf3;
  padding-left: 60px;
}
```

화면 왼쪽에만 여백이 생겨 글자가 오른쪽으로 밀립니다

```
.pad-mixed {
  background-color: #f3dcf3;
  padding: 10px;
  padding-left: 60px;
}
```

화면 네 방향 10px 을 준 뒤 왼쪽만 60px 로 덮어씹습니다

padding 을 주면 상자가 커집니다

섹션 5

```
.size-base {  
  width: 300px;  
  background-color: #cfe3ff;  
}  
.size-pad {  
  width: 300px;  
  padding: 20px;  
  background-color: #ffd9a8;  
}
```

화면 아래쪽 주황 상자가 위쪽 파란 상자보다 눈에 띄게 넓습니다

단위를 빠뜨린 예

섹션 6

```
.bad-unit {  
  background-color: #fbe0e0;  
  padding: 20;  
}
```

화면 아무 여백도 생기지 않습니다. 이 줄이 통째로 버려지기 때문입니다

border (테두리)

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

F12 개발자도구도 함께 열어 두세요 (개념01 섹션5).



실행하면 나오는 화면 — border (테두리)

네 겹 중 세 번째 겹인 border 를 다룹니다.

콘텐츠 → padding → border → margin
└ 여기

border 는 padding 바로 바깥에 그어지는 선입니다. 선이라고 했지만 두께가 있는 ‘겹’이라는 점이 중요합니다. 그래서 border 를 주면 padding 과 마찬가지로 상자가 커집니다.

테두리 하나를 그리려면 정할 것이 세 가지입니다.

| | | |
|---------|--------------|----------------|
| 얼마나 두껍게 | border-width | |
| 어떤 모양으로 | border-style | (실선? 점선? 이중선?) |
| 무슨 색으로 | border-color | |

이 셋 중에서 style 이 특히 중요합니다. style 을 안 주면 두께와 색을 아무리 정해도 선이 안 보입니다.
 “테두리를 줬는데 안 나와요” 의 원인은 거의 항상 이것입니다.

세 개를 매번 따로 쓰기는 번거로우니 보통은 한 줄로 줄여 씁니다.

```
border: 1px solid #333;
      └──┬──┬──┘
        |  |  |
        |  |  └─ color
        |  └─── style
        └─── width
```

이 한 줄이 웹에서 가장 많이 쓰이는 CSS 중 하나입니다.

이 파일은 테두리를 다룹니다. 두께 · 모양 · 색 세 가지를 정하는 법, 한 줄로 줄여 쓰는 법, 네 방향 중 한쪽만 굵는 법, 그리고 모서리를 둥글게 깎아 동그라미를 만드는 법까지 봅니다.

테두리는 세 가지를 정한다

섹션 1

border-width · border-style · border-color. 이 중 style 이 없으면 선이 안 보입니다.

아래 상자 세 개에 각각 이렇게 썼습니다.

```
.bd-three {
  border-width: 4px;
  border-style: solid;
  border-color: #c0392b;
}
.bd-nostyle {
  border-width: 10px;      두께를 10px 이나 줬는데도
  border-color: #c0392b;   색까지 줬는데도
                           style 이 없어서 안 보입니다
}
.bd-styleonly {
  border-style: solid;     style 만 줬는데 보입니다
}
```

결과를 재 보면 이렇습니다.

| | | |
|---------------|------------|----------------------|
| .bd-three | 테두리 두께 4px | |
| .bd-nostyle | 테두리 두께 0px | ← 10px 이라고 썼는데 0 입니다 |
| .bd-styleonly | 테두리 두께 3px | ← 아무것도 안 썼는데 3 입니다 |

왜 이럴까요? border-style 의 기본값이 none 이기 때문입니다. none 은 “선을 긋지 않는다” 는 뜻이고, 선을 안 그으면 두께도 0 이 됩니다. 브라우저가 두께를 무시한 게 아니라, 그을 선 자체가 없는 것입니다.

반대로 style 만 주면 나머지 둘은 기본값으로 채워집니다.

border-width 의 기본값은 medium → 크롬에서 3px 입니다
border-color 의 기본값은 그 요소의 글자색입니다

세 번째 상자의 선이 거무스름한 이유가 여기 있습니다. 공통.css 가 글자색을 #222 로 정해 뒀는데, 테두리가 그 색을 그대로 따라간 것입니다.

정리하면 이렇습니다.

style 은 반드시 쓴다. width 와 color 는 생략하면 기본값이 들어간다.

화면 첫 상자는 굵은 빨간 실선 테두리,

둘째 상자는 테두리가 아예 없고 분홍 배경만 보이며, 셋째 상자는 가늘고 거무스름한 실선 테두리가 보입니다

width 4px + style solid + color 빨강
width 10px + color 빨강 (style 없음)
style solid 하나만

▹ 직접 해보기 1

파일 위쪽 <style> 의 .bd-nostyle 에 border-style: solid; 한 줄을 추가하고 새로고침(F5) 해 보세요.
10px 짜리 두꺼운 빨간 테두리가 나타납니다.

단축 border

섹션 2

border: 1px solid #333; 이 세 줄을 대신합니다. 실무에서 거의 이 형태만 씁니다.

세 줄을 한 줄로 합칠 수 있습니다.

```
border-width: 1px;
border-style: solid;    → border: 1px solid #333;
border-color: #333;
```

이렇게 여러 속성을 한 줄로 합쳐 주는 것을 단축 속성이라고 합니다. 개념02 에서 본 padding 도 단축 속성이었습니다.

값의 순서는 사실 바꿔 써도 동작합니다.

```
border: solid 1px #333; ← 이것도 됩니다
```

하지만 모두가 width → style → color 순서로 씁니다. 읽는 사람이 그 순서를 기대하고 있으니 그대로 따르세요.

색을 생략할 수도 있습니다.

```
border: 1px solid;      ← 색이 글자색으로 정해집니다
```

생략하면 나중에 글자색을 바꿨을 때 테두리 색까지 같이 바뀝니다. 예상 밖의 일이 생기기 쉬우니 색은 되도록 적어 주세요.

아래 두 상자를 비교해 보세요. 두께만 다릅니다.

화면 위 상자는 아주 얇은 회색빛 검정 테두리,

아래 상자는 굵은 파란 테두리를 두르고 있습니다

```
border: 1px solid #333;
border: 6px solid #2d6cdf;
```

border 도 padding 처럼 상자를 키웁니다. 위 두 상자의 실제 가로 길이는 둘 다 824px 로 같지만, 글자가 들어갈 수 있는 안쪽 폭은 6px 짜리 상자가 10px 더 좁습니다. (왼쪽 5px + 오른쪽 5px 만큼 테두리에 자리를 뺏겼습니다)

▫ 직접 해보기 2

.bd-short 를 border: 3px dashed #c0392b; 로 바꿔 보세요. 한 줄만 고쳤는데 두께 · 모양 · 색이 한꺼번에 바뀝니다.

border-style 종류

섹션 3

solid · dashed · dotted · double · none 을 주로 씁니다.

style 에 넣을 수 있는 값은 여러 가지입니다. 자주 쓰는 것만 봅니다.

| | | |
|--------|---------------|-------------|
| solid | 한 줄로 이어진 실선 | ← 가장 많이 씁니다 |
| dashed | 짧은 선이 끊어진 점선 | |
| dotted | 둥그란 점이 이어진 점선 | |
| double | 두 줄짜리 이중선 | |
| none | 긋지 않음 (기본값) | |

이 밖에 groove, ridge, inset, outset 이 있습니다. 입체감을 흉내 내는 옛날 모양이라 요즘은 거의 안 씁니다. 이름만 알아 두세요.

double 에는 조건이 하나 있습니다. 두께가 최소 3px 은 되어야 두 줄로 보입니다. 1px 이면 두 줄과 사이 간격을 넣을 자리가 없어서 그냥 실선처럼 보입니다. 아래 예에서 double 만 8px 을 준 이유입니다.

점선은 개념01에서 본 것처럼 배경이 비쳐 보인다는 성질이 있습니다. 선 사이의 틈에는 그 상자의 배경색이 그대로 보입니다.

화면 파란 테두리 상자가 위에서부터 실선, 짧은 막대 점선, 동그란 점선,

두 줄짜리 이중선으로 보이고, 마지막 상자에는 테두리가 없습니다

solid : 이어진 실선
dashed : 끊어진 점선
dotted : 동그란 점선
double : 두 줄짜리 이중선 (8px)
none : 아무 선도 안 그림

▣ 직접 해보기 3

.bd-double 의 두께를 8px 에서 2px 로 줄여 보세요. 두 줄이 한 줄처럼 보이는 것을 확인하고 되돌리세요.

한쪽만 긋기

섹션 4

border-top · border-right · border-bottom · border-left 로 방향을 골라 그을 수 있습니다.

네 방향 전부 테두리를 두르는 일보다, 한쪽에만 긋는 일이 실무에서는 더 많습니다.

border-bottom: 5px solid #27853f; 글자 아래 밑줄 같은 선
border-left: 6px solid #f0c040; 왼쪽에 세로 막대 (인용문·안내 상자)
border-top: 3px solid #c0392b; 구분선

방향 속성도 단축입니다. 안에 width, style, color 세 값을 그대로 씁니다.

여러분은 이 모양을 이미 여러 번 봤습니다. 이 자료의 노란 안내 상자, 파란 '직접 해보기' 상자, 회색 결다리 메모가 전부 border-left 하나로 만든 것입니다. 공통.css 를 열어 보면 .doc-guide 에 border-left: 4px solid #f0c040 이 적혀 있습니다.

두 방향을 같이 줄 수도 있습니다.

border-top: 3px solid #c0392b;
border-bottom: 3px solid #c0392b;

여기서도 단축과 개별의 순서 규칙이 그대로 적용됩니다(개념02 섹션4).

border: 1px solid #333; 먼저 네 방향
border-bottom: 5px solid; 그중 아래만 덮어쓰기

순서를 거꾸로 쓰면 border-bottom 이 지워집니다.

화면 첫 줄 아래에 초록색 굵은 선이 그어지고,

둘째 상자는 왼쪽에 노란 세로 막대가 붙은 연노랑 상자이며, 셋째 상자는 위아래에만 빨간 가로선이 있고 좌우는 뚫려 있습니다

```
border-bottom: 5px solid #27853f;
```

```
border-left: 6px solid #f0c040; 이 자료의 노란 안내 상자와 같은 방식입니다.
```

```
border-top 과 border-bottom 을 같이 준 상자
```

⇒ 직접 해보기

4

.bd-left-accent 의 border-left 를 border-right 로 바꿔 보세요. 노란 막대가 반대쪽으로 옮겨 갑니다.

border-radius 로 모서리 깎기

섹션 5

border-radius 는 모서리를 둥글게 깎습니다. 50% 를 주면 동그라미가 됩니다.

border-radius 에 주는 값은 '모서리를 깎는 반지름' 입니다. 숫자가 클수록 많이 깎입니다.

```
border-radius: 16px;    적당히 둥근 카드
border-radius: 999px;   양 끝이 완전히 둥근 알약 모양
border-radius: 50%;     동그라미
```

999px 처럼 터무니없이 큰 값을 줘도 에러가 나지 않습니다. 브라우저가 알아서 깎을 수 있는 최대치까지만 깎습니다. 그래서 높이가 얼마든 상관없이 알약 모양이 나옵니다. 자주 쓰는 요령입니다.

모서리마다 다르게 깎을 수도 있습니다. 값 네 개를 쓰면 됩니다. 다만 이번에는 시계 방향의 출발점이 '왼쪽 위' 입니다.

```
border-radius: 24px 0 24px 0;
      ┌──┴──┐ ┌──┴──┐ ┌──┴──┐
      │   │   │   │   └─ 왼쪽 아래
      │   │   │   │   └─ 오른쪽 아래
      │   └─ 오른쪽 위
      └─ 왼쪽 위
```

padding 과 margin 은 '위' 에서 출발했지만, radius 는 '왼쪽 위' 에서 출발합니다. 둘 다 시계 방향이라는 것만 같습니다. 헛갈리면 값을 하나씩 바꿔 보면 바로 압니다.

동그라미를 만드는 법은 이렇습니다.

```
width 와 height 를 같은 값으로 맞추고
border-radius: 50% 를 준다
```

50% 는 '가로의 절반, 세로의 절반' 이라는 뜻입니다. 정사각형에서 네 모서리를 절반씩 깎으면 남은 모양이 원입니다. 가로세로가 다르면 원이 아니라 타원이 됩니다. 아래에서 둘 다 보여 드립니다.

(width 와 height 는 개념05 에서 제대로 배웁니다. 지금은 “가로 길이와 세로 길이를 숫자로 못 박는 속성”으로만 보세요)

마지막으로 outline 이야기를 한 줄 합니다.

outline 은 border 처럼 선을 긋지만 상자 크기에 포함되지 않습니다.
그래서 outline 을 줘도 상자가 커지지 않고 옆 요소가 밀리지도 않습니다.

아래 마지막 두 상자가 그 차이입니다. 둘 다 width 를 200px 로 줬는데, border 를 준 상자는 실제 가로가 220px 이고 outline 을 준 상자는 200px 그대로입니다.

화면 첫 상자는 네 모서리가 부드럽게 둥글고,

둘째 상자는 좌우 끝이 반원처럼 완전히 둥글며, 셋째 상자는 왼쪽 위와 오른쪽 아래만 둥글고 나머지는 각져 있습니다

```
border-radius: 16px;
border-radius: 999px;
border-radius: 24px 0 24px 0;
```

화면 지름 120px 짜리 보라색 원이 하나, 그 아래에 가로 200px 세로 80px 짜리

연보라 타원이 하나 보입니다

화면 두 상자 모두 파란 테두리가 둘러져 있지만

위 상자가 아래 상자보다 조금 더 넓습니다

```
border 10px : 실제 가로 220px
outline 10px : 실제 가로 200px
```

outline 은 주로 키보드로 이동할 때 지금 어디에 있는지 보여 주는 용도로 브라우저가 자동으로 씁니다. 직접 쓸 일은 드물지만, 크기를 안 바꾸는 선이 필요할 때 유용합니다.

▢ 직접 해보기

5

.bd-ellipse 의 width: 200px; 를 width: 80px; 로 바꿔 보세요. 타원이 동그라미가 됩니다. 왜 그런지 말로 설명해 보세요.

자주 하는 실수

섹션 6

테두리가 안 보이거나, 보였는데 배치가 어긋나는 일이 자주 생깁니다. 에러는 나지 않습니다.

이런 실수를 합니다
style 을 안 줌

이런 실수를 합니다

실수 1 — style 을 안 줘서 테두리가 안 보임

```
border-width: 10px;  
border-color: red;
```

두께도 색도 줬는데 아무 선도 안 그어집니다. border-style 의 기본값이 none 이라서, 그을 선 자체가 없기 때문입니다. 섹션 1 의 두 번째 상자가 바로 이 상태입니다. 테두리가 안 보이면 style 부터 확인하세요.

이렇게 쓰세요

```
border: 10px solid red;
```

이런 실수를 합니다

테두리를 줬더니 배치가 밀림

이런 실수를 합니다

실수 2 — 테두리를 줬더니 배치가 어긋남

:hover 로 테두리를 붙였더니 마우스를 올릴 때마다 글자가 흔들리고 옆 상자가 밀립니다. border 는 두께가 있는 겹이라 상자를 키우기 때문입니다. 평소에도 같은 두께의 투명한 테두리를 미리 깔아 두거나, 크기를 안 바꾸는 outline 을 쓰면 흔들리지 않습니다.

이런 실수를 합니다

색을 생략

이런 실수를 합니다

실수 3 — 색을 생략함

```
border: 2px solid;
```

지금은 잘 나옵니다. 그런데 나중에 그 요소의 color 를 빨강으로 바꾸면 테두리까지 빨강으로 변합니다. 건드린 적 없는 테두리가 바뀌니 원인을 찾기 어렵습니다. 색은 되도록 적어 주세요.

이런 실수를 합니다

단축을 나중에 써서 한쪽 지정이 지워짐

이런 실수를 합니다

실수 4 — 단축을 개별 지정 뒤에 씌

```
border-bottom: 5px solid green;
border: 1px solid #333;
```

초록 밑줄이 조용히 사라지고 네 방향이 전부 얇은 회색이 됩니다. 나중에 나온 단축 border 가 네 방향을 통째로 다시 정하기 때문입니다. 단축을 먼저, 개별을 나중에 쓰세요. padding 과 똑같은 규칙입니다.

이런 실수를 합니다

원을 만들려는데 타원이 됨

이런 실수를 합니다

실수 5 — 원을 만들려는데 타원이 나옴

```
width: 200px;
height: 80px;
border-radius: 50%;
```

동그라미가 아니라 길쭉한 타원이 나옵니다. 50% 는 가로와 세로의 절반을 각각 깎는다는 뜻이라서, 가로세로가 다르면 원이 될 수 없습니다. 원을 원하면 width 와 height 를 같은 값으로 맞추세요.

이런 실수를 합니다

단위 빠뜨림

이런 실수를 합니다

실수 6 — 두께에 단위를 안 붙임

```
border: 3 solid #333;
```

3 은 두께로 인정되지 않고, 이 줄 전체가 버려집니다. 한 값만 틀려도 단축 속성은 통째로 무시됩니다. 테두리가 아예 안 나오면 단위를 의심하세요.

정리

▢ 직접 해보기 6

정답

1) style 안의 .bd-nostyle 을 이렇게 바꿉니다.

```
.bd-nostyle {
  border-width: 10px;
  border-style: solid;
  border-color: #c0392b;
  padding: 10px;
  background-color: #fdf0ef;
}
```

→ 안 보이던 테두리가 10px 짜리 굵은 빨간 실선으로 나타납니다.

동시에 상자가 위아래로 20px 두꺼워집니다(47px → 67px).
 가로는 어떨까요? 겹보기 가로 길이는 824px 그대로입니다.
 이 상자는 한 줄을 통째로 차지하는 상자라 자리가 이미 824px 로 정해져 있고,
 테두리는 그 824px 안에서 자리를 가져가기 때문입니다.
 대신 글자가 들어갈 안쪽 폭이 824px 에서 804px 로 20px 좁아집니다.
 (섹션 2 의 겹다리 메모에서 본 것과 같은 이야기입니다)
 style 한 줄이 두께까지 살려 났다는 점을 확인하세요.

2) style 안의 .bd-short 를 이렇게 바꿉니다.

```
.bd-short {
  border: 3px dashed #c0392b;
  padding: 10px;
  background-color: #ffffff;
}
```

→ 얇은 검정 실선이 굵은 빨간 점선으로 바뀝니다.

한 줄에 세 가지가 다 들어 있어서 한 번에 다 바꿉니다.

3) style 안의 .bd-double 의 두께를 2px 로 줄입니다.

```
.bd-double {
  border: 2px double #2d6cdf;
  ...
}
```

→ 두 줄이 붙어 버려 한 줄처럼 보입니다.

double 은 '선 · 틈 · 선' 세 부분을 넣어야 하므로
 최소 3px 은 되어야 두 줄로 보입니다. 확인했으면 8px 로 되돌리세요.

4) style 안의 .bd-left-accent 를 이렇게 바꿉니다.


```
.bd-left-accent {
  border-right: 6px solid #f0c040;
  background-color: #fff8dc;
  padding: 10px 14px;
}
```

→ 노란 세로 막대가 왼쪽에서 오른쪽으로 옮겨 갑니다.

글자 위치도 왼쪽으로 6px 만큼 당겨집니다.
왼쪽 테두리가 차지하던 자리가 없어졌기 때문입니다.

5) style 안의 .bd-ellipse 를 이렇게 바꿉니다.

```
.bd-ellipse {
  width: 80px;
  height: 80px;
  border-radius: 50%;
  background-color: #b09ae0;
}
```

→ 타원이 지름 80px 짜리 동그라미가 됩니다.

50% 는 '가로의 절반, 세로의 절반' 만큼 깎으라는 뜻인데,
가로와 세로가 같아지면 깎이는 양도 같아져서 원이 됩니다.
가로세로가 다를 때 타원이 되는 이유도 같은 설명입니다.

세 가지를 따로 정하기

섹션 1

```
.bd-three {
  border-width: 4px;
  border-style: solid;
  border-color: #c0392b;
  padding: 10px;
  background-color: #fdf0ef;
}
```

화면 빨간 실선 테두리가 4px 두께로 사방에 그어집니다

```
.bd-nostyle {
  border-width: 10px;
  border-color: #c0392b;
  padding: 10px;
  background-color: #fdf0ef;
}
```

화면 테두리가 전혀 안 보입니다. style 을 안 줬기 때문입니다

```
.bd-styleonly {  
  border-style: solid;  
  padding: 10px;  
  background-color: #fdf0ef;  
}
```

화면 검은색에 가까운 실선이 얇게 그어집니다 (기본 두께 3px, 기본 색은 글자색)

단축 border

섹션 2

```
.bd-short {  
  border: 1px solid #333;  
  padding: 10px;  
  background-color: #ffffff;  
}  
.bd-short-thick {  
  border: 6px solid #2d6cdf;  
  padding: 10px;  
  background-color: #ffffff;  
}
```

border-style 종류

섹션 3

```
.bd-solid {  
  border: 4px solid #2d6cdf;  
  padding: 8px;  
  background-color: #ffffff;  
}  
.bd-dashed {  
  border: 4px dashed #2d6cdf;  
  padding: 8px;  
  background-color: #ffffff;  
}  
.bd-dotted {  
  border: 4px dotted #2d6cdf;  
  padding: 8px;  
  background-color: #ffffff;  
}  
.bd-double {  
  border: 8px double #2d6cdf;  
  padding: 8px;
```

```

}

.bd-nonestyle {
  border: 4px none #2d6cdf;
  padding: 8px;
  background-color: #ffffff;
}

```

화면 마지막 상자만 테두리가 없습니다. none 은 "긋지 않는다" 는 뜻입니다

한쪽만

섹션 4

```

.bd-bottom {
  border-bottom: 5px solid #27853f;
  padding: 8px 0;
}

```

화면 글자 아래에만 초록 밑줄 같은 선이 하나 그어집니다

```

.bd-left-accent {
  border-left: 6px solid #f0c040;
  background-color: #fff8dc;
  padding: 10px 14px;
}

```

화면 왼쪽에만 굵은 노란 세로 막대가 붙은 안내 상자가 됩니다

```

.bd-two-side {
  border-top: 3px solid #c0392b;
  border-bottom: 3px solid #c0392b;
  padding: 10px;
}

```

border-radius

섹션 5

```

.bd-radius {
  border: 3px solid #6b3fc0;
  border-radius: 16px;
  padding: 12px;
  background-color: #f3ecff;
}

```

화면 네 모서리가 둥글게 깎인 상자가 됩니다

```
.bd-pill {
  border: 3px solid #6b3fc0;
  border-radius: 999px;
  padding: 8px 24px;
  background-color: #f3ecff;
}
```

화면 알약처럼 양 끝이 완전히 둥근 상자가 됩니다

```
.bd-corner {
  border: 3px solid #c0392b;
  border-radius: 24px 0 24px 0;
  padding: 12px;
  background-color: #fdf0ef;
}
```

화면 왼쪽 위와 오른쪽 아래만 둥글고 나머지 두 모서리는 각집니다

```
.bd-circle {
  width: 120px;
  height: 120px;
  border-radius: 50%;
  background-color: #6b3fc0;
}
```

화면 지름 120px 짜리 보라색 동그라미가 하나 보입니다

```
.bd-ellipse {
  width: 200px;
  height: 80px;
  border-radius: 50%;
  background-color: #b09ae0;
}
```

화면 가로로 길쭉한 타원이 됩니다. 가로세로가 다르기 때문입니다

outline 과의 비교

섹션 5

```
.bd-vs-border {  
  border: 10px solid #2d6cdf;  
  width: 200px;  
  background-color: #eaf1ff;  
}  
.bd-vs-outline {  
  outline: 10px solid #2d6cdf;  
  width: 200px;  
  background-color: #eaf1ff;  
}
```

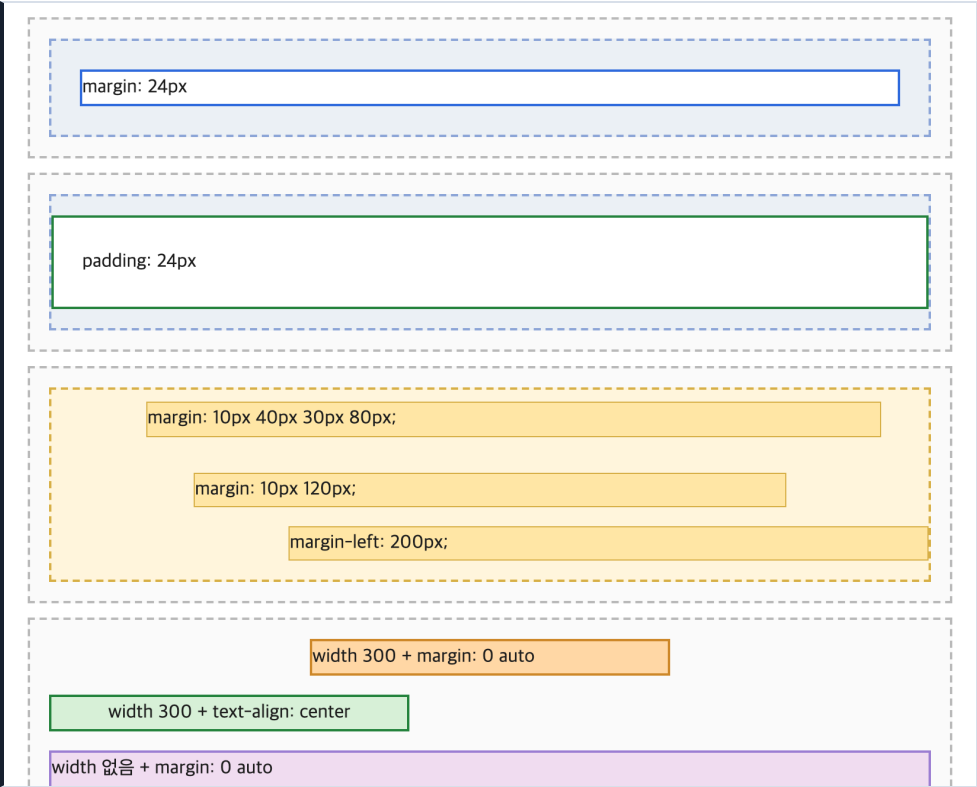
화면 두 상자의 테두리 두께는 같아 보이지만

아래 상자는 실제 가로가 200px 그대로입니다

margin (바깥 여백)

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

F12 개발자도구도 함께 열어 두세요 (개념01 섹션5).



실행하면 나오는 화면 — margin (바깥 여백)

네 겹 중 가장 바깥 겹인 margin 을 다룹니다.

컨텐츠 → padding → border → margin
└ 여기

padding 이 ‘상자 안쪽’ 여백이었다면 margin 은 ‘상자 바깥’ 여백입니다. 상자와 상자 사이를 얼마나 떨어뜨릴지 정하는 것이 margin 입니다.

padding 안쪽으로 밀어 넣는다 → 상자가 뚱뚱해진다
margin 바깥으로 밀어낸다 → 상자끼리 멀어진다

값을 쓰는 방법(값 1~4개, 방향별 지정)은 padding 과 완전히 같습니다. 그래서 이 파일의 앞부분은 빨리 지나갑니다.

진짜 내용은 뒷부분입니다. margin 에는 padding 에 없는 성질이 셋 있습니다.

- 1 auto 를 쓸 수 있습니다. margin: 0 auto 로 상자를 가운데 보냅니다.
- 2 음수를 쓸 수 있습니다. 상자를 반대로 끌어당깁니다.
- 3 위아래 margin 이 겹칩니다. 40 과 30 을 붙여 놓으면 70 이 아니라 40 입니다.

특히 3번은 초보자가 가장 오래 헤매는 부분입니다. “분명히 40 이랑 30 을 줬는데 왜 40 밖에 안 벌어지지?” 버그가 아니라 CSS 가 원래 그렇게 만들어졌습니다. 섹션 4 에서 직접 재 봅니다.

이 파일은 상자 바깥 여백을 다룹니다. 값 쓰는 법은 padding 과 같으니 앞부분은 가볍게 보고, 섹션 3(가운데 정렬) 과 섹션 4(margin 겹침) 에 시간을 쓰세요. 이 두 개가 초보자를 가장 오래 붙잡는 내용입니다.

margin 은 바깥 여백

섹션 1

같은 24px 을 줘도 padding 은 상자를 부풀리고 margin 은 상자를 밀어냅니다.

아래에 똑같이 생긴 바깥 상자 두 개를 뒀습니다. 안에 든 흰 상자 하나는 margin: 24px, 다른 하나는 padding: 24px 입니다.

```
.mg-box { margin: 24px; border: 2px solid #2d6cdf; background-color: #fff; }
.pd-box { padding: 24px; border: 2px solid #27853f; background-color: #fff; }
```

결과를 재 보면 이렇습니다.

```
.mg-box  가로 772px  바깥 상자 안쪽(820px)에서 좌우 24px 씩 물러났습니다
.pd-box  가로 820px  바깥 상자에 딱 붙어 있습니다
```

눈으로도 구분됩니다.

margin 을 준 상자 → 흰 상자과 바깥 상자 사이에 '연회색 띠' 가 보입니다
padding 을 준 상자 → 흰 상자가 벽에 붙어 있고, 글자가 안쪽으로 들어갔습니다

핵심은 여백이 파란 테두리의 ‘바깥’ 에 생겼느냐 ‘안쪽’ 에 생겼느냐입니다. 테두리를 기준으로 나뉘다고 생각하면 절대 안 헷갈립니다.

테두리 바깥의 빈 자리 = margin
테두리 안쪽의 빈 자리 = padding

그리고 개념01 에서 본 그대로, margin 자리에는 색이 안 칠해집니다. 흰 상자의 배경색은 흰색인데 margin 자리에는 바깥 상자의 연회색이 보입니다.

화면 연회색 바깥 상자 안에서 흰 상자가 사방으로 24px 물러나 있습니다.

그 틈에 연회색이 띠처럼 보입니다

```
margin: 24px
```

화면 흰 상자가 바깥 상자에 딱 붙어 있고, 상자 안에서 글자가

사방으로 24px 씩 안으로 들어가 있습니다

```
padding: 24px
```

▹ 직접 해보기

1

파일 위쪽 <style> 에서 .mg-box 의 margin: 24px; 을 margin: 60px; 로 바꾸고 새로고침(F5) 해 보세요. 흰 상자의 가로 길이가 얼마가 되는지 먼저 계산해 본 뒤 개발자도구로 확인하세요.

값 개수는 padding 과 같다

섹션 2

위 → 오른쪽 → 아래 → 왼쪽 시계 방향, 빠진 값은 맞은편에서 빌려 옵니다.

개념02 섹션2·3 에서 배운 규칙이 그대로 적용됩니다. 새로 외울 것이 없습니다.

| | |
|------------------------------|----------------------------|
| margin: 20px; | 네 방향 모두 20px |
| margin: 10px 120px; | 위아래 10, 좌우 120 |
| margin: 10px 40px 30px; | 위 10, 좌우 40, 아래 30 |
| margin: 10px 40px 30px 80px; | 위 10, 오른쪽 40, 아래 30, 왼쪽 80 |

방향별 속성도 똑같이 있습니다.

```
margin-top / margin-right / margin-bottom / margin-left
```

단축을 먼저, 개별을 나중에 쓰는 규칙도 같습니다.

아래 세 상자를 보세요. 바깥 노란 점선이 기준선입니다.

첫째 왼쪽이 80px, 오른쪽이 40px 물러나 있습니다
둘째 좌우가 똑같이 120px 씩 물러나 있습니다
셋째 왼쪽만 200px 물러나고 오른쪽은 기준선에 붙어 있습니다

실제로 재 보면 이렇습니다.

| | | |
|---------------|----------|-------------------|
| .mg-four | 가로 700px | (820 - 80 - 40) |
| .mg-two | 가로 580px | (820 - 120 - 120) |
| .mg-left-only | 가로 620px | (820 - 200) |

margin 을 주면 상자가 그만큼 '작아진다' 는 점을 눈여겨보세요. padding 은 상자를 키웠는데 margin 은 반대입니다. 한 줄을 통째로 차지하는 상자는 자리가 정해져 있어서, 바깥 여백을 늘리면 그만큼 자기 몫이 줄어듭니다.

화면 노란 점선 상자 안에 연노랑 상자 세 개가 들어 있습니다.

들여쓰기 폭이 서로 다르고, 세 번째 상자만 오른쪽 끝이 점선에 붙어 있습니다

```
margin: 10px 40px 30px 80px;
margin: 10px 120px;
margin-left: 200px;
```

▹ 직접 해보기 2

.mg-left-only 에 margin-right: 200px; 한 줄을 더해 보세요. 상자가 좌우 대칭으로 줄어드는 것을 확인하고 되돌리세요.

margin 0 auto 로 가운데 보내기

섹션 3

글자를 가운데 두는 것과 상자를 가운데 두는 것은 다른 일입니다. 09단원의 text-align: center 와 헷갈리기 쉽습니다.

09단원에서 text-align: center 를 배웠습니다. 그것은 '상자 안에 든 글자' 를 가운데로 보내는 속성입니다. 상자 자체는 한 발짝도 움직이지 않습니다.

상자 자체를 가운데로 보내려면 margin 에 auto 를 씁니다.

```
margin: 0 auto;
  └─┬─┬─
    │  └─ 좌우 : 남은 자리를 반씩 나눠 가지세요
    └─ 위아래 : 0
```

auto 는 “브라우저가 알아서 정하라” 는 뜻입니다. 왼쪽과 오른쪽을 둘 다 auto 로 두면 브라우저는 남은 자리를 정확히 반으로 나눕니다. 그 결과 상자가 한가운데에 놓입니다.

여기에 조건이 하나 붙습니다. 아주 중요합니다.

상자에 width 가 정해져 있어야 합니다.

왜냐하면 width 가 없는 상자는 한 줄을 통째로 차지해 버리기 때문입니다. 자리가 남지 않으니 반으로 나눌 것도 없습니다. auto 는 0 이 됩니다.

(p, div 같은 태그가 왜 한 줄을 통째로 차지하는지는 11단원에서 배웁니다. 지금은 “p, div 같은 태그는 한 줄을 통째로 차지한다” 만 기억하세요. 02단원 개념04 에서 본 블록 이야기와 같은 것입니다)

아래 세 상자를 비교하세요.

| | | |
|----|--|----------------------|
| 첫째 | <code>width: 300px + margin: 0 auto</code> | → 상자가 가운데로 갔습니다 |
| 둘째 | <code>width: 300px + text-align: center</code> | → 상자는 왼쪽, 글자만 가운데입니다 |
| 셋째 | <code>width 없음 + margin: 0 auto</code> | → 아무 일도 안 일어났습니다 |

실제로 재 보면 이렇습니다.

| | | |
|----|-------------------|--------|
| 첫째 | 왼쪽 끝이 화면 x 좌표 488 | 가로 304 |
| 둘째 | 왼쪽 끝이 화면 x 좌표 228 | 가로 304 |
| 셋째 | 왼쪽 끝이 화면 x 좌표 228 | 가로 824 |

둘째 상자는 첫째와 가로가 똑같은데 자리는 왼쪽 그대로입니다. `text-align` 이 옮긴 것은 상자가 아니라 글자였다는 증거입니다.

정리하면 이렇습니다.

| | | |
|----------|---------------------------------------|--------|
| 글자를 가운데로 | → <code>text-align: center</code> | (09단원) |
| 상자를 가운데로 | → <code>width + margin: 0 auto</code> | (지금) |

참고로 `width` 대신 `max-width` 를 써도 됩니다. 개념05 에서 이어서 봅니다. 여러분이 지금 보고 있는 이 자료 페이지 자체가 `max-width` 와 `margin: 40px auto` 로 가운데 놓인 것입니다.

화면 주황 상자만 좌우 한가운데에 있습니다.

초록 상자는 왼쪽에 붙어 있고 글자만 상자 안에서 가운데에 있습니다. 보라 상자는 한 줄을 다 차지하며 왼쪽 끝에서 시작합니다

| |
|---|
| <code>width 300 + margin: 0 auto</code> |
| <code>width 300 + text-align: center</code> |
| <code>width 없음 + margin: 0 auto</code> |

가운데로 안 가면 `width` 부터 확인하세요. `margin: 0 auto` 가 안 먹는 이유의 열에 아홉은 `width` 가 없어서입니다. 나머지 하나는 그 요소가 한 줄을 통째로 차지하는 종류가 아니어서인데, 그 이야기는 11단원에서 합니다.

▹ 직접 해보기 3

.center-nowidth 에 `width: 400px`; 한 줄을 더해 보세요. 보라 상자가 가운데로 이동합니다. `auto` 가 갑자기 일을 시작한 이유를 말로 설명해 보세요.

margin 겹침

섹션 4

위아래로 맞닿은 `margin` 은 더해지지 않고 큰 쪽만 남습니다. 40 과 30 이 만나면 70 이 아니라 40 입니다.

아래 두 상자를 준비했습니다. 값은 완전히 똑같습니다. 40 과 30 입니다. 다른 것은 그 값을 margin 으로 줬느냐 padding 으로 줬느냐뿐입니다.

노란 상자 · `.col-top { margin: 0 0 40px; }` 아래쪽 margin 40

`.col-bottom { margin: 30px 0 0; }` 위쪽 margin 30

초록 상자 · `.pcol-top { padding: 0 0 40px; }` 아래쪽 padding 40

`.pcol-bottom { padding: 30px 0 0; }` 위쪽 padding 30

실제로 재 보면 바깥 상자의 세로 길이가 이렇게 다릅니다.

| | |
|----------------|-------|
| 노란 상자(margin) | 102px |
| 초록 상자(padding) | 132px |

차이는 정확히 30px 입니다. margin 쪽에서 30 이 통째로 사라진 것입니다.

무슨 일이 일어났을까요?

| | |
|-----------|---|
| padding 쪽 | $40 + 30 = 70$ 만큼 벌어졌습니다. 예상대로입니다. |
| margin 쪽 | 40 과 30 중 큰 쪽인 40 만 남았습니다. 30 은 사라졌습니다. |

위아래로 맞닿은 margin 두 개가 하나로 합쳐지는 이 현상을 margin 겹침(margin collapse) 이라고 부릅니다. 합쳐진 뒤의 크기는 '둘을 더한 값' 이 아니라 '둘 중 큰 값' 입니다.

왜 이런 규칙이 있을까요? 버그가 아니라 일부러 만든 것입니다.

문단(p) 에는 원래 위아래 margin 이 붙어 있습니다. 문단을 여러 개 이어 쓰면 앞 문단의 아래 margin 과 뒤 문단의 위 margin 이 매번 맞닿습니다. 겹치지 않는다면 문단 사이만 간격이 두 배로 벌어져서 글의 첫 문단과 마지막 문단만 간격이 다른 이상한 문서가 됩니다. 그래서 CSS 는 맞닿은 위아래 여백을 하나로 합치기로 정했습니다.

기억할 것은 세 가지입니다.

- 1 겹치는 것은 위아래뿐입니다. 좌우 margin 은 절대 안 겹칩니다.
- 2 겹친 결과는 큰 쪽입니다. 더하지 않습니다.
- 3 padding 과 border 는 절대 안 겹칩니다. 언제나 그대로 더해집니다.

3번이 해결책의 실마리입니다. 사이를 확실히 벌리고 싶으면 한쪽에만 margin 을 주거나(예: 아래쪽 margin 만 쓰기), 아예 padding 으로 벌리면 됩니다.

화면 노란 점선 상자 안에서 흰 상자 두 개가 40px 만큼 떨어져 있습니다

아래쪽 margin 40px
위쪽 margin 30px

화면 초록 점선 상자 안에서 흰 상자 두 개가 서로 딱 붙어 있고,

각 상자 안쪽에 여백이 들어가 전체가 더 길어졌습니다

아래쪽 padding 40px
위쪽 padding 30px

개발자도구로 노란 상자의 colTop 을 찍어 보세요. margin 칸 아래쪽에는 40 이 제대로 적혀 있습니다. colBottom 의 margin 칸 위쪽에도 30 이 적혀 있습니다. 값이 지워진 것이 아니라, 둘이 같은 자리를 나눠 쓰고 있는 것입니다.

겹침에는 또 한 가지 경우가 있습니다. 부모와 자식 사이입니다.

부모 상자에 padding 도 border 도 없으면, 자식의 위쪽 margin 이 부모를 뚫고 밖으로 새어 나갑니다. 부모가 자식을 감싸지 못하고 자식과 함께 아래로 밀려 내려갑니다.

아래 분홍 상자가 그 상태입니다. 자식에게 margin-top: 40px 을 줬는데 분홍색이 한 점도 안 보입니다. 분홍 상자의 세로 길이는 29px 로, 자식 높이와 똑같습니다. 여백이 사라진 것이 아니라 분홍 상자 위쪽 바깥으로 빠져나간 것입니다.

초록 상자에는 border 를 1px 만 줬습니다. 그것만으로 새어 나가지 못합니다. 세로 길이가 71px 이 되고 위쪽에 초록 띠 40px 이 보입니다.

이 자료의 데모 상자들에 굳이 점선 테두리를 둘러 둔 이유가 이것입니다. 테두리가 없으면 안쪽 상자의 margin 이 밖으로 새어 나가서 여백이 어디로 갔는지 알 수 없게 됩니다.

화면 위쪽 분홍 상자에는 분홍색이 한 점도 안 보입니다.

흰 상자가 분홍 상자를 통째로 덮고 있어서, 40px 짜리 여백은 분홍 상자 위쪽 바깥으로 빠져나가 버렸습니다. 아래 초록 상자에는 흰 상자 위로 초록 띠가 40px 만큼 보입니다

부모에 테두리가 없습니다 (margin-top: 40px)

부모에 테두리가 1px 있습니다 (margin-top: 40px)

◁ 직접 해보기 4

.col-bottom 의 margin: 30px 0 0; 을 margin: 90px 0 0; 으로 바꿔 보세요. 이번에는 90 이 40 보다 크므로 90 이 이깁니다. 개발자도구로 상자 사이가 90px 이 되는지 확인하고 되돌리세요.

음수 margin

섹션 5

margin 에는 음수를 쓸 수 있습니다. 밀어내는 대신 끌어당깁니다.

padding 에는 음수를 못 씁니다. 상자 안쪽을 마이너스만큼 채운다는 말이 성립하지 않습니다. 하지만 margin 은 '거리' 이므로 음수가 말이 됩니다.

```
margin-top: -14px;    위쪽으로 14px 끌어올립니다
margin-left: -12px;   왼쪽으로 12px 잡아당깁니다
```

아래 첫 데모에서 주황 상자에 margin-top: -14px 을 줬습니다. 주황 상자가 파란 상자 위로 14px 올라가 살짝 겹칩니다. 나중에 쓴 상자가 위에 그려지므로 주황 배경이 파란 배경을 덮습니다.

한 가지만 덧붙입니다. 덮이는 것은 '배경' 이고 '글자' 는 아닙니다. 브라우저는 상자들의 배경을 먼저 다 칠한 다음에 글자를 엽니다. 그래서 파란 상자의 글자는 주황 배경 위에 그대로 남습니다. 많이 겹치면 두 상자의 글자가 뒤엉켜 보이게 됩니다. 직접 해보기 5 에서 확인합니다.

두 번째 상자에는 좌우로 -12px 씩 줬습니다.

```
margin: 12px -12px 0;
```

좌우 margin 이 음수라서 상자가 부모보다 넓어집니다. 부모 안쪽 폭이 820px 인데 이 상자는 844px 입니다. 점선 밖으로 살짝 튀어나온 것이 보입니다.

음수 margin 은 강력하지만 위험합니다. 상자가 겹치면 아래에 깔린 상자는 클릭이 안 되는 일이 생깁니다. 요즘은 이런 배치를 12~13단원의 flex, grid 로 하는 것이 보통입니다. 여기서는 "이런 것도 된다" 는 것만 알아 두세요.

화면 파란 상자 위로 주황 상자가 올라타 살짝 겹칩니다.

맨 아래 분홍 상자는 점선 테두리 밖으로 좌우가 조금씩 튀어나옵니다

```
위쪽 상자입니다
margin-top: -14px 로 위로 끌어올렸습니다
margin: 12px -12px 0 로 부모보다 넓어졌습니다
```

◀ 직접 해보기 5

.neg-b 의 margin: -14px 0 0; 을 margin: -29px 0 0; 으로 바꿔 보세요. 파란 배경이 통째로 사라집니다. 그런데 파란 상자의 글자는 사라지지 않습니다. 무슨 일이 벌어지는지 눈으로 확인하고 되돌리세요.

자주 하는 실수

섹션 6

margin 은 안 보이는 여백이라 잘못돼도 알아채기 어렵습니다. 에러는 나지 않습니다.

이런 실수를 합니다

width 없이 margin: 0 auto

이런 실수를 합니다

실수 1 — width 없이 margin: 0 auto 를 씀

```
.card {  
  margin: 0 auto; /* width 가 없습니다 */  
}
```

아무 일도 안 일어납니다. 상자는 왼쪽에 그대로 있습니다. 한 줄을 통째로 차지하는 상자에는 남는 자리가 없어서 반으로 나눌 것이 없기 때문입니다. width 나 max-width 를 먼저 정하세요.

이렇게 쓰세요

```
.card {  
  width: 300px;  
  margin: 0 auto;  
}
```

이런 실수를 합니다

text-align 으로 상자를 옮기려 함

이런 실수를 합니다

실수 2 — text-align: center 로 상자를 옮기려 함

```
.card {  
  width: 300px;  
  text-align: center;  
}
```

글자만 가운데로 가고 상자는 왼쪽에 그대로 있습니다. text-align 은 상자 안의 글자를 정렬하는 속성입니다. 상자 자체를 옮기는 것은 margin: 0 auto 의 일입니다. 섹션 3 의 초록 상자가 정확히 이 상태입니다.

이런 실수를 합니다

margin 겹침을 버그로 오해

이런 실수를 합니다

실수 3 — margin 겹침을 버그로 오해함

```
.a { margin-bottom: 40px; }
.b { margin-top: 30px; }    /* 70px 벌어질 거라고 기대 */
```

40px 만 벌어집니다. 값이 무시된 게 아니라 둘이 합쳐진 것입니다. 여기서 30 을 40 으로 늘려도 여전히 40 이라, 값을 바꿔도 화면이 안 변해 “CSS 가 안 먹는다” 고 오해하기 딱 좋습니다. 꼭 70 이 필요하면 한쪽을 padding 으로 주세요.

이런 실수를 합니다

자식 margin 이 부모 밖으로 샘

이런 실수를 합니다

실수 4 — 안쪽 상자의 margin 이 밖으로 새어 나감

```
.parent { background-color: pink; }
.child { margin-top: 40px; }
```

안쪽 상자를 아래로 40px 내리려고 했는데 부모까지 통째로 40px 내려가 버립니다. 둘 사이는 그대로 붙어 있습니다. 부모에 padding 이나 border 를 조금이라도 주면 막힙니다. 섹션 4 의 분홍 상자와 초록 상자를 비교해 보세요.

이런 실수를 합니다

단위 빠뜨림

이런 실수를 합니다

실수 5 — 단위를 빠뜨림

```
margin: 20;
```

여백이 하나도 안 생깁니다. 이 줄이 통째로 버려집니다. margin: 0 auto 처럼 0 은 단위 없이 써도 되지만 나머지 숫자에는 반드시 단위를 붙이세요.

이런 실수를 합니다

margin 에 색이 칠해질 거라고 생각함

이런 실수를 합니다

실수 6 — margin 자리에도 색이 칠해질 거라고 생각함

상자를 넓혀 보려고 margin 을 키웠는데 색칠된 면적은 오히려 줄어듭니다. margin 은 언제나 투명하고, 한 줄을 통째로 차지하는 상자에서는 바깥 여백이 커진 만큼 자기 자리가 좁아지기 때문입니다. 색까지 넓히려면 padding 을 쓰세요.

정리

▹ 직접 해보기 6

정답

1) style 안의 .mg-box 를 이렇게 바꿉니다.

```
.mg-box {  
  margin: 60px;  
  border: 2px solid #2d6cdf;  
  background-color: #ffffff;  
}
```

→ 계산: 바깥 상자 안쪽 폭 820 에서 좌우 60 씩 빼면 700px 입니다.

개발자도구로 재도 700 이 나옵니다.

연회색 띠가 24px 에서 60px 로 두꺼워집니다. 되돌려 놓으세요.

2) style 안의 .mg-left-only 를 이렇게 바꿉니다.

```
.mg-left-only {  
  margin-left: 200px;  
  margin-right: 200px;  
  background-color: #ffe9a8;  
  border: 1px solid #d9b34a;  
}
```

→ 가로가 620px 에서 420px 로 줄고, 상자가 좌우 대칭이 됩니다.

$820 - 200 - 200 = 420$ 입니다. 되돌려 놓으세요.

3) style 안의 .center-nowidth 를 이렇게 바꿉니다.


```
.center-nowidth {
  width: 400px;
  margin: 0 auto;
  border: 2px solid #9b7fd4;
  background-color: #f3dcf3;
}
```

→ 보라 상자가 가운데로 이동합니다.

설명: width 가 없을 때는 상자가 한 줄(824px)을 통째로 먹어서 남는 자리가 0 이었습니다. auto 는 남는 자리를 반씩 나누는 값이므로 나눌 것이 없으면 0 이 됩니다.
width: 400px 을 주는 순간 좌우에 남는 자리가 생기고, auto 가 그것을 반씩 나눠 가져서 상자가 가운데로 갑니다.

4) style 안의 .col-bottom 을 이렇게 바꿉니다.

```
.col-bottom {
  margin: 90px 0 0;
  border: 1px solid #bbbbbb;
  background-color: #ffffff;
}
```

→ 두 상자 사이가 40px 에서 90px 로 벌어집니다.

노란 바깥 상자의 세로 길이도 102px 에서 152px 이 됩니다.
40 과 90 중 큰 쪽인 90 이 남은 것입니다. 더한 값 130 이 아닙니다.
되돌려 놓으세요.

5) style 안의 .neg-b 를 이렇게 바꿉니다.

```
.neg-b {
  margin: -29px 0 0;
  border: 1px solid #d08a2a;
  background-color: #ffd9a8;
}
```

→ 파란 상자의 세로 길이가 29px 이므로, 주황 상자가 정확히 그만큼 올라와

파란 상자와 같은 자리에 겹칩니다.

파란 배경은 주황 배경에 완전히 덮여서 화면에서 사라집니다.
그런데 파란 상자의 글자 "위쪽 상자입니다" 는 그대로 남아,
주황 상자의 글자와 같은 줄에서 서로 뒤엉켜 보입니다.
배경은 덮이지만 글자는 덮이지 않는다는 것을 여기서 확인하세요.

파란 상자가 지워진 것은 아닙니다. 개발자도구 Elements 에서
파란 상자 줄에 마우스를 올리면 주황 상자 자리에서 반짝이는 것이 보입니다.
확인했으면 되돌리세요.

margin 과 padding 을 나란히

섹션 1

```
.mg-stage {  
  background-color: #eef2f7;  
  border: 2px dashed #90a8d8;  
}
```

화면 연회색 파란빛 바탕에 점선 테두리를 두른 바깥 상자입니다

```
.mg-box {  
  margin: 24px;  
  border: 2px solid #2d6cdf;  
  background-color: #ffffff;  
}
```

화면 바깥 상자와 24px 떨어진 흰 상자. 그 틈에는 바깥 상자 색이 보입니다

```
.pd-box {  
  padding: 24px;  
  border: 2px solid #27853f;  
  background-color: #ffffff;  
}
```

화면 바깥 상자에 딱 붙은 흰 상자. 여백이 상자 '안쪽' 에 생깁니다

값 개수

섹션 2

```
.mg-four {  
  margin: 10px 40px 30px 80px;  
  background-color: #ffe9a8;  
  border: 1px solid #d9b34a;  
}
```

```
.mg-two {
  margin: 10px 120px;
  background-color: #ffe9a8;
  border: 1px solid #d9b34a;
}
.mg-left-only {
  margin-left: 200px;
  background-color: #ffe9a8;
  border: 1px solid #d9b34a;
}
```

margin 0 auto 로 가운데 보내기

섹션 3

```
.center-box {
  width: 300px;
  margin: 0 auto;
  border: 2px solid #d08a2a;
  background-color: #ffd9a8;
}
```

화면 상자 자체가 좌우 한가운데로 옮겨집니다

```
.center-text {
  width: 300px;
  text-align: center;
  border: 2px solid #27853f;
  background-color: #d7f0d7;
}
```

화면 상자는 왼쪽에 그대로 있고 글자만 상자 안에서 가운데로 옵니다

```
.center-nowidth {
  margin: 0 auto;
  border: 2px solid #9b7fd4;
  background-color: #f3dcf3;
}
```

화면 width 가 없어서 상자가 한 줄을 다 먹습니다. 가운데로 갈 자리가 없습니다

margin 겹침

섹션 4

```
.col-stage {
  background-color: #fff8dc;
```

```

}

.col-top {
  margin: 0 0 40px;
  border: 1px solid #bbbbbb;
  background-color: #ffffff;
}

.col-bottom {
  margin: 30px 0 0;
  border: 1px solid #bbbbbb;
  background-color: #ffffff;
}

.pcol-stage {
  background-color: #eaf7ee;
  border: 2px dashed #7fbf90;
}

.pcol-top {
  margin: 0;
  padding: 0 0 40px;
  border: 1px solid #bbbbbb;
  background-color: #ffffff;
}

.pcol-bottom {
  margin: 0;
  padding: 30px 0 0;
  border: 1px solid #bbbbbb;
  background-color: #ffffff;
}

```

자식 margin 이 부모 밖으로 새는 경우

섹션 4

```

.esc-stage {
  background-color: #fbe0e0;
}

.esc-fixed {
  background-color: #eaf7ee;
  border: 1px solid #7fbf90;
}

.esc-box {
  margin: 40px 0 0;
  border: 1px solid #bbbbbb;
  background-color: #ffffff;
}

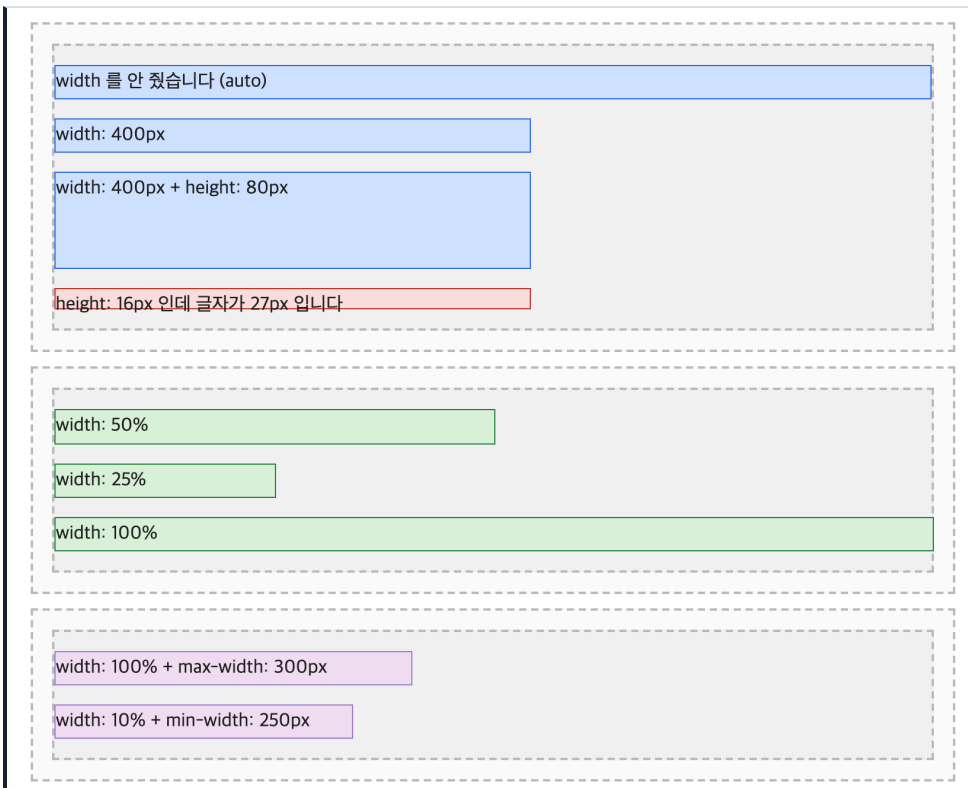
```

```
.neg-stage {  
  background-color: #f4f4f4;  
  border: 2px dashed #bbbbbb;  
}  
.neg-a {  
  margin: 0;  
  border: 1px solid #2d6cdf;  
  background-color: #cfe3ff;  
}  
.neg-b {  
  margin: -14px 0 0;  
  border: 1px solid #d08a2a;  
  background-color: #ffd9a8;  
}  
.neg-wide {  
  margin: 12px -12px 0;  
  border: 1px solid #c0392b;  
  background-color: #fbe0e0;  
}
```

width 와 box-sizing

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

F12 개발자도구도 함께 열어 두세요 (개념01 섹션5).



실행하면 나오는 화면 — width 와 box-sizing

지금까지 padding, border, margin 세 겹을 배웠습니다. 이제 가장 안쪽 겹인 콘텐츠 영역의 크기를 정합니다.

콘텐츠 → padding → border → margin
 ↳ 여기. width 와 height 로 정합니다.

그런데 이 파일의 진짜 주제는 width 자체가 아닙니다.

width: 300px 이라고 썼는데 화면에서 재면 340px 이다.

이 사고를 이해하고 고치는 것이 목표입니다. 개념02 섹션5 와 개념03 에서 이미 예고했던 그 이야기입니다.

원인은 한 줄로 말할 수 있습니다.

CSS 의 width 는 '상자 전체' 가 아니라 '콘텐츠 영역' 의 가로 길이이다.

그래서 padding 과 border 를 더할 때마다 상자가 부풀어 오릅니다. 옆에 나란히 놓으려던 상자가 줄바꿈되고, 부모 밖으로 빠져나옵니다.

해결책은 box-sizing 이라는 속성 한 줄입니다.

```
box-sizing: content-box;    기본값. width 는 콘텐츠만.
box-sizing: border-box;    width 를 '테두리까지 포함한 전체' 로 세게 합니다.
```

요즘 만들어지는 거의 모든 웹사이트가 border-box 를 켜고 시작합니다. 왜 그런지는 섹션 5 에서 직접 재 보면 납득이 됩니다.

이 파일은 상자의 크기를 정하는 법과, 정한 크기가 왜 안 지켜지는지를 다룹니다. 섹션 4 와 섹션 5 를 나란히 놓고 보세요. 같은 값을 줬는데 결과가 다릅니다. 이 단원에서 실무로 바로 이어지는 내용이 여기에 가장 많습니다.

width 와 height

섹션 1

width 는 가로, height 는 세로 길이입니다. 둘 다 콘텐츠 영역의 크기입니다.

아무것도 안 정하면 width 와 height 는 auto 입니다. auto 는 브라우저가 알아서 정한다는 뜻인데, 방향에 따라 뜻이 다릅니다.

```
width: auto    한 줄을 통째로 차지합니다. 부모 안쪽 폭을 다 씁니다.
height: auto   안에 든 내용의 높이만큼만 차지합니다.
```

가로는 최대한 넓게, 세로는 최소한만. 방향에 따라 정반대입니다. 글이 위에서 아래로 흐르는 문서라서 그렇습니다.

(p, div 같은 태그가 한 줄을 통째로 차지한다는 이야기는 02단원 개념04 의 블록 이야기입니다. 왜 그런지는 11단원에서 배웁니다)

여기에 숫자를 넣으면 그 크기로 못 박힙니다.

```
width: 400px;    가로를 400px 로
height: 80px;    세로를 80px 로
```

아래 상자들을 재 보면 이렇습니다.

```
.w-auto    가로 820px   (부모 안쪽을 다 씁니다)
.w-fixed   가로 402px   (400 + 테두리 1 + 1)
.wh-fixed  가로 402px, 세로 82px
```

height 를 줄 때는 조심할 것이 하나 있습니다. 내용이 그 높이보다 크면 그냥 밖으로 빠져나옵니다. 상자가 늘어나지 않습니다. 아래 마지막 빨간 상자가 그 예입니다. 높이를 16px 로 줬더니 27px 짜리 글자가 상자

밖으로 흘러나왔습니다.

그래서 실무에서는 height 를 잘 안 씁니다. 세로는 내용이 정하도록 두고, padding 으로 여유를 주는 편이 안전합니다. 빠져나온 내용을 어떻게 다룰지는 개념06 의 overflow 에서 배웁니다.

화면 첫 상자는 점선 안쪽을 꽉 채우고,

둘째 셋째는 왼쪽 400px 만 차지하며 셋째가 세로로 더 길니다. 마지막 빨간 상자는 글자가 상자 아래로 빠져나와 있습니다

```
width 를 안 줬습니다 (auto)
width: 400px
width: 400px + height: 80px
height: 16px 인데 글자가 27px 입니다
```

height 를 억지로 정하기보다 padding 으로 위아래 여유를 주는 편이 안전합니다. 글자 크기가 바뀌거나 글이 길어져도 상자가 알아서 늘어나기 때문입니다.

◦ 직접 해보기

1

파일 위쪽 <style> 에서 .h-tight 의 height: 16px; 을 height: 60px; 로 바꾸고 새로고침(F5) 해 보세요. 글자가 상자 안으로 들어옵니다.

% 단위

섹션 2

% 는 부모 상자의 안쪽 폭을 기준으로 한 비율입니다.

px 은 화면 크기가 어떻든 늘 같은 길이입니다. % 는 부모를 기준으로 한 비율이라, 부모가 넓어지면 같이 넓어집니다.

```
width: 50%;      부모 안쪽 폭의 절반
```

여기서 '부모 안쪽 폭' 이란 부모의 콘텐츠 영역 가로 길이입니다. 아래 점선 상자의 안쪽 폭은 820px 입니다. 그래서 50% 는 410px, 25% 는 205px 이 됩니다.

실제로 재면 테두리 1px 이 좌우에 붙어서 이렇게 나옵니다.

| | | |
|------------|-------|---------------|
| .w-half | 412px | (410 + 1 + 1) |
| .w-quarter | 207px | (205 + 1 + 1) |
| .w-full | 822px | (820 + 1 + 1) |

마지막 줄을 잘 보세요. 100% 를 줬는데 822px 입니다. 부모 안쪽은 820px 인데 2px 이 더 큼니다. 테두리 두께가 width 에 포함되지 않아서 그렇습니다. 이 2px 이 섹션 4 에서 40px 이 되고, 진짜 문제가 됩니다.

브라우저 창의 가로 폭을 마우스로 줄였다 늘렸다 해 보세요. px 로 정한 상자는 그대로인데 % 로 정한 상자는 같이 늘었다 줄었다 합니다. (이 자료 페이지 자체에는 max-width 가 걸려 있어서 창을 아주 넓게 키우면 어느 지점부터 더 넓어지지 않습니다)

참고로 화면 크기를 직접 기준으로 삼는 vw, vh 같은 단위도 있습니다. 그것은 13단원 반응형에서 배웁니다. 여기서는 % 만 씁니다.

화면 초록 상자 세 개의 길이가 절반, 4분의 1, 짝 참 순으로 다릅니다.

마지막 상자는 점선 테두리를 아주 살짝 넘어갑니다

```
width: 50%
width: 25%
width: 100%
```

직접 해보기 2

.w-quarter 의 width: 25%; 를 width: 75%; 로 바꾸고, 실제 가로가 몇 px 이 될지 먼저 계산한 뒤 개발자도구로 확인해 보세요.

max-width 와 min-width

섹션 3

max-width 는 여기까지만, min-width 는 여기 아래로는 안 됨이라는 한계선입니다.

width 는 “이 크기로 하라” 는 명령이고, max-width 와 min-width 는 “이 선을 넘지 마라” 는 한계선입니다. 둘을 같이 쓰면 한계선이 항상 이깁니다.

```
width: 100%;
max-width: 300px;
→ 부모가 아무리 넓어도 300px 을 넘지 않습니다.
  부모가 300px 보다 좁아지면 그때는 부모를 따라 같이 좁아집니다.

width: 10%;
min-width: 250px;
→ 10% 는 82px 이지만, 250px 아래로는 안 내려갑니다.
```

이 조합이 왜 유용할까요? 넓은 화면에서는 글줄이 너무 길어지면 읽기 힘들고, 좁은 화면에서는 상자가 화면 밖으로 나가면 안 됩니다. max-width 는 그 두 가지를 한 줄로 해결합니다.

```
width: 100%;      좁은 화면에서는 화면에 맞추고
max-width: 860px; 넓은 화면에서는 860px 에서 멈춘다
```

여러분이 지금 보고 있는 이 자료 페이지가 정확히 이 방식입니다. 공통.css 에 max-width: 860px 과 margin: 40px auto 가 적혀 있습니다. 개념04 섹션3 에서 배운 가운데 정렬과 짝을 이루는 것입니다.

max-width 는 13단원 반응형의 기초가 되는 속성이기도 합니다.

화면 첫 보라 상자는 100% 라고 썼는데도 왼쪽 300px 정도만 차지하고,

둘째 보라 상자는 10% 라고 썼는데도 그보다 훨씬 넓습니다

```
width: 100% + max-width: 300px
width: 10% + min-width: 250px
```

max-height 와 min-height 도 있습니다. 쓰는 방법은 같지만 훨씬 덜 씁니다. 세로는 내용이 정하도록 두는 것이 보통이기 때문입니다.

▹ 직접 해보기 3

.w-max 의 max-width: 300px; 을 max-width: 900px; 으로 바꿔 보세요. 한계선이 부모보다 커지면 어떤 쪽이 되는지 확인하고 되돌리세요.

300px 이라고 썼는데 340px

섹션 4

CSS 의 width 는 콘텐츠 영역만의 가로 길이입니다. padding 과 border 는 그 바깥에 따로 붙습니다.

아래에 자를 하나 놓았습니다. 회색 상자는 width 가 340px 이고 테두리가 없어서 실제 가로도 정확히 340px 입니다. 그 아래 파란 상자에는 이렇게 썼습니다.

```
.bs-content {
  width: 300px;
  padding: 15px;
  border: 5px solid #2d6cdf;
}
```

결과를 보면 파란 상자의 오른쪽 끝이 회색 자와 정확히 같은 자리에서 끝납니다. 즉 파란 상자의 실제 가로는 300px 이 아니라 340px 입니다.

계산은 이렇습니다.

| | |
|---------|--------------|
| 콘텐츠 | 300 |
| padding | 15 + 15 = 30 |
| border | 5 + 5 = 10 |

| | |
|----|-----|
| 합계 | 340 |
|----|-----|

코드 어디에도 340 이라는 숫자는 없습니다. 브라우저가 더해서 만든 값입니다.

이 계산 방식에는 이름이 있습니다.

```
box-sizing: content-box
```

content-box 는 “width 는 콘텐츠 상자의 크기다” 라는 뜻이고, 아무것도 안 쓰면 이것이 기본값입니다. 그래서 지금까지 만든 모든 상자가 이 방식으로 계산되어 왔습니다.

문제는 이 방식이 사람의 직관과 정반대라는 것입니다.

사람의 생각 "가로 300px 짜리 상자를 만들었다"
브라우저의 계산 "속을 300px 로 하고 겉을 340px 로 만들었다"

상자 세 개를 가로로 나란히 놓으려고 각각 33% 를 줬는데 padding 때문에 넘쳐서 줄바꿈되는 사고가 여기서 나옵니다. 여백 하나 바꿀 때마다 width 를 다시 계산해야 한다면 아무도 못 버팁니다.

다음 섹션에 해결책이 있습니다.

화면 파란 상자의 오른쪽 끝이 위 회색 상자의 오른쪽 끝과 정확히 맞습니다.

300 이라고 썼는데 340 짜리 자와 길이가 같습니다

이 회색 자는 정확히 340px 입니다
width: 300px + padding 15 + border 5

개발자도구로 파란 상자를 찍어 보세요. 박스 모델 그림의 콘텐츠 칸에는 300 이라고 적혀 있는데, Elements 패널에서 그 줄에 마우스를 올리면 상자 옆에 340 으로 표시됩니다. 두 숫자가 다른 것이 정상입니다.

▹ 직접 해보기

4

.bs-content 의 padding: 15px; 을 padding: 40px; 으로 바꿔 보세요. 회색 자보다 얼마나 튀어나오는지 보고, 실제 가로가 몇 px 이 되는지 계산해 보세요.

box-sizing: border-box

섹션 5

box-sizing: border-box; 한 줄이면 width 가 테두리까지 포함한 전체 크기가 됩니다.

아래 초록 상자에는 섹션 4 의 파란 상자과 똑같은 값을 줬습니다. 딱 한 줄만 다릅니다.

```
.bs-border {
  box-sizing: border-box;  ← 이 한 줄만 추가
  width: 300px;
  padding: 15px;
  border: 5px solid #27853f;
}
```

실제로 재면 초록 상자의 가로는 정확히 300px 입니다. 파란 상자는 340px 이었습니다. 같은 값을 줬는데 40px 이 다릅니다.

border-box 에서는 계산이 반대 방향으로 일어납니다.

| | |
|-------------|---|
| content-box | 콘텐츠 300 에 padding 과 border 를 '더한다' → 340 |
| border-box | 전체 300 에서 padding 과 border 를 '빼고' 남는 만큼이 콘텐츠
$300 - 30 - 10 = 260$ 이 콘텐츠 영역이 됩니다 |

즉 padding 과 border 를 아무리 바꿔도 상자의 겉 크기는 300px 그대로입니다. 안쪽 여유만 줄었다 늘었다 합니다. 이것이 사람이 원래 기대하던 계산 방식입니다.

효과가 가장 극적으로 드러나는 경우가 width: 100% 입니다.

```
width: 100%;  
padding: 20px;
```

content-box 라면 부모 안쪽 폭 820 에 padding 40 과 테두리 2 가 더해져 862 가 됩니다. 부모 밖으로 좌우가 빠져나옵니다. 아래 빨간 상자가 그 모습입니다. border-box 라면 820 안에서 padding 을 빼고 쓰므로 딱 맞습니다. 초록 상자입니다.

실무 관행이 여기서 나옵니다. 요즘은 파일 맨 위에 이 한 줄을 깔고 시작합니다.

```
* {  
  box-sizing: border-box;  
}
```

여기서 별표(*) 는 '모든 요소' 를 뜻하는 선택자입니다. 페이지의 모든 상자를 border-box 방식으로 바꾸겠다는 뜻입니다. 부트스트랩을 비롯한 거의 모든 CSS 도구가 이렇게 시작합니다.

이 자료 파일에서는 일부터 켜 두지 않았습니다. 기본값인 content-box 가 어떻게 동작하는지 눈으로 봐야 border-box 가 무엇을 해 주는지 알 수 있기 때문입니다. 여러분이 직접 만드는 페이지에서는 맨 위에 * 규칙을 깔고 시작하세요.

화면 위 파란 상자가 아래 초록 상자보다 눈에 띄게 깁니다.

코드에는 둘 다 width: 300px 이라고 적혀 있습니다

| |
|------------------------------|
| content-box (기본값) : 실제 340px |
| border-box : 실제 300px |

화면 빨간 상자는 점선 테두리를 좌우로 뚫고 나가 있고,

초록 상자는 점선 안쪽에 정확히 맞춰 들어가 있습니다

```

content-box : 부모 밖으로 나갑니다
border-box : 부모 안에 딱 맞습니다

* {
  box-sizing: border-box;
}

```

위 세 줄이 요즘 CSS 파일의 첫 규칙입니다. 한 번 깔아 두면 이 파일에서 본 사고가 통째로 사라집니다. 여러분이 만드는 페이지에는 이 규칙을 넣고 시작하세요.

▹ 직접 해보기

5

.bs-overflow 에 box-sizing: border-box; 한 줄을 추가해 보세요. 빨간 상자가 점선 안으로 들어옵니다. 확인했으면 그 줄을 지우세요.

자주 하는 실수

섹션 6

크기 문제는 화면이 조금씩 어긋나는 형태로 나타납니다. 에러 메시지는 없습니다.

이런 실수를 합니다

width 100% 에 padding 을 더함

이런 실수를 합니다

실수 1 — width: 100% 에 padding 을 더함

```

width: 100%;
padding: 20px;

```

상자가 부모 밖으로 빠져나옵니다. 화면 아래에 가로 스크롤바가 생기기도 합니다. 100% 는 콘텐츠 영역만의 100% 이고, padding 은 그 바깥에 따로 붙기 때문입니다. box-sizing: border-box; 한 줄이면 해결됩니다.

이렇게 쓰세요

```

box-sizing: border-box;
width: 100%;
padding: 20px;

```

이런 실수를 합니다

단위 빠뜨림

이런 실수를 합니다

실수 2 — 단위를 빠뜨림

```
width: 300;
```

폭이 전혀 안 줄어듭니다. 이 줄이 통째로 버려지고 width 는 auto 그대로 남습니다. 아래 상자가 진짜로 그렇게 써 본 결과입니다. 한 줄을 다 차지하고 있으니 “width 가 안 먹는다” 고 오해하기 쉽습니다. width: 300; 이라고 썼습니다

이런 실수를 합니다

height 를 % 로 줌

이런 실수를 합니다

실수 3 — height 를 % 로 줌

```
height: 50%;
```

아무 일도 안 일어납니다. 세로 비율은 부모의 세로 길이가 정해져 있어야 계산할 수 있는데, 부모의 height 가 auto 이면 기준이 없습니다. 기준이 없으면 브라우저는 그냥 auto 로 둡니다. 아래 상자가 그 결과입니다. 세로가 글자 높이 그대로입니다. height: 50% 라고 썼습니다

이런 실수를 합니다

max-width 와 width 를 헷갈림

이런 실수를 합니다

실수 4 — max-width 를 “언제나 이 크기” 로 읽음

```
max-width: 300px; /* 어떤 화면에서도 300px 일 거라고 기대 */
```

지금 화면에서는 정말 300px 이라 맞는 것처럼 보입니다. 그런데 부모가 300px 보다 좁아지면 상자도 같이 좁아집니다. max-width 는 “여기까지만” 이라는 한계선이지 “이 크기로 하라” 가 아니기 때문입니다. 어떤 화면에서도 정확히 300px 이어야 한다면 width: 300px; 을 쓰세요. 반대로 좁은 화면에서 같이 줄어드는 것이 목적이라면 max-width 가 맞습니다.

이런 실수를 합니다

box-sizing 을 한 요소에만 걸고 있음

이런 실수를 합니다

실수 5 — box-sizing 을 한 곳에만 걸어 둠

문제가 생긴 상자 하나에만 box-sizing: border-box; 를 붙이면 그 상자만 규칙이 다른 페이지가 됩니다. 어떤 상자는 340 이 되고 어떤 상자는 300 이 되니, 나중에 계산이 안 맞아 원인을 찾기 어렵습니다. 처음부터 * 규칙으로 전부 통일하세요.

이런 실수를 합니다

height 로 세로 여백을 만들려 함

이런 실수를 합니다

실수 6 — height 로 세로 여백을 만들려 함

```
height: 60px; /* 글자가 위쪽에 붙습니다 */
```

상자는 60px 이 되지만 글자는 맨 위에 붙어 있습니다. height 는 자리를 만들 뿐 내용을 가운데로 옮겨 주지 않습니다. 위아래로 숨통을 틔우고 싶으면 padding: 20px 0; 처럼 padding 을 쓰세요. 글자를 기준으로 위아래가 같이 벌어집니다.

정리

▫ 직접 해보기

6

정답

1) style 안의 .h-tight 를 이렇게 바꿉니다.

```
.h-tight {
  width: 400px;
  height: 60px;
  border: 1px solid #c0392b;
  background-color: #fbe0e0;
}
```

→ 27px 짜리 글자가 60px 짜리 상자 안에 들어가서 더 이상 빠져나오지 않습니다.

다만 글자는 상자 위쪽에 붙어 있습니다. height 는 자리를 만들 뿐 내용을 가운데로 옮겨 주지는 않기 때문입니다(섹션6 실수 6).

2) style 안의 .w-quarter 를 이렇게 바꿉니다.

```
.w-quarter {
  width: 75%;
  border: 1px solid #27853f;
  background-color: #d7f0d7;
}
```

→ 계산: 부모 안쪽 폭 820 의 75% 는 615px 입니다.

여기에 테두리 1px 이 좌우로 붙어 실제 가로는 617px 이 됩니다.
개발자도구로 재면 617 이 나옵니다. 되돌려 놓으세요.

3) style 안의 .w-max 를 이렇게 바꿉니다.

```
.w-max {
  width: 100%;
  max-width: 900px;
  border: 1px solid #9b7fd4;
  background-color: #f3dcf3;
}
```

→ 상자가 점선 안쪽을 꽉 채웁니다. 실제 가로는 822px 입니다.

한계선 900 이 부모 안쪽 폭 820 보다 크기 때문에 한계선이 아무 일도 하지 않고,
width: 100% 가 그대로 적용된 것입니다.
max-width 는 '넘지 마라' 일 뿐 '이만큼 되라' 가 아니라는 점을 확인하세요.

4) style 안의 .bs-content 의 padding 을 40px 로 바꿉니다.

→ 계산: $300 + 40 + 40 + 5 + 5 = 390\text{px}$ 입니다.

회색 자(340px)보다 50px 만큼 오른쪽으로 튀어나옵니다.
width 는 여전히 300 이라고 적혀 있는데 상자만 계속 커집니다.
이것이 content-box 의 문제입니다. 되돌려 놓으세요.

5) style 안의 .bs-overflow 를 이렇게 바꿉니다.

```
.bs-overflow {
  box-sizing: border-box;
  width: 100%;
  padding: 20px;
  border: 1px solid #c0392b;
  background-color: #fbe0e0;
}
```

→ 빨간 상자가 점선 밖으로 튀어나오지 않고 안쪽에 딱 맞게 들어옵니다.

가로가 862px 에서 820px 로 줄어듭니다.
padding 20 과 border 1 이 820 안에서 자리를 나눠 쓰게 되었기 때문입니다.
확인했으면 box-sizing 줄을 지워 원래대로 되돌리세요.

여러 섹션에서 쓰는 바깥 상자

```
.size-stage {
  background-color: #f4f4f4;
  border: 2px dashed #bbbbbb;
}
```

화면 연회색 바탕에 점선 테두리. 안쪽 가로가 820px 인 기준 상자입니다

width 와 height

섹션 1

```
.w-auto {
  border: 1px solid #2d6cdf;
  background-color: #cfe3ff;
}
.w-fixed {
  width: 400px;
  border: 1px solid #2d6cdf;
  background-color: #cfe3ff;
}
.wh-fixed {
  width: 400px;
  height: 80px;
  border: 1px solid #2d6cdf;
  background-color: #cfe3ff;
}
```

화면 가로 402px, 세로 82px 짜리 파란 상자가 됩니다

```
.h-tight {
  width: 400px;
  height: 16px;
  border: 1px solid #c0392b;
  background-color: #fbe0e0;
}
```

화면 상자가 글자보다 낮아서 글자가 상자 아래로 빠져나옵니다

```
.w-half {
  width: 50%;
  border: 1px solid #27853f;
  background-color: #d7f0d7;
}
.w-quarter {
  width: 25%;
  border: 1px solid #27853f;
  background-color: #d7f0d7;
}
.w-full {
  width: 100%;
  border: 1px solid #27853f;
  background-color: #d7f0d7;
}
```

max-width / min-width

```
.w-max {
  width: 100%;
  max-width: 300px;
  border: 1px solid #9b7fd4;
  background-color: #f3dcf3;
}
```

화면 100% 라고 썼지만 300px 을 넘지 못합니다

```
.w-min {
  width: 10%;
  min-width: 250px;
  border: 1px solid #9b7fd4;
  background-color: #f3dcf3;
}
```

화면 10% 는 82px 이지만 250px 아래로는 안 내려갑니다

width 300px 인데 340px 이 되는 문제

섹션 4

```
.bs-ruler {
  width: 340px;
  background-color: #dddddd;
}
```

화면 테두리가 없어서 실제 가로도 정확히 340px 인 회색 자입니다

```
.bs-content {
  width: 300px;
  padding: 15px;
  border: 5px solid #2d6cdf;
  background-color: #cfe3ff;
}
```

화면 width 를 300 이라고 썼는데 실제 가로는 340px 입니다

box-sizing: border-box

섹션 5

```
.bs-border {
  box-sizing: border-box;
  width: 300px;
  padding: 15px;
  border: 5px solid #27853f;
  background-color: #d7f0d7;
}
```

화면 같은 padding 과 border 를 줬는데 실제 가로가 정확히 300px 입니다

```
.bs-overflow {
  width: 100%;
  padding: 20px;
  border: 1px solid #c0392b;
  background-color: #fbe0e0;
}
```

화면 부모 밖으로 좌우가 튀어나옵니다

```
.bs-fixed {
  box-sizing: border-box;
  width: 100%;
  padding: 20px;
  border: 1px solid #27853f;
  background-color: #d7f0d7;
}
```

화면 부모 안에 딱 맞습니다

실수 데모

섹션 6

```
.bad-nounit {
  width: 300;
  border: 1px solid #c0392b;
  background-color: #fbe0e0;
}
```

화면 폭이 전혀 안 줄어듭니다. 이 줄이 버려지기 때문입니다

```
.bad-hpct {
  height: 50%;
  border: 1px solid #c0392b;
  background-color: #fbe0e0;
}
```

화면 세로 길이가 안 바뀝니다. 부모의 세로 길이가 정해져 있지 않기 때문입니다

배경과 넘침

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

같은 폴더에 있는 무늬.svg와 풍경.svg를 배경으로 씁니다.

인터넷 없이도 그대로 보입니다.



실행하면 나오는 화면 — 배경과 넘침

이 단원의 마지막 파일입니다. 두 가지를 다룹니다.

- 1 상자의 배경을 꾸미는 법 background 계열
- 2 상자에서 넘친 내용을 다루는 법 overflow

07단원에서 background-color로 배경색을 칠해 봤습니다. 배경에는 색뿐 아니라 그림도 깔 수 있습니다.

| | |
|---------------------|--------------|
| background-color | 배경색 |
| background-image | 배경 그림 |
| background-repeat | 그림을 반복할지 말지 |
| background-position | 그림을 어디에 둘지 |
| background-size | 그림을 얼마나 키울지 |
| background | 위 다섯 개를 한 줄로 |

뒷부분의 overflow 는 개념05 에서 미뤄 둔 숙제입니다. height 를 정해 놓았는데 내용이 그보다 길면 밖으로 빠져나왔습니다. 그 빠져나온 부분을 어떻게 할지 정하는 것이 overflow 입니다.

| | |
|---------|---------------------|
| visible | 그냥 밖으로 나오게 둔다 (기본값) |
| hidden | 잘라서 안 보이게 한다 |
| auto | 넘칠 때만 스크롤바를 붙인다 |
| scroll | 항상 스크롤바 자리를 잡아 둔다 |

배경과 넘침은 사실 서로 다른 주제입니다. 둘 다 '상자를 어떻게 보이게 할 것인가' 에 속해서 한 파일에 모았습니다.

이 파일은 배경 꾸미기와 넘친 내용 다루기 두 가지를 다룹니다. 배경 그림은 같은 폴더의 무늬.svg 와 풍경.svg 를 씁니다. 인터넷 주소를 쓰지 않았으므로 인터넷이 끊겨 있어도 그대로 보입니다.

배경이 칠해지는 범위

섹션 1

배경은 콘텐츠 + padding + 테두리 아래까지 깔립니다. margin 자리에는 안 깔립니다.

개념01 섹션4 에서 배경색으로 확인했던 내용입니다. 여기서 한 번 더 짚고 넘어가는 이유는, 배경 그림도 똑같은 범위에 깔리기 때문입니다.

| | |
|------------------|---------------|
| background-color | 이 범위에 색을 칠합니다 |
| background-image | 이 범위에 그림을 깎니다 |

아래 상자는 테두리를 굵은 점선으로 그었습니다. 점선의 굵긴 틈으로 연파랑 배경이 보입니다. 배경이 테두리 아래까지 깔려 있다는 뜻입니다.

색과 그림을 둘 다 주면 그림이 색 위에 올라갑니다. 그래서 그림이 상자를 다 못 채우면 빈 자리에 배경색이 보입니다. 섹션 5 의 단축 예에서 그 모습을 확인할 수 있습니다.

그림이 안 뜰 때를 대비해 배경색을 같이 주는 것은 좋은 습관입니다. 그림 파일 경로가 틀려도 최소한 글자는 읽히기 때문입니다.

화면 파란 굵은 점선 상자 안이 연파랑이고, 점선의 틈에도 연파랑이 보입니다

background-color 는 테두리 아래까지 깔립니다

▸ 직접 해보기 1

파일 위쪽 <style> 에서 .bgc-box 에 margin: 20px; 한 줄을 추가해 보세요. 새로 생긴 20px 자리에는 연파랑이 안 칠해지는 것을 확인하고 되돌리세요.

background-image 와 repeat

섹션 2

background-image: url("파일이름"); 으로 그림을 깎니다. 그림은 기본적으로 반복됩니다.

배경 그림은 이렇게 씁니다.

```
background-image: url("무늬.svg");
```

url() 안에는 그림 파일의 경로를 씁니다. 경로 쓰는 법은 04단원에서 img 태그로 배운 것과 똑같습니다.

| | |
|-------------------|---------------|
| url("무늬.svg") | 같은 폴더에 있는 파일 |
| url("이미지/무늬.svg") | 이미지 폴더 안의 파일 |
| url("../무늬.svg") | 한 단계 위 폴더의 파일 |

주의할 점이 하나 있습니다. CSS 파일 안에서 쓰는 경로는 'HTML 파일' 이 아니라 'CSS 파일' 기준입니다. 지금은 HTML 파일 안의 style 태그에 쓰고 있으므로 HTML 기준입니다.

무늬.svg 는 가로세로 40px 짜리 작은 그림입니다. 그런데 아래 첫 번째 상자는 그림이 깔리는 안쪽이 가로 822px, 세로 120px 입니다. 그림이 훨씬 작는데 어떻게 될까요?

브라우저는 그림을 바둑판처럼 반복해서 상자를 채웁니다.

이것이 background-repeat 의 기본값 repeat 입니다. 타일을 붙이듯 채우기 때문에 작은 그림 하나로 넓은 면을 꾸밀 수 있습니다.

반복을 조절하는 값은 네 가지입니다.

| | |
|-----------|------------------|
| repeat | 가로세로 모두 반복 (기본값) |
| no-repeat | 반복하지 않고 하나만 |
| repeat-x | 가로로만 반복 |
| repeat-y | 세로로만 반복 |

no-repeat 를 주면 그림이 왼쪽 위 구석에 하나만 놓입니다. 왜 왼쪽 위일까요? background-position 의 기본값이 왼쪽 위이기 때문입니다. 그 이야기는 다음 섹션에서 합니다.

화면 첫 상자는 물방울 무늬가 상자를 가득 채우고,

둘째 상자는 왼쪽 위에 물방울 하나만 있으며, 셋째 상자는 맨 윗줄에만 물방울이 가로로 늘어서 있습니다. 배경 그림은 태그와 역할이 다릅니다. 내용으로서 의미가 있는 사진은 로 넣고(04단원), 장식용 무늬나 배경은 CSS 의 background-image 로 넣습니다.

▣ 직접 해보기 2

.bgi-repeatx 의 background-repeat: repeat-x; 를 repeat-y; 로 바꿔 보세요. 무늬가 어느 방향으로 늘어서는지 확인하고 되돌리세요.

background-position

섹션 3

background-position 은 가로 위치와 세로 위치를 차례로 씁니다.

값은 두 개를 씁니다. 앞이 가로, 뒤가 세로입니다.

| | |
|---|----------------------|
| <code>background-position: right top;</code> | 오른쪽 · 위 |
| <code>background-position: center;</code> | 가운데 (가로세로 둘 다) |
| <code>background-position: 120px 40px;</code> | 왼쪽에서 120px, 위에서 40px |

말로 쓸 수 있는 값은 이렇습니다.

| | | | |
|------|------|--------|--------|
| 가로 : | left | center | right |
| 세로 : | top | center | bottom |

기본값은 left top 입니다. 그래서 앞 섹션의 no-repeat 상자에서 무늬가 왼쪽 위에 놓였던 것입니다.

숫자로 쓰면 왼쪽 위 구석에서 얼마나 떨어뜨릴지를 뜻합니다. px 뿐 아니라 % 도 쓸 수 있는데, % 는 계산 방식이 조금 특이해서 처음에는 말로 쓰는 값과 px 만 써도 충분합니다.

position 은 no-repeat 와 거의 항상 같이 씁니다. 반복해서 다 채워 버리면 위치를 정하는 의미가 없기 때문입니다.

아래 세 상자에서 무늬가 어디에 놓였는지 확인해 보세요.

화면 첫 상자는 오른쪽 위 구석,

둘째 상자는 정확히 한가운데, 셋째 상자는 왼쪽에서 조금 떨어진 위쪽에 물방울이 하나씩 있습니다

▣ 직접 해보기 3

.bgp-righttop 의 background-position: right top; 을 left bottom; 으로 바꿔 보세요. 무늬가 대각선 반대편으로 옮겨 갑니다.

background-size 와 cover / contain

섹션 4

cover 는 빈틈없이 채우기, contain 은 다 보이게 넣기입니다.

풍경.svg 는 가로 300px, 세로 200px 짜리 그림입니다. 아래 상자들은 그림이 깔리는 안쪽이 가로 822px, 세로 140px 입니다. 모양이 전혀 다릅니다. 이럴 때 그림을 어떻게 앞힐지 정하는 것이 background-size 입니다.

| | |
|-------------|--|
| (아무것도 안 씌) | 그림을 원래 크기 그대로 놓습니다. 300 x 200 입니다.
상자보다 세로가 커서 아래쪽이 잘립니다. |
| cover | 상자를 빈틈없이 채울 때까지 그림을 키웁니다.
가로세로 비율은 유지합니다. 대신 넘치는 쪽이 잘립니다. |
| contain | 그림 전체가 다 보이도록 상자 안에 넣습니다.
가로세로 비율은 유지합니다. 대신 빈 자리가 남습니다. |
| 150px 100px | 가로세로를 직접 지정합니다. 비율이 깨질 수 있습니다. |

cover 와 contain 은 이렇게 나눠 쓰면 됩니다.

사진을 배경으로 꽉 채우고 싶다 → cover (잘려도 괜찮은 경우)
로고나 도표가 다 보여야 한다 → contain (잘리면 안 되는 경우)

cover 를 쓸 때는 background-position 을 같이 주는 일이 많습니다. 어느 쪽이 잘릴지 정할 수 있기 때문입니다. 사람 얼굴이 위쪽에 있는 사진이라면 position 을 top 으로 두는 식입니다.

아래 네 상자를 비교해 보세요. 같은 그림인데 결과가 전부 다릅니다.

화면 첫 상자는 왼쪽에 그림이 원래 크기로 놓고 아래쪽 땅이 잘려 있습니다.

둘째 상자는 하늘이 전체를 덮고, 오른쪽에 커다란 해의 윗부분만 보입니다. 셋째 상자는 그림 전체가 세로에 맞춰 작게 들어가고 오른쪽이 비어 있습니다. 넷째 상자는 그림이 작게 줄어 왼쪽 위에 놓입니다

▹ 직접 해보기

4

.bgs-cover 에 background-position: center bottom; 한 줄을 추가해 보세요. 잘리는 부분이 위쪽으로 바뀌어 땅이 보이기 시작합니다.

background 단축

섹션 5

background 하나에 색 · 그림 · 반복 · 위치를 몰아서 쓸 수 있습니다.

네 줄을 한 줄로 줄일 수 있습니다.

```
background-color: #fff8dc;
background-image: url("무늬.svg");
background-repeat: no-repeat;
background-position: right center;
→ background: #fff8dc url("무늬.svg")
no-repeat right center;
```

padding 이나 border 와 같은 단축 속성입니다. 값의 순서는 자유롭지만, 보통 색 → 그림 → 반복 → 위치 순으로 씁니다.

여기에도 단축 속성의 함정이 그대로 있습니다.

```
background: #fff8dc;
background-image: url("무늬.svg");    이 순서는 괜찮습니다

background-image: url("무늬.svg");
background: #fff8dc;                  그림이 사라집니다
```

단축 background 는 자기가 안 적은 항목까지 전부 기본값으로 되돌립니다. 아래 줄의 background 에 그림이 안 적혀 있으니 그림이 없어지는 것입니다. padding, border 에서 본 것과 똑같은 규칙입니다.

단축을 먼저, 개별을 나중에.

아래 상자에는 배경색과 배경 그림을 한 줄로 같이 썼습니다. 무늬가 오른쪽 한가운데에만 놓이고, 나머지 자리는 배경색이 보입니다.

화면 연노랑 상자의 오른쪽 한가운데에 물방울 무늬 하나가 놓여 있습니다

```
background: #fff8dc url("무늬.svg") no-repeat right center;
```

◀ 직접 해보기 5

.bg-short 의 값에서 no-repeat 를 빼고 새로고침해 보세요. 무늬가 상자를 가득 채우고, 위치 지정이 아무 의미가 없어집니다.

overflow

섹션 6

상자보다 내용이 길 때 넘친 부분을 어떻게 할지 정합니다.

개념05 에서 height 를 정해 놓으면 내용이 밖으로 빠져나온다고 했습니다. 아래 네 상자는 전부 가로 300px, 세로 90px 로 똑같고 안에 든 글도 똑같습니다. overflow 값만 다릅니다.

| | |
|---------|--|
| visible | 기본값. 넘친 글이 상자 밖으로 그대로 흘러나옵니다.
상자 아래에 있던 다른 요소와 글자가 겹칠 수 있습니다. |
| hidden | 넘친 부분을 잘라 냅니다. 안 보이게 됩니다.
스크롤바가 없으므로 잘린 내용은 읽을 방법이 없습니다. |
| auto | 넘칠 때만 스크롤바가 생깁니다. 넘치지 않으면 안 생깁니다.
대부분의 경우 이것이 정답입니다. |
| scroll | 넘치든 안 넘치든 스크롤바 자리를 항상 잡아 둡니다.
내용이 바뀌어도 상자 크기가 안 흔들린다는 장점이 있습니다. |

네 번째 상자를 잘 보세요. 스크롤바가 두 개(가로·세로) 다 있습니다. scroll 은 안 넘치는 방향에도 자리를 잡기 때문입니다.

스크롤바가 생겨도 상자 자체의 크기는 안 바뀝니다. 네 상자 모두 가로 320px, 세로 110px 로 똑같습니다. 스크롤바는 상자 안쪽에 자리를 잡으므로 글이 들어갈 폭만 조금 줄어듭니다. (스크롤바의 굵기와 생김새는 브라우저와 운영체제에 따라 다릅니다)

overflow 는 가로세로를 따로 정할 수도 있습니다.

```
overflow-x: hidden;   가로로 넘치는 것만 자르기
overflow-y: auto;     세로로 넘칠 때만 스크롤바
```

페이지 전체에 가로 스크롤바가 생겼을 때 overflow-x: hidden 으로 덮어 버리고 싶어질 때가 있습니다. 그것은 원인을 가리는 것일 뿐 고치는 것이 아닙니다. 대개는 개념05 에서 본 width 100% + padding 문제가 진짜 원인입니다.

화면 회색 상자 아래로 글자 세 줄 정도가 테두리를 뚫고 흘러나와 있습니다

이 상자의 세로 길이는 90px 입니다. 그런데 안에 든 글이 그보다 훨씬 깁니다. 남는 부분이 어떻게 되는지 보세요. overflow 속성이 그것을 정합니다. 지금은 아무것도 안 정했으므로 기본값 visible 이 쓰이고 있습니다.

화면 글이 상자 아래 테두리에서 뚫 끊깁니다. 스크롤바는 없습니다

이 상자의 세로 길이는 90px 입니다. 그런데 안에 든 글이 그보다 훨씬 깁니다. 남는 부분이 어떻게 되는지 보세요. overflow 속성이 그것을 정합니다. 지금은 hidden 이라서 넘친 글이 잘려 나갑니다.

화면 상자 오른쪽에 세로 스크롤바가 하나 생기고, 끌어내리면 나머지 글이 보입니다

이 상자의 세로 길이는 90px 입니다. 그런데 안에 든 글이 그보다 훨씬 깁니다. 남는 부분이 어떻게 되는지 보세요. overflow 속성이 그것을 정합니다. 지금은 auto 라서 오른쪽에 세로 스크롤바가 생겼습니다.

화면 오른쪽에 세로 스크롤바, 아래쪽에 가로 스크롤바가 둘 다 보입니다.

가로로는 넘친 것이 없는데도 자리가 잡혀 있습니다

이 상자의 세로 길이는 90px 입니다. 그런데 안에 든 글이 그보다 훨씬 깁니다. 남는 부분이 어떻게 되는지 보세요. overflow 속성이 그것을 정합니다. 지금은 scroll 이라서 가로 스크롤바 자리까지 잡혀 있습니다.

헛갈리면 auto 를 쓰세요. 필요할 때만 스크롤바를 붙여 주므로 대부분의 경우에 맞습니다. hidden 은 잘린 내용을 읽을 방법이 없어지므로, 정말 안 보여도 되는 내용에만 쓰세요.

.ov-hidden 의 height: 90px 을 지우려면 어디를 고쳐야 할까요? 위쪽 <style> 에서 네 상자가 공유하는 규칙을 찾아 height 줄을 잠깐 지워 보고, 네 상자가 어떻게 변하는지 확인한 뒤 되돌리세요.

자주 하는 실수

섹션 7

배경 그림은 안 보여도 에러가 안 납니다. 그냥 빈 상자가 보일 뿐이라 원인을 찾기 어렵습니다.

이런 실수를 합니다

경로가 틀림

이런 실수를 합니다

실수 1 — 그림 경로가 틀림

```
background-image: url("없는그림.svg");
```

아무 그림도 안 보입니다. 화면에는 배경색만 남습니다. 깨진 그림 아이콘조차 안 나옵니다. 태그와 다른 점입니다. 아래 상자가 그때의 모습입니다. 배경색만 있고 그림 자리는 그냥 비어 있습니다. 여러분 파일에서 진짜로 경로가 틀리면 F12 개발자도구의 Console 탭에 그 파일을 못 찾았다는 빨간 줄이 뜹니다. 배경이 안 보이면 Console 부터 확인하세요. (아래 상자는 모습만 흉내 낸 것이라 이 페이지의 Console 에는 아무것도 안 뜹니다)

이런 실수를 합니다

height 를 안 줌

이런 실수를 합니다

실수 2 — 빈 상자에 배경만 주고 height 를 안 줌

```
<div class="hero"></div>

.hero {
  background-image: url("풍경.svg");
}
```

아무것도 안 보입니다. 안에 내용이 없으면 상자의 세로 길이가 0 이 되고, 세로가 0 인 상자에는 배경을 칠할 자리가 없기 때문입니다. 빈 상자에 배경 그림을 깔 때는 height 를 꼭 같이 주세요.

이런 실수를 합니다

단축을 나중에 써서 그림이 사라짐

이런 실수를 합니다

실수 3 — 단축 background 를 나중에 씀

```
background-image: url("무늬.svg");
background: #fff8dc;
```

배경 그림이 조용히 사라집니다. 단축 background 는 자기가 안 적은 항목까지 전부 기본값으로 되돌리기 때문입니다. padding, border 와 똑같은 규칙입니다. 단축을 먼저, 개별을 나중에 쓰세요.

이런 실수를 합니다

cover 와 contain 을 바꿔 씀

이런 실수를 합니다

실수 4 — cover 와 contain 을 바꿔 씀

로고에 cover 를 쓰면 로고 가장자리가 잘려 나갑니다. 배경 사진에 contain 을 쓰면 옆에 빈 자리가 생겨 어색해집니다. 잘려도 되면 cover, 다 보여야 하면 contain 입니다.

이런 실수를 합니다

넘치는 것을 overflow: hidden 으로 덮음

이런 실수를 합니다

실수 5 — 넘치는 원인을 안 고치고 hidden 으로 덮음

```
overflow-x: hidden; /* 가로 스크롤바만 안 보이게 */
```

가로 스크롤바는 사라지지만 내용은 여전히 밖에 있습니다. 잘려서 안 보일 뿐입니다. 화면이 좁아지면 글자가 통째로 안 읽히기도 합니다. 진짜 원인은 대개 개념05 의 width: 100% + padding 입니다. box-sizing: border-box 로 원인을 고치세요.

이런 실수를 합니다

url 안에 따옴표와 괄호를 잘못 씀

이런 실수를 합니다

실수 6 — url 을 문자열처럼만 씀

```
background-image: "무늬.svg";
```

url() 을 빼먹었습니다. 이 줄이 통째로 버려집니다. 배경이 아예 안 나옵니다. 파일 이름만 적는 것이 아니라 url("무늬.svg") 형태로 감싸야 합니다. 따옴표는 없어도 되지만 파일 이름에 공백이나 한글이 있으면 붙이는 편이 안전합니다.

정답

1) style 안의 .bgc-box 를 이렇게 바꿉니다.

```
.bgc-box {  
  background-color: #dbe7fb;  
  margin: 20px;  
  padding: 20px;  
  border: 10px dashed #2d6cdf;  
}
```

→ 상자가 사방으로 20px 씩 안쪽으로 물러납니다.

새로 생긴 20px 자리는 흰색입니다. 연파랑이 안 칠해집니다.
margin 은 배경이 안 깔린다는 것을 다시 확인한 것입니다. 되돌리세요.

2) style 안의 .bgi-repeatx 를 이렇게 바꿉니다.

```
.bgi-repeatx {  
  background-image: url("무늬.svg");  
  background-repeat: repeat-y;  
  height: 120px;  
  border: 1px solid #999999;  
}
```

→ 무늬가 가로 한 줄이 아니라 왼쪽 세로 한 줄로 늘어섭니다.

x 는 가로축, y 는 세로축입니다. 되돌려 놓으세요.

3) style 안의 .bgp-righttop 의 position 을 이렇게 바꿉니다.

```
background-position: left bottom;
```

→ 무늬가 오른쪽 위에서 왼쪽 아래로, 대각선 반대편으로 옮겨 갑니다.

앞 값이 가로, 뒤 값이 세로라는 것을 확인하세요.

4) style 안의 .bgs-cover 에 한 줄을 추가합니다.

```
.bgs-cover {
  background-image: url("풍경.svg");
  background-repeat: no-repeat;
  background-size: cover;
  background-position: center bottom;
  height: 140px;
  border: 1px solid #999999;
}
```

→ 잘리는 부분이 아래쪽에서 위쪽으로 바뀝니다.

초록색 땅이 보이기 시작하고 하늘이 덜 보입니다.
cover 는 반드시 어딘가를 자르므로, 어디를 자를지 position 으로 고릅니다.

5) style 안의 .bg-short 를 이렇게 바꿉니다.

```
.bg-short {
  background: #fff8dc url("무늬.svg") right center;
  height: 110px;
  padding: 16px;
  border: 1px solid #999999;
}
```

→ 무늬가 상자를 가득 채웁니다. 배경색은 무늬에 완전히 가려 안 보입니다.

반복이 커지면 상자가 그림으로 다 덮이므로 위치 지정이 의미가 없어집니다.
되돌려 놓으세요.

6) style 안에서 네 상자가 공유하는 규칙은 이것입니다.

```
.ov-visible,
.ov-hidden,
.ov-auto,
.ov-scroll {
  width: 300px;
  height: 90px;
  ...
}
```

여기서 height: 90px; 줄을 잠깐 지웁니다.

→ 네 상자 모두 글 길이에 맞게 세로로 늘어나고, 넘치는 것이 없어집니다.

그래서 visible 과 hidden 은 아무 차이가 없어 보이고,
auto 의 스크롤바도 사라집니다. scroll 만 스크롤바 자리를 계속 잡고 있습니다.
넘치는 것이 있어야 overflow 가 일을 한다는 뜻입니다. 되돌려 놓으세요.

background-color 가 칠해지는 범위

섹션 1

```
.bgc-box {  
  background-color: #dbe7fb;  
  padding: 20px;  
  border: 10px dashed #2d6cdf;  
}
```

화면 점선의 틈 사이로 연파랑이 비칩니다. 배경이 테두리 아래까지 깔려 있습니다

background-image 와 repeat

섹션 2

```
.bgi-repeat {  
  background-image: url("무늬.svg");  
  height: 120px;  
  border: 1px solid #999999;  
}
```

화면 40px 짜리 물방울 무늬가 상자 전체에 바둑판처럼 반복됩니다

```
.bgi-norepeat {  
  background-image: url("무늬.svg");  
  background-repeat: no-repeat;  
  height: 120px;  
  border: 1px solid #999999;  
}
```

화면 왼쪽 위 구석에 물방울 하나만 보입니다

```
.bgi-repeatx {  
  background-image: url("무늬.svg");  
  background-repeat: repeat-x;  
  height: 120px;  
  border: 1px solid #999999;  
}
```

화면 맨 윗줄에만 가로로 한 줄 반복됩니다

background-position

섹션 3

```
.bgp-righttop {  
  background-image: url("무늬.svg");  
}
```



```
background-repeat: no-repeat;
    background-position: right top;
    height: 110px;
    border: 1px solid #999999;
}
.bgp-center {
    background-image: url("무늬.svg");
    background-repeat: no-repeat;
    background-position: center;
    height: 110px;
    border: 1px solid #999999;
}
.bgp-px {
    background-image: url("무늬.svg");
    background-repeat: no-repeat;
    background-position: 120px 40px;
    height: 110px;
    border: 1px solid #999999;
}
```

background-size

섹션 4

```
.bgs-auto {
    background-image: url("풍경.svg");
    background-repeat: no-repeat;
    height: 140px;
    border: 1px solid #999999;
}
```

화면 그림이 원래 크기(300 x 200)로 깔려서 아래쪽이 잘립니다

```
.bgs-cover {
    background-image: url("풍경.svg");
    background-repeat: no-repeat;
    background-size: cover;
    height: 140px;
    border: 1px solid #999999;
}
```

화면 하늘이 상자를 빈틈없이 덮고 해의 윗부분만 크게 보입니다.

그림이 확대되어 아래쪽 산과 땅은 잘려 나갑니다

```
.bgs-contain {
  background-image: url("풍경.svg");
  background-repeat: no-repeat;
  background-size: contain;
  height: 140px;
  border: 1px solid #999999;
}
```

화면 그림 전체가 다 보입니다. 대신 오른쪽에 빈 자리가 남습니다

```
.bgs-px {
  background-image: url("풍경.svg");
  background-repeat: no-repeat;
  background-size: 150px 100px;
  height: 140px;
  border: 1px solid #999999;
}
```

background 단축

섹션 5

```
.bg-short {
  background: #fff8dc url("무늬.svg") no-repeat right center;
  height: 110px;
  padding: 16px;
  border: 1px solid #999999;
}
```

화면 연노랑 바탕 오른쪽 한가운데에 물방울 하나가 놓입니다

overflow

섹션 6

```
.ov-visible,
.ov-hidden,
.ov-auto,
.ov-scroll {
  width: 300px;
  height: 90px;
  padding: 8px;
  border: 2px solid #888888;
  background-color: #f4f4f4;
}
.ov-visible {
  overflow: visible;
```

```
margin-bottom: 80px;
}
```

화면 넘친 글자가 상자 밖 아래로 그대로 흘러나옵니다

```
.ov-hidden {
  overflow: hidden;
}
```

화면 넘친 부분이 잘려 안 보입니다. 스크롤바도 없습니다

```
.ov-auto {
  overflow: auto;
}
```

화면 오른쪽에 세로 스크롤바가 생깁니다

```
.ov-scroll {
  overflow: scroll;
}
```

화면 넘치지 않아도 가로세로 스크롤바 자리가 항상 잡혀 있습니다

```
.ov-text {
  margin: 0;
}
```

실수 데모

섹션 7

```
.bad-url {
  background-color: #fbe0e0;
  height: 60px;
  border: 1px solid #c0392b;
}
```

화면 그림이 안 뜬 상자가 어떻게 보이는지 훑내 낸 상자입니다.

배경색만 남고 아무 그림도 없습니다

박스 모델

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

VS Code 로도 열어 두고, 위쪽 <style> 안의 TODO 자리에 코드를 씁니다.

저장(Ctrl+S) 하고 브라우저에서 새로고침(F5) 하면 결과가 바뀝니다.

F12 개발자도구로 크기를 재 가며 푸세요.



실행하면 나오는 화면 — 박스 모델

1~10 은 기본, 11~13 은 응용, 14~15 는 도전, 16 은 고장 난 코드 고치기입니다. 각 문제 위의 주석에 무엇을 만들어야 하는지와 제대로 됐을 때 화면이 어떻게 보여야 하는지가 적혀 있습니다.

위쪽 <style> 안의 각 문제 자리에 코드를 쓰세요. 문제 설명은 이 파일을 VS Code 로 열면 각 상자 위의 주석에 있습니다. 1~10 기본 · 11~13 응용 · 14~15 도전 · 16 고장 난 코드 고치기 순서입니다. 크기를 확인할 때는 F12 개발자도구를 쓰세요.

문제 1 (개념02 섹션1)

#q1 상자에 네 방향 모두 20px 짜리 안쪽 여백을 주세요.

기대 출력 글자가 상자의 네 벽에서 20px 씩 떨어지고,
상자의 세로 길이가 27px 에서 67px 로 두꺼워집니다.
이렇게 되면 틀린 것: 아무 변화가 없으면 단위(px) 를 빠뜨린 것입니다.

안쪽 여백을 사방 20px 주세요

문제 2 (개념02 섹션3)

#q2 상자에 위아래 10px, 좌우 40px 짜리 안쪽 여백을 주세요. 값을 두 개만 써서 해결하세요.

기대 출력 위아래는 살짝, 좌우는 넉넉하게 여백이 생깁니다.
글자가 왼쪽 벽에서 40px 떨어진 곳에서 시작합니다.
이렇게 되면 틀린 것: 왼쪽 여백이 10px 이면 값의 순서를 반대로 쓴 것입니다.

위아래 10px, 좌우 40px

문제 3 (개념02 섹션4)

#q3 상자에 왼쪽에만 60px 짜리 안쪽 여백을 주세요. 다른 세 방향은 0 이어야 합니다.

기대 출력 글자가 오른쪽으로 밀려 시작하고, 위아래에는 여백이 없습니다.
이렇게 되면 틀린 것: 위아래도 두꺼워졌다면 padding 단축을 쓴 것입니다.
방향 이름이 붙은 속성을 쓰세요.

왼쪽에만 60px

문제 4 (개념03 섹션2)

#q4 상자에 3px 두께의 빨간색(#c0392b) 실선 테두리를 두르세요. 단축 속성 한 줄로 쓰세요.

기대 출력 흰 상자 둘레에 빨간 실선이 그어집니다.
이렇게 되면 틀린 것: 테두리가 아예 안 보이면 style 값(solid) 을 빠뜨린 것입니다.

3px 빨간 실선 테두리

문제 5 (개념03 섹션3)

#q5 상자에 4px 두께의 파란색(#2d6cdf) 점선 테두리를 두르세요. 점선은 짧은 막대가 이어진 모양입니다.

기대 출력 흰 상자 둘레에 파란 막대 점선이 그어집니다.
이렇게 되면 틀린 것: 동그란 점이 이어진 모양이면 dotted 를 쓴 것입니다.

4px 파란 막대 점선 테두리

문제 6 (개념03 섹션4)

#q6 상자에 아래쪽에만 5px 두께의 초록색(#27853f) 실선을 그으세요. 나머지 세 방향에는 테두리가 없어야 합니다.

기대 출력 글자 아래에 초록색 굵은 선이 한 줄만 그어집니다.
이렇게 되면 틀린 것: 네 방향이 다 그어졌으면 border 단축을 쓴 것입니다.

아래쪽에만 초록 실선

문제 7 (개념03 섹션5)

#q7 상자의 네 모서리를 12px 만큼 둥글게 깎으세요.

기대 출력 보라 테두리 상자의 네 귀퉁이가 부드럽게 둥글어집니다.
이렇게 되면 틀린 것: 완전히 동그래졌다면 50% 를 쓴 것입니다. 12px 입니다.

모서리를 12px 만큼 둥글게

문제 8 (개념04 섹션2)

#q8 상자에 위아래 30px, 좌우 0 짜리 바깥 여백을 주세요. 값을 두 개만 써서 해결하세요.

기대 출력 노란 상자가 점선 상자의 위아래 벽에서 30px 씩 떨어집니다.
좌우로는 점선에 딱 붙어 있습니다.
이렇게 되면 틀린 것: 좌우도 떨어졌다면 값을 한 개만 쓴 것입니다.
노란 면적이 넓어졌다면 padding 을 쓴 것입니다.

위아래 30px, 좌우 0

문제 9 (개념04 섹션3)

#q9 상자를 가로 320px 로 만들고 좌우 한가운데에 놓으세요. 위아래 바깥 여백은 0 입니다.

기대 출력 주황 상자가 점선 상자의 한가운데에 놓입니다.
 왼쪽과 오른쪽에 남는 흰 자리의 폭이 똑같습니다.
 이렇게 되면 틀린 것: 글자만 가운데로 갔다면 text-align 을 쓴 것입니다.
 상자가 왼쪽에 그대로 있다면 width 를 안 준 것입니다.

가로 320px, 가운데로

문제 10 (개념05 섹션1)

#q10 상자의 가로를 250px, 세로를 100px 로 만드세요.

기대 출력 초록 상자가 왼쪽에 네모나게 자리 잡습니다.
 글자는 상자 위쪽에 붙어 있습니다.
 이렇게 되면 틀린 것: 상자가 한 줄을 다 차지한다면 단위를 빠뜨린 것입니다.

가로 250px, 세로 100px

문제 11 [응용] (개념05 섹션3)

#q11 상자가 부모 폭을 꽉 채우되, 400px 보다는 넓어지지 않게 하세요. 한 줄이면 됩니다. (뜻을 더 분명히 하려고 두 줄로 쓰기도 합니다. 정답에서 함께 설명합니다)

기대 출력 보라 상자의 가로가 정확히 400px 이 됩니다.
 브라우저 창을 아주 좁게 줄이면 상자도 같이 좁아집니다.
 이렇게 되면 틀린 것: 상자가 점선 안쪽을 다 차지한다면 한계선을 안 준 것이고,
 창을 좁혀도 400px 그대로라면 폭을 px 로 못 박은 것입니다.

꽉 채우되 400px 까지만

문제 12 [응용] (개념05 섹션5)

#q12 상자에는 이미 padding: 20px 과 border: 5px 이 주어져 있습니다. 여기에 코드를 더해서 상자의 실제 가로 길이가 정확히 300px 이 되게 하세요. width 값도 300px 이라고 쓸 수 있어야 합니다.

힌트: 아무 생각 없이 width: 300px 만 쓰면 실제로는 350px 이 됩니다.

$300 + 20 + 20 + 5 + 5 = 350$ 이기 때문입니다.

기대 출력 초록 상자의 실제 가로가 개발자도구에서 300 으로 나옵니다.
이렇게 되면 틀린 것: 350 이 나오면 box-sizing 을 안 쓴 것입니다.

실제 가로를 300px 로

문제 13 [응용] (개념06 섹션2·3)

#q13 상자의 배경에 같은 폴더의 무늬.svg 를 깔되, 반복하지 말고 오른쪽 위 구석에 하나만 놓으세요. 속성 세 개를 씁니다.

기대 출력 연노랑 상자의 오른쪽 위 구석에 물방울 무늬가 하나만 보입니다.
이렇게 되면 틀린 것: 무늬가 상자를 가득 채웠다면 반복을 안 끈 것입니다.
아무것도 안 보이면 url() 을 빠뜨렸거나 파일 이름이 틀린 것입니다.

문제 14 [도전] (개념06 섹션6)

#q14 상자의 세로를 90px 로 고정하고, 내용이 넘칠 때만 스크롤바가 생기도록 만드세요.

기대 출력 회색 상자의 세로가 짧아지고, 오른쪽에 세로 스크롤바가 생깁니다.
스크롤바를 끌어내리면 나머지 글이 보입니다.
이렇게 되면 틀린 것: 글이 상자 밖으로 흘러나오면 overflow 를 안 준 것입니다.
글이 잘리고 스크롤바가 없으면 hidden 을 쓴 것입니다.
가로 스크롤바까지 생겼다면 scroll 을 쓴 것입니다.

이 상자의 세로를 90px 로 고정하면 이 글이 상자보다 길어집니다.
넘친 부분을 어떻게 할지 정하는 속성이 무엇이었는지 떠올려 보세요.
넘칠 때만 스크롤바가 생기는 값을 골라야 합니다.

문제 15 [도전] (개념02~06 종합)

#q15 를 아래 조건을 모두 만족하는 카드로 만드세요.

- 실제 가로 길이가 정확히 360px - 좌우 한가운데에 놓임 - 안쪽 여백은 네 방향 20px - 테두리는 1px 짜리 회색(#cccccc) 실선 - 모서리는 8px 만큼 둥글게 - 배경색은 흰색(#ffffff)

기대 출력 점선 상자 한가운데에 흰 카드가 놓입니다.
모서리가 둥글고 얇은 회색 테두리가 있으며,
글자는 카드 벽에서 20px 떨어져 있습니다.
개발자도구로 재면 가로가 정확히 360 입니다.
이렇게 되면 틀린 것: 가로가 402 로 나오면 box-sizing 을 안 쓴 것입니다.
카드가 왼쪽에 붙어 있으면 margin 의 좌우가 auto 가 아닙니다.

가운데 놓인 360px 짜리 카드

문제 16 [확인] (개념02·03·04)

위쪽 <style> 의 #q16 규칙에는 잘못된 곳이 세 군데 있습니다. 지금 화면을 보면 안쪽 여백도 없고, 테두리도 안 보이고, 상자도 가운데로 안 가 있습니다. 그런데 에러 메시지는 어디에도 없습니다.

세 곳을 찾아 고치세요. 목표는 이렇습니다.

- 네 방향 24px 짜리 안쪽 여백 - 4px 두께의 빨간색(#c0392b) 실선 테두리 - 가로 400px 짜리 상자가 좌우 한가운데에 놓임

힌트: “아무 일도 안 일어난다” 는 세 가지 원인이 각각 다릅니다.

하나는 단위, 하나는 빠진 값, 하나는 빠진 속성입니다.

기대 출력 연노랑 상자가 가운데에 놓이고, 빨간 실선 테두리가 둘러지며,
글자가 상자 벽에서 24px 떨어집니다.
이렇게 되면 틀린 것: 여백이 안 생기면 숫자에 단위(px) 가 빠진 것입니다.
테두리가 안 보이면 굵기·색만 있고 선 종류(solid) 가 빠진 것입니다.
테두리는 보이는데 상자가 왼쪽에 붙어 있으면 width 를 아직 안 준 것입니다.
(margin: 0 auto 는 width 가 있어야 가운데로 갑니다)

세 군데를 고쳐 주세요

정리

아래 각 문제의 TODO 자리에 코드를 쓰세요.
저장한 뒤 브라우저에서 F5 를 누르면 결과가 바뀝니다.

문제 8·9·15 에서 기준선 역할을 하는 바깥 상자입니다. 고치지 마세요.

```
.q-stage {
  border: 2px dashed #bbbbbb;
}
```

문제 1

#q1 {
background-color: #dbe7fb;

여기에 코드를 쓰세요

}

문제 2

#q2 {
background-color: #dbe7fb;

여기에 코드를 쓰세요

}

문제 3

#q3 {
background-color: #dbe7fb;

여기에 코드를 쓰세요

}

문제 4

```
#q4 {
  background-color: #ffffff;
  padding: 10px;
```

여기에 코드를 쓰세요

}

문제 5

```
#q5 {
  background-color: #ffffff;
  padding: 10px;
```

여기에 코드를 쓰세요

}

문제 6

```
#q6 {
  background-color: #ffffff;
  padding: 8px 0;
```

여기에 코드를 쓰세요

```
}
```

문제 7

```
#q7 {  
  background-color: #f3ecff;  
  border: 2px solid #6b3fc0;  
  padding: 12px;
```

여기에 코드를 쓰세요

```
}
```

문제 8

```
#q8 {  
  background-color: #ffe9a8;
```

여기에 코드를 쓰세요

```
}
```

문제 9

```
#q9 {
  background-color: #ffd9a8;
```

여기에 코드를 쓰세요

```
}
```

문제 10

```
#q10 {
  background-color: #d7f0d7;
```

여기에 코드를 쓰세요

```
}
```

문제 11

```
#q11 {
  background-color: #f3dcf3;
```

여기에 코드를 쓰세요

```
}
```

문제 12

```
#q12 {  
  background-color: #d7f0d7;  
  padding: 20px;  
  border: 5px solid #27853f;
```

여기에 코드를 쓰세요

```
}
```

문제 13

```
#q13 {  
  height: 120px;  
  border: 1px solid #999999;  
  background-color: #fff8dc;
```

여기에 코드를 쓰세요

```
}
```

문제 14

```
#q14 {
  width: 300px;
  padding: 8px;
  border: 2px solid #888888;
  background-color: #f4f4f4;
```

여기에 코드를 쓰세요

```
}
.q14-text {
  margin: 0;
}
```

문제 15

```
#q15 {
```

여기에 코드를 쓰세요

```
}
```

문제 16

아래 규칙에는 잘못된 곳이 세 군데 있습니다. 찾아서 고치세요.

```
#q16 {  
  background-color: #fff8dc;  
  padding: 24;  
  border-width: 4px;  
  border-color: #c0392b;  
  margin: auto;  
}
```

10단원 마무리 박스모델

자주 넘어지는 자리

- 1 **F12 → Computed** 의 박스 모델 그림. 이 단원은 이것 없이는 안 됩니다
- 2 `margin` 겹침 — 40 과 30 을 주면 70 이 아니라 **40** 이 되는 것

부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

06단원 연습문제 정답 시맨틱 구조

보는 법

- 1) 먼저 연습문제.html 을 끝까지 풀어 보세요.
- 2) 그 다음에 이 파일을 열어 코드를 나란히 대조하세요.



실행하면 나오는 화면 — 시맨틱 구조

★ 이 단원의 정답 확인법은 다른 단원과 다릅니다.

06단원 태그는 화면을 바꾸지 않습니다. 그래서 “화면이 같으니 맞았다” 라고 볼 수 없습니다. div 로 써도 화면은 똑같이 나오니까요.

반드시 코드를 대조하세요. 볼 곳은 두 군데입니다.

- 1 태그 이름을 알맞게 골랐는가
- 2 어느 태그가 어느 태그 안에 들어 있는가 (중첩 관계)

★ 이 파일에도 main 은 하나뿐입니다.

문제 14 의 답에만 살아 있는 main 이 들어 있습니다. 문제 15 의 답은 코드 글자로만 적었습니다. 정답 파일이 스스로 규칙을 어기면 안 되기 때문입니다.

각 문제의 답과 왜 그 태그를 골랐는지가 함께 적혀 있습니다. 화면 결과는 대부분 맛있습니다. 코드를 대조하는 것이 채점입니다. 태그 이름이 달라도 화면은 같게 나온다는 점을 계속 떠올리세요.

문제 1 (개념01 섹션1)

제목 한 줄과 문단 두 줄을 div 하나로 묶으세요.

문제 1 — 세 줄을 div 로 묶기

제목 한 줄과 문단 두 줄을 <div> 하나로 묶으세요.

화면 굵고 조금 큰 글씨 "동네 축구 모임" 아래에

문단 두 줄이 차례로 보입니다. 모두 세 줄입니다

동네 축구 모임
매주 토요일 아침 8시에 모입니다.
초보자도 환영합니다.

<div> 를 씌워도 세 줄은 그대로입니다. div 는 묶기만 할 뿐 화면을 안 바꿉니다. 제목이 굵고 큰 것은 <h3> 덕분이지 div 덕분이 아닙니다.

문제 2 (개념01 섹션3)

문장에서 가격 부분만 span 으로 묶고 class 이름을 price 로 붙이세요.

문제 2 — 문장 속 값을 span 으로 묶기

가격 부분만 로 묶으세요.

화면 "오늘의 국수는 7,000원입니다." 가 한 줄로 보입니다.

7,000원 도 나머지와 똑같은 보통 글씨입니다

오늘의 국수는 7,000원입니다.

 은 인라인이라 줄이 안 나뉩니다. 여기에 <div> 를 썼다면 줄이 세 줄로 쪼개졌을 것입니다. class="price" 는 08단원에서 이 숫자만 골라 꾸밀 때 쓸 손잡이입니다.

문제 3 (개념02 섹션2)

같은 내용을 div 판과 section 판으로 두 번 쓰세요.

문제 3 — div 판과 section 판을 같이 써 보기

같은 내용을 <div> 와 <section> 으로 각각 감싸고 결과가 같은지 확인하세요.

화면 굵은 소제목과 문단으로 된 덩어리가 두 벌 이어서 보입니다.

두 덩어리는 글자 크기도 줄 간격도 완전히 같습니다

여는 시간
평일은 9시부터 18시까지입니다.

여는 시간
평일은 9시부터 18시까지입니다.

두 덩어리의 높이가 정확히 같습니다. 여기서는 “여는 시간” 이라는 제목이 있으니 <section> 쪽이 더 정확한 답입니다. 제목을 붙일 수 없는 묶음이었다면 <div> 가 맞습니다.

문제 4 (개념03 섹션2)

가게 사이트의 머리말을 header 로 만드세요.

문제 4 — header 만들기

가게 이름과 한 줄 소개를 <header> 로 묶으세요.

화면 굵고 조금 큰 글씨 “골목 분식” 아래에

보통 글씨로 “1998년부터 한자리에서” 한 줄이 보입니다

골목 분식
1998년부터 한자리에서

<header> 는 글자를 하나도 안 키웁니다. 큰 글씨는 <h3> 가 만든 것입니다. header 가 한 일은 “여기부터 여기까지가 이 사이트의 머리말” 이라고 적어 둔 것뿐입니다.

문제 5 (개념03 섹션3)

nav 안에 ul, li, a 로 메뉴 세 개를 만드세요.

문제 5 — nav 로 메뉴 만들기

<nav> 안에 , , <a> 로 메뉴 세 개를 만드세요.

화면 점(•) 이 붙은 세 줄이 보이고 글자는 파란색 밑줄 링크입니다

메뉴판
오시는 길
사진

메뉴는 실제로 항목이 여러 개인 목록이라 을 씁니다. 그래야 화면 낭독기가 “목록, 항목 3개” 라고 미리 알려 줍니다. 메뉴가 세로로 쌓여 있는 것이 정상입니다. 가로로 눕히는 것은 12단원에서 합니다.

문제 6 (개념03 섹션5)

주소와 저작권 표시를 footer 로 묶으세요.

문제 6 — footer 만들기

주소와 저작권 표시를 <footer> 로 묶으세요.

화면 보통 글씨 두 줄이 보이고, 둘째 줄은 동그라미 안에 c 가 든

기호로 시작해서 “2026 골목 분식” 으로 이어집니다

골목 분식 · 서울시 어딘가 5길 12
© 2026 골목 분식

저작권 기호는 02단원에서 배운 특수문자 표기로 씁니다. 이름을 잘못 적으면 브라우저가 고쳐 주지 않고 글자를 그대로 그림니다. <footer> 는 화면 맨 아래로 내려가지 않고 적힌 자리에 놓입니다.

문제 7 (개념04 섹션3)

제목이 있는 section 을 만드세요.

문제 7 — 제목이 있는 section 만들기

<section> 을 쓰고 제목을 반드시 같이 넣으세요.

화면 굵고 조금 큰 글씨 "주차 안내" 아래에 문단 한 줄이 보입니다

주차 안내
가게 뒤편에 세 자리가 있습니다.

<section> 은 주제로 묶은 덩어리이므로 그 주제를 적은 제목이 함께 있어야 합니다. 제목이 떠오르지 않는 묶음이라면 그건 section 이 아니라 <div> 입니다.

문제 8 (개념04 섹션2)

후기 하나를 article 로 감싸세요.

문제 8 — article 로 후기 하나 만들기

후기 하나를 <article> 로 감싸세요.

화면 굵고 조금 큰 글씨 "김밥이 맛있어요" 아래에

날짜 한 줄과 본문 한 줄이 보입니다. 모두 세 줄입니다

김밥이 맛있어요
2026년 8월 5일
재료가 아주 신선했습니다.

후기 하나는 떼어 내도 그 자체로 말이 되는 덩어리라서 article 입니다. 개념04 섹션5 흐름도의 1번 질문에서 바로 멈춥니다. article 을 신문 기사에만 쓴다고 오해하면 이런 자리를 놓칩니다.

문제 9 (개념04 섹션4)

추천 목록을 aside 로 감싸세요.

문제 9 — aside 로 결다리 만들기

추천 목록을 <aside> 로 감싸세요.

화면 굵은 소제목 "오늘의 추천" 아래에 점(•) 이 붙은 세 줄이

왼쪽에 그대로 쌓여서 보입니다

오늘의 추천

라면
김밥
떡볶이

추천 목록은 지위도 본문을 읽는 데 지장이 없는 걸다리라서 `aside` 입니다. `<aside>` 를 썼는데 오른쪽으로 안 갔다고 놀라지 마세요. `aside` 는 위치가 아니라 관계를 적는 태그입니다.

문제 10 [응용] (개념02 섹션1)

`div` 만 있는 코드를 시맨틱 태그로 바꿔 쓰세요.

문제 10 [응용] — `div` 만 있는 코드를 시맨틱으로

`<div>` 만 쓴 코드를 알맞은 시맨틱 태그로 바꿔 쓰세요. 화면은 하나도 안 바뀌어야 합니다.

화면 굵은 소제목 두 개와 문단 두 줄이 위에서 아래로 이어져

모두 네 줄로 보입니다

골목 분식

오늘의 국수
멸치 육수로 끓였습니다.

문의 02-000-0000

바깥 `<div>` 는 그대로 두는 것이 맞습니다. 아무 뜻 없이 세 덩어리를 한 번 묶은 껍데기니까요. 안쪽 셋은 각각 머리말 · 주제 묶음 · 꼬리말이라 `header` · `section` · `footer` 가 됩니다. 바꾸기 전과 화면이 똑같은 것이 정답의 증거입니다.

문제 11 [응용] (개념04 섹션5)

네 상황에 무엇을 쓸지 고르는 판단 문제입니다.

문제 11 [응용] — 어떤 태그를 쓸지 고르기

네 가지 상황에 article · section · aside · div 중 무엇을 쓸지 고르세요.

정답

네 개를 전부 section 으로 골랐다면 흐름도의 1번과 2번을 건너뛴 것입니다. 순서대로 물어야 합니다.

문제 12 [응용] (개념03 섹션6)

article 안에 header 와 footer 를 각각 넣으세요.

문제 12 [응용] — 글 한 편에 머리말과 꼬리말 붙이기

<article> 안에 <header> 와 <footer> 를 넣으세요.

화면 굵고 조금 큰 글씨 제목 아래에 날짜 한 줄, 본문 한 줄,

“글쓴이 : 최은지” 한 줄이 차례로 보입니다. 모두 네 줄입니다

떡볶이 국물 남기지 않는 법
2026년 8월 8일

마지막에 밥을 볶으면 됩니다.

글쓴이 : 최은지

여기의 <header> 는 글 한 편의 머리말입니다. 문제 4 의 header 는 사이트 전체의 머리말이었습니다. 태그는 같아도 뜻이 다릅니다. header 를 article 밖에 두면 화면은 같지만 “이 글의 머리말”이라는 정보가 사라집니다.

문제 13 [도전] (개념04 섹션2)

article 두 개와 aside 하나로 글 목록을 만드세요.

문제 13 [도전] — 글 목록 만들기

<article> 두 개와 <aside> 하나로 글 목록을 만드세요.

화면 제목 · 날짜 · 본문 · 글쓴이 순서의 덩어리가 두 벌 이어지고,

그 아래에 굵은 소제목 “많이 본 글” 과 점(•) 목록 두 줄이 보입니다. 두 글 사이에 구분선은 없습니다

여름 김치 담그기

2026년 7월 30일

소금 양을 줄이는 것이 요령입니다.

글쓴이 : 김민준

냉면 육수 내기

2026년 8월 2일

동치미 국물을 반 섞으면 시원합니다.

글쓴이 : 이서연

많이 본 글

김밥 싸기

된장 끓이기

글이 여러 편이면 <article> 을 여러 개 씁니다. 글 하나하나가 각각 완결된 덩어리이기 때문입니다. 두 글 사이에 선이 없어 경계가 안 보이는 것이 정상입니다. 선을 그으려고 <hr /> 를 넣으면 “주제가 바뀐다” 는 다른 뜻이 붙습니다.

문제 14 [도전] (개념05 섹션5)

페이지 한 장을 통째로 조립하세요. 이 파일의 유일한 main 입니다.

문제 14 [도전] — 페이지 한 장 통째로 조립하기

header · nav · main · article · section · aside · footer 를 모두 써서 페이지 한 장을 만드세요.

화면 가게 이름과 소개, 점(•) 메뉴 세 줄, 글 제목과 날짜,

“재료” 와 점 목록 세 줄, “끓이는 순서” 와 1. 2. 번호 목록 두 줄, 글쓴이 한 줄, “결들이면 좋아요” 와 점 목록 두 줄, 마지막에 주소와 저작권 두 줄이 위에서 아래로 이어집니다

골목 분식
1998년부터 한자리에서

메뉴판
오시는 길
사진

오늘의 국수
2026년 8월 11일

재료

멸치
다시마
소면

끓이는 순서

육수를 20분 끓입니다.
면을 3분 삶습니다.

글쓴이 : 사장님

결들이면 좋아요

김밥
단무지

골목 분식 · 서울시 어딘가 5길 12
© 2026 골목 분식

확인할 것은 넷입니다. main 은 하나, header 와 footer 는 각각 두 개 (가게 것 하나, 글 한 편의 것 하나), nav 는 위쪽 메뉴에만, 재료와 순서는 제목이 붙는 주제 묶음이라 section. 재료는 순서가 없으니 , 품이는 순서는 순서가 있으니 입니다.

문제 15 [확인] (개념03 섹션7, 개념04 섹션7)

잘못된 곳 네 군데를 찾아 고치는 문제입니다. 살아 있는 코드로 쓰면 이 파일에 main 이 두 개가 되므로 글자로만 적었습니다.

문제 15 [확인] — 잘못된 곳 네 군데 찾아 고치기

에러도 안 나고 화면도 멀쩡한데 잘못된 코드입니다. 네 군데를 찾아 고친 코드를 적으세요.

이런 실수를 합니다

잘못된 코드

```

<main>
<header>골목 분식</header>
<span> <div>오늘의 국수</div>
</span>
</main>

<section>
<p>멸치 육수로 끓였습니다.</p>
</section>

<main>
<p>문의 02-000-0000</p>
</main>

```

고친 코드

```

<header>골목 분식</header> <main> <section> <h3>오늘의 국수</h3> <p>멸치 육수로
끓였습니다.</p> </section>
</main> <footer> <p>문의 02-000-0000</p>
</footer>

```

- (1) main 이 두 개였습니다. 아래쪽 main 은 사실 연락처라서 꼬리말입니다. <footer> 로 바꿉니다.
- (2) header 가 main 안에 있었습니다. 가게 이름은 모든 페이지에 똑같이 나오므로 이 페이지만의 알맹이가 아닙니다. main 밖으로 빼야 "본문으로 건너뛰기" 가 제 자리로 갑니다.
- (3) span 안에 div 가 들어 있었습니다. 인라인 안에 블록이 들어가 줄이 쪼개집니다. 게다가 "오늘의 국수" 는 제목이므로 <h3> 가 맞습니다. span 과 div 를 둘 다 없었습니다.
- (4) section 에 제목이 없었습니다. 무슨 주제의 구역인지 아무 데도 안 적혀 있었습니다. 위에서 꺼낸 <h3> 오늘의 국수</h3> 를 section 안으로 넣으면 (3) 과 (4) 가 한 번에 풀립니다.

07단원 · 연습문제 정답

07단원 연습문제 정답 CSS 시작하기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

답은 이 파일 맨 위 <style> 안에 들어 있습니다.

문제 7 만 아래 본문 안에 있습니다.



실행하면 나오는 화면 — CSS 시작하기

답은 이 파일 맨 위 <style> 안에 있습니다. 각 답 아래에 왜 그렇게 되는지 짧은 설명을 붙여 두었습니다. 다만 옮겨 적지 말고 설명을 읽은 뒤, 연습문제.html 로 돌아가 직접 다시 쳐 보세요.

문제 1

화면 파란 글자 한 줄이 보입니다

우리 동네 빵집

문제 2

화면 노란 배경 위에 검은 글자 한 줄이 보입니다

오늘 구운 빵

문제 3

화면 목록 세 줄이 전부 초록 글자입니다

단팥빵
소금빵
크림빵

문제 4

화면 연한 노란 배경 위에 벽돌색 글자 한 줄이 보입니다

오늘의 추천

문제 5

화면 주황 배경 위에 흰 글자 한 줄이 보입니다

할인 중

문제 6

화면 파란 배경 위에 흰 글자 한 줄이 보입니다

rgb 로 칠했습니다

문제 7 (개념02 섹션 2)

아래 문단에 인라인 style 속성을 붙여 글자색을 #8e44ad 로 만드세요. <p 와 id 사이에 한 칸 띄고 style="..." 을 씁니다. 중괄호도 선택자도 쓰지 않습니다. 속성과 값만 적습니다.

기대 출력 "인라인으로 칠해 보세요" 가 보라색 글자로 보입니다.

답 설명: 인라인 자리는 이미 "이 요소" 라고 정해져 있어서 선택자를 쓸 자리가 없습니다. 그래서 중괄호 없이 속성과 값만 적습니다. style 속성 값 안에서는 큰따옴표를 다시 쓰면 안 됩니다.

화면 보라색 글자 한 줄이 보입니다

인라인으로 칠해 보세요

문제 8

화면 벽돌색 글자 한 줄. 배경은 상자의 연한 회색 그대로입니다

주석으로 꺼 봅니다

문제 9

화면 세 줄이 모두 보라색 글자입니다

첫 번째 줄입니다.
두 번째 줄입니다.
세 번째 줄입니다.

문제 10

화면 파란 덩어리 안 첫 줄은 흰 글자이고, 둘째 줄 자리에만

가로로 긴 흰 띠가 생기며 그 위에 파란 글자가 보입니다

저는 부모에게서 글자색만 받았습니다.
저는 안쪽 문단입니다.

문제 11

화면 두 줄 모두 벽돌색 글자입니다. 아래 링크에는 밑줄이 그어져 있습니다

저는 부모 색을 받았습니다.
가게 위치 보러 가기

문제 12

화면 아주 연한 분홍빛 배경 위에 진한 벽돌색 글자가 또렷하게 보입니다

연한 배경 위의 진한 글자

문제 13

화면 보라색 덩어리 안에 흰 글자 한 줄과, 밑줄 그어진 흰 글자 링크가 보입니다

공지 사항이 하나 있습니다.
공지 보러 가기

문제 14

화면 파랑, 빨강, 초록 띠 세 개가 위에서 아래로 놓이고 글자는 모두 흰색입니다

새 글이 등록되었습니다.
저장하지 않은 내용이 있습니다.
처리가 모두 끝났습니다.

문제 15

화면 첫 줄은 파란 배경에 흰 글자, 둘째 줄은 노란 배경에 검은 글자,

셋째 줄은 벽돌색 글자, 넷째 줄은 초록 배경에 흰 글자입니다

파란 배경에 흰 글자가 되어야 합니다.
 노란 배경이 되어야 합니다.
 벽돌색 글자가 되어야 합니다.
 초록 배경에 흰 글자가 되어야 합니다.

정리

07단원 연습문제 정답 - CSS 시작하기

문제 파일과 같은 뼈대에 답을 채운 파일입니다.
 다만 보지 말고, 답 아래에 붙은 짧은 설명을 꼭 같이 읽으세요.

문제 1 (개념03 섹션 1)

id 가 q1Title 인 요소의 글자색을 #2d6cdf 로 만드세요.

기대 화면: "우리 동네 빵집" 이 파란 글자로 보입니다.

```
#q1Title {
  color: #2d6cdf;
}
```

화면 "우리 동네 빵집" 이 파란 글자로 보입니다.

id 선택자는 # 을 붙입니다. HTML 쪽 id="q1Title" 에는 # 을 안 붙입니다.

기호를 붙이는 자리는 CSS 안뿐입니다.

문제 2 (개념03 섹션 2, 개념04 섹션 1)

class 가 q2-box 인 요소의 배경색을 색 이름 gold 로 만드세요.

기대 화면: "오늘 구운 빵" 줄의 뒷배경이 노랗게 칠해집니다. 글자는 검은색 그대로입니다.

```
.q2-box {
  background-color: gold;
}
```

화면 "오늘 구운 빵" 의 뒷배경만 노랗게 칠해지고 글자는 검은색 그대로입니다.

클래스 선택자는 점을 붙입니다.

color 를 안 건드렸으니 글자색은 원래 값이 그대로 남습니다.

문제 3 (개념03 섹션 2)

태그 이름 선택자를 써서 이 파일의 모든 li 의 글자색을 #27853f 로 만드세요.
선택자 앞에 점이나 # 을 붙이지 않습니다.

기대 화면: 문제 3 상자의 목록 세 줄이 전부 초록 글자가 됩니다.

```
li {  
  color: #27853f;  
}
```

화면 목록 세 줄이 전부 초록 글자가 됩니다.

태그 이름 선택자는 기호를 안 붙입니다.
이 규칙은 파일 안의 li 를 하나도 빠짐없이 전부 고칩니다.

이 파일에는 문제 3 상자에만 li 가 있어서 거기서만 효과가 보입니다.

문제 4 (개념04 섹션 2)

id 가 q4Line 인 요소를 글자색 #c0392b, 배경색 #fff3a0 으로 만드세요.
규칙 하나 안에 선언 두 개를 쓰면 됩니다. 선언마다 끝에 세미콜론을 찍으세요.

기대 화면: "오늘의 추천" 이 연한 노란 배경 위에 벽돌색 글자로 보입니다.

```
#q4Line {  
  color: #c0392b;  
  background-color: #fff3a0;  
}
```

화면 연한 노란 배경 위에 벽돌색 글자 한 줄이 보입니다.

첫 줄 끝의 세미콜론이 없으면 두 줄이 한 덩어리로 묶여 둘 다 죽습니다.

(개념03 섹션 5 의 ㉔ 상자와 같은 사고입니다)

문제 5 (개념04 섹션 2)

class 가 q5-box 인 요소를 글자색 흰색, 배경색 주황색으로 만드세요.
두 색 모두 세 자리 축약형으로 쓰세요. 흰색은 #ffffff, 주황색은 #ff6600
입니다.

기대 화면: "할인 중" 이 주황 배경 위에 흰 글자로 보입니다.

```
.q5-box {
  color: #fff;
  background-color: #f60;
}
```

화면 주황 배경 위에 흰 글자 한 줄이 보입니다.

#ffffff 는 ff, ff, ff 라서 #fff 로, #ff6600 은 ff, 66, 00 이라서 #f60 으로
줄입니다.

두 글자가 같을 때만 줄일 수 있습니다.

문제 6 (개념04 섹션 3)

id 가 q6Line 인 요소를 글자색 흰색, 배경색 파란색으로 만드세요.
이번에는 두 색 모두 rgb() 표기로 쓰세요.
흰색은 rgb(255, 255, 255), 파란색은 rgb(45, 108, 223) 입니다.

기대 화면: "rgb 로 칠했습니다" 가 파란 배경 위에 흰 글자로 보입니다.

```
#q6Line {
  color: rgb(255, 255, 255);
  background-color: rgb(45, 108, 223);
}
```

화면 파란 배경 위에 흰 글자 한 줄이 보입니다.

rgb(45, 108, 223) 은 #2d6cdf 와 완전히 같은 색입니다.

F12 의 Computed 탭으로 보면 두 표기 모두 rgb(45, 108, 223) 으로 나옵니다.

문제 7 (개념02 섹션 2)

이 문제만 style 이 아니라 body 안에서 풁니다. 문제 7 상자를 찾아가세요.

문제 8 (개념03 섹션 4)

아래 #q8Line 규칙에서 background-color 줄만 CSS 주석으로 감싸 꺼 두세요.
줄을 지우면 안 됩니다. 개념03 섹션 4 에서 배운 CSS 주석 기호로 감싸세요.

기대 화면: "주석으로 꺼 봅니다" 가 벽돌색 글자로 보이고 배경은 안 칠해집니다.

```
#q8Line {  
  color: #c0392b;
```

```
background-color: #fff3a0;
```

```
}
```

화면 벽돌색 글자 한 줄이 보이고 배경은 상자의 연한 회색 그대로입니다.

주석 안에 들어간 줄은 브라우저가 아예 안 읽습니다.

지운 것과 결과는 같지만, 다시 살리기 쉬워서 원인을 찾을 때 편합니다.

문제 9 (개념05 섹션 1)

class 가 q9-card 인 바깥 div 에만 규칙을 하나 써서
그 안의 문단 세 줄이 모두 글자색 #8e44ad 가 되게 하세요.
문단마다 규칙을 쓰면 안 됩니다. 규칙은 딱 하나만 씁니다.

기대 화면: 문제 9 상자 안의 세 줄이 모두 보라색 글자가 됩니다.

```
.q9-card {  
  color: #8e44ad;  
}
```

화면 상자 안의 세 줄이 모두 보라색 글자가 됩니다.

color 는 상속되는 속성이라 부모에만 주면 자식이 알아서 받습니다.

규칙 세 개를 쓸 일을 한 개로 줄인 것입니다.

문제 10 (개념05 섹션 2)

아래 .q10-card 규칙은 이미 주어져 있습니다. 고치지 마세요.
 배경색은 자식에게 상속되지 않습니다. 그래서 안쪽 문단은 자기 배경이 없습니다.
 id 가 q10Kid 인 문단에 직접 규칙을 써서
 배경색 #ffffff, 글자색 #2d6cdf 로 만드세요.

기대 화면: 파란 카드 안에 흰 띠가 하나 생기고 그 위에 파란 글자가 보입니다.

```
.q10-card {
  color: #ffffff;
  background-color: #2d6cdf;
}
#q10Kid {
  color: #2d6cdf;
  background-color: #ffffff;
}
```

화면 파란 카드 안에 가로로 긴 흰 띠가 생기고 그 위에 파란 글자가 보입니다.

첫 번째 문단은 배경을 안 줬으니 여전히 투명해서 파랑이 비칩니다.

자식에게 배경을 주려면 이렇게 직접 줘야 합니다. 상속으로는 안 옵니다.

문제 11 (개념05 섹션 3, 섹션 6)

아래 .q11-card 규칙은 이미 주어져 있습니다. 고치지 마세요.
 그런데 상자 안의 링크만 파란색으로 남아 있습니다.
 브라우저가 a 의 색을 직접 정해 뒀기 때문입니다.
 id 가 q11Link 인 링크에 규칙을 써서 부모 색을 그대로 받아 오게 하세요.
 색 값을 직접 적지 말고 inherit 를 쓰세요.

기대 화면: 상자 안의 링크가 벽돌색 글자가 됩니다. 밑줄은 그대로 남습니다.

```
.q11-card {
  color: #c0392b;
}
#q11Link {
  color: inherit;
}
```

화면 상자 안의 링크가 파란색에서 벽돌색으로 바뀝니다. 밑줄은 그대로 남습니다.

상속으로 내려온 값은 가장 약해서, 브라우저가 a 에 직접 준 파란색에 집니다. a 를 직접 골라 규칙을 줘야 이깁니다. inherit 는 "부모 것을 그대로" 라는 뜻입니다.

밑줄이 남는 것은 text-decoration 이라는 다른 속성이라 안 건드렸기 때문입니다.

문제 12 [응용] (개념04 섹션 3, 섹션 6)

id 가 q12Line 인 요소에 아래 두 가지를 주세요.

- 배경색: 벽돌색 rgb(192, 57, 43) 에 투명도 0.15 를 붙인 rgba 값
- 글자색: 그 연한 배경 위에서 잘 읽히도록 #c0392b

기대 화면: 아주 연한 분홍빛 배경 위에 진한 벽돌색 글자가 또렷하게 보입니다.

```
#q12Line {
  color: #c0392b;
  background-color: rgba(192, 57, 43, 0.15);
}
```

화면 아주 연한 분홍빛 배경 위에 진한 벽돌색 글자가 또렷하게 보입니다.

투명도를 0.15 로 낮추면 같은 색이라도 아주 연하게 깔립니다.
연한 색을 새로 고르지 않고 같은 색의 투명도만 바꿨다는 점이 요령입니다.

글자는 투명도를 안 준 진한 색이라 대비가 충분합니다.

문제 13 [응용] (개념05 섹션 1, 섹션 6)

class 가 q13-panel 인 바깥 div 를 배경색 #8e44ad, 글자색 #ffffff 로 만드세요.
그런데 그것만으로는 안쪽 링크가 파란색으로 남습니다.

id 가 q13Link 인 링크도 흰 글자로 보이게 하세요.

링크 색은 inherit 를 써도 되고 #ffffff 를 직접 적어도 됩니다.

기대 화면: 보라색 상자 안에 흰 글자 한 줄과 흰 글자 링크(밑줄 있음)가 보입니다.

```
.q13-panel {
  color: #ffffff;
  background-color: #8e44ad;
}
#q13Link {
  color: inherit;
}
```

화면 보라색 덩어리 안에 흰 글자 한 줄과, 밑줄이 그어진 흰 글자 링크가 보입니다.

첫 줄은 상속만으로 흰색이 됐지만 링크는 규칙을 따로 줘야 했습니다. #ffffff 를 직접 적어도 결과는 같습니다. 다만 나중에 패널 색을 바꿀 때

inherit 쪽이 자동으로 따라오므로 고칠 곳이 한 군데로 줄어듭니다.

문제 14 [도전] (개념01 ~ 개념05 종합)

알림 상자 세 종류를 만듭니다. 클래스는 이미 HTML 에 붙어 있습니다. 세 클래스 모두 글자색은 #ffffff 이고, 배경색만 다릅니다. 그리고 배경색은 세 가지를 일부러 서로 다른 표기법으로 쓰세요.

| | | | |
|-------------|-------|-------------|------------------|
| .alert-info | 배경 파랑 | 16진수 여섯 자리로 | #2d6cdf |
| .alert-warn | 배경 빨강 | rgb() 표기로 | rgb(192, 57, 43) |
| .alert-done | 배경 초록 | 색 이름으로 | seagreen |

기대 화면: 파랑·빨강·초록 띠 세 개가 위에서 아래로 놓이고 글자는 모두 흰색입니다.

```
.alert-info {
  color: #ffffff;
  background-color: #2d6cdf;
}
.alert-warn {
  color: #ffffff;
  background-color: rgb(192, 57, 43);
}
.alert-done {
  color: #ffffff;
  background-color: seagreen;
}
```

화면 파랑, 빨강, 초록 띠 세 개가 위에서 아래로 놓이고 글자는 모두 흰색입니다.

표기법이 셋 다 다른데도 결과는 똑같이 잘 나옵니다.
브라우저는 어떤 표기로 적든 안에서는 rgb 값 하나로 바꿔 기억하기 때문입니다.

실무에서는 이렇게 섞어 쓰지 말고 한 가지로 통일하세요. 보통 #rrggbb 를 씁니다.

문제 15 [확인] (개념03 섹션 5, 섹션 6 / 개념04 섹션 7)

아래 네 규칙은 전부 일부러 틀리게 써 두었습니다.
브라우저는 에러를 하나도 안 냅니다. 틀린 선언만 조용히 버릴 뿐입니다.
무엇이 틀렸는지 찾아 고치세요. 규칙마다 잘못된 곳은 딱 한 군데씩입니다.

넷 중 하나는 두 줄 중 한 줄만 죽어서 반쯤은 먹은 것처럼 보입니다.
어느 것인지도 찾아내세요.

★ "반쯤 먹은 것처럼 보이는" 하나는 #q15d 입니다.
등호를 쓴 color 줄만 버려지고 background-color 줄은 멀쩡히 살아남아서,
고치기 전에도 초록 배경은 이미 나와 있었습니다. 글자만 검은색이었습니다.
#q15a 는 세미콜론이 없어서 두 줄이 한 덩어리로 묶여 둘 다 죽었고,
#q15b 와 #q15c 는 선언이 원래 한 줄뿐이라 그 한 줄이 통째로 죽었습니다.

기대 화면: 고치고 나면 네 줄이 이렇게 보입니다.

q15a 파란 배경 위에 흰 글자
q15b 노란 배경 위에 검은 글자
q15c 벽돌색 글자
q15d 초록 배경 위에 흰 글자

고친 곳: color 줄 끝에 세미콜론을 찍었습니다. 고치기 전에는 두 줄이 한 선언으로 묶여 둘 다 죽어 있었습니다.

```
#q15a {  
  color: #ffffff;  
  background-color: #2d6cdf;  
}
```

고친 곳: 선택자 앞에 점을 붙였습니다. 점이 없으면 q15-tag 라는 이름의 태그를 찾는 뜻이 되어 아무도 못 고릅니다.

```
.q15-tag {  
  background-color: #fff3a0;  
}
```

고친 곳: 색 값 앞에 # 을 붙였습니다. # 이 없으면 c0392b 라는 이름의 색을 찾는데 그런 이름은 없습니다.


```
#q15c {
  color: #c0392b;
}
```

고친 곳: 등호를 콜론으로 바꿨습니다. 등호는 HTML 속성에서 쓰는 기호입니다. CSS 는 콜론을 씁니다.

```
#q15d {
  color: #ffffff;
  background-color: #27853f;
}
```

화면 네 줄이 각각 파란 배경에 흰 글자, 노란 배경에 검은 글자,

벽돌색 글자, 초록 배경에 흰 글자로 보입니다.
네 가지 모두 브라우저가 아무 말도 안 해 주는 종류의 실수입니다.

“안 먹으면 세미콜론, 점, #, 콜론” 순서로 훑는 습관을 들이세요.

08단원 연습문제 정답 선택자

제목 두 줄을 빨간 글자로 만드세요.

오늘의 메뉴

아메리카노, 카페라떼

내일의 메뉴

딸기라떼, 자몽차

notice 를 붙인 두 줄에 노란 배경을 넣으세요.

문 여는 시간이 바뀌었습니다.

평일은 8시에 엽니다.

일요일은 쉽니다.

id 가 mainTitle 인 줄만 초록 글자로 만드세요.

봄내 카페

강원도 춘천시 봄내로 12

실행하면 나오는 화면 — 정답

문제 상자 안의 HTML 은 문제 파일과 한 글자도 다르지 않습니다. (제목과 이 안내 상자, 맨 아래 정리 상자만 다릅니다) 답은 <style> 안에만 채워져 있습니다. 답을 보기 전에 먼저 스스로 풀어 보고, 화면이 다르면 <style> 의 해설을 읽으세요.

문제 1 (개념01 섹션1)

화면 제목 두 줄이 빨간 글자, 그 아래 메뉴 문단은 검은 글자입니다

h3 제목 두 줄을 빨간 글자로 만드세요.

오늘의 메뉴

아메리카노, 카페라떼

내일의 메뉴

딸기라떼, 자몽차

문제 2 (개념01 섹션2)

화면 첫째 줄과 셋째 줄만 노란 배경입니다

notice 를 붙인 두 줄에 노란 배경을 넣으세요.
 문 여는 시간이 바뀌었습니다.
 평일은 8시에 엽니다.
 일요일은 쉽니다.

문제 3 (개념01 섹션3)

화면 첫 줄만 초록 글자입니다

id 가 mainTitle 인 줄만 초록 글자로 만드세요.
 봄내 카페
 강원도 춘천시 봄내로 12

문제 4 (개념02 섹션1)

화면 첫 줄과 셋째 줄만 연한 하늘색 배경입니다

차와 케이크 두 줄만 규칙 하나로 하늘색 배경을 넣으세요.
 캐모마일차
 아메리카노
 치즈케이크

문제 5 (개념02 섹션4)

화면 셋째 줄만 연한 분홍 배경입니다

card 와 big 을 둘 다 가진 줄만 분홍 배경으로 만드세요.
 card 만 가진 줄
 big 만 가진 줄
 card 와 big 을 둘 다 가진 줄

문제 6 (개념03 섹션2)

화면 위 두 줄이 빨간 글자, 마지막 줄은 검은 글자입니다

notice-box 안의 문단을 깊이 상관없이 전부 빨강게 만드세요.

상자 안에 바로 있는 문단

한 겹 더 안에 있는 문단

상자 밖 문단

문제 7 (개념03 섹션3)

화면 첫 줄만 노란 배경입니다

crumb 의 바로 아래 단계 문단만 노란 배경으로 만드세요.

바로 아래 단계 문단

한 겹 더 들어간 문단

문제 8 (개념03 섹션5)

화면 가운데 줄만 연한 하늘색 배경입니다

질문 줄 바로 다음 문단 하나만 하늘색 배경으로 만드세요.

주차할 수 있나요?

건물 뒤에 두 자리 있습니다.

주말에는 만차일 수 있습니다.

문제 9 (개념04 섹션1)

화면 평소에는 색이 없고, 마우스를 올린 동안만 파란 배경에 흰 글자입니다

마우스를 올린 동안만 파란 배경에 흰 글자가 되게 하세요.

여기에 마우스를 올려 보세요

문제 10 (개념04 섹션5)

화면 위 글자만 노란 배경입니다. 아래 체크박스를 누르면 아래도 노래집니다

체크된 칸의 글자에만 노란 배경을 넣으세요.

얼음을 추가합니다

시럽을 추가합니다

문제 11 (개념05 섹션1)

화면 1등 줄은 노란 배경, 3등 줄은 연한 하늘색 배경, 2등 줄은 색이 없습니다

첫 줄은 노란 배경, 마지막 줄은 하늘색 배경으로 만드세요.

1등 아메리카노

2등 카페라떼

3등 딸기라떼

문제 12 (개념05 섹션3)

화면 2, 4, 6번 줄에만 연한 회청색 배경이 깔립니다

짝수 번째 줄에만 연한 회청색 배경을 깔아 주세요.

1번 줄

2번 줄

3번 줄

4번 줄

5번 줄

6번 줄

문제 13 [응용] (개념05 섹션5)

화면 1번째와 3번째 줄만 연한 초록 배경입니다

품절이 아닌 항목만 연한 초록 배경으로 만드세요.

아메리카노
카페라떼 (품절)
딸기라떼
자몽차 (품절)

문제 14 [응용] (개념05 섹션2)

화면 제목 아래 첫 문단만 노란 배경입니다

제목이 먼저 있는 상자에서 첫 번째 문단만 노란 배경으로 만드세요.

가게 소개
2019년에 문을 열었습니다.
원두는 매주 볶습니다.

문제 15 [도전] (개념06)

화면 위 줄은 흐린 회색, 아래 줄만 빨간 글자입니다

아래 두 줄은 이미 써 둔 규칙 때문에 흐린 회색입니다.
strong-tip 줄만 빨강게 바꾸세요. !important 와 id 는 금지입니다.

보통 안내 문단입니다.
이 줄만 빨간 글자로 만드세요.

문제 16 [확인]

화면 (가) 줄은 노란 배경, (나) 두 줄은 빨간 글자,

(다) 체크박스 옆 글자는 노란 배경입니다

<style> 맨 아래 세 규칙은 전부 아무 일도 하지 않습니다.
각각 한 군데씩 고치세요.

- (가) 이 줄에 노란 배경이 깔려야 합니다.
- (나) 이 줄이 빨간 글자가 되어야 합니다.
- (나) 이 줄도 빨간 글자가 되어야 합니다.
- (다) 이 글자에 노란 배경이 깔려야 합니다.

정리

08단원 연습문제 정답 - 선택자

문제 파일과 body 는 완전히 같습니다. style 안만 채워져 있습니다.
답을 베끼기 전에 먼저 스스로 풀어 보세요.

문제 1 (개념01 섹션1)

h3 의 글자색을 crimson 으로.

```
h3 {
  color: crimson;
}
```

화면 문제 1 상자의 "오늘의 메뉴", "내일의 메뉴" 두 줄이 빨간 글자입니다.

태그 선택자는 이름만 그대로 씁니다. 점도 우물정자도 붙이지 않습니다.
이 규칙은 이 파일 안의 h3 를 전부 고칩니다.

여기서는 h3 가 문제 1 상자에만 있어서 그 두 줄만 바뀐 것입니다.

문제 2 (개념01 섹션2)

class="notice" 인 두 줄의 배경색을 #fff3a0 으로.

```
.notice {
  background-color: #fff3a0;
}
```

화면 문제 2 상자의 첫째 줄과 셋째 줄에 노란 배경이 깔립니다.

CSS 에서는 클래스 이름 앞에 점을 붙입니다. HTML 에는 안 붙입니다.

같은 클래스를 몇 번이든 다시 쓸 수 있어서 규칙 한 개가 두 줄을 칠합니다.

문제 3 (개념01 섹션3)

id 가 mainTitle 인 줄의 글자색을 #1a7f5a 로.

```
#mainTitle {  
  color: #1a7f5a;  
}
```

화면 "봄내 카페" 한 줄만 초록 글자이고 주소 줄은 검은 글자 그대로입니다.

id 는 우물정자를 붙입니다. 한 페이지에 하나뿐이라 딱 한 줄만 걸립니다.

문제 4 (개념02 섹션1)

tea 와 cake 두 줄만 배경색 #eef7ff 로. 규칙 한 개.

```
.tea,  
.cake {  
  background-color: #eef7ff;  
}
```

화면 "캐모마일차" 와 "치즈케이크" 두 줄이 연한 하늘색 배경이고,

가운데 "아메리카노" 줄은 배경색이 없습니다.
심표는 "또는" 입니다. .tea 뒤의 심표를 빠뜨리면

"tea 안의 cake" 라는 뜻이 되어 한 줄도 안 바뀝니다.

문제 5 (개념02 섹션4)

card 와 big 을 둘 다 가진 줄만 배경색 #ffe9e9 로.

```
.card.big {  
  background-color: #ffe9e9;  
}
```

화면 문제 5 상자의 셋째 줄만 연한 분홍 배경입니다.

사이에 공백이 없어야 "한 요소가 둘 다 가진 경우" 가 됩니다.

공백을 넣으면 "card 안의 big" 이 되어 한 줄도 안 걸립니다.

문제 6 (개념03 섹션2)

notice-box 안의 문단을 깊이 상관없이 전부 글자색 #c0392b 로.

```
.notice-box p {
  color: #c0392b;
}
```

화면 위 두 줄이 빨간 글자가 되고 "상자 밖 문단" 은 검은 글자입니다.

공백은 자손 결합자라서 몇 겹 안에 있어도 걸립니다.

빨개지는 것은 오른쪽 p 이고, .notice-box 자체는 아무 변화가 없습니다.

문제 7 (개념03 섹션3)

crumb 의 바로 아래 단계 문단만 배경색 #fff3a0 으로.

```
.crumb > p {
  background-color: #fff3a0;
}
```

화면 첫 줄만 노란 배경이고 한 겹 더 들어간 줄은 배경색이 없습니다.

부등호는 "바로 한 단계 아래" 만 봅니다.

안쪽 문단의 부모는 crumb 가 아니라 가운데 div 라서 안 걸립니다.

문제 8 (개념03 섹션5)

class="q" 바로 다음 문단 하나만 배경색 #eef7ff 로.

```
.q + p {
  background-color: #eef7ff;
}
```

화면 "건물 뒤에 두 자리 있습니다." 한 줄만 연한 하늘색 배경입니다.

더하기는 바로 다음 형제 하나만 봅니다.

물결(~)을 쓰면 "주말에는" 줄까지 두 줄이 칠해집니다.

문제 9 (개념04 섹션1)

마우스를 올린 동안만 배경색 #2d6cdf, 글자색 white.

```
.hover-item:hover {  
  background-color: #2d6cdf;  
  color: white;  
}
```

화면 평소에는 아무 색도 없다가 마우스를 올린 동안만

파란 배경에 흰 글자가 되고, 치우면 곧바로 돌아옵니다.

콜론은 반드시 붙여 씁니다. 앞에 공백을 넣으면 뜻이 달라집니다.

문제 10 (개념04 섹션5)

체크된 체크박스 바로 다음 label 의 배경색을 #fff3a0 으로.

```
.opt:checked + label {  
  background-color: #fff3a0;  
}
```

화면 처음 열면 "얼음을 추가합니다" 글자만 노란 배경입니다.

아래 체크박스를 누르면 "시럽을 추가합니다" 에도 노란 배경이 생깁니다.
가상 클래스(:checked)와 형제 결합자(+)를 이어 붙여 쓴 것입니다.

체크박스 자체는 색이 잘 안 먹어서 옆의 label 을 대신 꾸밉니다.

문제 11 (개념05 섹션1)

첫 줄은 #fff3a0, 마지막 줄은 #eef7ff.

```
.rank li:first-child {  
  background-color: #fff3a0;  
}  
.rank li:last-child {  
  background-color: #eef7ff;  
}
```

화면 "1등" 줄은 노란 배경, "3등" 줄은 연한 하늘색 배경이고

가운데 "2등" 줄은 배경색이 없습니다.

항목을 더 넣거나 빼도 CSS 를 안 고쳐도 됩니다. 브라우저가 다시 셉니다.

문제 12 (개념05 섹션3)

짝수 번째 줄에만 배경색 #eef2f7.

```
.stripe li:nth-child(even) {
  background-color: #eef2f7;
}
```

화면 2, 4, 6번 줄에만 연한 회청색 배경이 깔려 줄무늬가 됩니다.

even 대신 2n 이라고 써도 결과가 같습니다.

순서는 1 부터 세기 때문에 첫 줄은 홀수입니다.

문제 13 [응용] (개념05 섹션5)

품절이 아닌 항목만 배경색 #e2f4ea.

```
.stock li:not(.sold) {
  background-color: #e2f4ea;
}
```

화면 1번째와 3번째 줄만 연한 초록 배경이고 품절 두 줄은 배경색이 없습니다.

:not() 을 쓰면 예외인 것에만 클래스를 붙이면 됩니다.

품절이 아닌 항목마다 클래스를 붙일 필요가 없어집니다.

문제 14 [응용] (개념05 섹션2)

intro 안의 첫 번째 문단만 배경색 #fff3a0.

```
.intro p:first-of-type {
  background-color: #fff3a0;
}
```

화면 "2019년에 문을 열었습니다." 줄만 노란 배경입니다.

:first-child 로 쓰면 한 줄도 안 걸립니다.
상자의 첫 자식이 h4 라서 "첫 자식이면서 p 인 것" 이 없기 때문입니다.

:first-of-type 은 p 끼리만 세므로 제목이 있어도 잘 걸립니다.

문제 15 [도전] (개념06)

아래 규칙은 문제 파일에 이미 있던 것입니다.

```
.tip-area p {  
  color: #8a8a8a;  
}
```

정답 — 명시도를 한 단계 올려서 이깁니다.

```
.tip-area p.strong-tip {  
  color: #c0392b;  
}
```

화면 아래 줄만 빨간 글자가 되고 위 줄은 흐린 회색 그대로입니다.

.tip-area p 는 0-1-1 이고, .strong-tip 만 쓰면 0-1-0 이라 집니다.
.tip-area p.strong-tip 은 0-2-1 이라 이깁니다.
.tip-area .strong-tip (0-2-0) 으로 써도 이깁니다.

!important 나 id 를 쓰지 않고도 이길 수 있다는 것이 이 문제의 요점입니다.

문제 16 [확인]

(가) 점을 빠뜨렸습니다. 클래스는 앞에 점을 붙입니다.

```
.menu-item {  
  background-color: #fff3a0;  
}
```

화면 "(가)" 줄에 노란 배경이 깔립니다.

고치기 전에는 menu-item 이라는 태그를 찾다가 못 찾고 규칙이 버려졌습니다.

(나) 심표를 빠뜨렸습니다. 두 요소는 나란히 있는 형제입니다.

```
.list-a,
.list-b {
  color: crimson;
}
```

화면 "(나)" 두 줄이 모두 빨간 글자가 됩니다.

고치기 전에는 "list-a 안에 있는 list-b" 를 찾았는데 그런 요소가 없었습니다.

(다) 콜론 앞에 공백이 있었습니다. 가상 클래스는 붙여 씁니다.

```
.chk:checked + label {
  background-color: #fff3a0;
}
```

화면 체크박스 옆 "(다)" 글자에 노란 배경이 깔립니다.

고치기 전에는 공백이 자손 결합자라서

"chk 안에 있는 체크된 요소" 를 찾았고, 체크박스 안에는 아무것도 없었습니다.

09단원 연습문제 정답 텍스트와 폰트



실행하면 나오는 화면 — 정답

문제 파일과 뼈대가 똑같은 파일입니다. 위쪽 <style> 에 답과 해설이 들어 있습니다. 두 파일을 나란히 열고 화면부터 비교하세요. 답이 달라도 화면이 같으면 맞은 것입니다.

화면 아래 줄이 위 기준 줄보다 눈에 띄게 큼니다

기준 줄입니다. 아무것도 지정하지 않았습니다.
이 줄을 24px 로 만드세요.

화면 문제 1 의 24px 줄보다 한 단계 더 큼니다

이 줄을 rem 으로 32px 로 만드세요.

화면 획 끝에 빠침이 있는 명조체로 보입니다

이 줄을 명조체로 만드세요. Hello 123

화면 굽어지면서 동시에 오른쪽으로 기울어집니다

이 줄을 굵고 기울게 만드세요. Hello

화면 줄 사이가 넉넉해지고 점선 상자가 세로로 길어집니다

이 문단의 줄 높이를 2.4 로 만드세요. 줄이 여러 개가 되도록 문장을 길게 씁니다.
줄과 줄 사이가 넓어지는 것을 확인하세요. 계속 이어 쓰는 문장입니다.

화면 글자마다 사이가 벌어져 문장이 옆으로 길어집니다

이 줄의 글자 사이를 벌리세요.

화면 글자가 점선 상자의 가로 가운데에 놓입니다

이 줄을 가운데로 보내세요.

화면 12,000원 한가운데에 가로줄이 그어집니다

12,000원

화면 NEW ARRIVAL TODAY 로 보입니다

new arrival today

화면 밑줄 없는 붉은 글자입니다. 눌러 보면 개념04 로 이동합니다

개념04 로 가는 링크입니다

화면 주소가 두 줄로 잘려서 노란 배경 안에 모두 들어옵니다

<https://www.example.com/blog/2026/01/very-long-article-name>

화면 회색 상자 안에서 둘째 줄이 첫 줄보다 조금 큼니다

부모입니다. 20px 입니다.
자식입니다. 24px 로 만드세요.

화면 파란 상자 안의 큰 글자가 두 줄이 되어도 위아래가 서로 닿지 않습니다

부모입니다.

자식은 28px 입니다. 줄이 여러 줄이 되도록 문장을 길게 씁니다.
글자가 위아래로 겹치지 않게 만들어야 합니다.

화면 "오늘의 공지사항입니다 자세한 내..." 까지만 보이고 점 세 개로 끝납니다

오늘의 공지사항입니다 자세한 내용은 게시판을 확인해 주세요

화면 파란 굵은 큰 글자가 가운데에 놓이고 글자 사이가 살짝 벌어집니다

이번 주 신상품 안내

화면 붉고 굵은 28px 글자가 가운데에 놓입니다

세 군데가 틀렸습니다. 찾아 보세요.

정리

09단원 연습문제 정답 - 텍스트와 폰트

문제 파일과 뼈대가 똑같고 TODO 자리만 채워 두었습니다.
두 파일을 나란히 열어 두고 비교하세요.

답이 여기와 달라도 화면이 같으면 맞은 것입니다.
값을 적는 방법이 여러 가지인 문제도 있습니다(예: bold 와 700).

아래 세 덩어리는 문제를 풀 준비물입니다. 고치지 마세요.

width 는 10단원에서 제대로 배웁니다.

지금은 '상자 폭을 정하는 것' 이라고만 알아 두세요.

```
.q-box {  
  width: 260px;  
  background-color: #fff8dc;  
}  
#q12-parent {  
  font-size: 20px;  
  background-color: #eef2f7;  
}  
#q13-child {  
  font-size: 28px;  
}
```


문제 1 (개념01 섹션2)

q1 문단의 글자 크기를 24px 로 만드세요.

기대 화면: 그 줄이 바로 위 기준 줄보다 눈에 띄게 커집니다.

```
#q1 {
  font-size: 24px;
}
```

화면 기준 줄보다 확실히 큰 글자가 됩니다

숫자 뒤에 단위 px 을 붙이는 것이 핵심입니다.

24 라고만 쓰면 이 줄이 통째로 버려져서 16px 그대로 남습니다.

문제 2 (개념01 섹션5)

q2 문단의 글자 크기를 32px 로 만드세요. px 이 아니라 rem 을 씁니다.

기대 화면: 문제 1 의 줄보다 더 큰 글자가 됩니다.

```
#q2 {
  font-size: 2rem;
}
```

화면 문제 1 의 24px 줄보다 한 단계 더 큰 글자가 됩니다

1rem 이 16px 이므로 $32 \div 16 = 2$, 즉 2rem 입니다.

rem 계산은 언제나 16 으로 나누거나 곱하면 됩니다.

문제 3 (개념02 섹션2)

q3 문단에 "없는글꼴" 과 serif 로 폰트 스택을 만드세요.

기대 화면: 없는 이름은 건너뛰고 명조체로 보입니다.

```
#q3 {
  font-family: "없는글꼴", serif;
}
```

화면 획 끝에 빠침이 있는 명조체로 바뀝니다. Hello 123 부분이 특히 다릅니다

브라우저가 첫 이름을 찾다가 없으니 조용히 다음 칸으로 넘어간 것입니다.

에러도 경고도 뜨지 않는다는 점을 꼭 확인하세요.

문제 4 (개념02 섹션5·6)

q4 문단을 굵게, 그리고 기울임으로 만드세요.

기대 화면: 굵어지면서 동시에 오른쪽으로 기울입니다.

```
#q4 {
  font-weight: bold;
  font-style: italic;
}
```

화면 글자가 굵어지고 동시에 오른쪽으로 비스듬히 눕습니다

font-weight 와 font-style 은 서로 다른 속성이라 함께 걸 수 있습니다.

bold 대신 700 이라고 써도 결과가 똑같습니다.

문제 5 (개념03 섹션1)

q5 문단의 줄 높이를 2.4 로 만드세요. 단위는 붙이지 않습니다.

기대 화면: 줄 사이가 눈에 띄게 벌어지고 그 문단의 세로가 54px 에서 77px 로 커집니다.

```
#q5 {
  line-height: 2.4;
}
```

화면 두 줄 사이가 확 벌어지고 그 문단의 세로가 54px 에서 77px 이 됩니다

(문단을 감싼 점선 상자는 143px 에서 165px 이 됩니다)
 $16\text{px} \times 2.4 = 38.4\text{px}$ 이 한 줄의 높이가 됩니다.
 이 페이지는 공통.css 가 이미 1.7 을 깔아 두었기 때문에,
 1.8 처럼 가까운 값을 주면 4px 밖에 안 달라져서 눈으로 구분이 안 됩니다.

line-height 는 단위를 안 붙이는 것이 원칙입니다(개념03 섹션2).

문제 6 (개념03 섹션4)

q6 문단의 글자 사이를 3px 로 벌리세요.

기대 화면: 글자 하나하나 사이가 벌어집니다.

```
#q6 {
  letter-spacing: 3px;
}
```

화면 글자마다 사이가 벌어져서 문장이 길어집니다

자간은 길이라서 단위가 반드시 필요합니다.

단위를 빼먹으면 아무 일도 일어나지 않습니다.

문제 7 (개념04 섹션1)

q7 문단의 글자를 가운데로 보내세요.

기대 화면: 글자만 가운데로 갑니다.

```
#q7 {
  text-align: center;
}
```

화면 글자가 상자 가운데로 옮겨집니다. 상자 자체는 움직이지 않습니다

가로 가운데는 center 입니다. middle 은 다른 속성의 값이라 통하지 않습니다.

문제 8 (개념04 섹션3)

q8 문단의 글자 가운데에 취소선을 그으세요.

기대 화면: 12,000원 한가운데에 가로줄이 그어집니다.

```
#q8 {  
  text-decoration: line-through;  
}
```

화면 12,000원 글자 한가운데에 가로줄이 그어집니다

밑줄은 underline, 취소선은 line-through 입니다.

할인 전 가격을 보여 줄 때 실제로 이렇게 씁니다.

문제 9 (개념04 섹션5)

q9 문단의 영어를 모두 대문자로 보이게 하세요. HTML 은 고치지 않습니다.

기대 화면: NEW ARRIVAL TODAY 로 보입니다.

```
#q9 {  
  text-transform: uppercase;  
}
```

화면 NEW ARRIVAL TODAY 로 보입니다

HTML 에 적힌 글자는 여전히 new arrival today 입니다.

마우스로 클릭해서 복사해 보면 소문자가 붙습니다. 보이는 모습만 바뀐 것입니다.

문제 10 (개념04 섹션4)

q10 링크의 밑줄을 없애고 글자색을 #c0392b 로 바꾸세요.

기대 화면: 밑줄이 사라지고 붉은 글자가 됩니다.

```
#q10 {
  text-decoration: none;
  color: #c0392b;
}
```

화면 밑줄이 사라지고 링크가 붉은색 글자가 됩니다

브라우저가 a 요소에 미리 걸어 둔 파란색과 밑줄을 우리가 덮어쓴 것입니다.

밑줄을 없앨 때는 이렇게 색이라도 남겨서 링크임을 알려 줘야 합니다.

문제 11 (개념05 섹션3)

q11 상자의 긴 주소를 잘라서 상자 안으로 들어오게 하세요.

기대 화면: 주소가 두 줄로 잘려 노란 배경 안에 모두 들어옵니다.

```
#q11 {
  overflow-wrap: break-word;
}
```

화면 주소가 두 줄로 잘려서 노란 배경 안에 모두 들어옵니다

break-word 는 한 줄에 도저히 안 들어가는 낱말만 자릅니다.

보통 문장에는 아무 영향이 없어서 본문에 걸어 두어도 안전합니다.

문제 12 [응용] (개념01 섹션4·5)

q12-parent 가 20px 일 때 q12-child 를 정확히 24px 로 만드세요. px 금지.

기대 화면: 회색 상자의 둘째 줄이 첫 줄보다 조금 큼니다.

```
#q12-child {
  font-size: 1.5rem;
}
```

화면 회색 상자 안 둘째 줄이 첫 줄(20px) 보다 조금 큰 24px 이 됩니다

rem 은 부모를 보지 않고 뿌리(html) 의 16px 만 봅니다. $16 \times 1.5 = 24$ 입니다.

em 을 썼다면 부모 20px 기준이라 1.5em 은 30px 이 되어 답이 틀립니다.

문제 13 [응용] (개념03 섹션2)

q13-parent 에 걸린 line-height: 26px 을 단위 없는 1.6 으로 고쳐
자식의 큰 글자가 안 겹치게 하세요.

기대 화면: 고치기 전에는 큰 글자 두 줄이 서로 닿아 있다가, 고치면 확
별어집니다.

(자식 줄 높이 26px → 44.8px)

```
#q13-parent {  
  background-color: #f0f7ff;  
  line-height: 1.6;  
}
```

화면 파란 상자 안의 28px 글자가 두 줄이 되어도 서로 닿지 않습니다

자식이 물려받는 것은 계산된 px 이 아니라 1.6 이라는 비율입니다.
그래서 자식이 자기 크기로 다시 계산합니다. $28 \times 1.6 = 44.8\text{px}$ 입니다.

만약 26px 처럼 단위를 붙였다면 26px 이 그대로 상속되어 글자가 겹칩니다.

문제 14 [도전] (개념05 섹션5)

q14 상자의 긴 제목을 한 줄로 줄이고 끝에 점 세 개가 나오게 하세요.

기대 화면: 한 줄만 보이고 끝이 ... 로 끝납니다.

```
#q14 {  
  overflow: hidden;  
  white-space: nowrap;  
  text-overflow: ellipsis;  
}
```

화면 "오늘의 공지사항입니다 자세한 내..." 까지만 보이고 점 세 개가 붙습니다

셋이 각각 다른 일을 합니다.
nowrap 이 한 줄로 만들고, hidden 이 넘친 부분을 감추고,

ellipsis 가 감춰진 자리에 점 세 개를 넣습니다. 하나라도 빠지면 안 됩니다.

문제 15 [도전] (개념01·02·03·04 종합)

q15 문단을 카드 제목처럼 꾸미세요.
1.5rem / 굵게 / 자간 2px / 가운데 / #2d6cdf / 줄 높이 1.3

기대 화면: 파란 굵은 큰 글자가 가운데에 놓입니다.

```
#q15 {
  font-size: 1.5rem;
  font-weight: bold;
  letter-spacing: 2px;
  text-align: center;
  color: #2d6cdf;
  line-height: 1.3;
}
```

화면 "이번 주 신상품 안내" 가 파란색 굵은 24px 글자로 가운데에 놓이고

글자 사이가 살짝 벌어집니다
여섯 줄의 순서는 아무래도 상관없습니다. 결과가 같습니다.

줄 높이 1.3 은 $24 \times 1.3 = 31.2\text{px}$ 입니다. 제목은 본문보다 좁게 잡습니다.

문제 16 [확인] (개념01 섹션7 · 개념04 섹션7)

원래 코드는 이랬습니다. 세 군데가 틀려 있었습니다.

```
#q16 {
  font-size: 28;          ← 단위 px 이 없습니다
  fontweight: bold;      ← 속성 이름에 하이픈이 빠졌습니다
  text-align: middle;    ← 그런 값은 없습니다. center 입니다
  color: #c0392b;        ← 이 줄만 멀쩡해서 색만 바뀌었습니다
}
```

틀린 곳 1: font-size 의 값에 단위가 없어서 그 줄이 버려졌습니다.
틀린 곳 2: fontweight 라는 속성은 없습니다. font-weight 입니다.
틀린 곳 3: 가로 가운데 정렬의 값은 middle 이 아니라 center 입니다.

기대 화면: 고치면 붉은 글자가 28px 굵기로 커지고 가운데로 갑니다.

```
#q16 {  
  font-size: 28px;  
  font-weight: bold;  
  text-align: center;  
  color: #c0392b;  
}
```

화면 붉고 굵은 28px 글자가 상자 가운데에 놓입니다

세 곳 모두 에러가 나지 않고 그 줄만 조용히 버려진 것이었습니다.

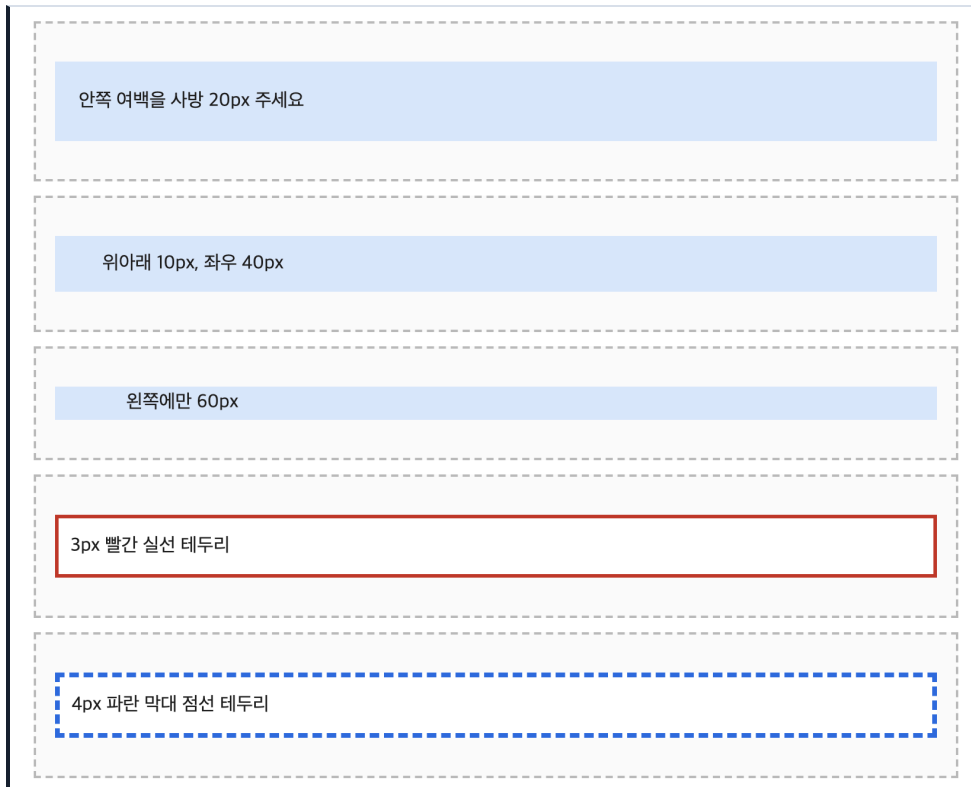
“일부만 적용됐다” 면 안 먹은 줄의 이름과 단위와 값을 하나씩 확인하세요.

10단원 · 연습문제 정답

10단원 연습문제 정답 박스모델

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

여러분이 푼 연습문제.html 과 나란히 열어 화면을 대조하세요.



실행하면 나오는 화면 — 박스 모델

답이 글자 하나까지 같을 필요는 없습니다. 화면이 같고, 개발자도로 잰 숫자가 같으면 맞은 것입니다. 예를 들어 문제 1 은 padding: 20px 20px 20px 20px 라고 써도 똑같습니다.

각 상자 위의 주석에 답과 해설이 있습니다. 답이 달라도 화면이 같으면 맞은 것입니다. 숫자가 애매하면 F12 개발자도로 재서 대조하세요.

문제 1 (개념02 섹션1)

답: padding: 20px;

해설: 값을 하나만 쓰면 네 방향 모두 같은 값이 됩니다.

padding: 20px 20px 20px 20px 라고 네 개를 다 써도 결과는 같습니다.
상자의 세로 길이가 27px 에서 67px 로 늘어납니다. $27 + 20 + 20 = 67$ 입니다.

화면 연파랑 상자가 위아래로 두꺼워지고 글자가 벽에서 20px 떨어집니다

안쪽 여백을 사방 20px 주세요

문제 2 (개념02 섹션3)

답: `padding: 10px 40px;`

해설: 값이 두 개면 “위아래 / 좌우” 입니다.

아래는 위의 맞은편이라 10px, 왼쪽은 오른쪽의 맞은편이라 40px 이 됩니다.
`padding: 10px 40px 10px 40px` 라고 써도 똑같습니다.

화면 글자가 왼쪽 벽에서 40px 떨어져 시작하고 위아래 여백은 얇습니다

위아래 10px, 좌우 40px

문제 3 (개념02 섹션4)

답: `padding-left: 60px;`

해설: 한 방향만 주고 싶을 때는 방향 이름이 붙은 속성을 씁니다.

`padding: 0 0 0 60px;` 라고 써도 결과는 같지만,
방향 이름을 쓰는 쪽이 무엇을 하려는지 한눈에 보입니다.

화면 글자만 오른쪽으로 밀리고 위아래 두께는 그대로입니다

왼쪽에만 60px

문제 4 (개념03 섹션2)

답: `border: 3px solid #c0392b;`

해설: 단축 border 는 두께 → 모양 → 색 순서로 씁니다.

`solid` 를 빠뜨리면 두께와 색을 아무리 정해도 선이 안 그어집니다.
`border-style` 의 기본값이 `none` 이기 때문입니다.

화면 흰 상자 둘레에 빨간 실선이 3px 두께로 그어집니다

3px 빨간 실선 테두리

문제 5 (개념03 섹션3)

답: `border: 4px dashed #2d6cdf;`

해설: dashed 는 짧은 막대가 끊어져 이어진 점선입니다.

dotted 는 동그란 점이라서 모양이 다릅니다.
점선의 굵긴 틈으로 상자의 배경색이 그대로 보입니다.

화면 흰 상자 둘레에 파란 막대 점선이 그어집니다

4px 파란 막대 점선 테두리

문제 6 (개념03 섹션4)

답: `border-bottom: 5px solid #27853f;`

해설: 방향 속성도 단축이라 안에 두께 · 모양 · 색을 그대로 씁니다.

border 단축을 쓰면 네 방향이 다 그어지므로 이 문제에서는 안 됩니다.
아래쪽만 그으면 상자의 세로 길이도 5px 만 늘어납니다.

화면 글자 아래에 초록색 굵은 선이 한 줄만 그어집니다

아래쪽에만 초록 실선

문제 7 (개념03 섹션5)

답: `border-radius: 12px;`

해설: 값 하나를 쓰면 네 모서리를 모두 12px 만큼 깎습니다.

50% 를 쓰면 완전히 둥근 모양이 되므로 이 문제의 답이 아닙니다.
테두리뿐 아니라 배경색도 같이 깎인다는 점을 확인하세요.

화면 보라 테두리 상자의 네 귀퉁이가 부드럽게 둥글어집니다

모서리를 12px 만큼 둥글게

문제 8 (개념04 섹션2)

답: `margin: 30px 0;`

해설: 값이 두 개면 “위아래 / 좌우” 입니다. padding 과 규칙이 같습니다.

좌우가 0 이라서 노란 상자의 가로는 점선 안쪽 폭 그대로 820px 입니다.
margin 자리에는 노란색이 안 칠해집니다. 흰 바탕이 그대로 보입니다.
p 요소에는 원래 위아래 margin 이 16px 씩 있는데, 이 답이 그것을 덮어씹습니다.

화면 노란 상자가 점선의 위아래 벽에서 30px 떨어지고 좌우로는 딱 붙습니다

위아래 30px, 좌우 0

문제 9 (개념04 섹션3)

답: `width: 320px;`

`margin: 0 auto;`

해설: 상자를 가운데로 보내려면 width 와 auto 가 둘 다 있어야 합니다.

width 가 없으면 상자가 한 줄을 다 먹어서 남는 자리가 0 이 되고,
auto 가 나눌 것이 없어 아무 일도 안 일어납니다.
text-align: center 를 쓰면 글자만 가운데로 가고 상자는 그대로입니다.
실제로 재면 상자의 왼쪽 끝이 화면 x 좌표 480 에 놓입니다.

화면 주황 상자가 점선 상자의 한가운데에 놓이고 좌우 흰 자리가 똑같습니다

가로 320px, 가운데로

문제 10 (개념05 섹션1)

답: `width: 250px;`

`height: 100px;`

해설: 숫자에는 반드시 단위를 붙입니다. width: 250 이라고 쓰면

그 줄이 버려져서 상자가 한 줄을 통째로 차지합니다.
 height 를 정하면 글자가 상자 위쪽에 붙습니다.
 height 는 자리를 만들 뿐 내용을 가운데로 옮겨 주지 않기 때문입니다.

화면 초록 상자가 왼쪽에 네모나게 자리 잡고 글자는 위쪽에 붙어 있습니다

가로 250px, 세로 100px

문제 11 [응용] (개념05 섹션3)

답: width: 100%;

max-width: 400px;

해설: width: 100% 는 "부모를 꽉 채워라",

max-width: 400px 은 "그래도 400 은 넘지 마라" 입니다.
 부모 안쪽이 824px 이므로 한계선이 이겨서 400px 이 됩니다.
 p 는 한 줄을 통째로 차지하는 상자라서 max-width: 400px 만 써도 화면은 같습니다.
 그래도 width: 100% 를 같이 적어 두면 "채우되 넘지는 마라" 라는 뜻이
 코드에 그대로 드러나서 읽기 좋습니다.
 width: 400px 으로 못 박으면 창을 좁혔을 때 같이 안 줄어듭니다.

화면 보라 상자의 가로가 정확히 400px 이 됩니다

꽉 채우되 400px 까지만

문제 12 [응용] (개념05 섹션5)

답: box-sizing: border-box;

width: 300px;

해설: box-sizing 없이 width: 300px 만 쓰면

$300 + 20 + 20 + 5 + 5 = 350\text{px}$ 이 됩니다.
 기본값 content-box 에서는 width 가 콘텐츠 영역만을 뜻하기 때문입니다.
 border-box 로 바꾸면 300 안에서 padding 과 border 가 자리를 나눠 쓰므로
 겉 크기가 정확히 300px 이 됩니다.
 이때 글자가 들어가는 콘텐츠 폭은 $300 - 40 - 10 = 250\text{px}$ 이 됩니다.

화면 초록 상자의 실제 가로가 개발자도구에서 300 으로 나옵니다

실제 가로를 300px 로

문제 13 [응용] (개념06 섹션2·3)

답: `background-image: url("무늬.svg");`

`background-repeat: no-repeat;`
`background-position: right top;`

해설: 세 줄을 단축으로 합쳐도 됩니다.

`background: #fff8dc url("무늬.svg") no-repeat right top;`
다만 단축을 쓰면 위에 이미 적어 둔 `background-color` 를 덮어쓰므로
색을 단축 안에 같이 적어야 합니다.
반복을 안 끄면 무늬가 상자를 가득 채워서 위치 지정이 의미가 없어집니다.

화면 연노랑 상자의 오른쪽 위 구석에 물방울 무늬가 하나만 보입니다

문제 14 [도전] (개념06 섹션6)

답: `height: 90px;`

`overflow: auto;`

해설: `overflow` 는 넘친 부분을 어떻게 할지 정합니다.

`auto` 는 넘칠 때만 스크롤바를 붙입니다.
`scroll` 은 안 넘쳐도 가로세로 스크롤바 자리를 잡아 버리고,
`hidden` 은 잘라서 읽을 방법을 없애 버립니다.
상자의 실제 세로는 $90 + 8 + 8 + 2 + 2 = 110\text{px}$ 입니다.

화면 회색 상자의 오른쪽에 세로 스크롤바가 생기고,

끌어내리면 나머지 글이 보입니다

이 상자의 세로를 90px 로 고정하면 이 글이 상자보다 길어집니다.
넘친 부분을 어떻게 할지 정하는 속성이 무엇이었는지 떠올려 보세요.
넘칠 때만 스크롤바가 생기는 값을 골라야 합니다.

문제 15 [도전] (개념02~06 종합)

답: `box-sizing: border-box;`

```
width: 360px;
margin: 0 auto;
padding: 20px;
border: 1px solid #cccccc;
border-radius: 8px;
background-color: #ffffff;
```

해설: 조건이 여섯 개지만 이 단원에서 배운 것을 하나씩 옮겨 적으면 됩니다.

함정은 첫 줄입니다. `box-sizing` 이 없으면
 $360 + 20 + 20 + 1 + 1 = 402\text{px}$ 이 되어 "정확히 360px " 조건이 깨집니다.
`margin: 0 auto` 는 `width` 가 있어야 일을 하므로 순서가 꼬여도 상관없지만,
`width` 를 빼먹으면 카드가 왼쪽에 붙은 채로 남습니다.
 실제로 재면 카드의 왼쪽 끝이 화면 x 좌표 460 에 놓입니다.

화면 점선 상자 한가운데에 모서리가 둥근 흰 카드가 놓입니다

가운데 놓인 360px 짜리 카드

문제 16 [확인] (개념02·03·04)

고친 곳 세 군데와 그 이유입니다.

1) `padding: 24;` → `padding: 24px;`

숫자에 단위가 없으면 CSS 가 그 줄을 통째로 버립니다.
 그래서 여백이 하나도 안 생겼습니다.

2) `border-width: 4px;` → `border: 4px solid #c0392b;`

`border-color: #c0392b;`
`style` 이 없으면 그을 선 자체가 없습니다.
`border-style` 의 기본값이 `none` 이라서 두께도 0 으로 계산됩니다.

3) `margin: auto;` → `width: 400px;`

```
margin: 0 auto;
```

width 가 없으면 상자가 한 줄을 다 먹어서 남는 자리가 0 입니다.
auto 가 나눌 것이 없으니 가운데로 못 갑니다.

셋 다 에러가 나지 않는다는 점이 핵심입니다. 브라우저는 못 알아듣는 줄을 조용히 버리고 나머지만 그립니다. 그래서 “아무 일도 안 일어난다” 를 보고 원인을 역추적할 줄 알아야 합니다.

실제로 재 보면 이렇습니다. 상자의 가로 $400 + 24 + 24 + 4 + 4 = 456\text{px}$ 왼쪽 끝 위치 화면 x 좌표 412

화면 연노랑 상자가 가운데에 놓이고 빨간 실선 테두리가 둘러지며

글자가 상자 벽에서 24px 떨어집니다

세 군데를 고쳐 주세요

정리

10단원 연습문제 정답
답이 달라도 화면이 같으면 맞은 것입니다. 가는 길은 여러 개입니다.

문제 8 · 9 · 15 에서 기준선 역할을 하는 바깥 상자입니다.

```
.q-stage {  
  border: 2px dashed #bbbbbb;  
}
```

문제 1

```
#q1 {  
  background-color: #dbe7fb;  
  padding: 20px;  
}
```

문제 2

```
#q2 {  
  background-color: #dbe7fb;  
  padding: 10px 40px;  
}
```


문제 3

```
#q3 {  
  background-color: #dbe7fb;  
  padding-left: 60px;  
}
```

문제 4

```
#q4 {  
  background-color: #ffffff;  
  padding: 10px;  
  border: 3px solid #c0392b;  
}
```

문제 5

```
#q5 {  
  background-color: #ffffff;  
  padding: 10px;  
  border: 4px dashed #2d6cdf;  
}
```

문제 6

```
#q6 {  
  background-color: #ffffff;  
  padding: 8px 0;  
  border-bottom: 5px solid #27853f;  
}
```

문제 7

```
#q7 {  
  background-color: #f3ecff;  
  border: 2px solid #6b3fc0;  
  padding: 12px;  
  border-radius: 12px;  
}
```

문제 8

```
#q8 {  
  background-color: #ffe9a8;  
  margin: 30px 0;  
}
```

문제 9

```
#q9 {  
  background-color: #ffd9a8;  
  width: 320px;  
  margin: 0 auto;  
}
```

문제 10

```
#q10 {  
  background-color: #d7f0d7;  
  width: 250px;  
  height: 100px;  
}
```

문제 11

```
#q11 {  
  background-color: #f3dcf3;  
  width: 100%;  
  max-width: 400px;  
}
```

문제 12

```
#q12 {  
  background-color: #d7f0d7;  
  padding: 20px;  
  border: 5px solid #27853f;  
  box-sizing: border-box;  
  width: 300px;  
}
```

문제 13

```
#q13 {  
  height: 120px;  
  border: 1px solid #999999;  
  background-color: #fff8dc;  
  background-image: url("무늬.svg");  
  background-repeat: no-repeat;  
  background-position: right top;  
}
```

문제 14

```
#q14 {
  width: 300px;
  padding: 8px;
  border: 2px solid #888888;
  background-color: #f4f4f4;
  height: 90px;
  overflow: auto;
}
.q14-text {
  margin: 0;
}
```

문제 15

```
#q15 {
  box-sizing: border-box;
  width: 360px;
  margin: 0 auto;
  padding: 20px;
  border: 1px solid #cccccc;
  border-radius: 8px;
  background-color: #ffffff;
}
```

문제 16

고친 곳 세 군데

- 1) padding: 24; → padding: 24px; 단위를 붙였습니다
- 2) border-width / border-color 만 있고 style 이 없었습니다
→ border: 4px solid #c0392b; 한 줄 단축으로 합쳤습니다
- 3) margin: auto; 만 있고 width 가 없었습니다

→ width: 400px; 를 더하고 margin: 0 auto; 로 바꿨습니다

```
#q16 {  
  background-color: #fff8dc;  
  padding: 24px;  
  border: 4px solid #c0392b;  
  width: 400px;  
  margin: 0 auto;  
}
```


부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.